

Business Process Technologies and Management

1. Introduction

Prof. Dr. Stefanie Rinderle-Ma

Dr. Jürgen Mangler

Technical University of Munich

Department of Informatics

Chair of Information Systems

and Business Process Management

stefanie.rinderle-ma@tum.de

juergen.mangler@tum.de



Overview

- Introduction
- Conceptual Process Modeling
- Process Verification
- Process Execution and Automation
- Interorganizational Processes
- Process Monitoring and Logging
- Summary and Outlook

Teaching objectives

This course shall

- develop your *analytical skills* in respect to the analysis, design and modeling of business processes
- give you insights into *fundamental concepts* of process-aware information systems
- enable you to create executable process models
- develop your *skills* in implementing process-aware applications
- *increase your knowledge* about current trends in BPM research and BPM tools

Literature

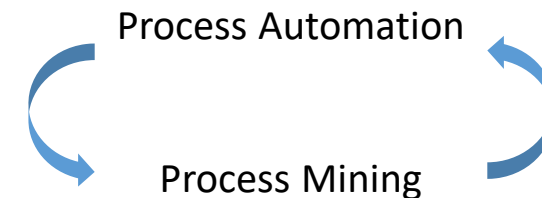


- Weske, M. (2019) Business Process Management - Concepts, Languages, Architectures, Third Edition. Springer 2019, ISBN 978-3-662-59431-5, pp. 1-417 2019
- Marlon Dumas, Marcello La Rosa, Jan Mendling, Hajo A. Reijers: Fundamentals of Business Process Management, Second Edition. Springer 2018, ISBN 978-3-662-56508-7, pp. 1-527
- Wil M. P. van der Aalst: Process Mining - Data Science in Action, Second Edition. Springer 2016, ISBN 978-3-662-49850-7, pp. 3-452
- Wilfried Grossmann, Stefanie Rinderle-Ma: Fundamentals of Business Intelligence. Data-Centric Systems and Applications, Springer 2015, ISBN 978-3-662-46530-1, pp. 1-327
- Manfred Reichert, Barbara Weber: Enabling Flexibility in Process-Aware Information Systems - Challenges, Methods, Technologies. Springer 2012, ISBN 978-3-642-30408-8, pp. I-XVIII, 1-515

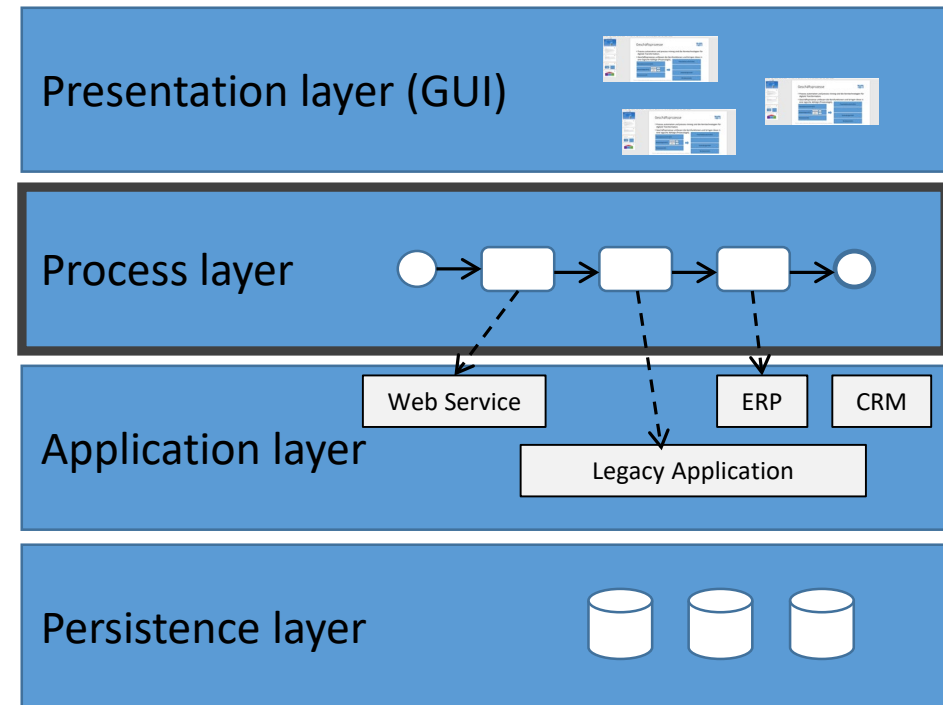
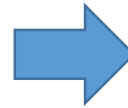
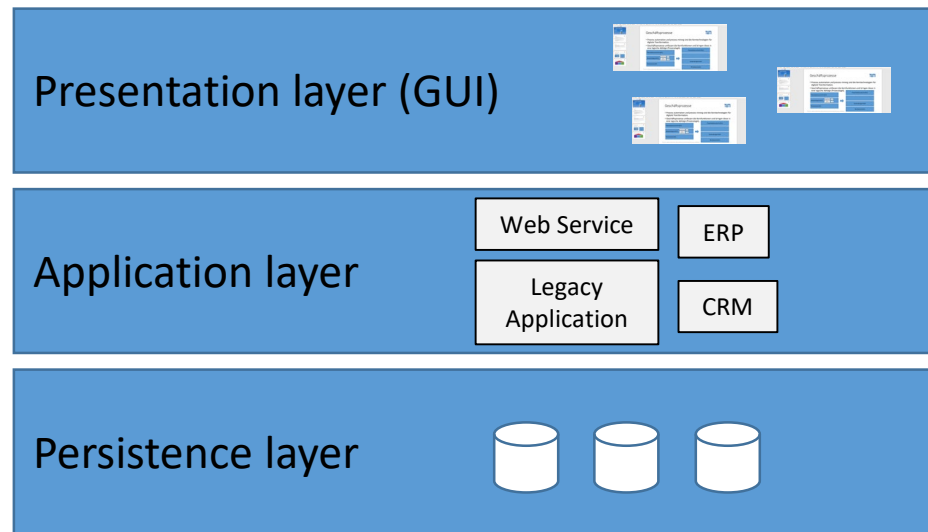
→ more literature provided in the chapters

Motivation

- Business process technologies and management enable
 - documentation
 - optimization
 - Process automation
 - separation of process logic and application code
 - application integration
 - contextualized data collection
 - Process mining:
 - improves transparency
 - provides insights
- key technologies for digital transformation
- independent of the application domain

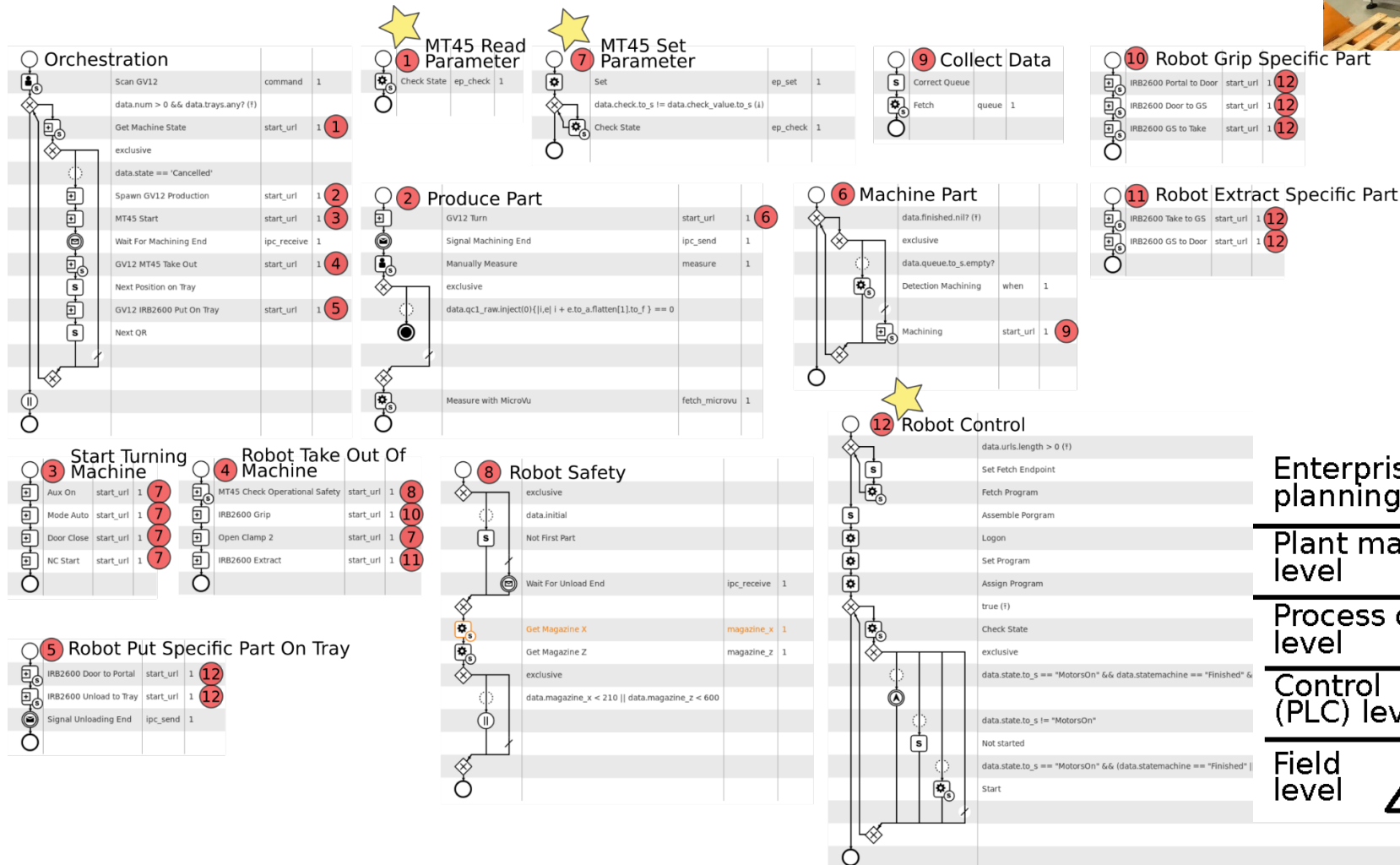


Enterprise architecture



Let us look at one of the most challenging application domains...

Manufacturing processes



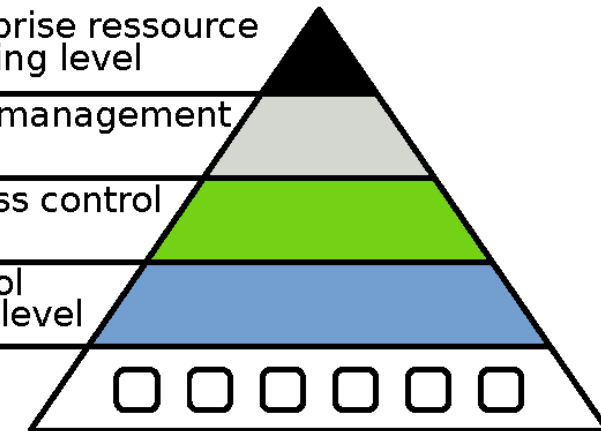
Enterprise resource planning level

Plant management level

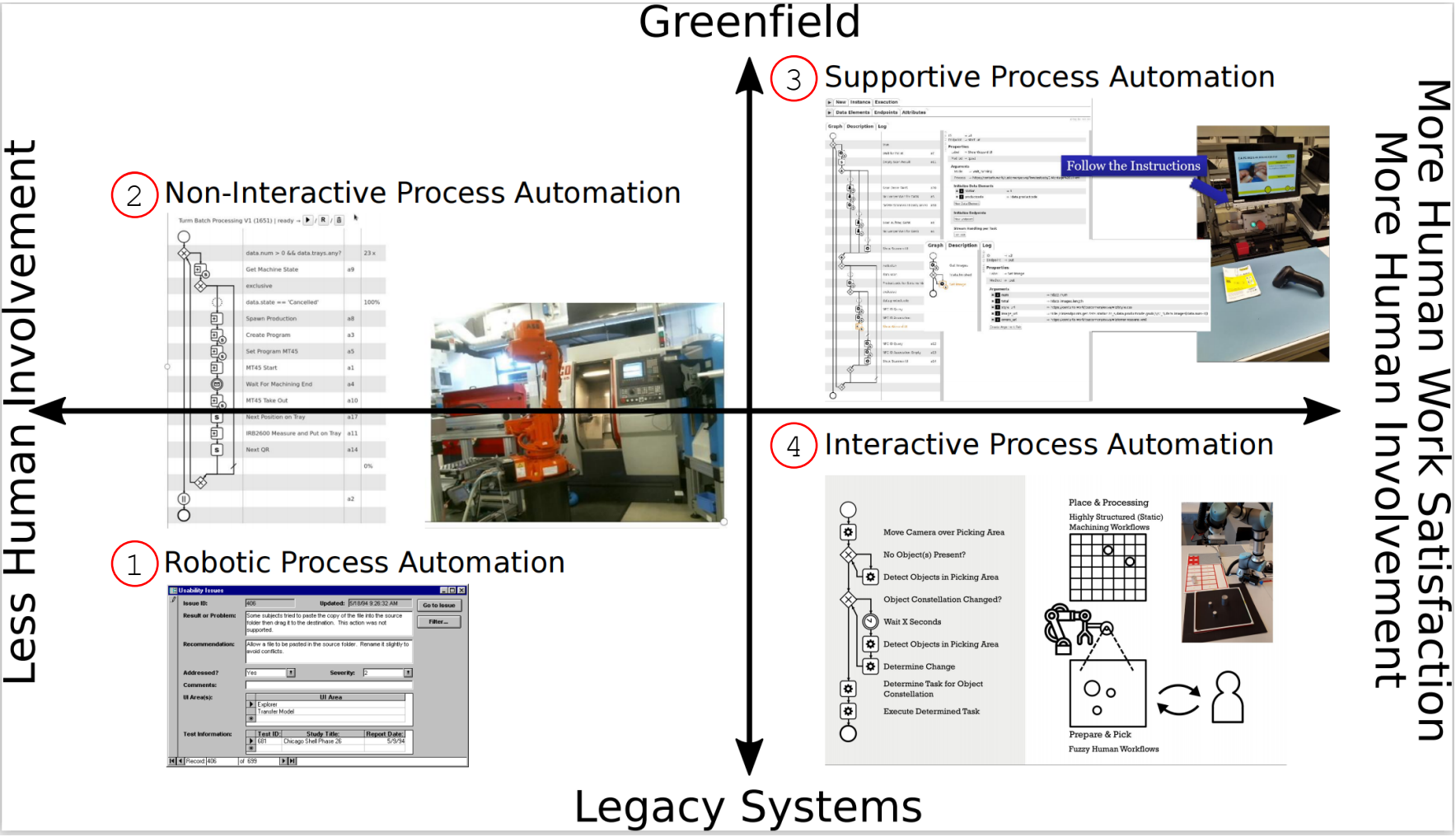
Process control level

Control (PLC) level

Field level



Process automation scenarios

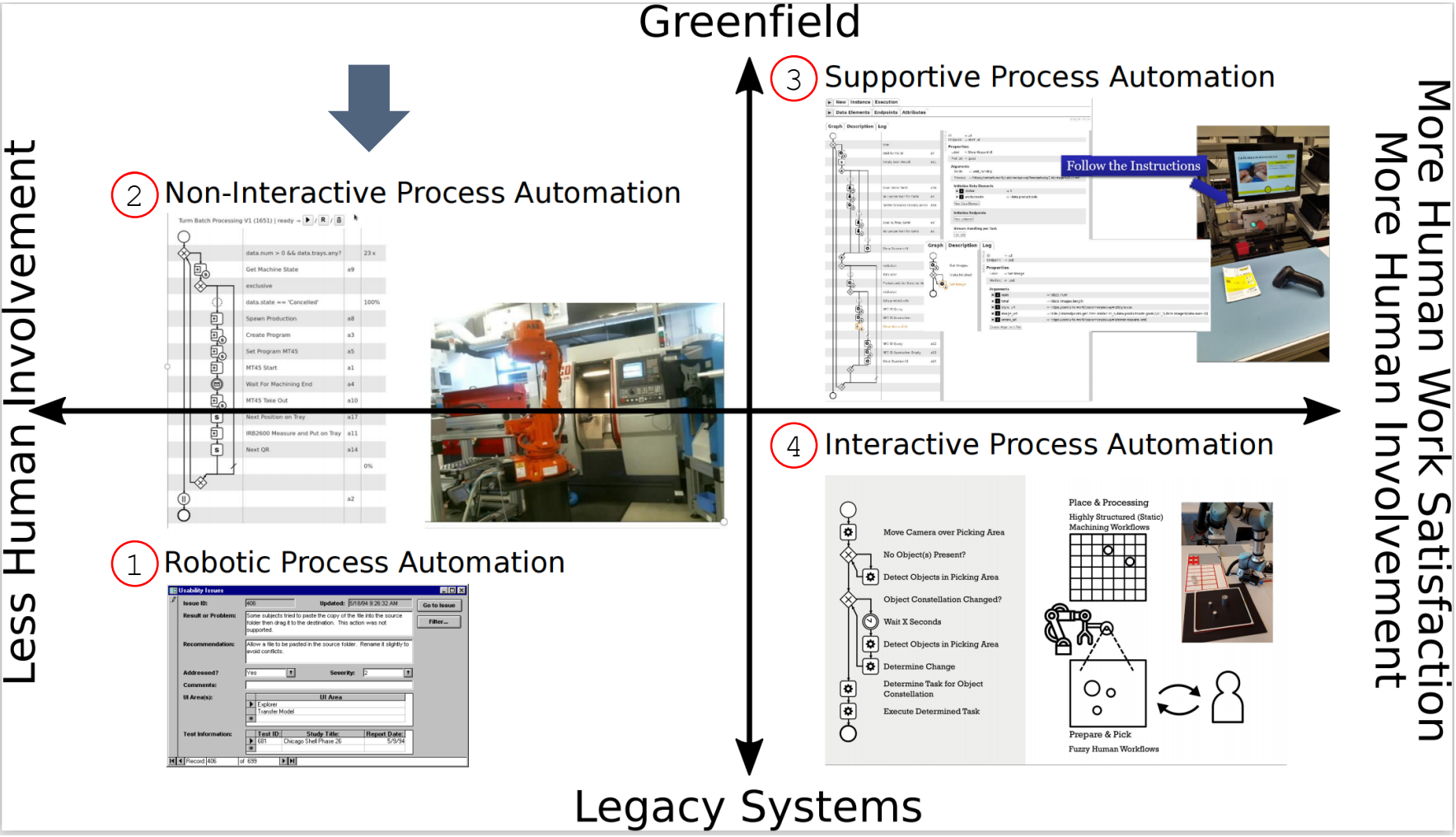


Experience



- Almost two decades of research
- Manufacturing process orchestration engine centurio.work based on cpee.org
- Currently 16 manufacturing process scenarios have been realized across 7 companies
- Benefits: traceability, flexibility, sustainability

Process automation scenarios

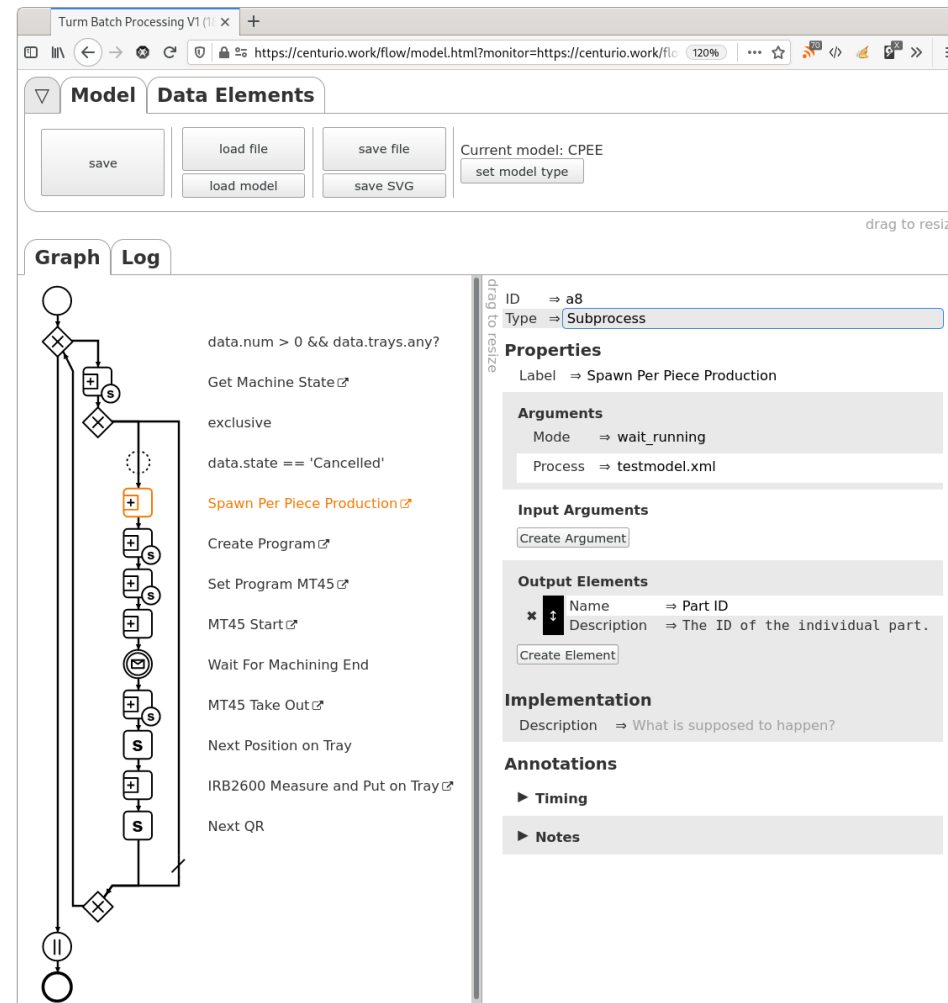


Non-interactive process automation

- Mix of legacy machines and hardware
- Goal: fully automatic production of batches
- What was automated?
 - Starting the machine (repetitive)
 - Taking manual measurements
- Benefits:
 - Skilled worker can work on other tasks
 - Reduction of errors by manual measurement

Conceptual design

- Requirements engineering
 - Textual description of input.
 - Textual description of output.
 - Limited set of task types.
- Specify control flow
- Batch View vs. Product View vs. Machine View
- Production vs. Production (Sensor) Data Collection



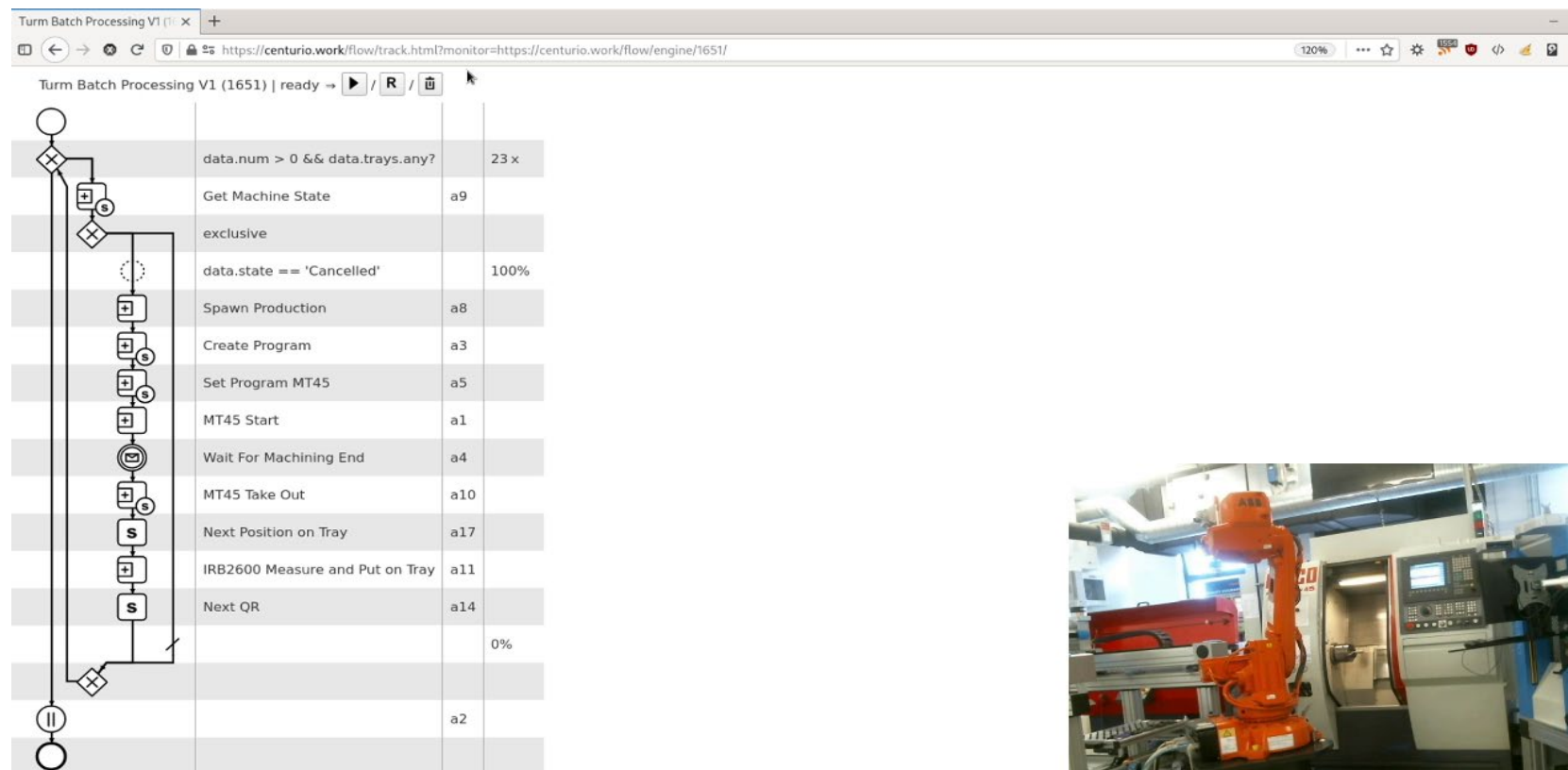
List of Models:

Ce-E-Pord_01_Nur_ein_Zylinder.xml	Juergen Mangler	Johannes Cech	2020-05-20, 08:53:11	[delete]
Ce-E-Pord_02_Nur_ein_Zylinder.xml	Johannes Cech	Johannes Cech	2020-05-25, 17:35:34	[delete]
Ce-E-Pord_03_Nur_ein_Zylinder.xml	Johannes Cech	Johannes Cech	2020-05-26, 15:12:14	[delete]
Ce-E-Pord_04_Nur_ein_Zylinder.xml	Juergen Mangler	Johannes Cech	2020-06-08, 10:46:23	[delete]
Ce-E-Pord_04_Nur_ein_Zylinder_ST1.xml	Juergen Mangler	Juergen Mangler	2020-05-28, 14:51:55	[delete]
Ce-E-Pord_04_Nur_ein_Zylinder_ST1_2.xml	Florian Pauker	Johannes Cech	2020-06-09, 11:52:42	[delete]
Ce-E-Pord_04_Nur_ein_Zylinder_ST2.xml	Juergen Mangler	Juergen Mangler	2020-05-28, 15:17:31	[delete]
Ce-E-Pord_04_Nur_ein_Zylinder_ST2_2.xml	Johannes Cech	Johannes Cech	2020-06-09, 11:52:42	[delete]
Ce-E-Pord_04_Nur_ein_Zylinder_ST3.xml	Juergen Mangler	Juergen Mangler	2020-05-28, 15:04:28	[delete]
Ce-E-Pord_04_Nur_ein_Zylinder_ST3_2.xml	Johannes Cech	Johannes Cech	2020-06-03, 09:43:54	[delete]
Ce-E-Pord_04_Nur_ein_Zylinder_ST3_21.xml	Johannes Cech	Johannes Cech	2020-06-09, 11:54:43	[delete]
Ce-E-Pord_04_Nur_ein_Zylinder_ST4.xml	Juergen Mangler	Johannes Cech	2020-06-02, 09:27:38	[delete]
Ce-E-Pord_04_Nur_ein_Zylinder_ST4_2.xml	Johannes Cech	Johannes Cech	2020-06-05, 12:59:22	[delete]
Ce-E-Pord_04_Nur_ein_Zylinder_ST4_21.xml	Johannes Cech	Johannes Cech	2020-06-09, 11:55:28	[delete]

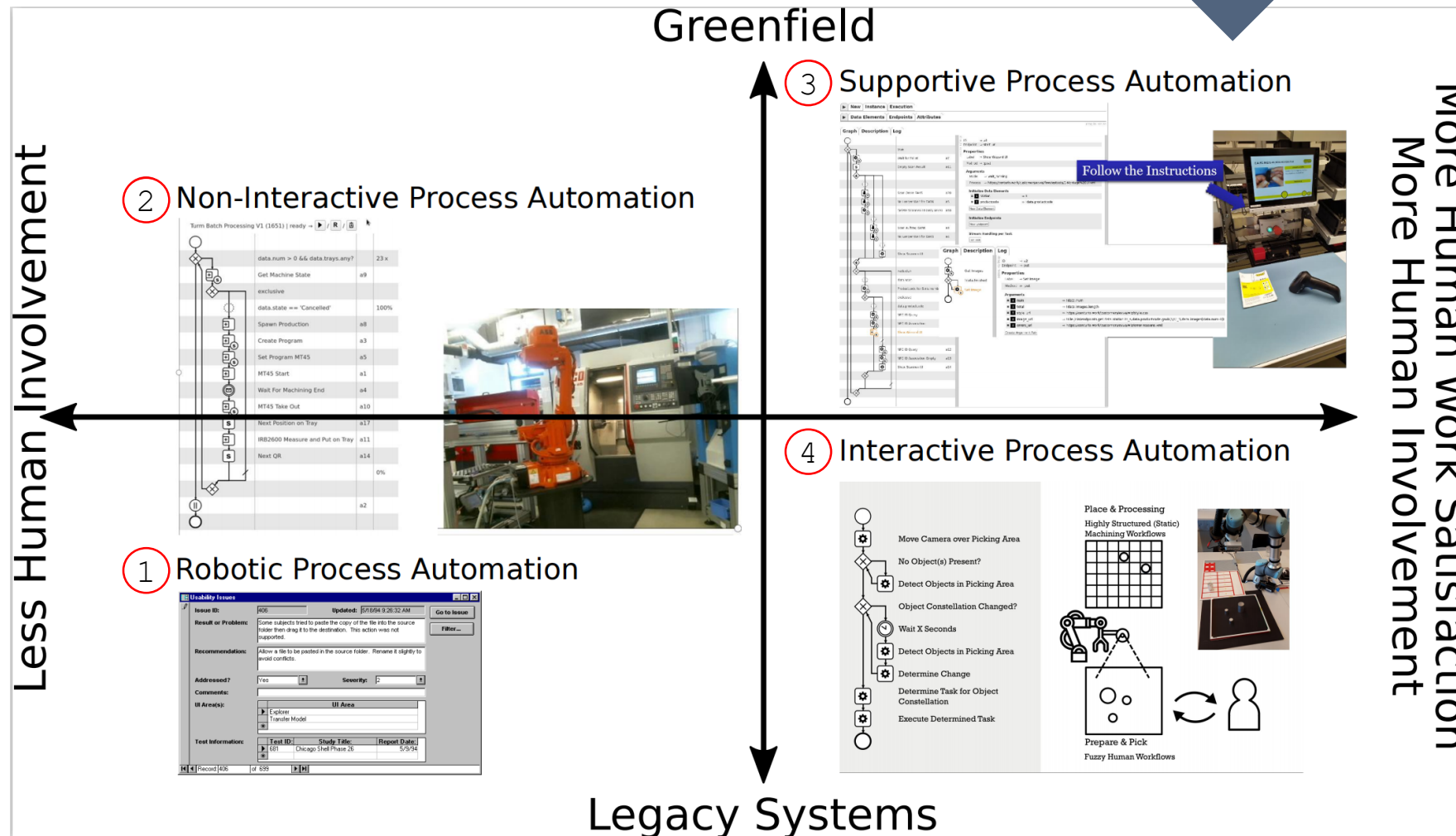
Execution design

- Bridge Execution Gap
 - Hook up functionalities & pass parameters
 - Handle response (Finalize)
 - Handle partial Response (Update)
 - Handle problems (Rescue)
 - Annotate for runtime compliance checking
 - Handle the collection of complementary sensor data, processing and automatic visualization (data stream handling)
- Step By Step Testing and Evolution

[illegible]



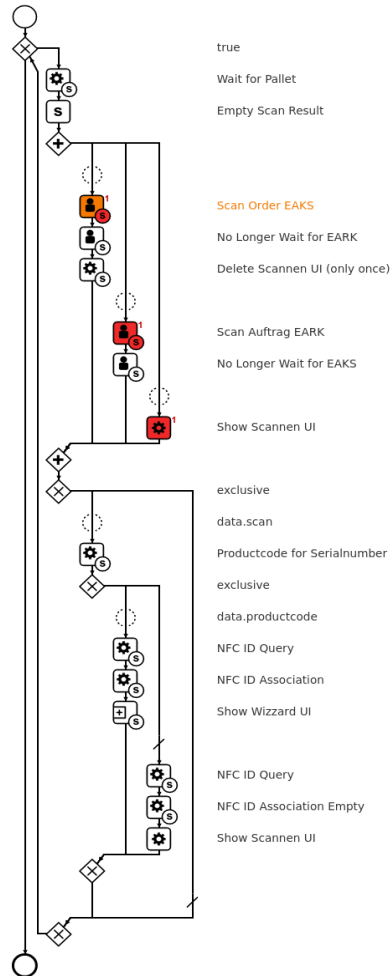
Process automation scenarios



Station 1: Scan Packing Label & Equipping Work Piece Carriers



EVVA E-Montage Station 1 (177) | running →



Station 1: Scan Packing Label & Equipping Work Piece Carriers



Follow the Instructions

Graph **Description** **Log**

Graph	Description	Log
Start		
Wait for Pallet	a7	
Empty Scan Result	a11	
Scan Order EAKS	a20	
No Longer Wait for EARK	a5	
Delete Scannen UI (only once)	a10	
Scan Auftrag EARK	a3	
No Longer Wait for EAKS	a4	
Show Scannen UI	a9	
Exclusive		
data.scan		
Productcode for Serialnumber	a8	
Exclusive		
data.productcode		
NFC ID Query	a2	
NFC ID Association	a6	
Show Wizard UI	a1	
NFC ID Query	a12	
NFC ID Association Empty	a13	
Show Scannen UI	a14	

Properties

ID → a1
Endpoint → start_url

Arguments

Mode → wait_running
Process → https://centurio.work/customers/evva/flow/testsets/E-Montage%20UI.xml

Initialize Data Elements

station → 1
productcode → !data.productcode

Initialize Endpoints

Stream Handling per Task

Data Stream Handling

Implementation

Annotations

Timing

Notes

Graph **Description** **Log**

Graph	Description	Log
Get Images		
!data.finished		
Set Image		

ID → a2
Endpoint → put

Properties

Label → Set Image
Method → :put

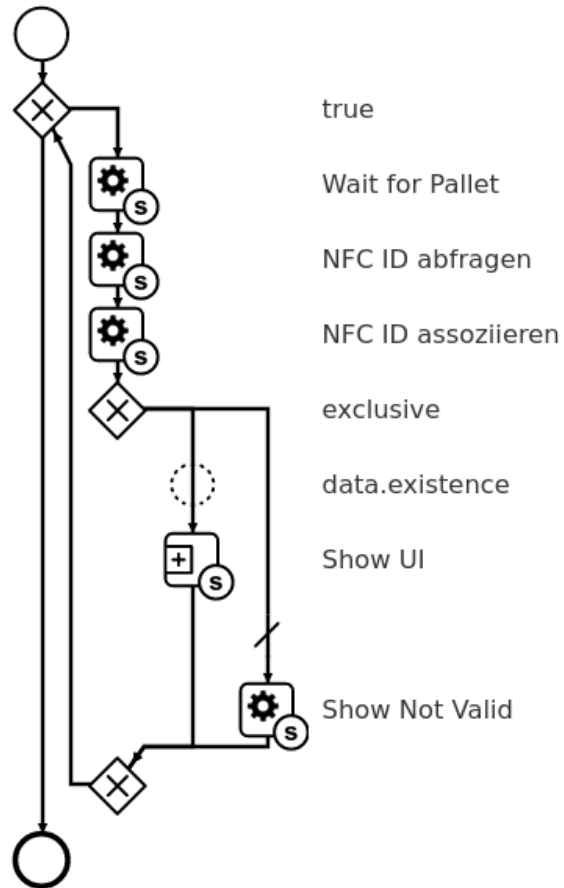
Arguments

num → !data.num
total → !data.images.length
style_url → https://centurio.work/customers/evva/wiz/style.css
image_url → !File.join(endpoints.get.data.station.to_s_data.productcode.gsub(/\/.*/, '_').data.images[data.num-1])
errors_url → https://centurio.work/customers/evva/wiz/error-reasons.xml

Station 2: Detect Work Piece Carrier, Show Associated Instructions



EVVA E-Montage Station 2 (960) | ready ⇒ ▶ / R / 🗑️



true

Wait for Pallet

NFC ID abfragen

NFC ID assoziieren

exclusive

data.existence

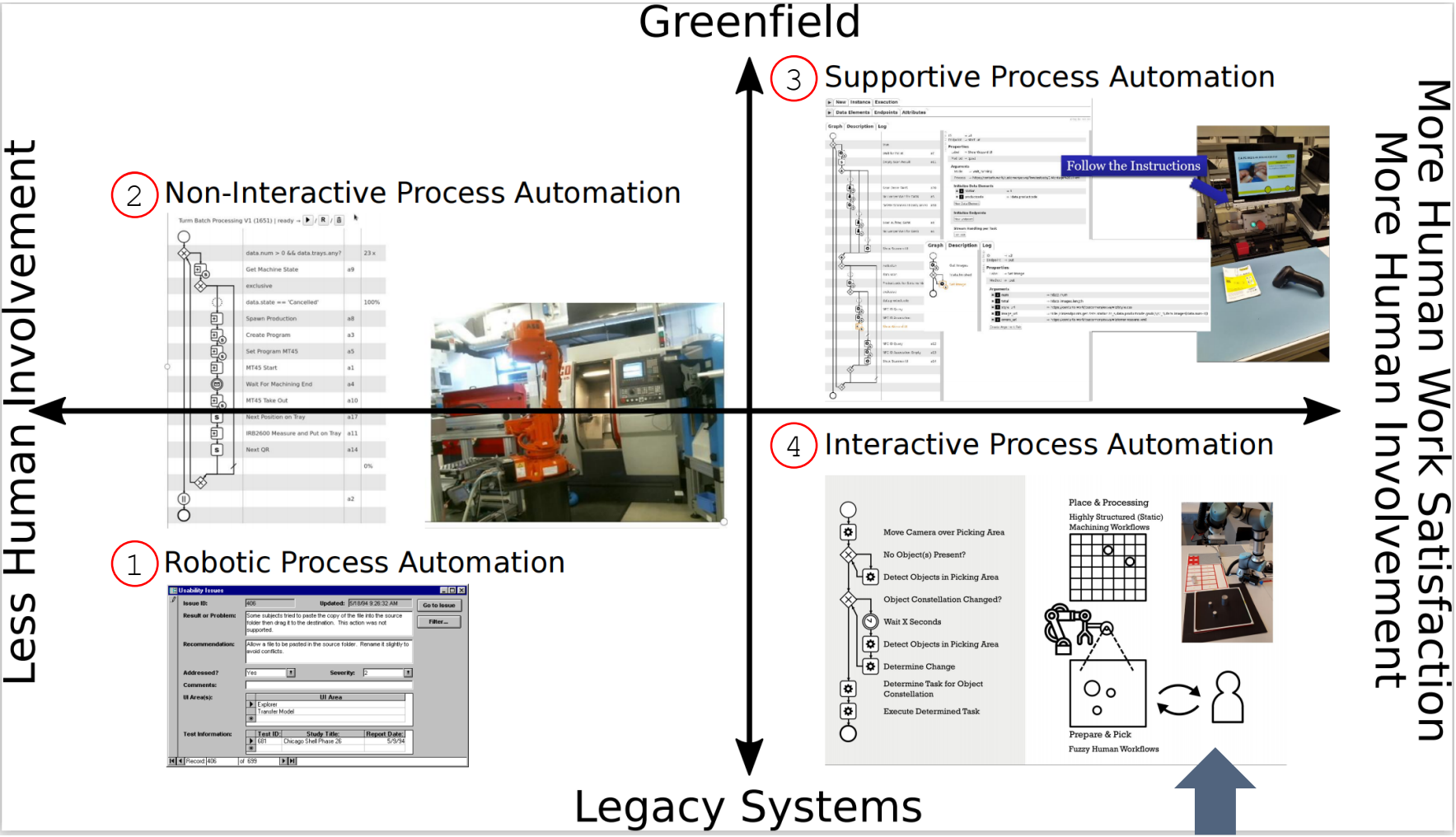
Show UI

Show Not Valid

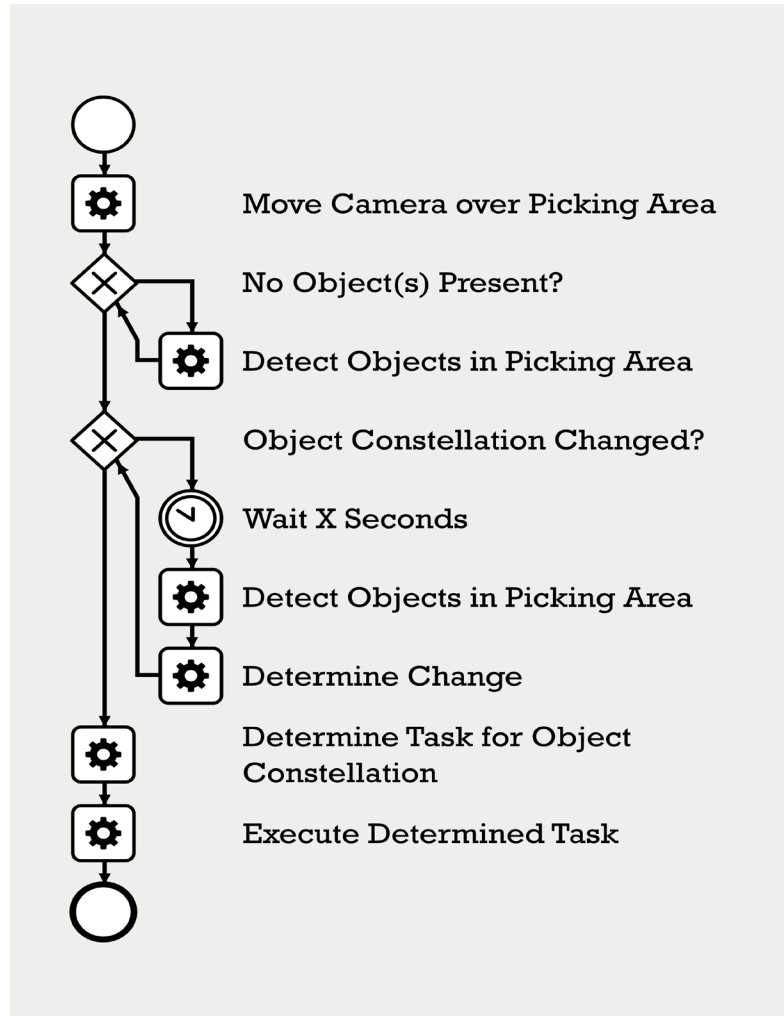
Follow the Instructions



Process automation scenarios

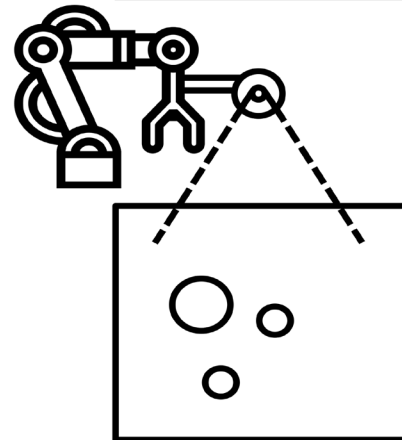
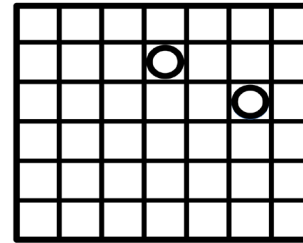


Interactive process automation



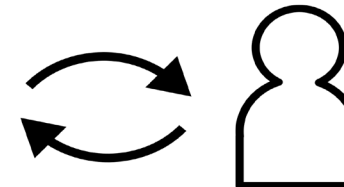
Place & Processing

Highly Structured (Static)
Machining Workflows



Prepare & Pick

Fuzzy Human Workflows

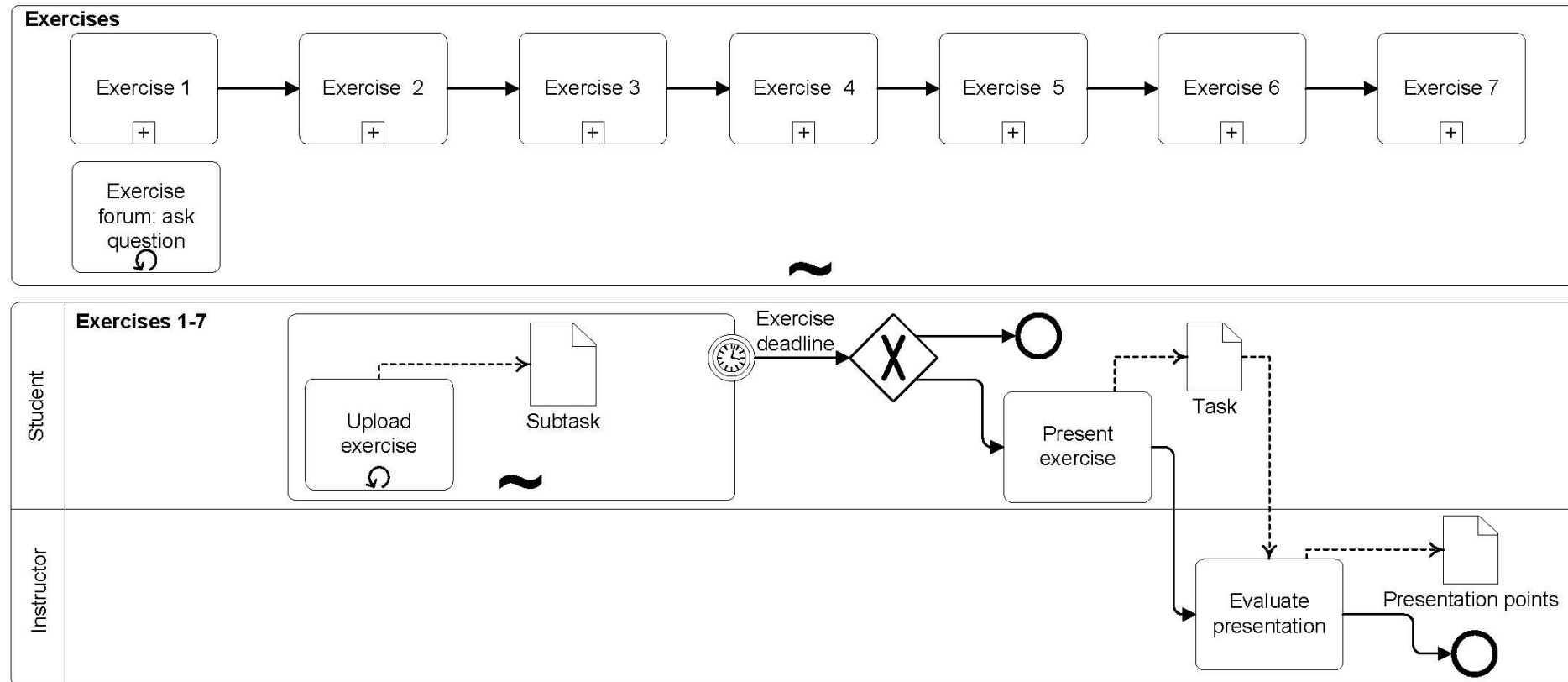


(c) A. Mangat

Read (open access): A. S. Mangat, J. Mangler, S. Rinderle-Ma:
Interactive Process Automation based on lightweight object
detection in manufacturing processes. Comput. Ind. 130: 103482
(2021)

Process technology and its application is domain-independent.

Higher education processes

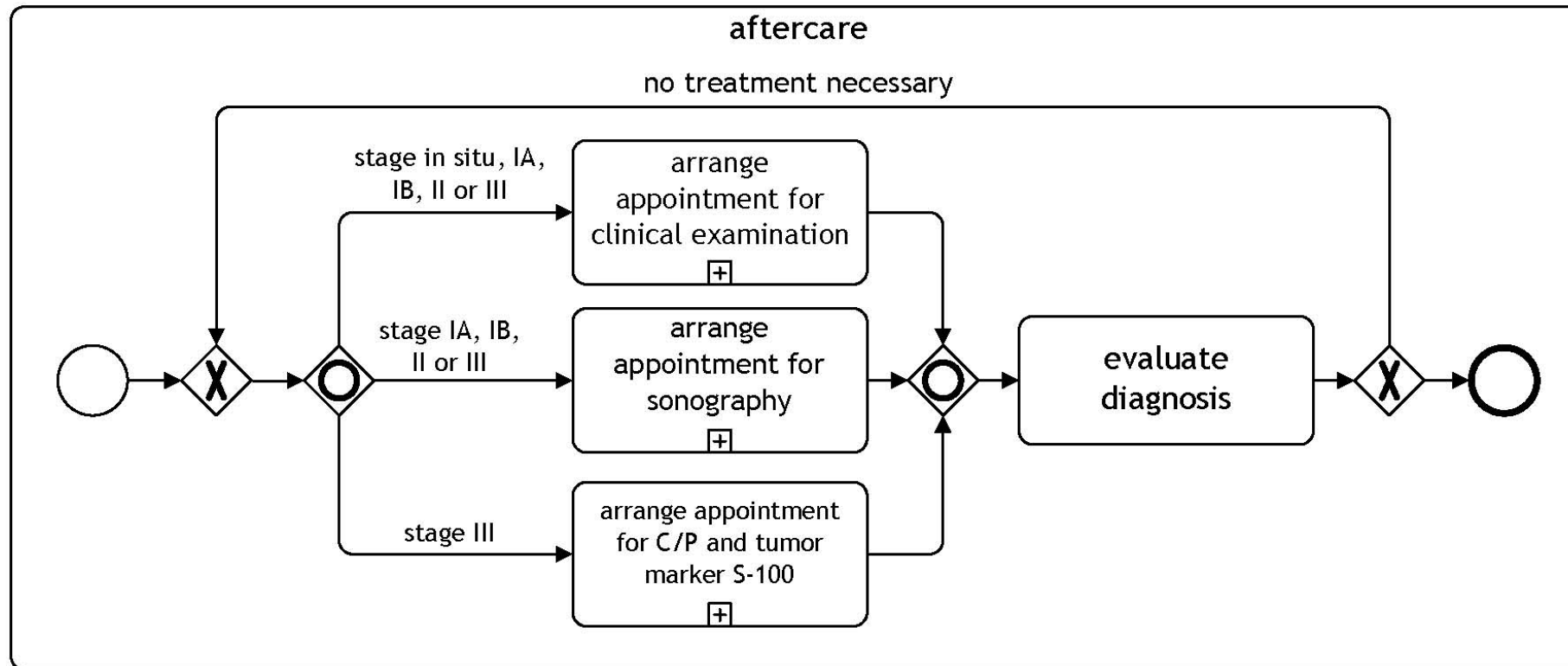


Source:

© Springer

Linh Thao Ly, Conrad Indiono, Jürgen Mangler, Stefanie Rinderle-Ma: Data Transformation and Semantic Log Purging for Process Mining. CAiSE 2012: 238-253

Skin cancer treatment processes



Source:
© Springer
Reinhold Dunkl, Karl Anton Fröschl, Wilfried Grossmann, Stefanie Rinderle-Ma: Assessing Medical Treatment Compliance Based on Formal Process Modeling. USAB 2011: 533-546

Process-aware Information Systems



- are aware of the logic of the implemented business process(es) (which is often defined in terms of an explicit *process model*)
- ensure that *process steps* (= *activities*) are executed in the right order during *runtime* (i.e., the PAIS manages the *flow of control*)
- control the *flow of data* between process activities (i.e., the data being exchanged between activities)
- provide an integrated view on *process and application data*
- know which *application component* shall be invoked when a particular activity is started
- pass input parameters to invoked applications and stores their output parameters in the *process data context*

Process-aware Information Systems



- add manual tasks (i.e., tasks requiring user interactions) to the *worklists* of those users who qualify for this task based on their assigned *roles* (e.g., secretary, medical doctor)
- *synchronize* concurrent changes of these worklists
- provide *reminders* if started activities are not finished in time
- enable *process participants* to *monitor* the progress of the executed processes and to record process-related events in a logfile (= *audit trail, execution log, event stream during runtime*)

Summary Chapter 1: Introduction

- Process technologies are key drivers for digital transformation
- Huge benefits through automation, integration, and mining
- Key skills:
 - process design and modeling, conceptual and execution
 - process-oriented application development
 - process analytics

Business Process Technologies and Management

2. Conceptual Process Modeling

Prof. Dr. Stefanie Rinderle-Ma

Dr. Jürgen Mangler

Technical University of Munich

Department of Informatics

Chair of Information Systems

and Business Process Management

stefanie.rinderle-ma@tum.de

juergen.mangler@tum.de



Literature and links



- Weske, M. (2019) Business Process Management - Concepts, Languages, Architectures, Third Edition. Springer 2019, ISBN 978-3-662-59431-5, pp. 1-417 2019
- Marlon Dumas, Marcello La Rosa, Jan Mendling, Hajo A. Reijers: Fundamentals of Business Process Management, Second Edition. Springer 2018, ISBN 978-3-662-56508-7, pp. 1-527
- <https://www.signavio.com/bpm-academic-initiative/>
- <https://cpee.org/>

Process modeling levels

- Semantic or conceptual process modeling
 - Event-driven process chains (EPC), ARIS
<https://www.ariscommunity.com/aris-express>
 - Business Process Modeling and Notation (BPMN), Standard,
www.bpmn.org
- Logic, formal languages:
 - Petri nets, Workflow Nets
 - Refined Process Structure Tree (RPST) → J. Vanhatalo, H. Völzer, J. Koehler:
 The refined process structure tree. Data Knowl. Eng. 68(9): 793-818 (2009)
- Execution languages: graphical programming
 - Examples: BPEL, WSM-Nets, CPEE-Trees
- Transformation between levels is possible, but has to be done carefully!

→ Capturing the process logic, communication

→ Verification, Analysis

→ Process implementation and execution

Process modeling levels

- Semantic or conceptual process modeling
 - Event-driven process chains (EPC), ARIS
<https://www.ariscommunity.com/aris-express>
 - **Business Process Modeling and Notation (BPMN)**, Standard,
www.bpmn.org
- Logic, formal languages:
 - **Petri nets, Workflow Nets**
 - **Refined Process Structure Tree (RPST)** → J. Vanhatalo, H. Völzer, J. Koehler: The refined process structure tree. Data Knowl. Eng. 68(9): 793-818 (2009)
- Execution languages: graphical programming
 - Examples: BPEL, WSM-Nets, **CPEE-Trees**
- Transformation between levels is possible, but has to be done carefully!

→ Capturing the process logic, communication

→ Verification, Analysis

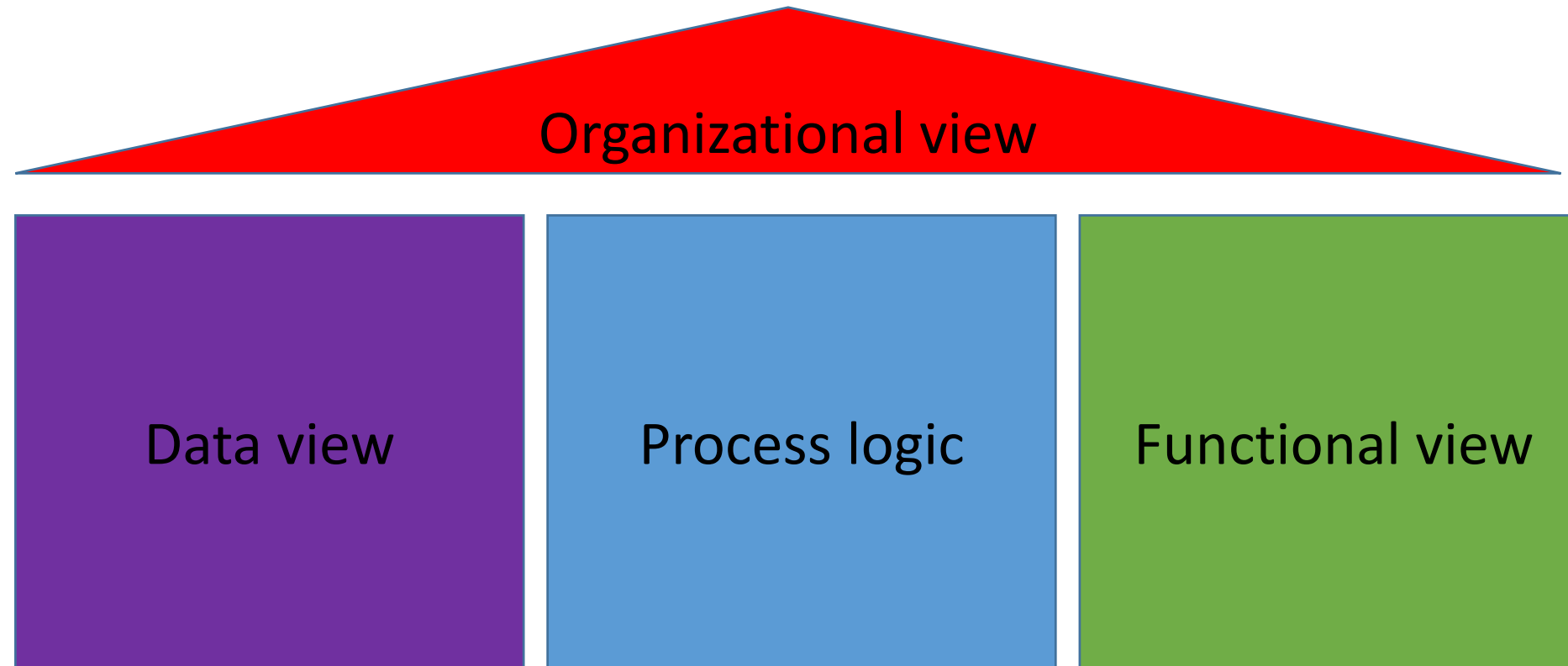
→ Process implementation and execution

Example: Customer complaint



Customers state their complaint via an online form. The form contains the insurance number, the amount of damage, and a description of the damage (mandatory). Optionally, customers can upload pictures for further documentation. If the amount of damage is ≤ 100 Euro, an automatic plausibility check is conducted and the complaint is approved. The customer is informed via email and the money transfer is initiated. If the damage amount is above 100 Euro, a clerk is checking the complaint. The customer is informed on the decision via email. If the complaint is rejected, the customer is offered an optional meeting.

Process perspectives



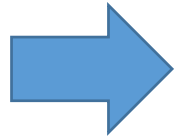
Example: Customer complaint

red:
green:
blue:
purple:



Customers state their complaint via an online form. The form contains the insurance number, the amount of damage, and a description of the damage (mandatory). Optionally, customers can upload pictures for further documentation. If the amount of damage is ≤ 100 Euro, an automatic plausibility check is conducted and the complaint is approved. The customer is informed via email and the money transfer is initiated. If the damage amount is above 100 Euro, a clerk checks the complaint. The customer is informed on the decision via email. If the complaint is rejected, the customer is offered an optional meeting.

Process modeling levels



- Semantic or conceptual process modeling
 - Event-driven process chains (EPC), ARIS
<https://www.ariscommunity.com/aris-express>
 - **Business Process Modeling and Notation (BPMN)**, Standard,
www.bpmn.org
- Logic, formal languages:
 - **Petri nets, Workflow Nets**
 - **Refined Process Structure Tree (RPST)** → J. Vanhatalo, H. Völzer, J. Koehler: The refined process structure tree. Data Knowl. Eng. 68(9): 793-818 (2009)
- Execution languages: graphical programming
 - Examples: BPEL, WSM-Nets, **CPEE-Trees**
- Transformation between levels is possible, but has to be done carefully!

→ Capturing the process logic, communication

→ Verification, Analysis

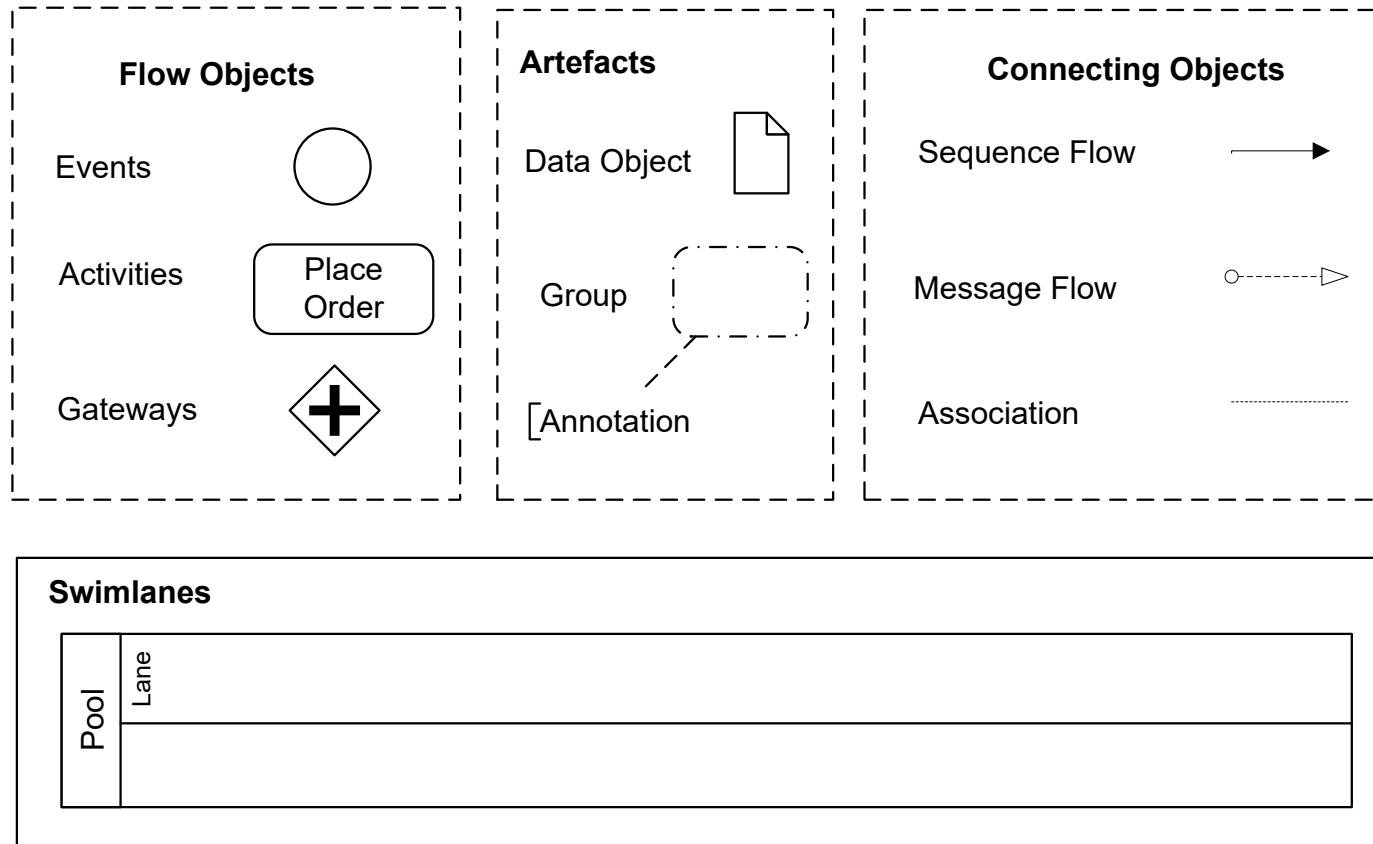
→ Process implementation and execution

BPMN (recap)



- Current process modeling standard: <https://www.bpmn.org/>
- Web-based modeling with Signavio: <https://academic.signavio.com/p/login>
- Many other tools available
- Selection of specific design goals:
 - Modeling of interorganizational processes → communication between partners via message exchanges
 - Previously: Mapping to process execution language WS-BPEL (Business Process Execution Language for Web Services), deprecated
 - Mapping to execution languages is necessary as formal semantics is only partly available
- In this course: mapping of BPMN to Petri Nets for formal analysis
- And to CPEE trees (<https://cpee.org/>) for process automation and execution

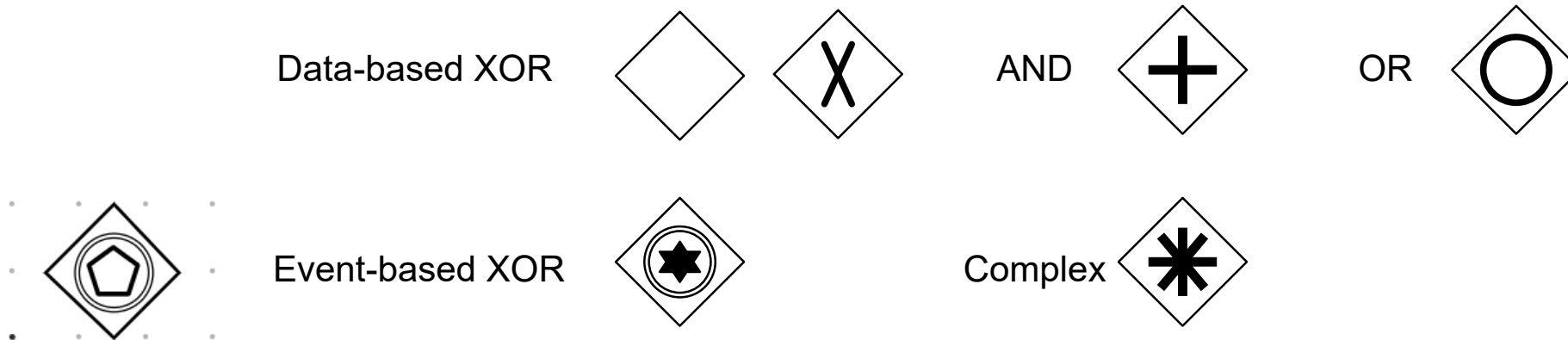
BPMN (recap)



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.78. Business Process Modeling Notation: categories of elements

BPMN (recap)



Gateway types in the BPMN, Object Management Group (2006)

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

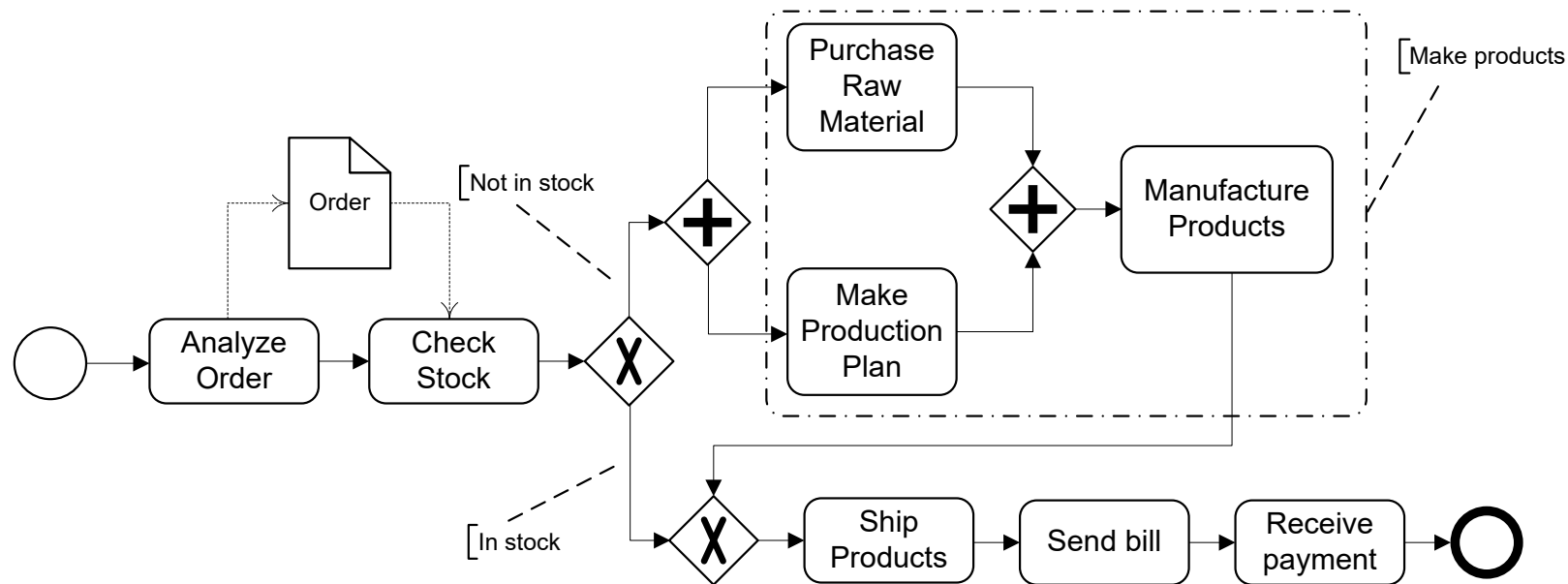
BPMN (recap)

		Message	Timer	Rule	Error	Link	Multiple
Start							
Intermediate							
End							
Termination							

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.80. Event types in the BPMN, Object Management Group (2006)

BPMN (recap)



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.77. Business process diagram expressed in BPMN

Example: Customer complaint

red:
green:
blue:
purple:

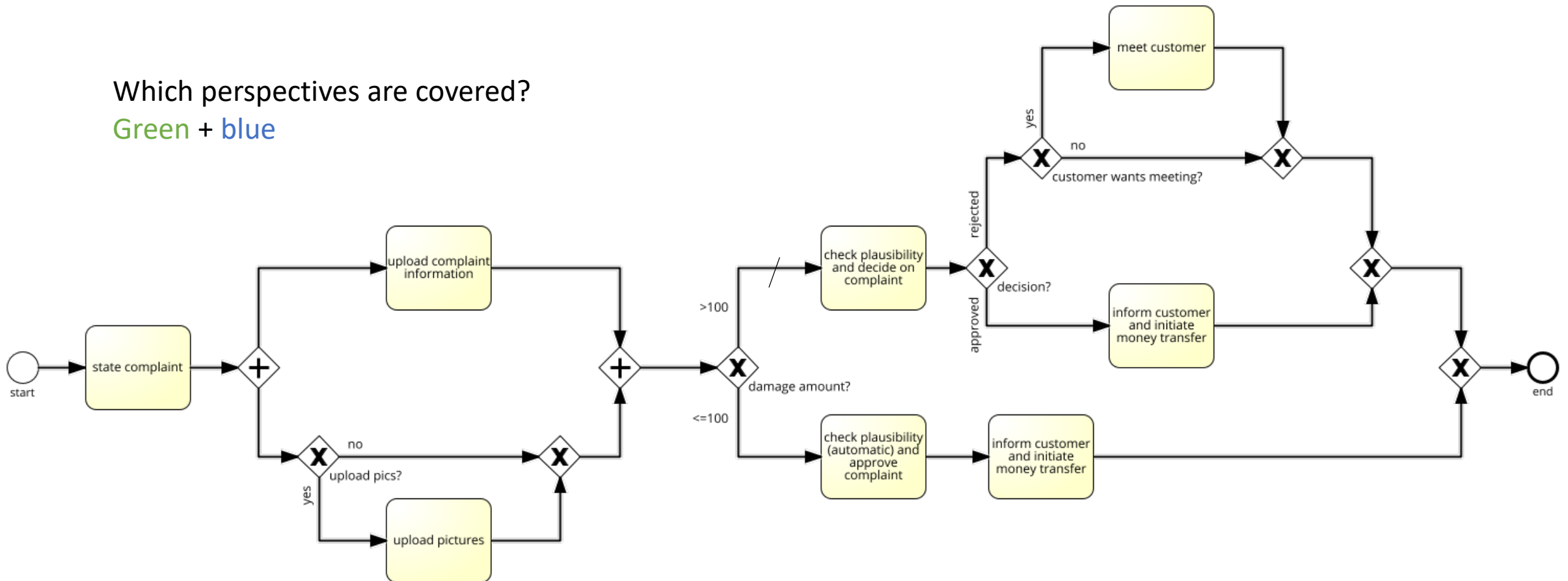


Customers state their complaint via an online form. The form contains the insurance number, the amount of damage, and a description of the damage (mandatory). Optionally, customers can upload pictures for further documentation. If the amount of damage is ≤ 100 Euro, an automatic plausibility check is conducted and the complaint is approved. The customer is informed via email and the money transfer is initiated. If the damage amount is above 100 Euro, a clerk checks the complaint. The customer is informed on the decision via email. If the complaint is rejected, the customer is offered an optional meeting.

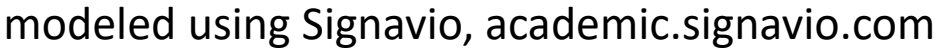
BPMN process model – control flow only

Which perspectives are covered?

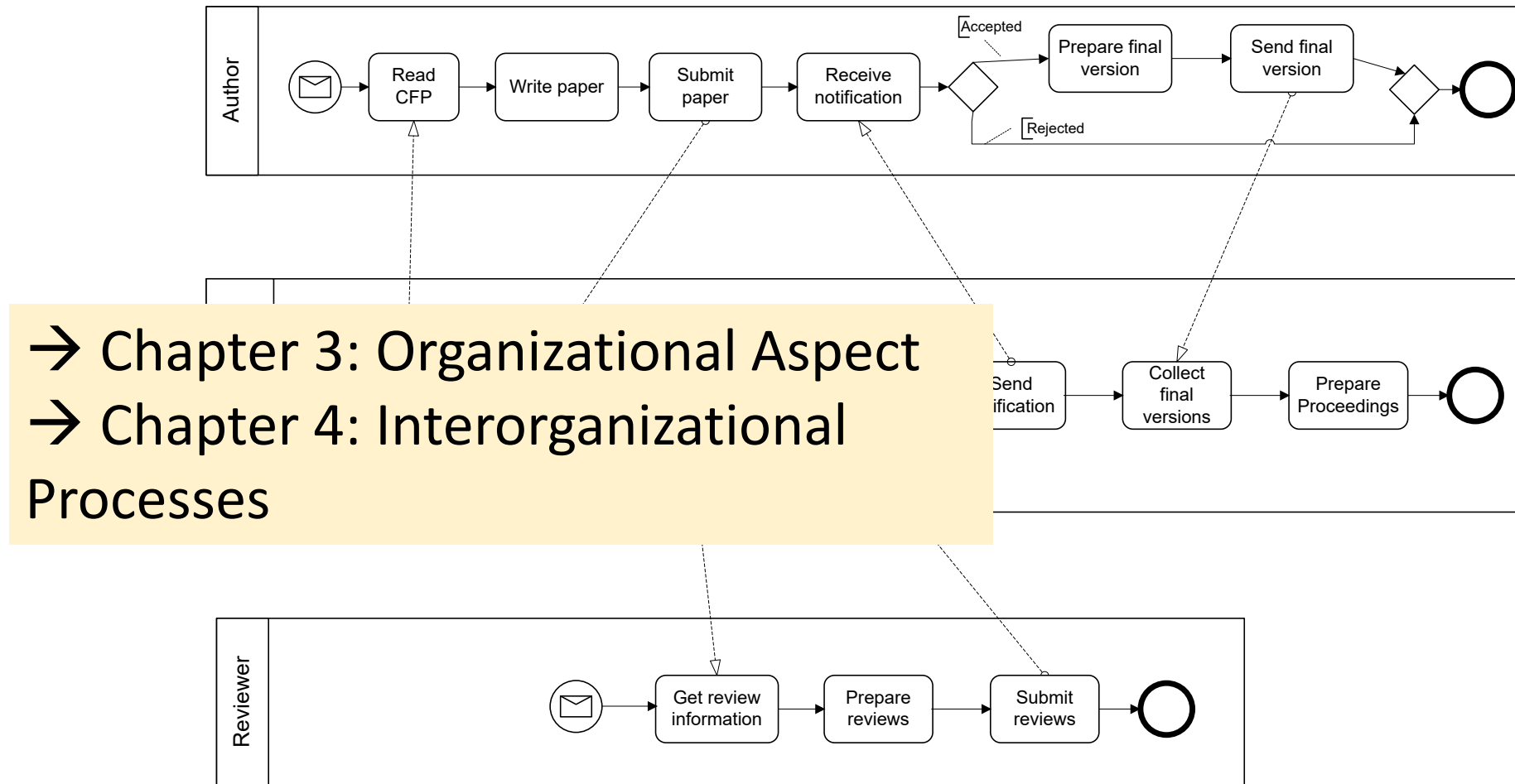
Green + blue



modeled using Signavio, academic.signavio.com



BPMN (recap)



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.79. Business process diagram of a scientific conference review process

BPMN (recap)

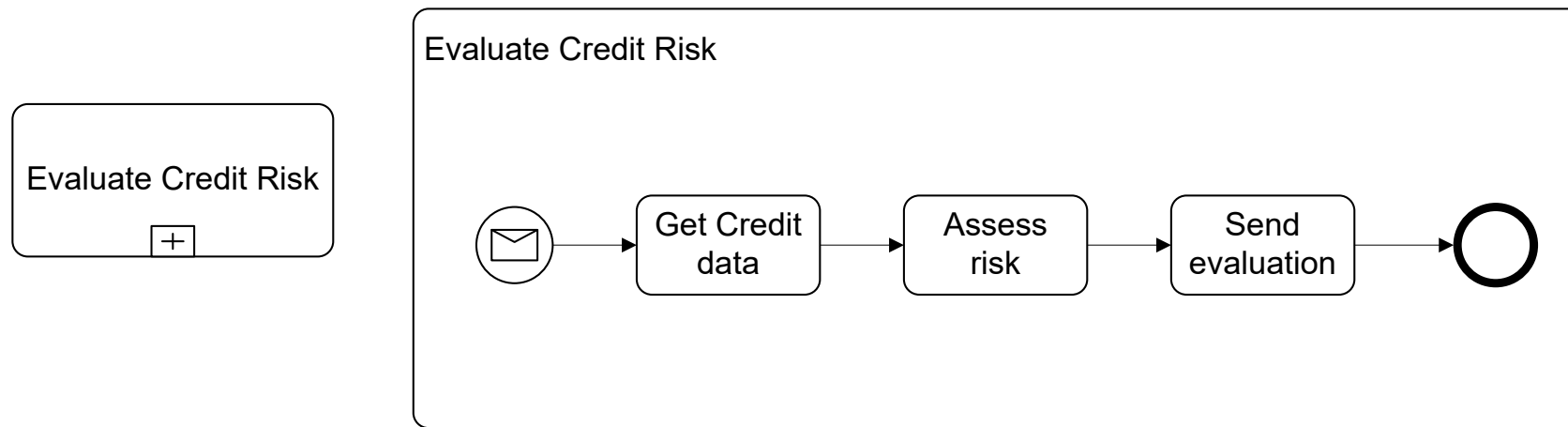


Fig 4.81. Collapsed and expanded subprocess

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

BPMN (recap)

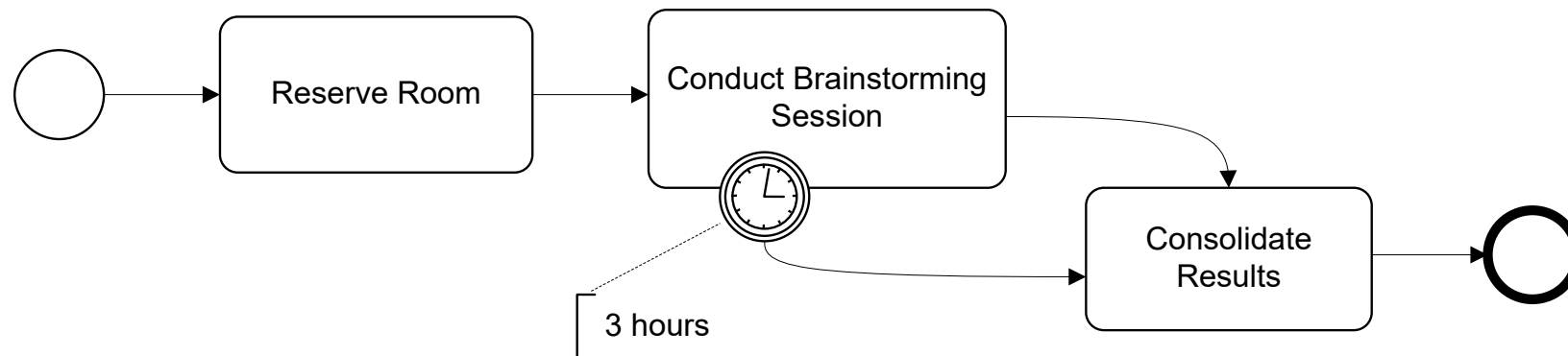
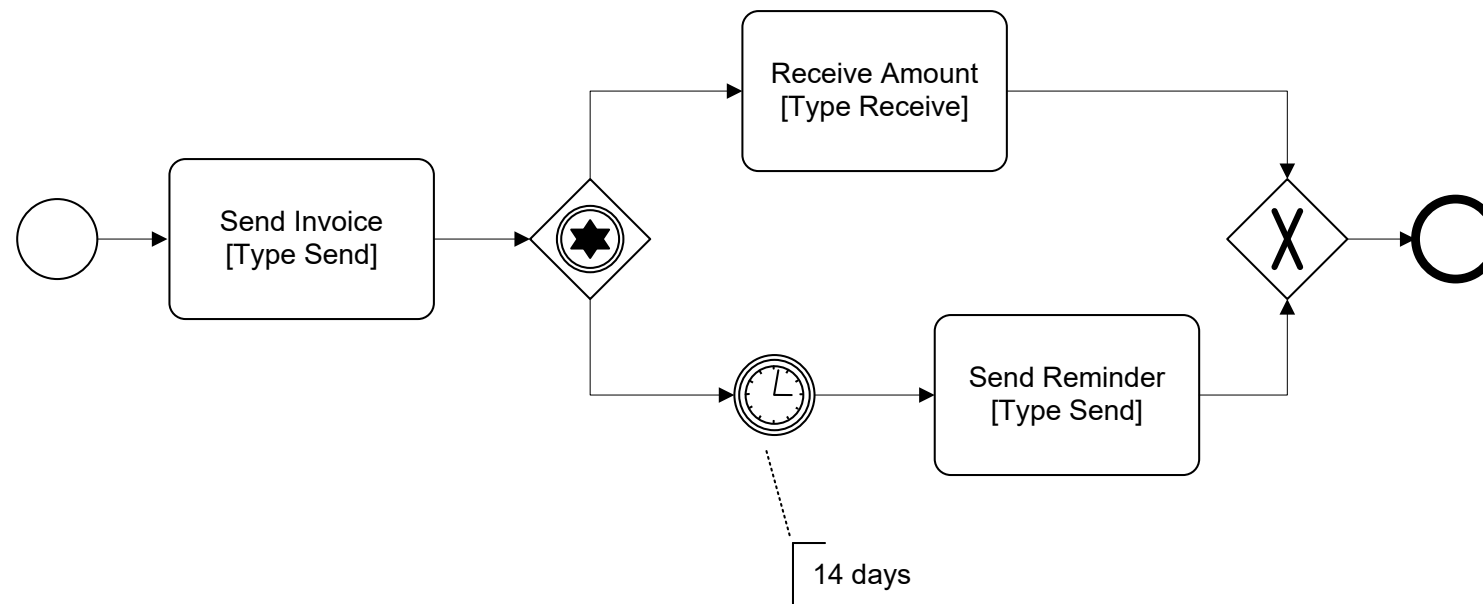


Fig 4.83. Exception flow, triggered by intermediate timer event

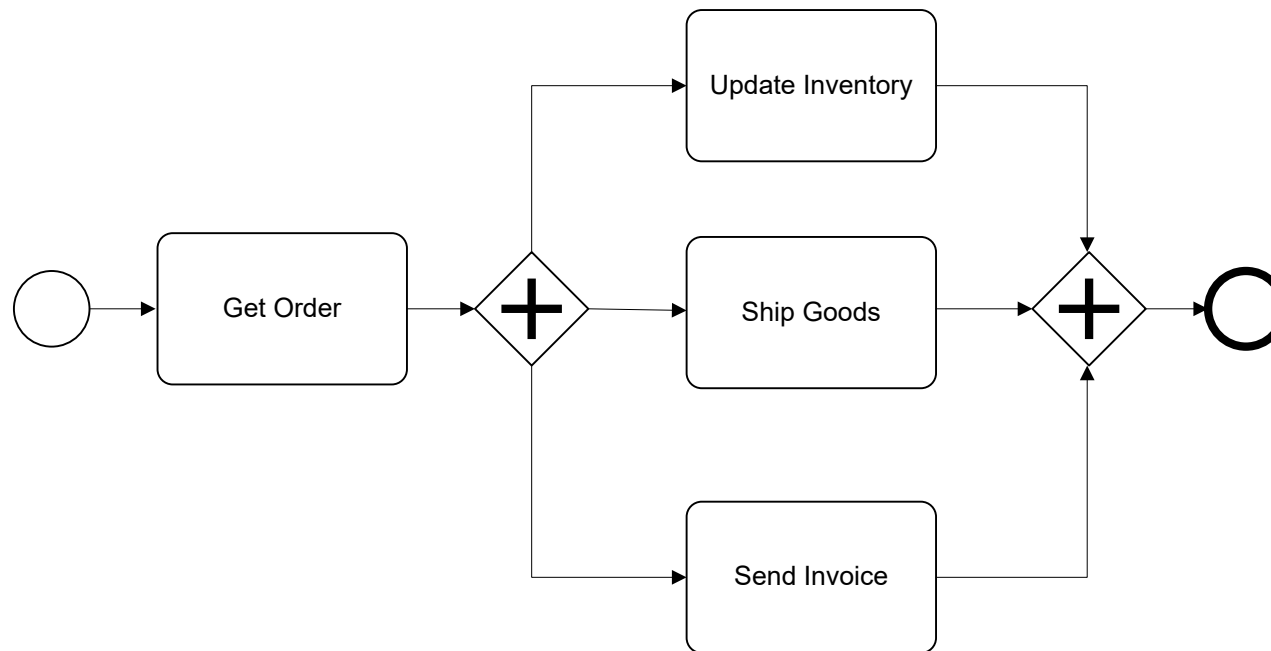
BPMN (recap)



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.85. Example of an *event-based exclusive or gateway*

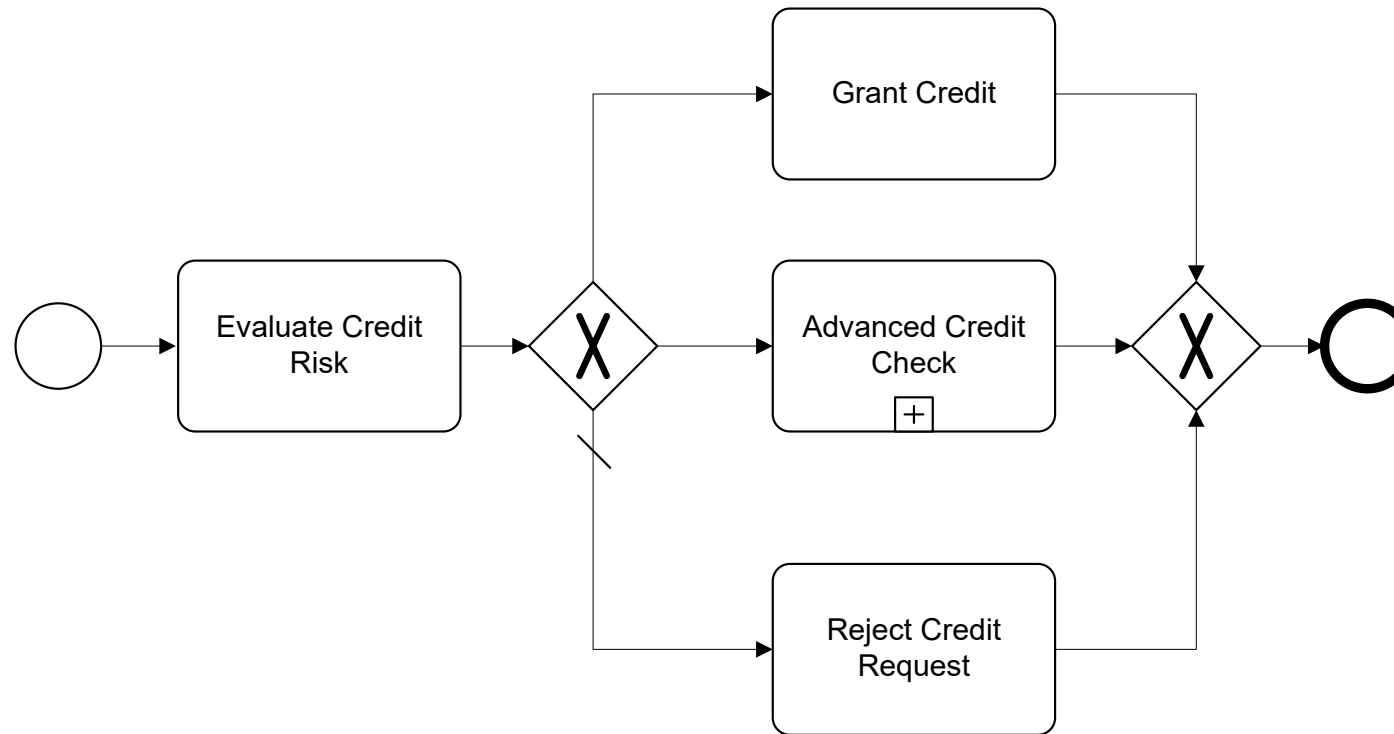
BPMN (recap)



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.87. Example of an *and split* and *and join* gateway

BPMN (recap)



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.88. Sample business process with sequence flow and default sequence flow

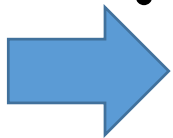
Process modeling levels

- Semantic or conceptual process modeling
 - Event-driven process chains (EPC), ARIS
<https://www.ariscommunity.com/aris-express>
 - **Business Process Modeling and Notation (BPMN)**, Standard,
www.bpmn.org
- Logic, formal languages:
 - **Petri nets, Workflow Nets**
 - **Refined Process Structure Tree (RPST)** → J. Vanhatalo, H. Völzer, J. Koehler: The refined process structure tree. Data Knowl. Eng. 68(9): 793-818 (2009)
- Execution languages: graphical programming
 - Examples: BPEL, WSM-Nets, **CPEE-Trees**
- Transformation between levels is possible, but has to be done carefully!

→ Capturing the process logic, communication

→ Verification, Analysis

→ Process implementation and execution



Petri Nets

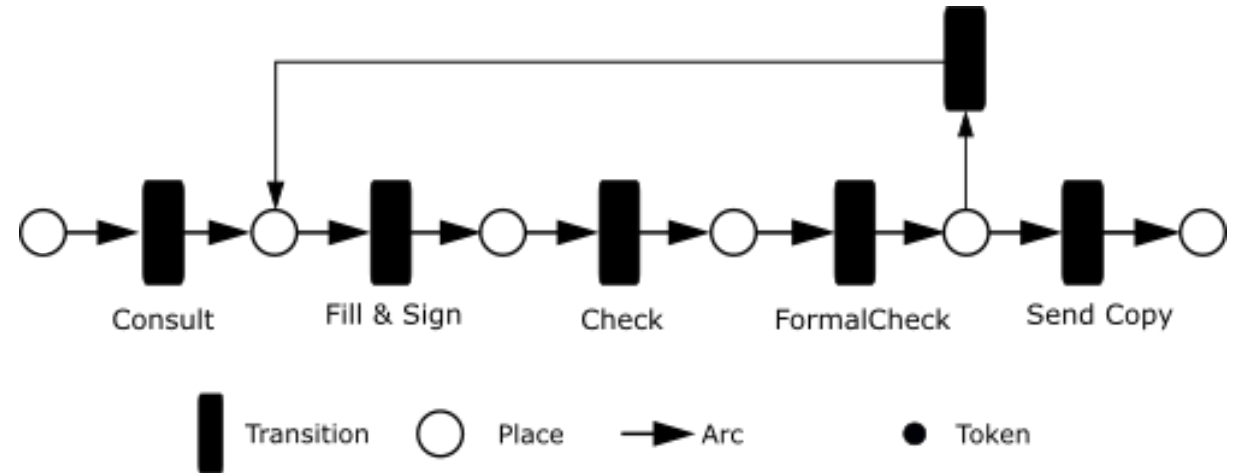
- Different classes with different expressiveness
- Syntax and formal semantics
- bipartite graph with transitions (active elements) and places (passive elements); transitions and places are arranged in an alternative way
- For process modeling: Workflow Nets -> later

DEFINITION 2.1 (Petri Net) A *(Petri-)Net* is a triple $N = (P, T, F)$ with

$$P \cap T = \emptyset \text{ and } F \subseteq (P \times T) \cup (T \times P)$$

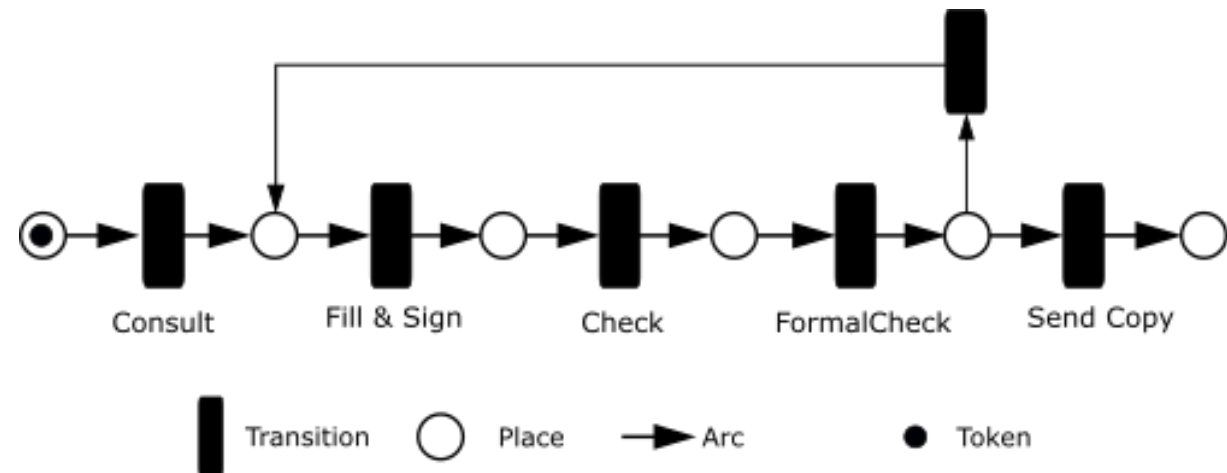
Petri Nets

- Pros:
 - formal analysis -> reachability
 - automation basically possible
- Cons:
 - understandability
 - data flow has to modeled in a separate manner; “higher” classes of Petri Nets, e.g., DB-Nets (M. Montali, A. Rivkin: DB-Nets: On the Marriage of Colored Petri Nets and Relational Databases. Trans. Petri Nets Other Model. Concurr. 12: 91-118 (2017))



Petri Nets

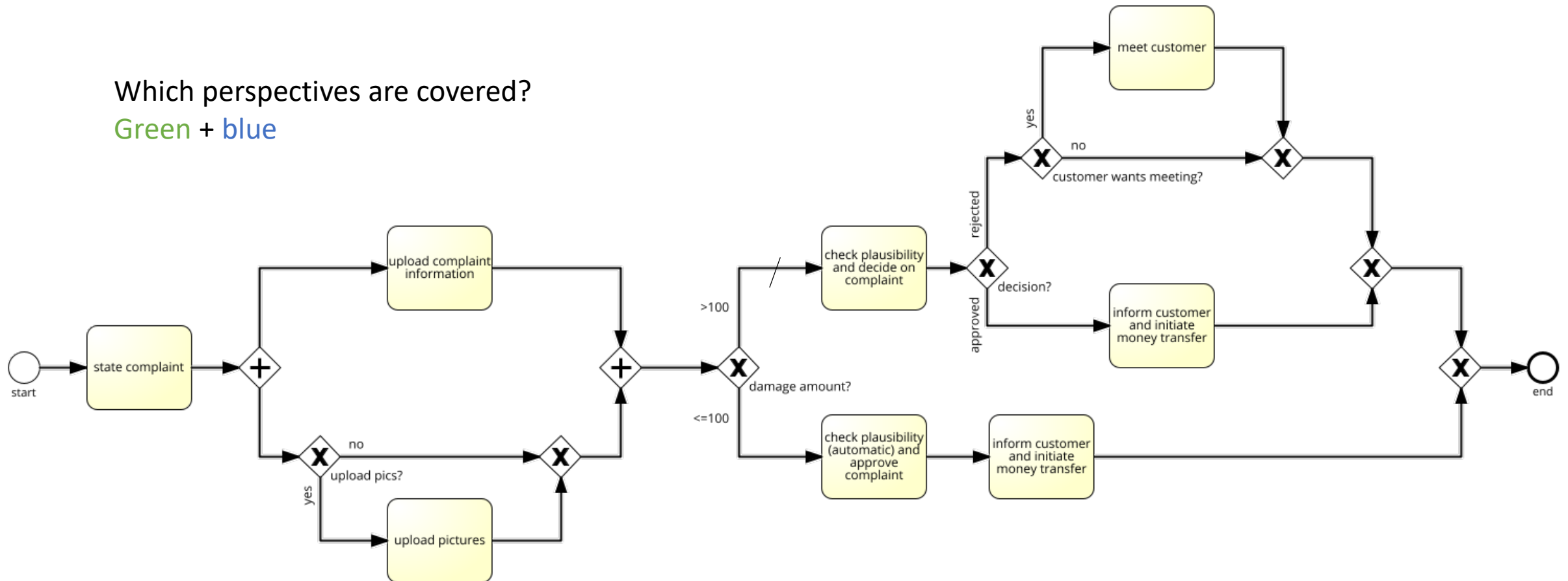
- Dynamic behavior is expressed by tokens
- Tokens mark places
- **Event/Condition Nets (EC Nets):** max. 1 token per place
- Interpretation of places as pre-conditions: if a place carries a token, the precondition is fulfilled



BPMN process model – control flow only

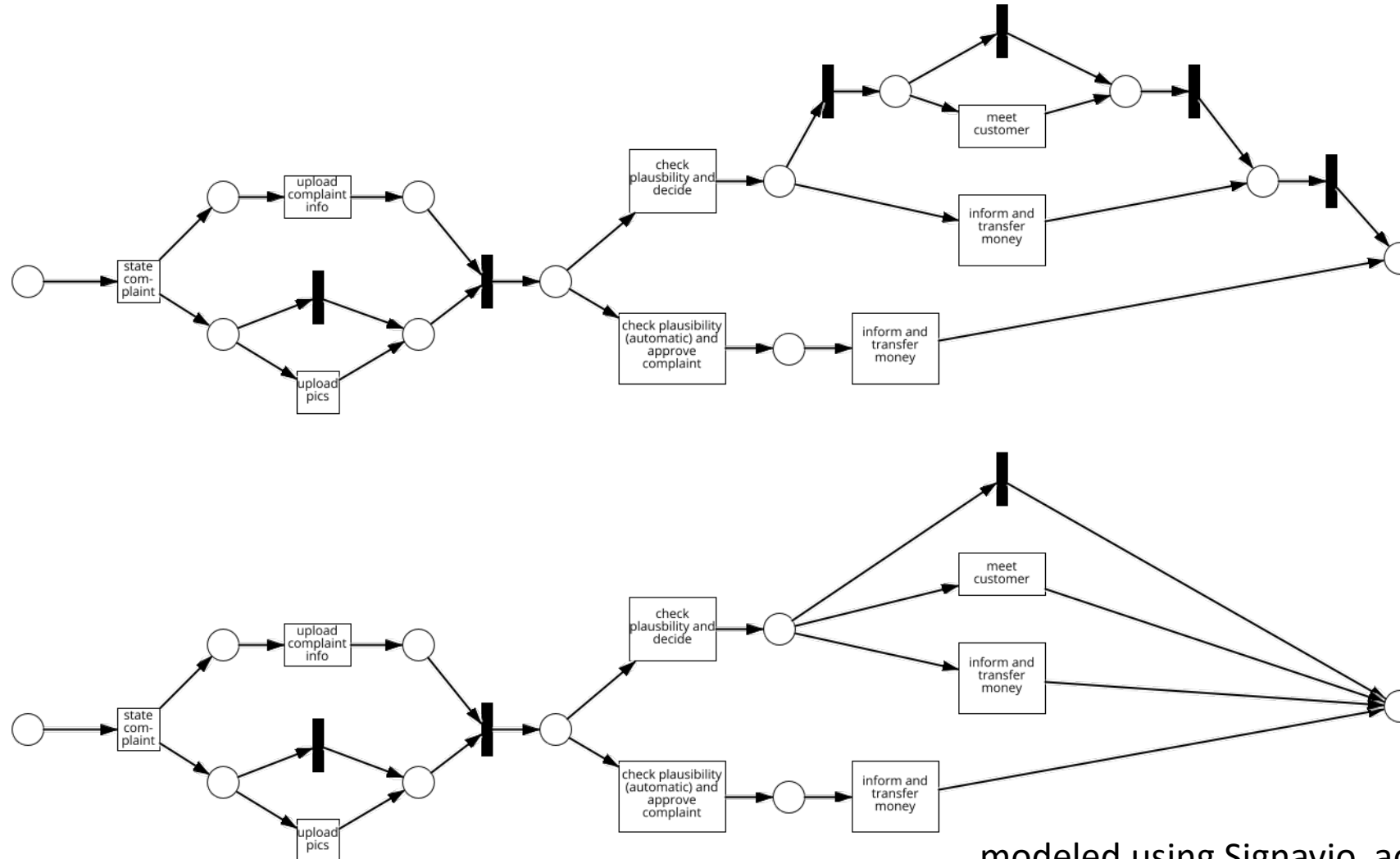
Which perspectives are covered?

Green + blue



modeled using Signavio, academic.signavio.com

Complaint process control Flow: EC Net



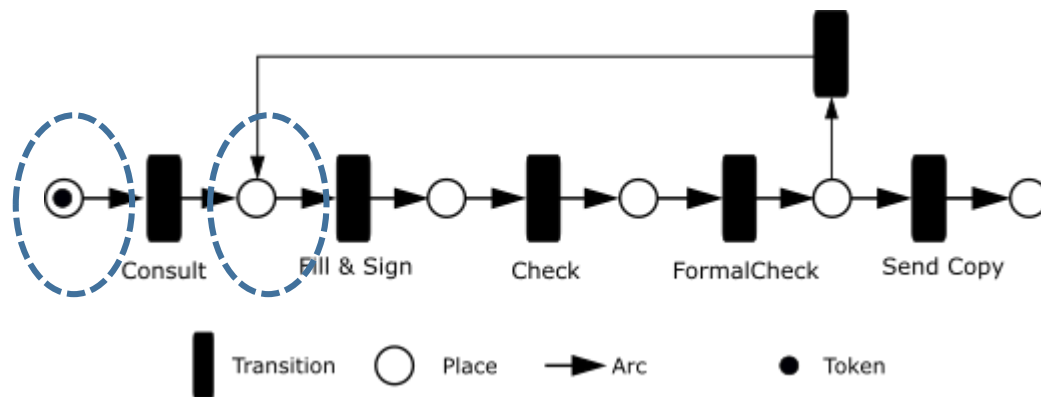
modeled using Signavio, academic.signavio.com

Petri Nets

DEFINITION 2.2 (Marking): A **marking function** M of an E/C-Net $PN = (P, T, F)$ is a mapping $M: P \rightarrow \{0,1\}$ or $M: P \rightarrow \{\text{True}, \text{False}\}$ respectively.

- $M(s) = \text{True}$: Place s carries one token, i.e., the condition described by s is satisfied
- $M(s) = \text{False}$: Place s carries no token, i.e., the condition described by s is dissatisfied

For marking M the set of marked places is given by $c := \{ p \in P \mid M(p) = 1 \}$. c is denoted as *case*.

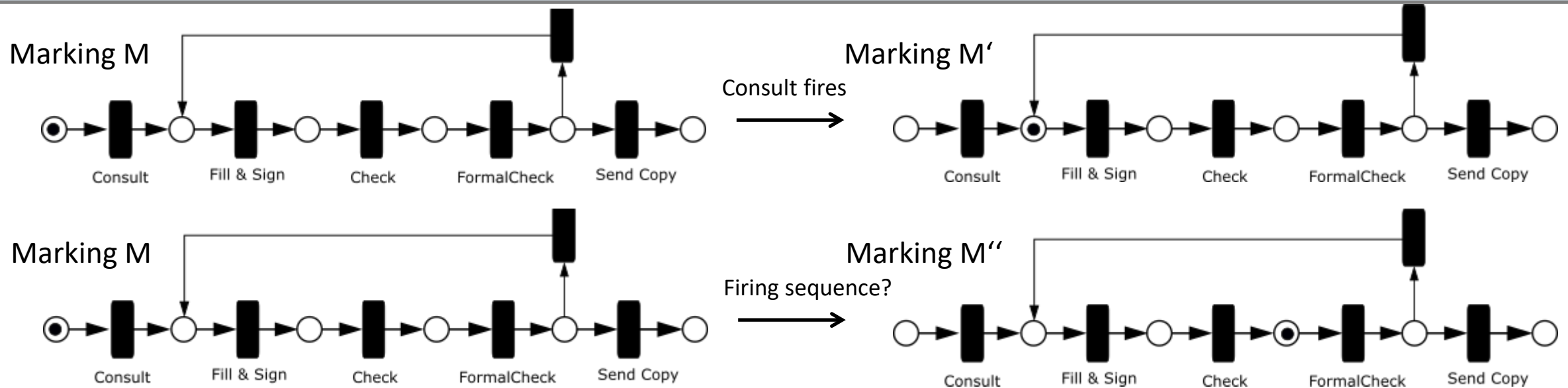


For $t \in T$, $\bullet t = \{p \mid (p, t) \in F\}$ defines the set of input places for t and $t\bullet = \{p \mid (t, p) \in F\}$ defines the set of output places.

Petri Nets

DEFINITION 2.3 (Marking/Firing): Let $M: P \rightarrow \{F, T\}$ be **marking function** of an E/C-Net $PN = (P, T, F)$.

- $M \xrightarrow{t} M'$: by firing t , the state of PN changes from M to M'
- $M \longrightarrow M'$: $\exists t \in T$ with $M \xrightarrow{t} M'$
- $M_1 \xrightarrow{*} M_n$: $\exists$ **firing sequence** $t_1, t_2, \dots, t_{n-1}$ such that $M_i \xrightarrow{t_i} M_{i+1}$ for $i = 1, \dots, n$
- M' is reachable from $M \Leftrightarrow M \xrightarrow{*} M'$



Petri Nets



- A transition t of an E/C Net will become *enabled* if all input places of t and no output place of t carry a token, formally:

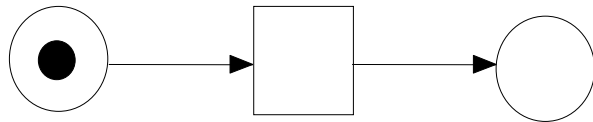
$$\forall p \in \bullet t : M(p) = \text{True} \wedge \forall p \in t \bullet : M(p) = \text{False}$$

- Only activated transitions may *fire* (i.e., be executed).
- When firing transition t all tokens from its input places are removed and one new token is added to each outplace of t , formally:

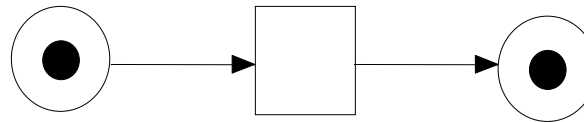
$$M \xrightarrow{t} M' \text{ with } M'(s) = \begin{cases} \text{False} & p \in \bullet t \setminus t \bullet \\ \text{True} & p \in t \bullet \setminus \bullet t \\ \text{True} & p \in \bullet t \cap t \bullet \\ M(p) & p \notin \bullet t \cup t \bullet \end{cases}$$

- **Marking Rule for E/C-Nets explained:** When firing a transition, all tokens from the input places are removed and on a token is put on every output place.
- By firing all transitions „through“ the net, a process instance (case) is executed.
- Recall: process instances describe specific cases for e.g., a patient or a product.
- Modeling of the process, i.e., the process template, happens during *design time*.
- Executing the process instances happens during *runtime*.

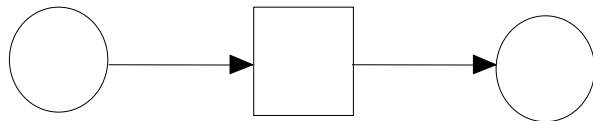
Petri Nets



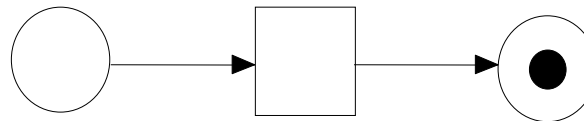
(a)



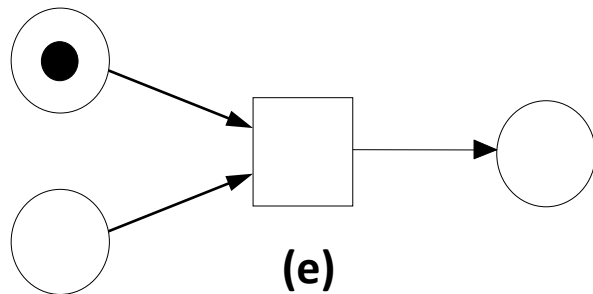
(b)



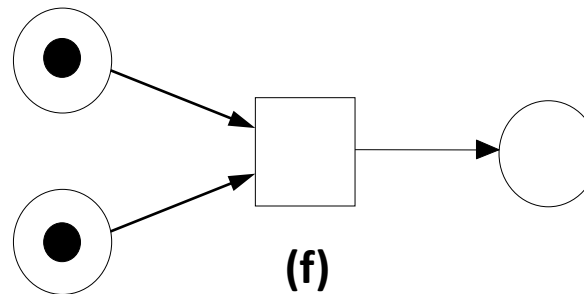
(c)



(d)



(e)



(f)

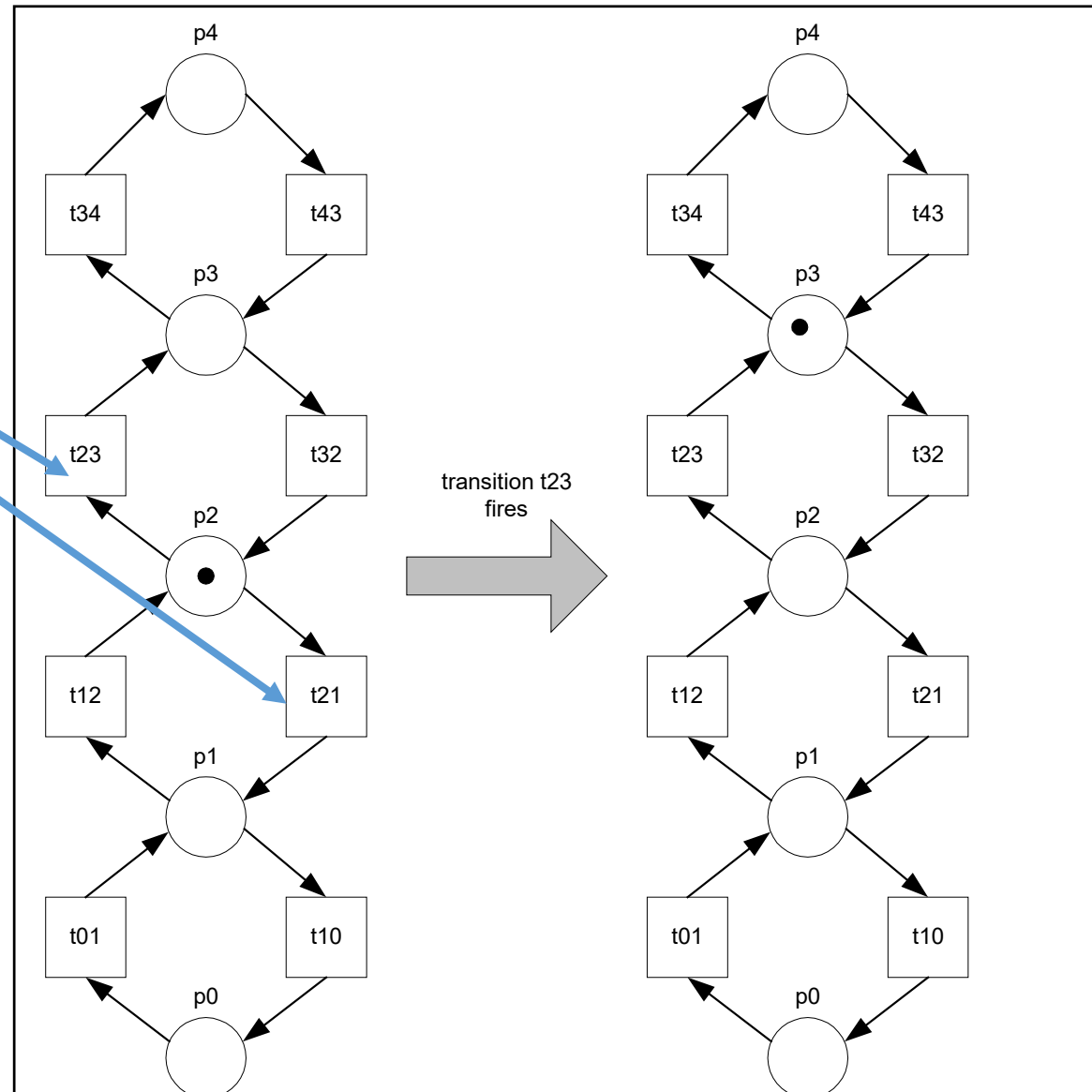
Exercise: Which transitions are enabled?

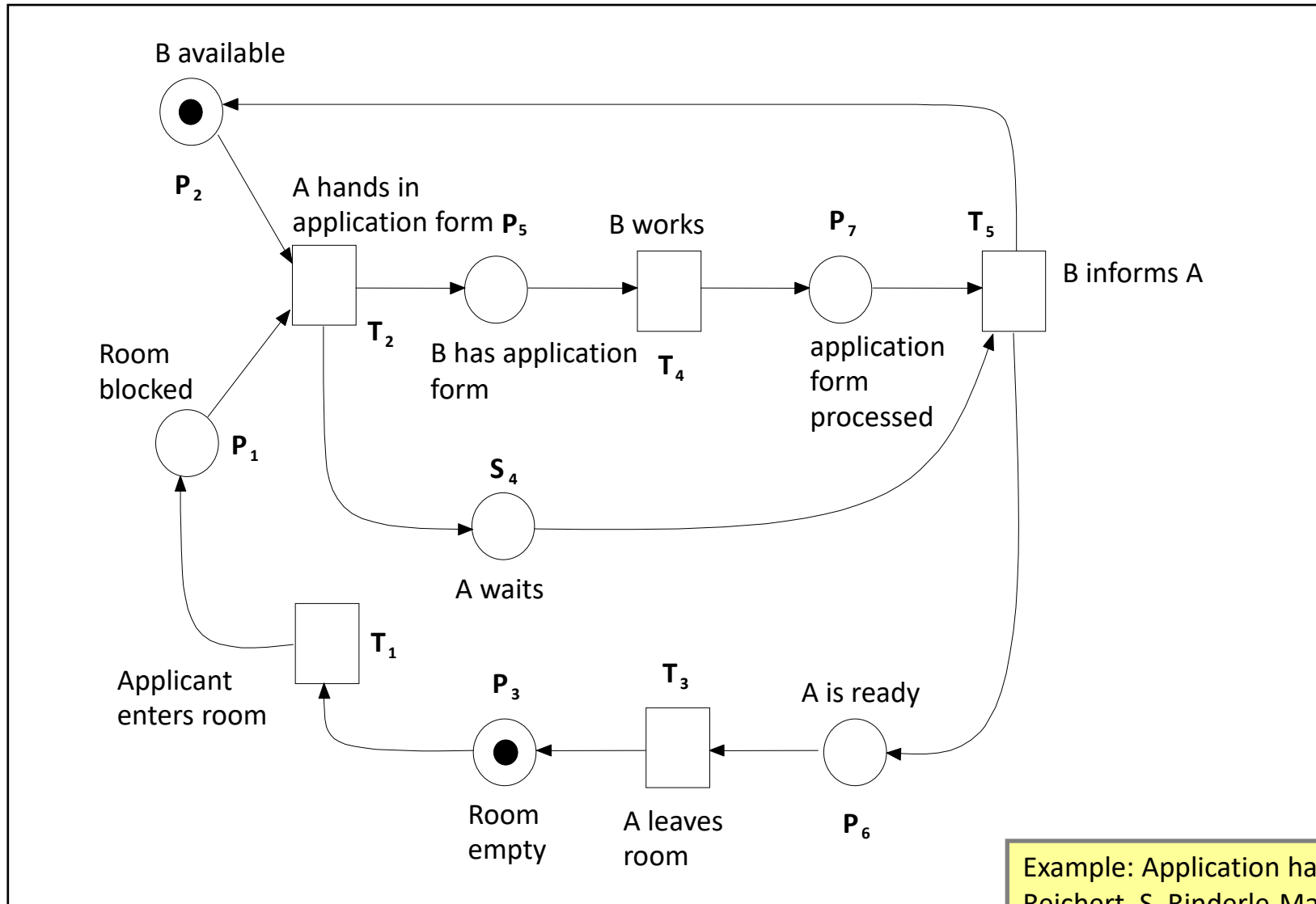
Petri Nets

Two transitions
are enabled, but
only one can fire

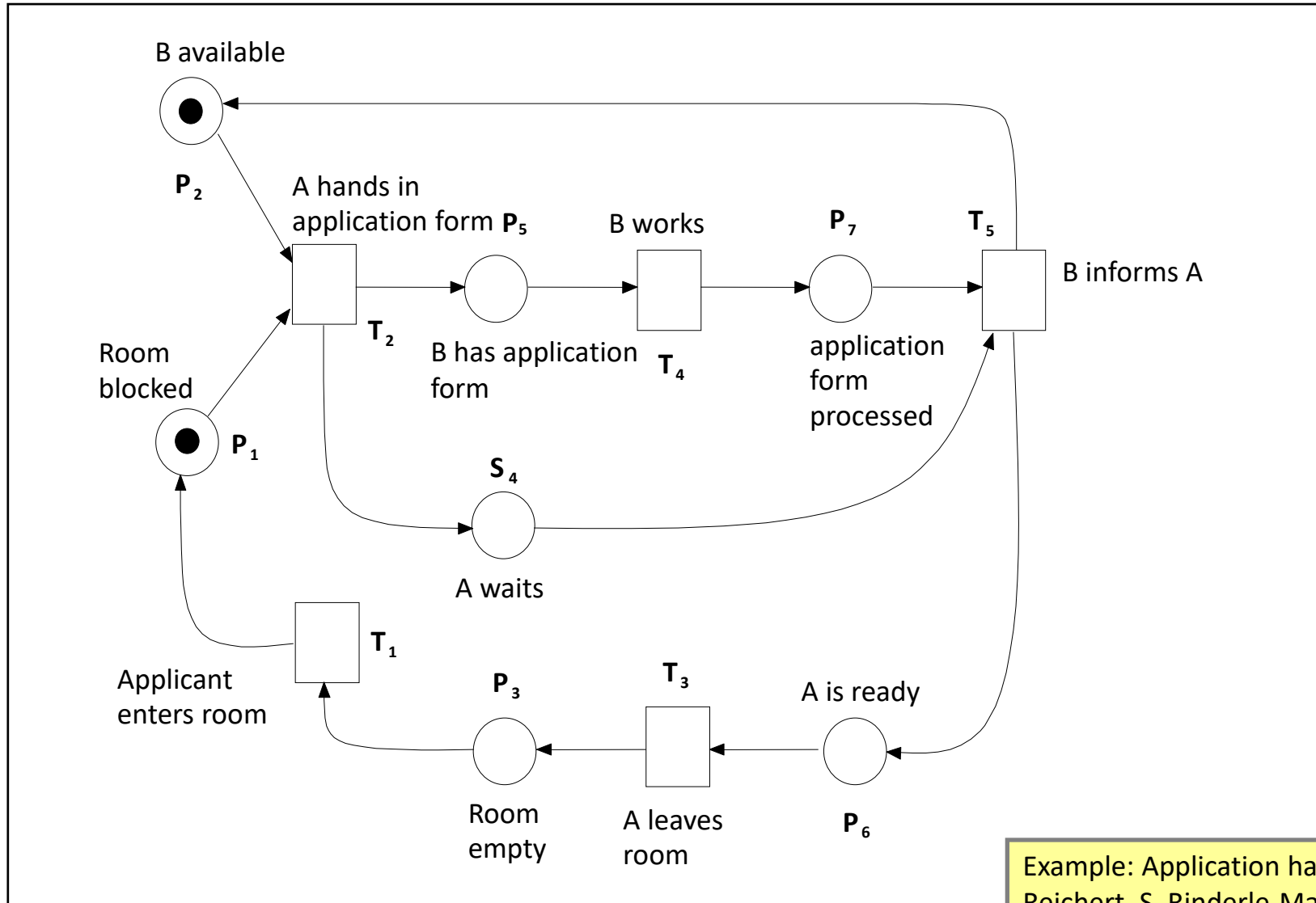
Non-determinism

Example taken from
www.workflowcourse.com
(W.M.P. van der Aalst)

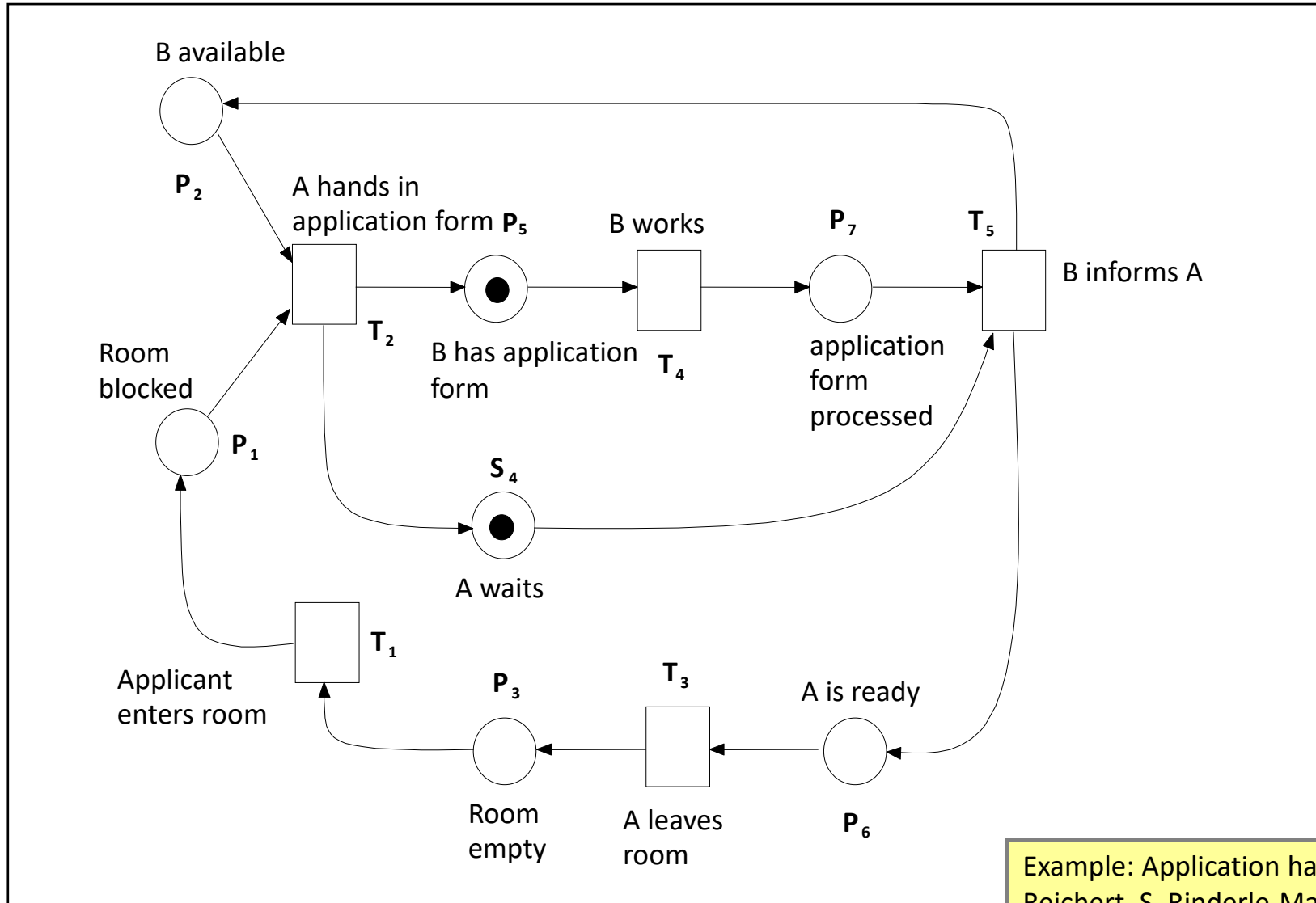




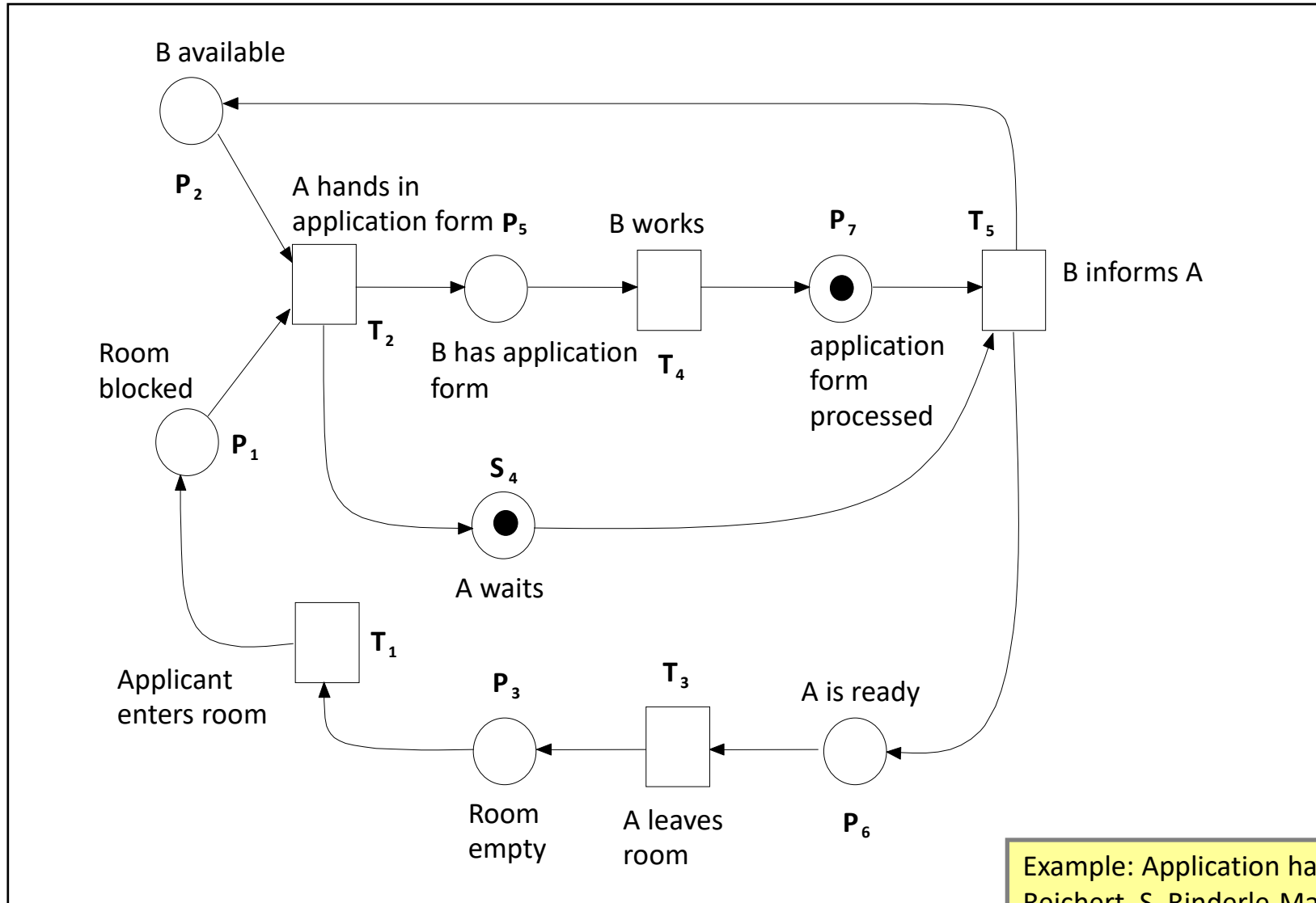
Example: Application handling (based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm)



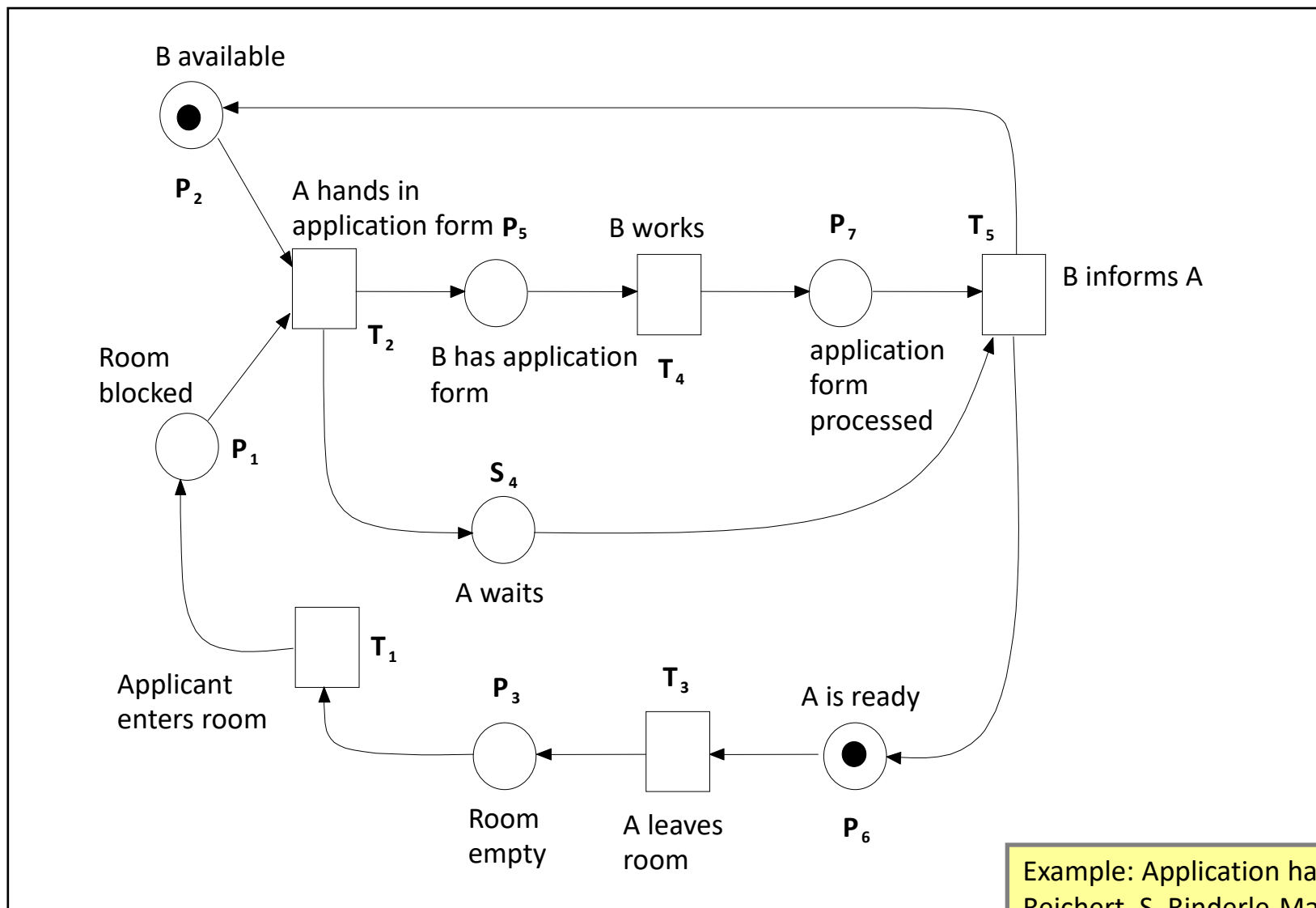
Example: Application handling (based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm)



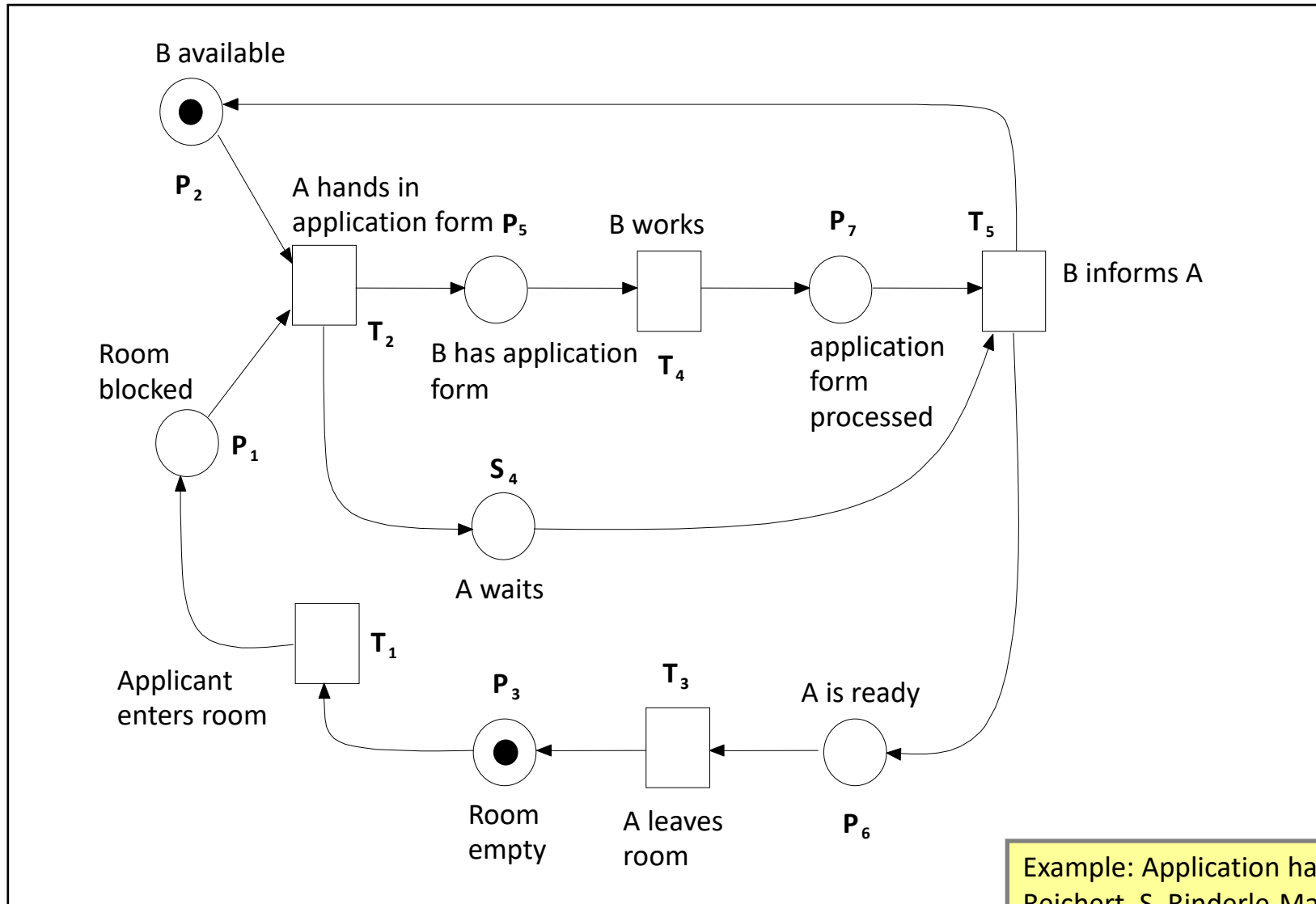
Example: Application handling (based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm)



Example: Application handling (based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm)



Example: Application handling (based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm)



Example: Application handling (based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm)

Structured analysis

- Reachable execution states

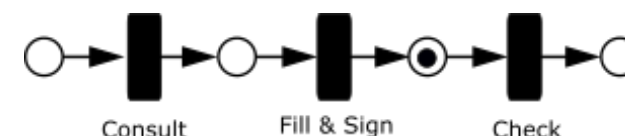


Time	Places							Transitions				
	P1	P2	P3	S4	P5	P6	P7	T1	T2	T3	T4	T5
0		X	X					X				
1	X	X							X			
2				X	X						X	
3				X			X					X
4		X				X				X		
5		X	X									

Example: Application handling (based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm)

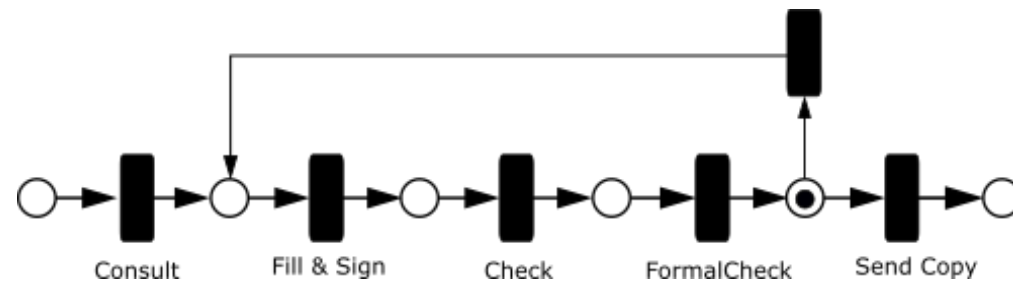
Petri net based logging

- The marking, i.e., the placement of tokens on the Petri net, signifies an execution state of a process instance.
- In which execution state is the Petri net ?
- Which transitions/activities have been executed is stored into a (process) execution log.
- Example execution log: `<Consult, Fill & Sign>`
- Execution logs enable to restore the execution state of process instances
- Execution logs provide the input for **Process Mining**.



Petri net based logging

- Determine the execution state of the Petri net below:



- Can you determine a process execution log for this instance in a unique way?
- Which execution logs are possible?

Place/Transition Nets

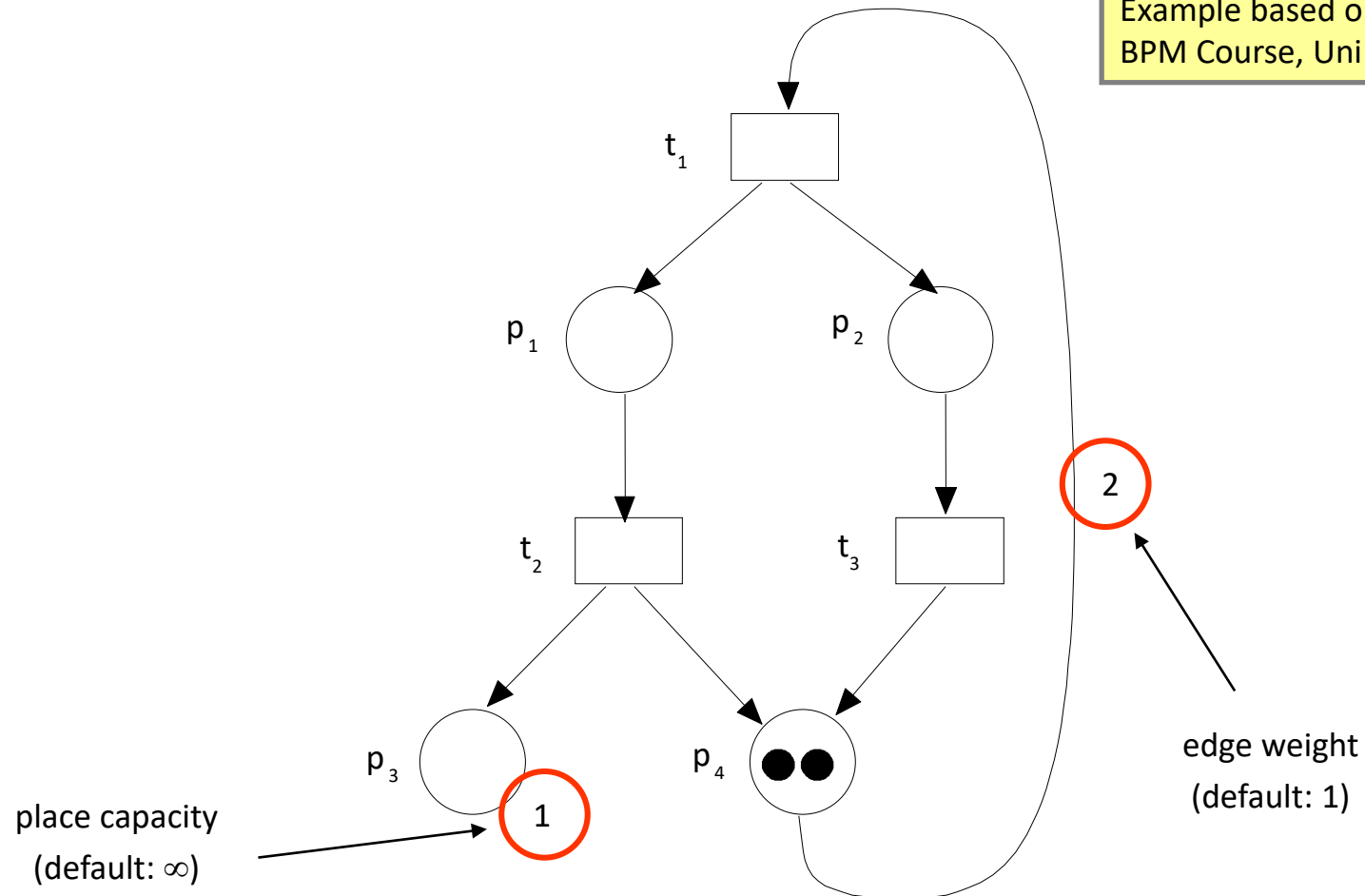
- E/C Nets are now extended by edge weights and place capacities
- Places can carry more than one token.
- Tokens remain anonymous.
- Useful to describe systems and processes with limited resources

DEFINITION 2.4 (Place/Transition Net): A tuple $Y = (P, T, F, C, W, M_0)$ is denoted as Place/Transition-Net (P/T-Net) if

- (P, T, F) is a Net
- $C: P \rightarrow \mathbb{N} \cup \{\infty\}$ assigns capacities to places
- $W: F \rightarrow \mathbb{N}$ assigns weights to edges
- $M_0: P \rightarrow \mathbb{N}_0$ denotes the initial marking with $\forall p \in P : M_0(p) \leq C(p)$.

Place/Transition Nets

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma:
BPM Course, Uni Ulm



Place/Transition Nets

DEFINITION 2.5 (Firing Rules for P/T Nets)

- A transition t of an P/T-Net is activated under a marking M (notation: $M[t\rangle$) if
 - $\forall p \in \bullet t : M(p) \geq W(p,t)$
 - $\forall p \in t\bullet : M(p) + W(t,p) \leq C(p)$
- $M \xrightarrow{t} M'$ with:

$$M'(p) = \begin{cases} M(p) - W(p,t), & \text{falls } p \in \bullet t \setminus t\bullet \\ M(p) + W(t,p), & \text{falls } p \in t\bullet \setminus \bullet t \\ M(p) - W(p,t) + W(t,p), & \text{falls } p \in t\bullet \cap \bullet t \\ M(p), & \text{sonst} \end{cases}$$

Place/Transition Nets: Reachability



- Analysis of dynamic properties by P/T-Net-example
- General target: Harvesting knowledge on execution behavior
 - Deadlocks
 - Unbounded places
 - Non-reachable, yet desired states
- Very important for process-oriented applications: Knowing possible problems in advance, not when they are already happening!!

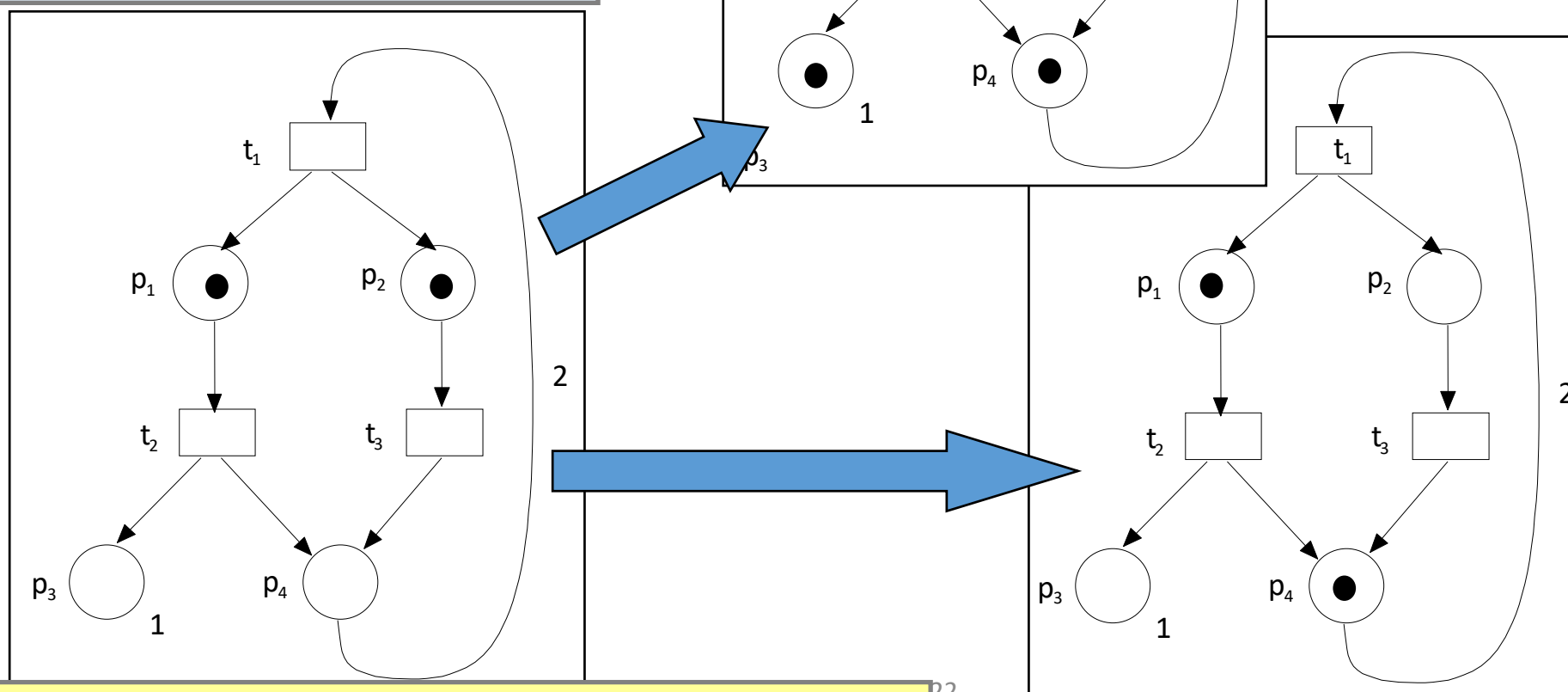
DEFINITION 2.6 (Reachability Set): Let M_0 be an initial marking. Then $[M_0\rangle$ denotes the set of all possible markings which can be reached starting from M_0 using the firing rules as defined before. $[M_0\rangle$ is denoted as reachability set of a P/T-Net.

For any marking M_i , $[M_i\rangle$ denotes the set of markings which can be reached from M_i .

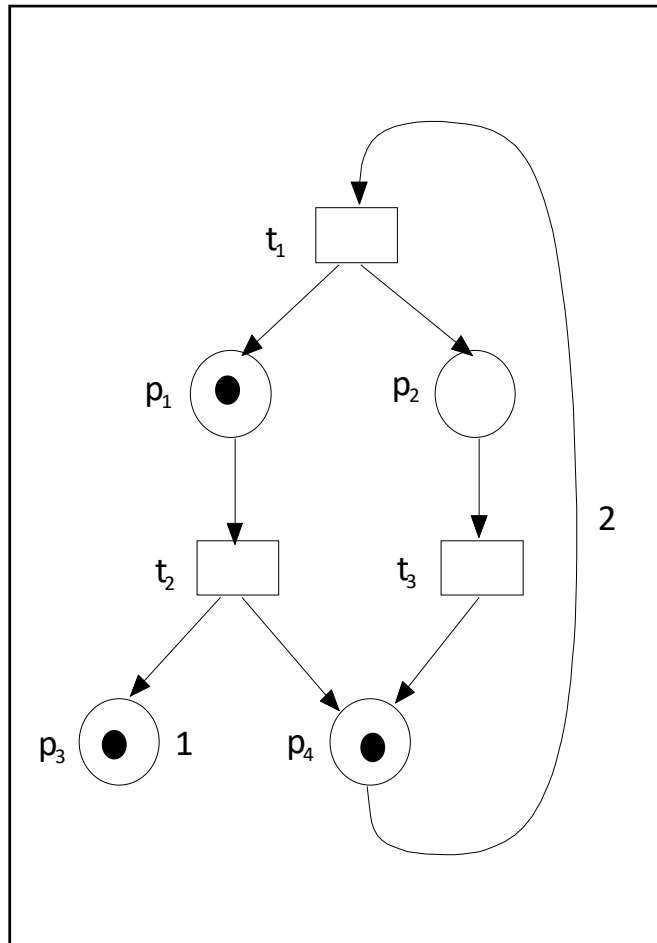
Place/Transition Nets

reachable state

A state reachable from the current state by firing a sequence of enabled transitions.



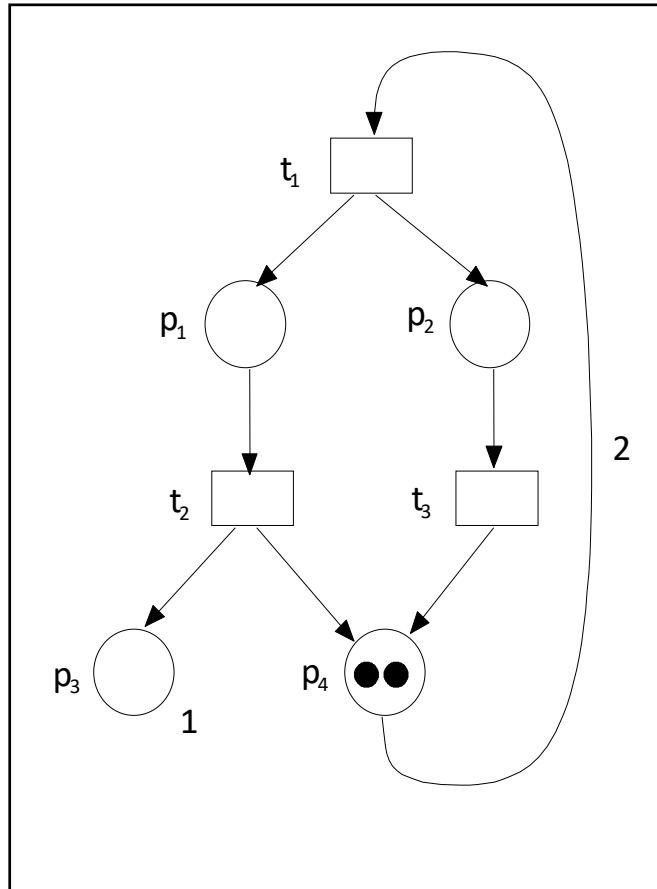
Place/Transition Nets



dead state

A state where no transition is enabled.

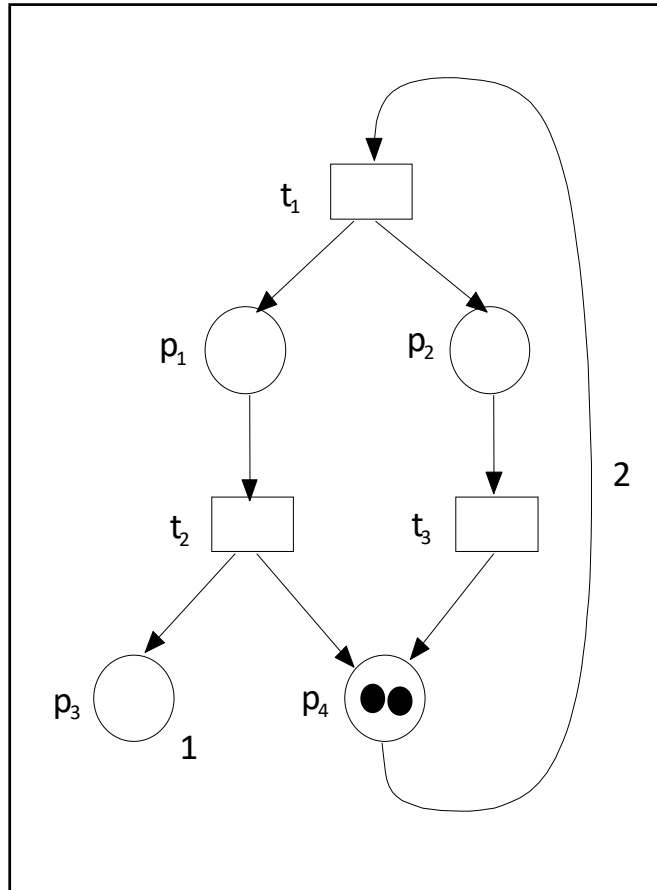
Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	

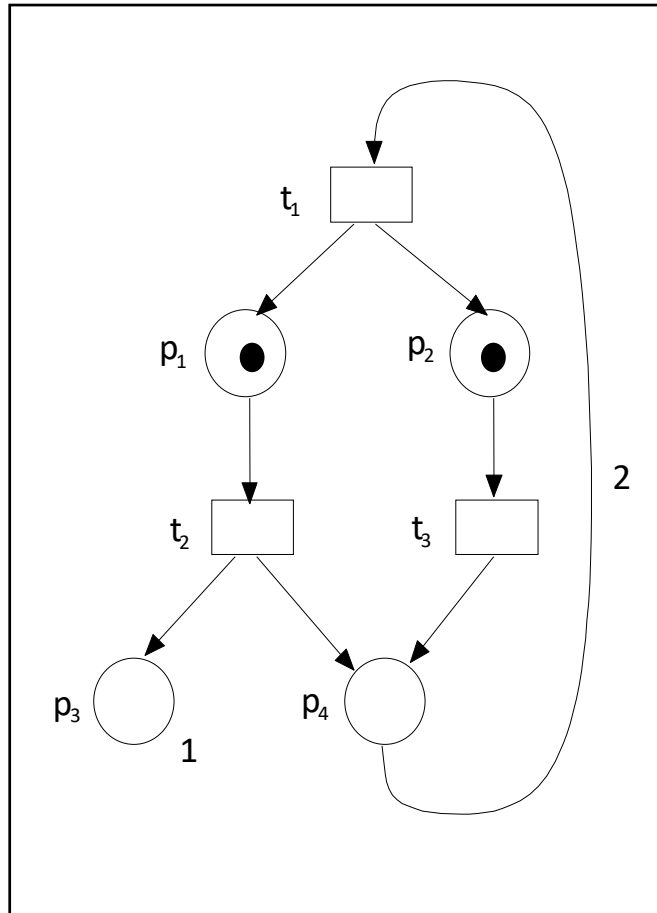
Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t1 \rightarrow M_1$
M_1					

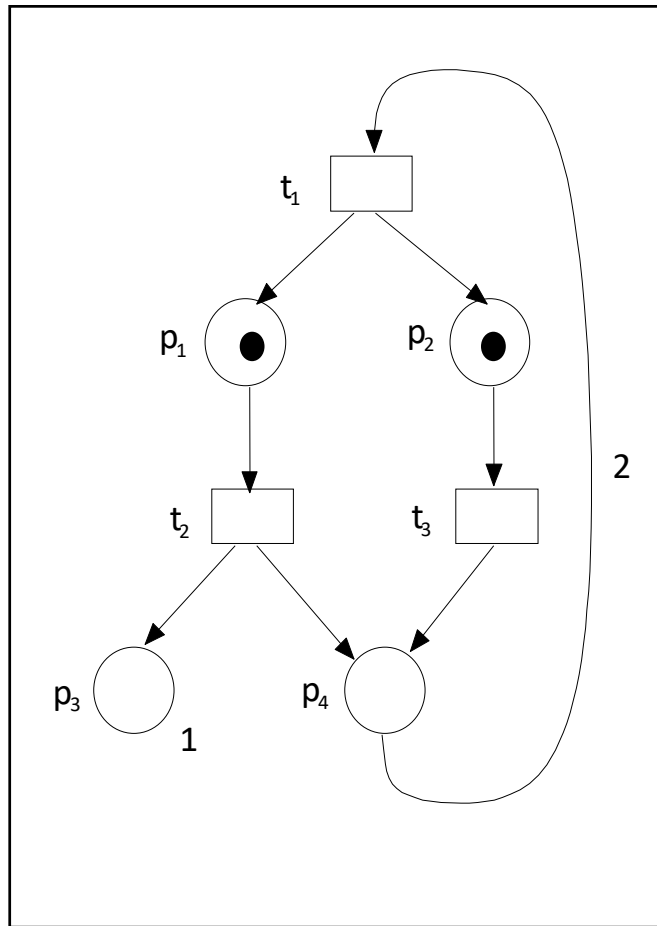
Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t1 \rightarrow M_1$
M_1	1	1	0	0	

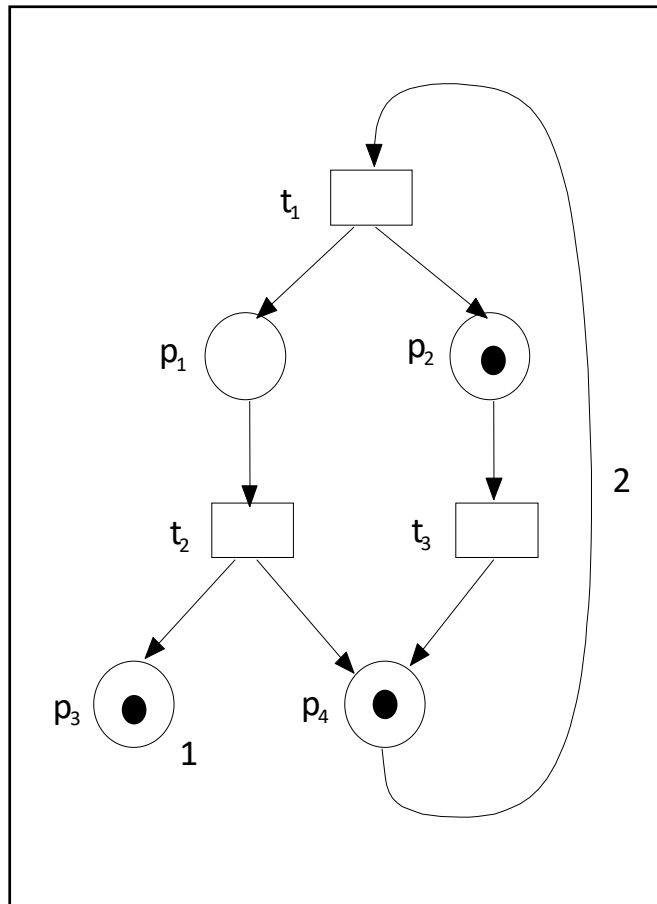
Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t_1 \rightarrow M_1$
M_1	1	1	0	0	$t_2 \rightarrow M_2$ $t_3 \rightarrow M_3$
M_2					
M_3					

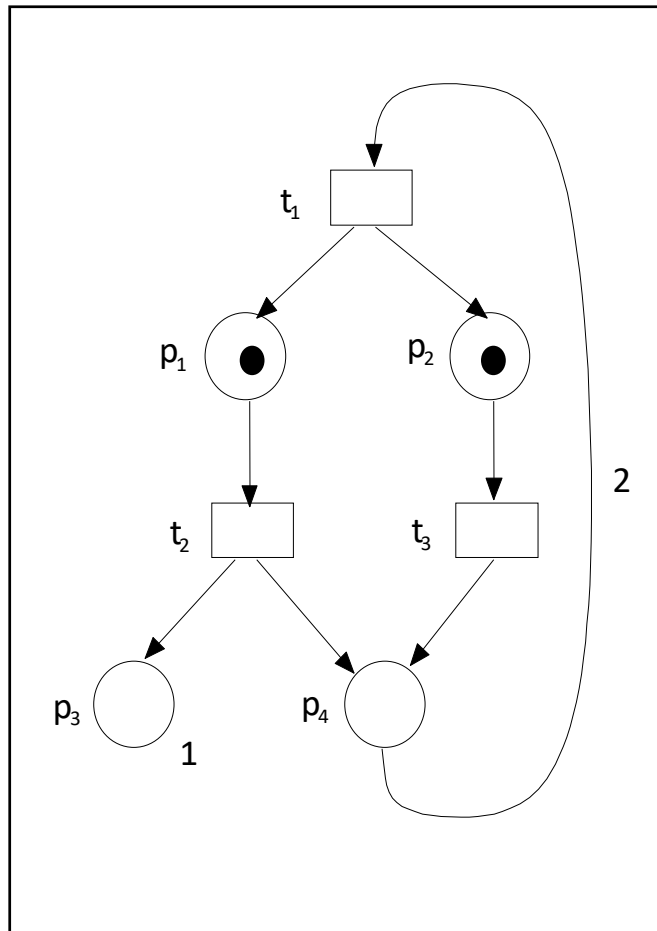
Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t_1 \rightarrow M_1$
M_1	1	1	0	0	$t_2 \rightarrow M_2$ $t_3 \rightarrow M_3$
M_2	0	1	1	1	
M_3					

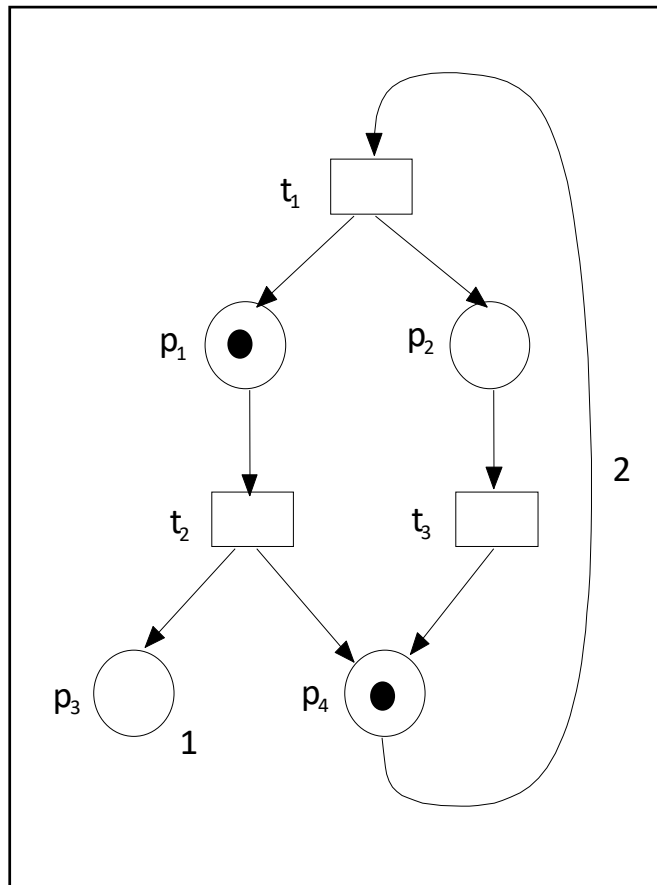
Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M ₀	0	0	0	2	t1 → M ₁
M ₁	1	1	0	0	t2 → M ₂ t3 → M ₃
M ₂	0	1	1	1	
M ₃					

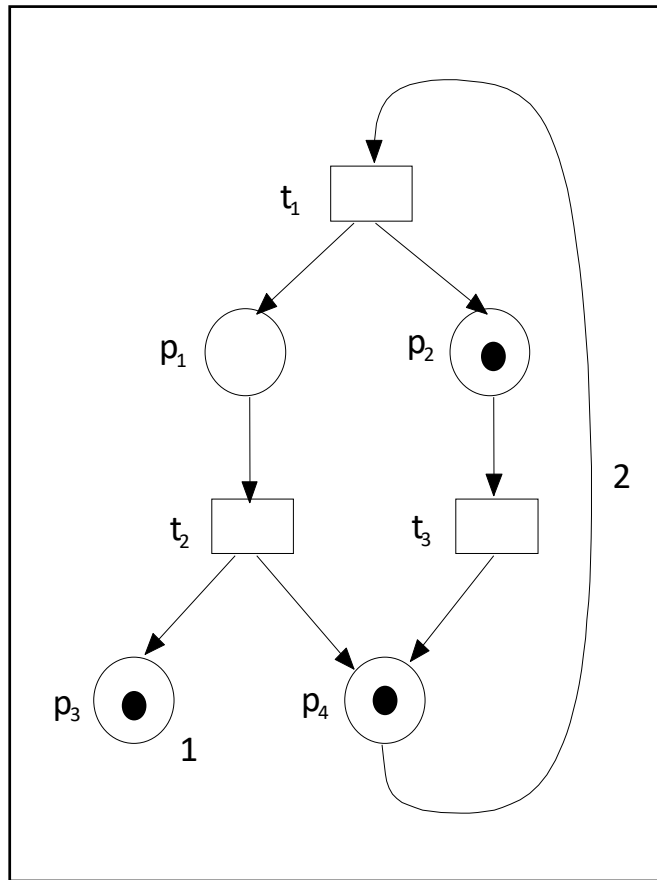
Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t_1 \rightarrow M_1$
M_1	1	1	0	0	$t_2 \rightarrow M_2$ $t_3 \rightarrow M_3$
M_2	0	1	1	1	
M_3	1	0	0	1	

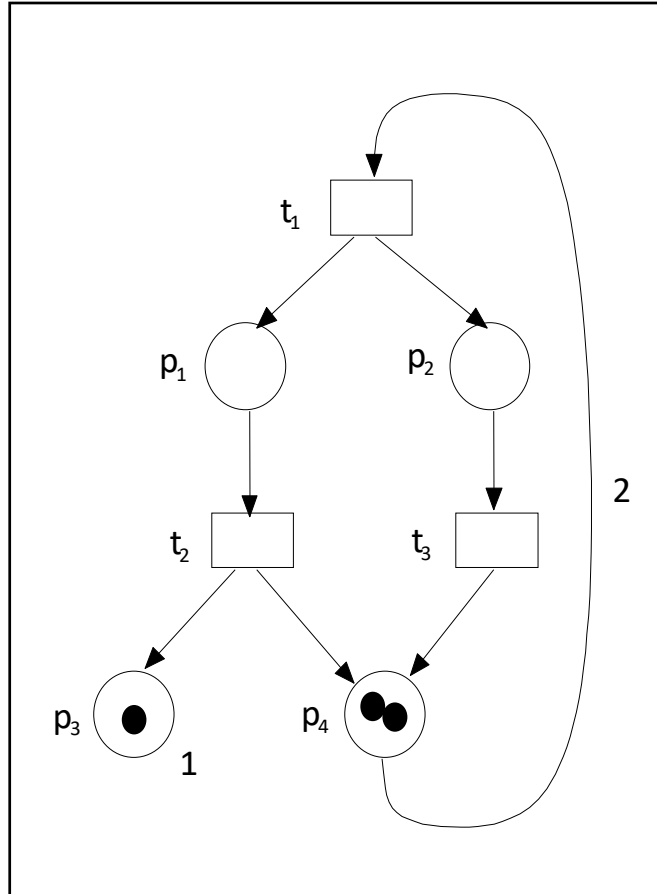
Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t_1 \rightarrow M_1$
M_1	1	1	0	0	$t_2 \rightarrow M_2$ $t_3 \rightarrow M_3$
M_2	0	1	1	1	$t_3 \rightarrow M_4$
M_3	1	0	0	1	
M_4					

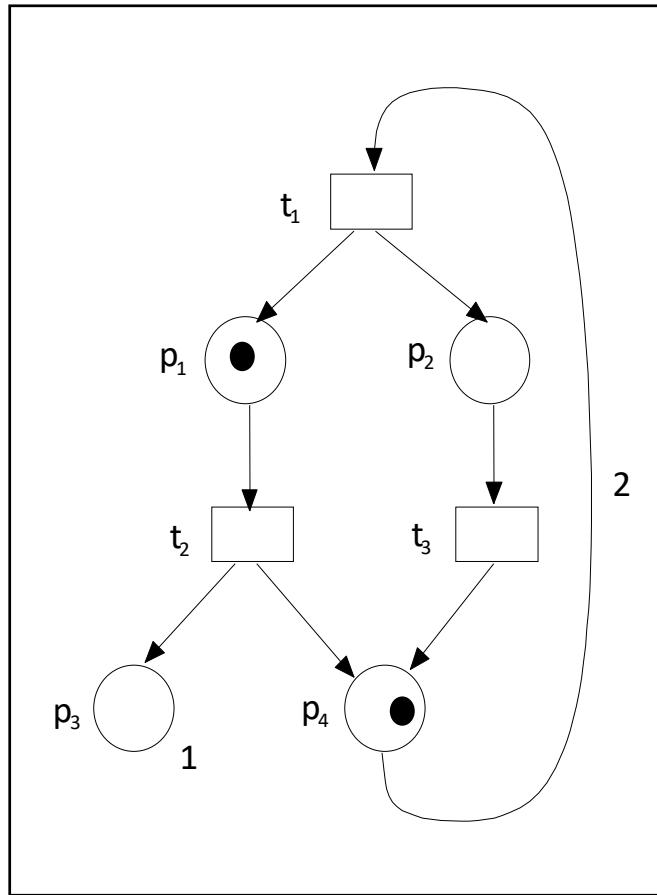
Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t_1 \rightarrow M_1$
M_1	1	1	0	0	$t_2 \rightarrow M_2$ $t_3 \rightarrow M_3$
M_2	0	1	1	1	$t_3 \rightarrow M_4$
M_3	1	0	0	1	
M_4	0	0	1	2	

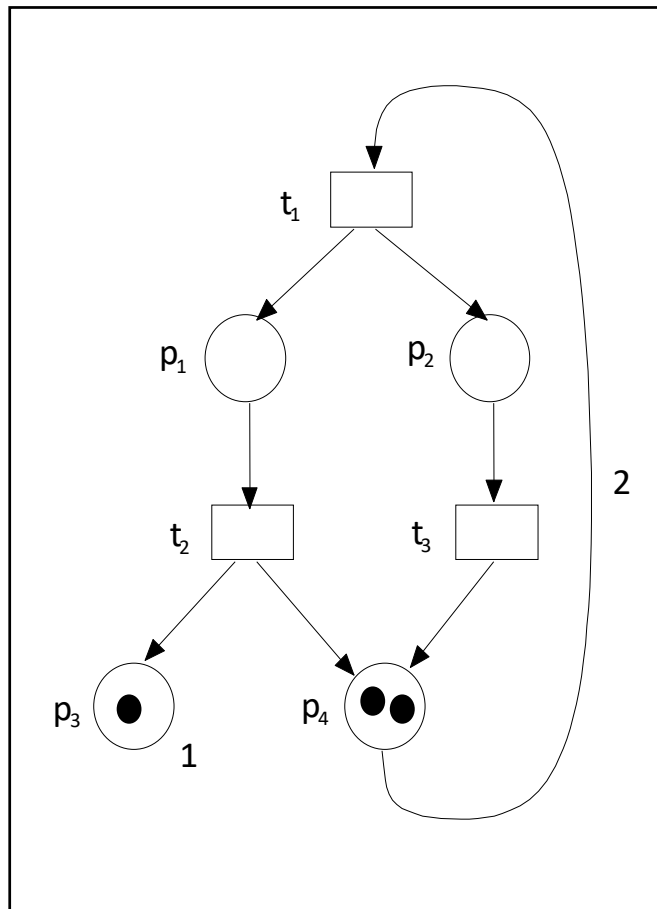
Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t_1 \rightarrow M_1$
M_1	1	1	0	0	$t_2 \rightarrow M_2$ $t_3 \rightarrow M_3$
M_2	0	1	1	1	$t_3 \rightarrow M_4$
M_3	1	0	0	1	$t_2 \rightarrow M_4$
M_4	0	0	1	2	

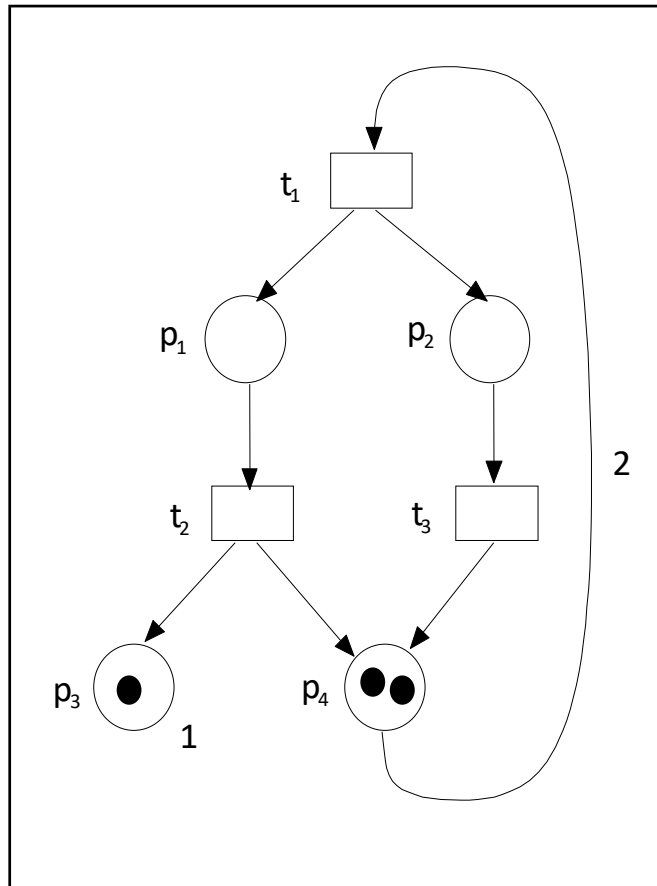
Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t_1 \rightarrow M_1$
M_1	1	1	0	0	$t_2 \rightarrow M_2$ $t_3 \rightarrow M_3$
M_2	0	1	1	1	$t_3 \rightarrow M_4$
M_3	1	0	0	1	$t_2 \rightarrow M_4$
M_4	0	0	1	2	

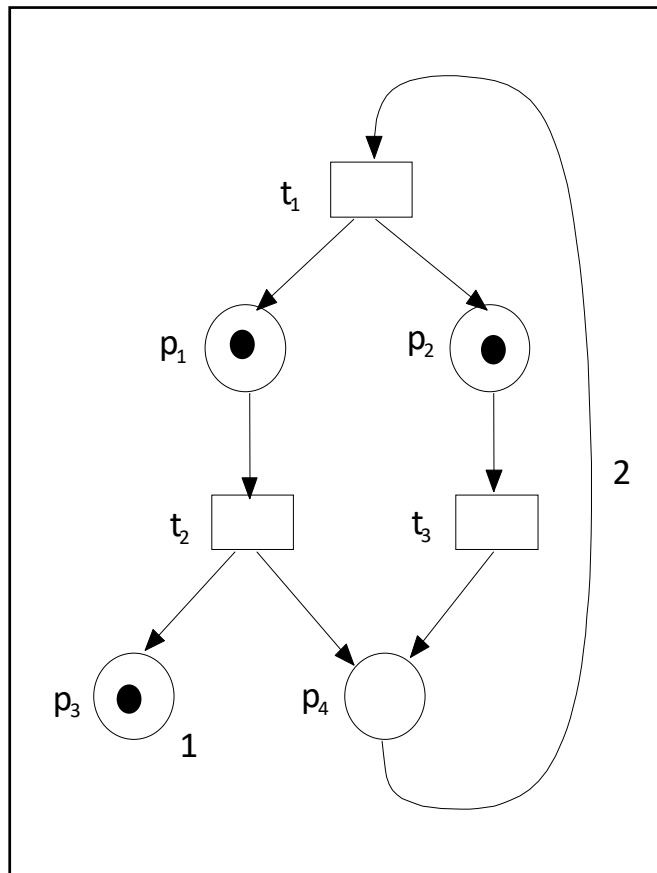
Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t_1 \rightarrow M_1$
M_1	1	1	0	0	$t_2 \rightarrow M_2$ $t_3 \rightarrow M_3$
M_2	0	1	1	1	$t_3 \rightarrow M_4$
M_3	1	0	0	1	$t_2 \rightarrow M_4$
M_4	0	0	1	2	$t_1 \rightarrow M_5$
M_5					

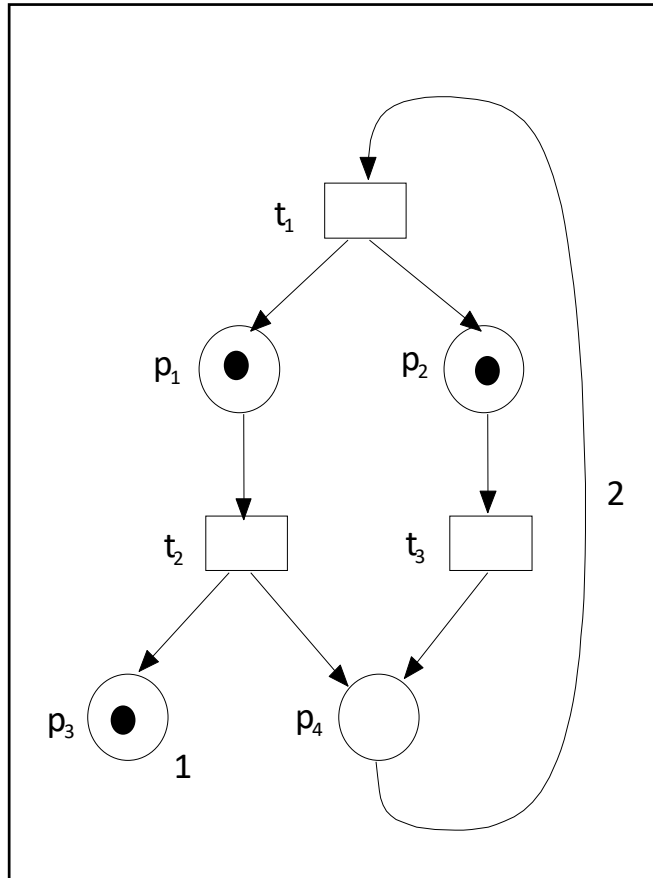
Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t_1 \rightarrow M_1$
M_1	1	1	0	0	$t_2 \rightarrow M_2$ $t_3 \rightarrow M_3$
M_2	0	1	1	1	$t_3 \rightarrow M_4$
M_3	1	0	0	1	$t_2 \rightarrow M_4$
M_4	0	0	1	2	$t_1 \rightarrow M_5$
M_5	1	1	1	0	

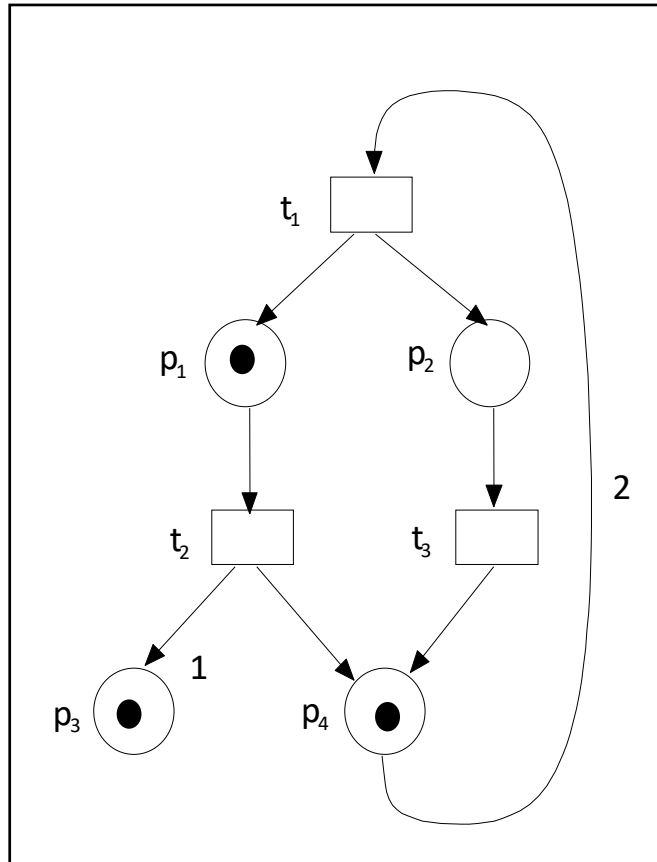
Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t_1 \rightarrow M_1$
M_1	1	1	0	0	$t_2 \rightarrow M_2$ $t_3 \rightarrow M_3$
M_2	0	1	1	1	$t_3 \rightarrow M_4$
M_3	1	0	0	1	$t_2 \rightarrow M_4$
M_4	0	0	1	2	$t_1 \rightarrow M_5$
M_5	1	1	1	0	$t_3 \rightarrow M_6$
M_6					

Place/Transition Nets



Reachability Analysis

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t_1 \rightarrow M_1$
M_1	1	1	0	0	$t_2 \rightarrow M_2$ $t_3 \rightarrow M_3$
M_2	0	1	1	1	$t_3 \rightarrow M_4$
M_3	1	0	0	1	$t_2 \rightarrow M_4$
M_4	0	0	1	2	$t_1 \rightarrow M_5$
M_5	1	1	1	0	$t_3 \rightarrow M_6$
M_6	1	0	1	1	

Place/Transition Nets



Reachability Graph

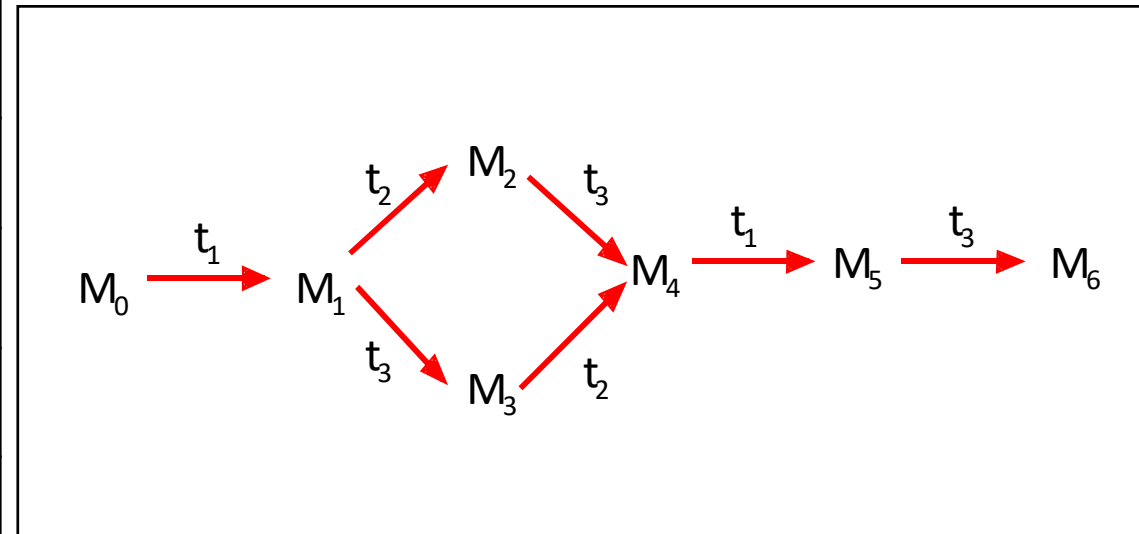
- The reachability graph of a Petri net is the part of the transition system reachable from the initial state in graph-like notation.
- The reachability graph can be calculated as follows:
 - Let X be the set containing just the initial state and let Y be the empty set.
 - Take an element x of X and add this to Y. Calculate all states reachable for x by firing some enabled transition. Each successor state that is not in Y is added to X.
 - If X is empty stop, otherwise goto 2.

DEFINITION 2.7 (Reachability Graph): The reachability graph $G(Y)$ of a P/T-Net Y is defined as follows:

$$G(Y) := \{(M_1, t, M_2) \in [M_0] \times T \times [M_0] \mid M_1 \xrightarrow{t} M_2\}$$

Place/Transition Nets

No.	p1	p2	p3	p4	Firing transition
M_0	0	0	0	2	$t_1 \rightarrow M_1$
M_1	1	1	0	0	$t_2 \rightarrow M_2$ $t_3 \rightarrow M_3$
M_2	0	1	1	1	$t_3 \rightarrow M_4$
M_3	1	0	0	1	$t_2 \rightarrow M_4$
M_4	0	0	1	2	$t_1 \rightarrow M_5$
M_5	1	1	1	0	$t_3 \rightarrow M_6$
M_6	1	0	1	1	



Reachability Graph

Example based on: P. Dadam, M. Reichert, S. Rinderle-Ma: BPM Course, Uni Ulm

Place/Transition Nets



DEFINITION 2.8 (Boundedness): Let $Y = (P, T, F, C, W, M_0)$ be a P/T-Net and $k \in \mathbb{N}$ a natural number. Then:

- Y is denoted as ***bounded*** if $\forall M \in [M_0], p \in P : M(p) \leq k$

Liveness¹

- $[M_0 \rangle$ all reachable markings from M_0
- $L(M_0)$ all possible firing sequences from M_0

A petri net is L_k -Live if all transitions are L_k -Live with $k = 0,1,2,3,4$

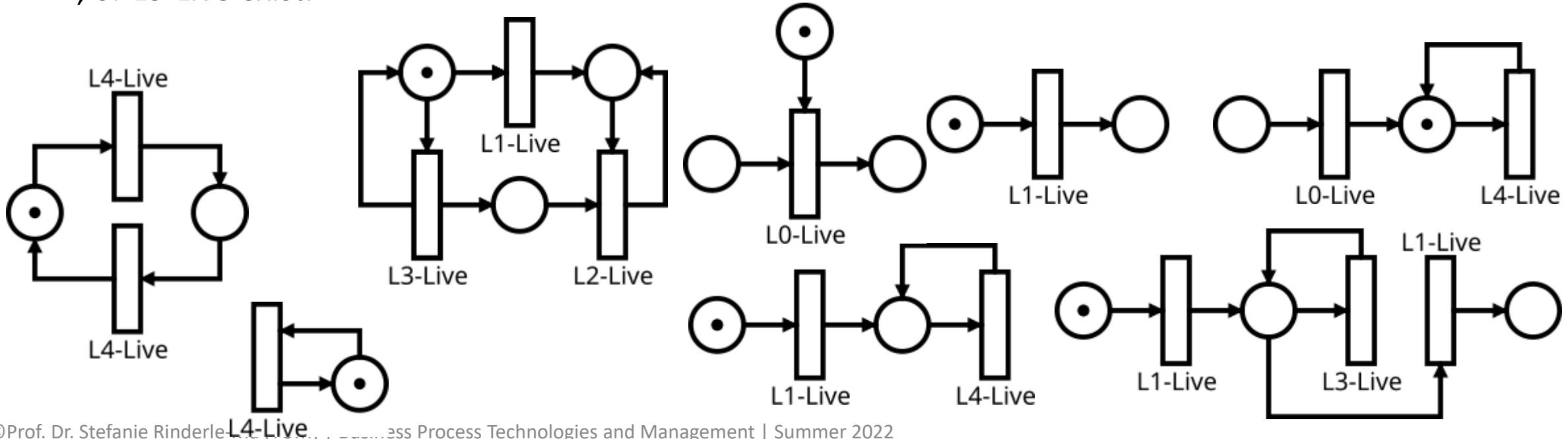
A transition is

- L0-Live: the transition can never fire in any $L(M_0)$
- L1-Live: the transition can fire at least once for a sequence in $L(M_0)$
- L2-Live: the transition can fire a finite number of times for a sequence in $L(M_0)$
- L3-Live: the transition can fire an infinite number of times for a sequence in $L(M_0)$
- L4-Live: the transition can fire an infinite number of times for $[M_0 \rangle$

¹ Murata, Tadao. "Petri nets: Properties, analysis and applications." Proceedings of the IEEE 77.4 (1989): 541-580. <http://www2.ing.unipi.it/~a009435/issw/extra/murata.pdf>

Liveness

- A petri net in M_0 is live or strong-live: all transitions are L4-Live
- A petri net in M_0 is weak-live or quasi-live: at least one transition is L4-Live, transitions which are L0-, L1-, L2-, or L3-Live exist.
- A petri net in M_0 has a livelock: at least one transition is L3-Live, transitions which are L0-, L1-, L2-, or L3-Live exist.



Place/Transition Nets



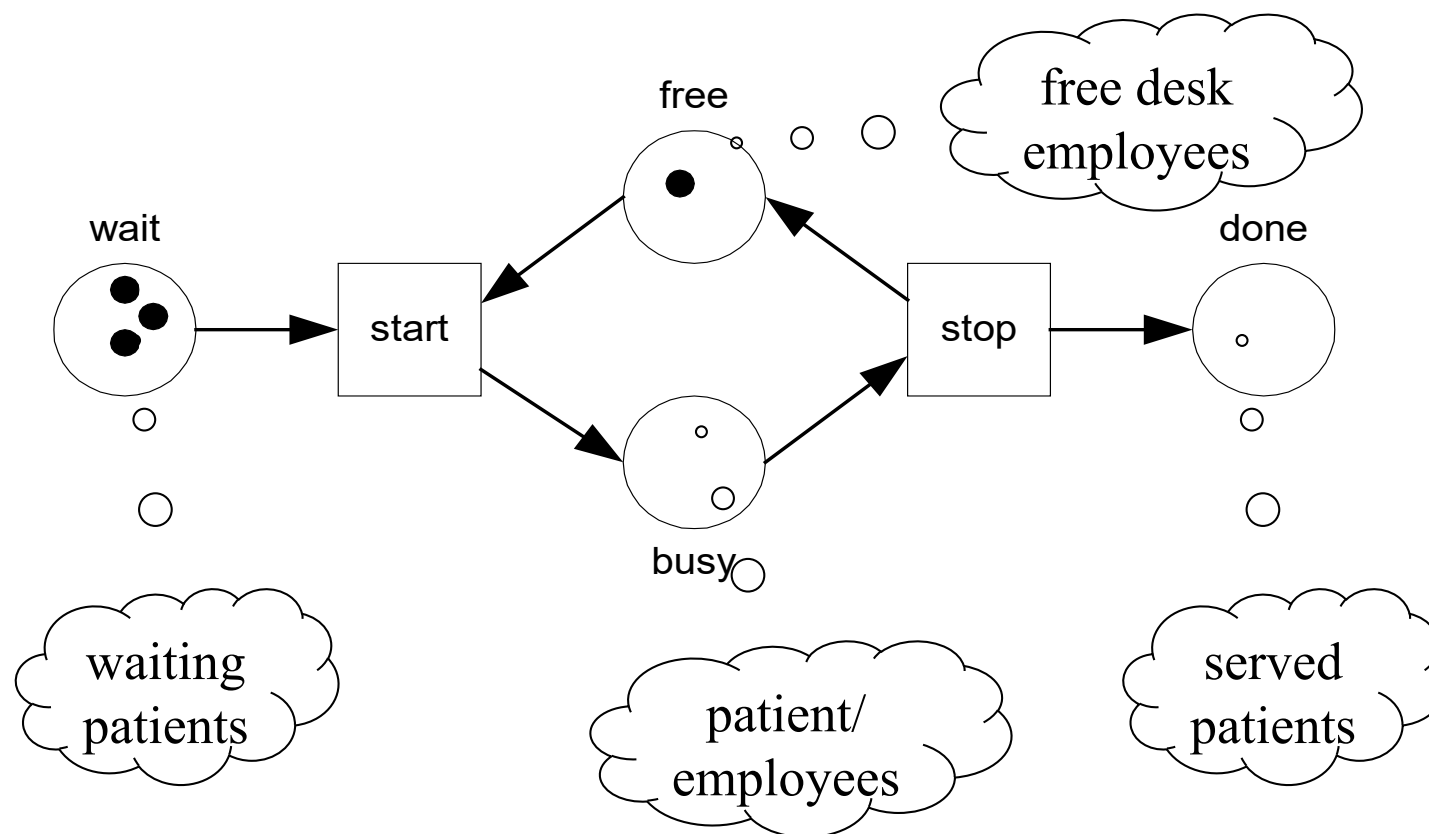
- **Different kinds of states:**
 - **Initial state:** Initial distribution of tokens.
 - **Reachable state:** Reachable from initial state.
 - **Final state** (also referred to as “dead states”): No transition is enabled.
 - **Home state** (also referred to as home marking): It is always possible to return (i.e., it is reachable from any reachable state).
- *How to recognize these states in the reachability graph?*

High-Level Petri Nets

- Limitations of classical Petri Nets:
 - Too large and complex
 - Takes too much time to model a given situation
 - It is not possible to handle time and data
- Petri nets extended with:
 - Color (i.e., data)
 - Time
 - Hierarchy

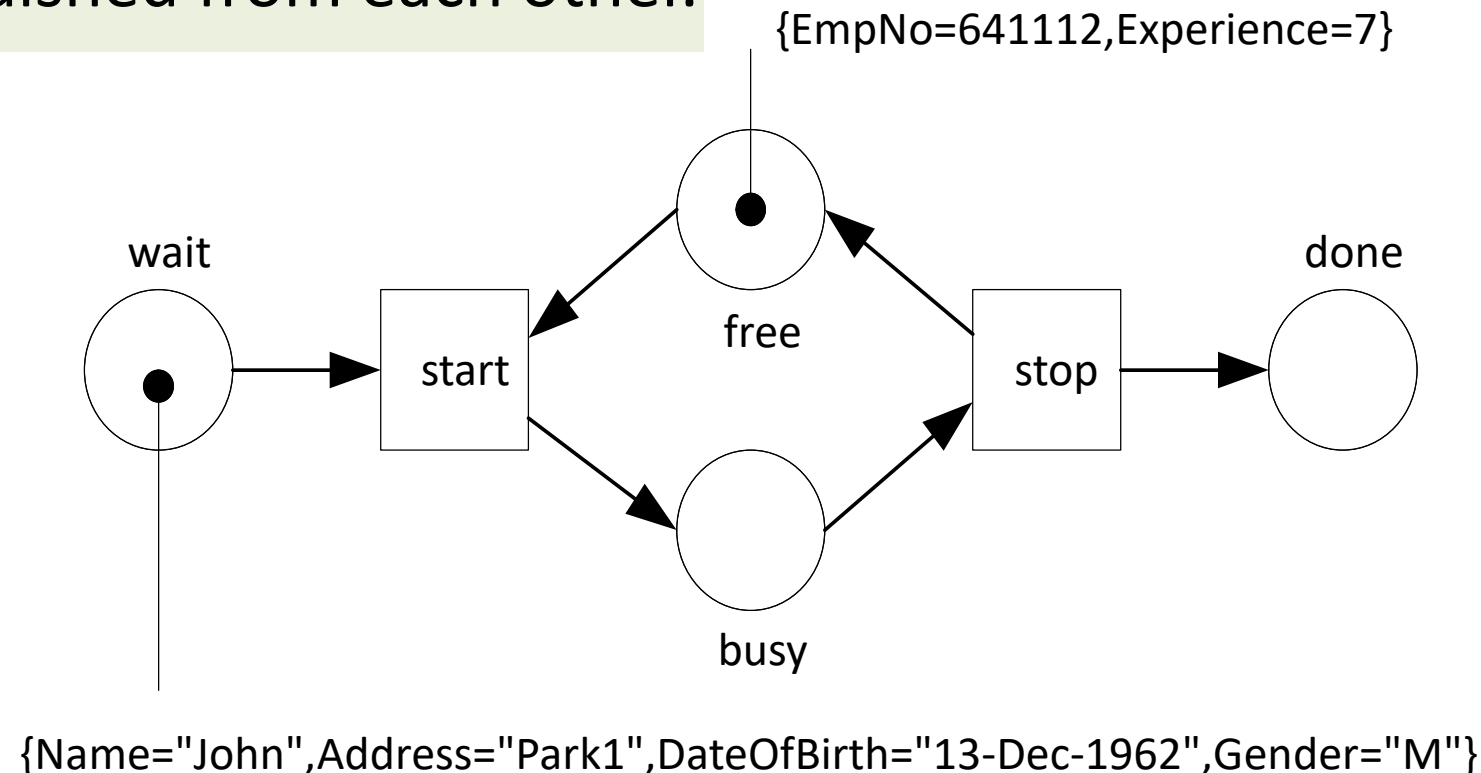
Colored Petri Nets (CPN)

In classical Petri Nets tokens cannot be distinguished



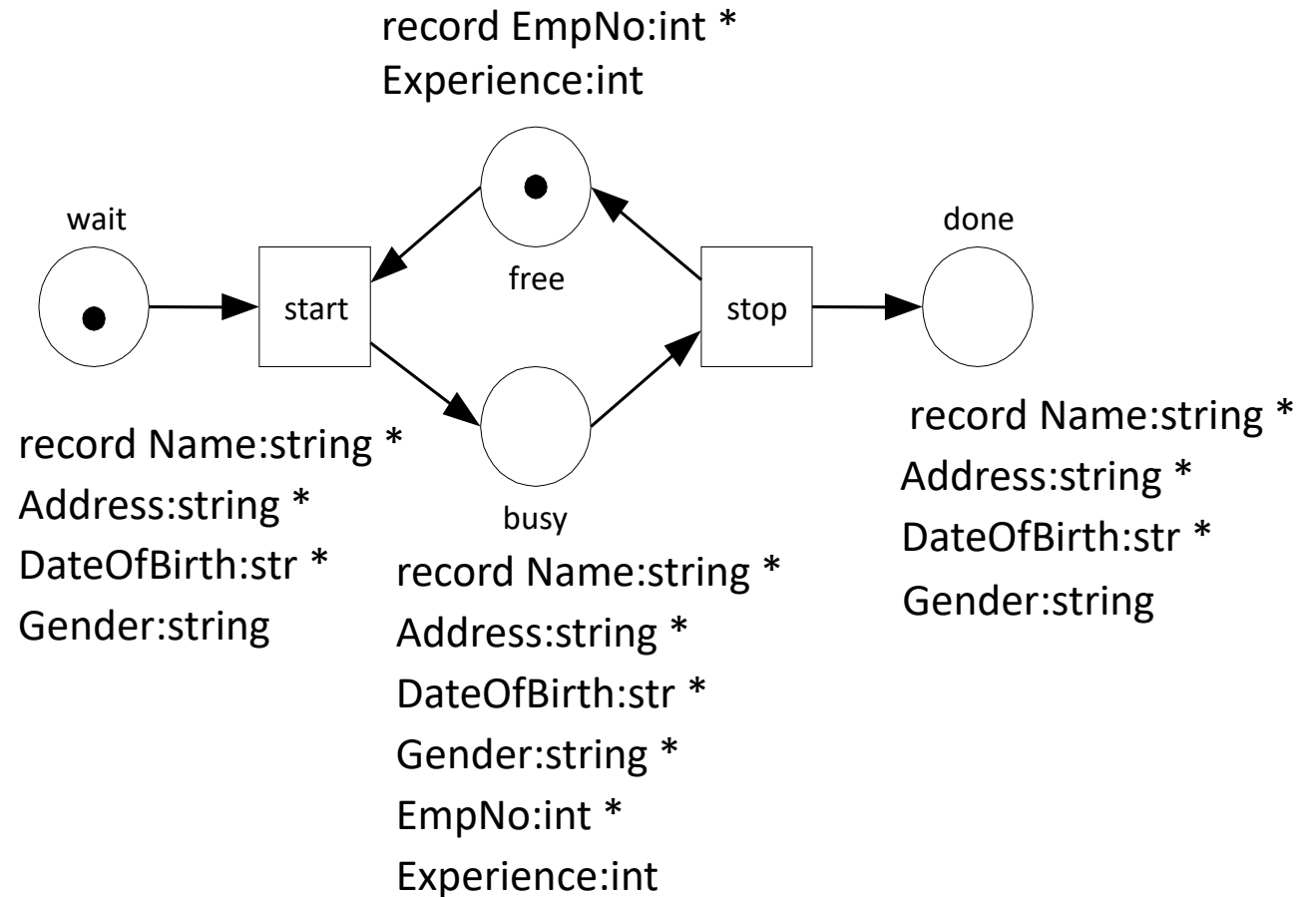
Colored Petri Nets (CPN)

Each token is provided with a **value** or **color** and can therefore be distinguished from each other.



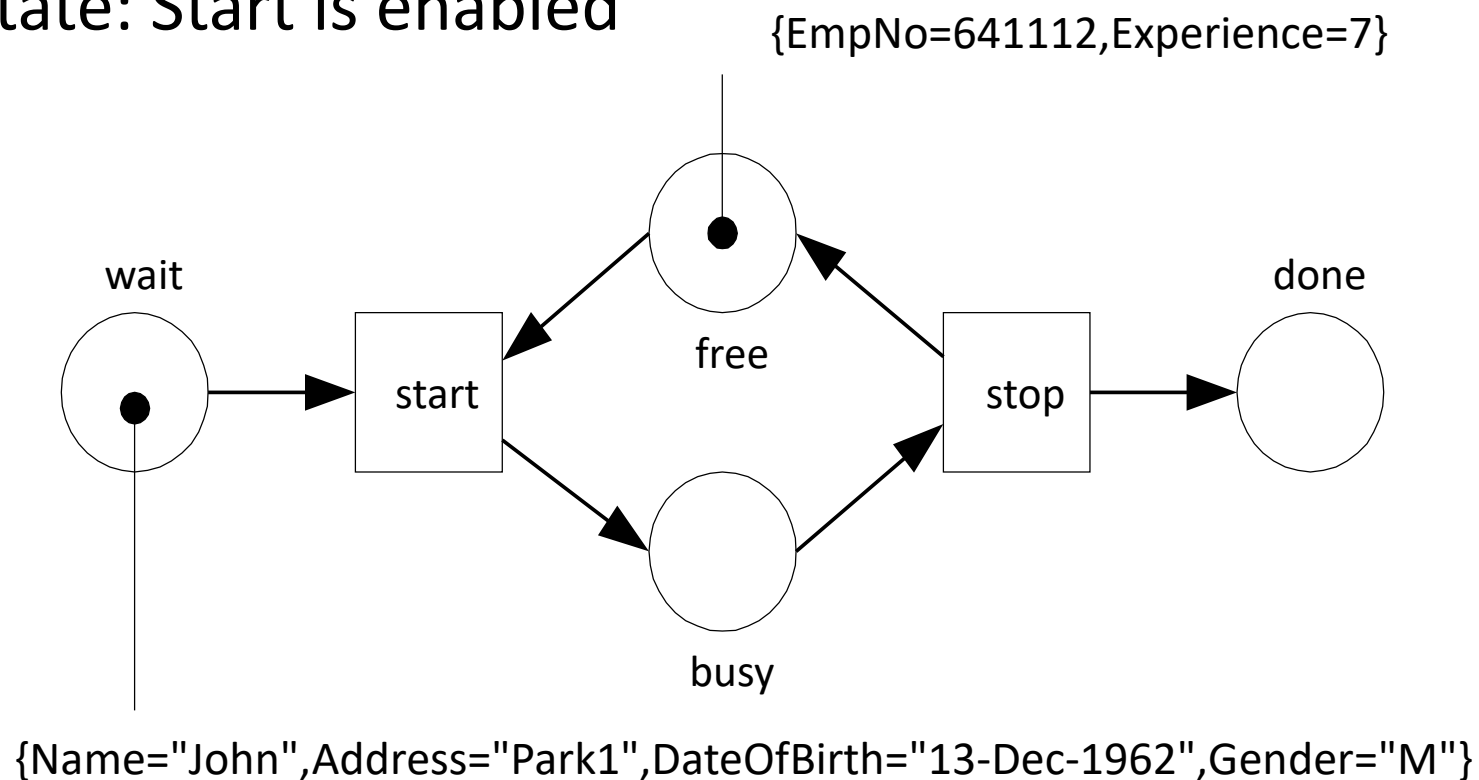
Colored Petri Nets

- Places are typed
- also referred to as color sets



Colored Petri Nets

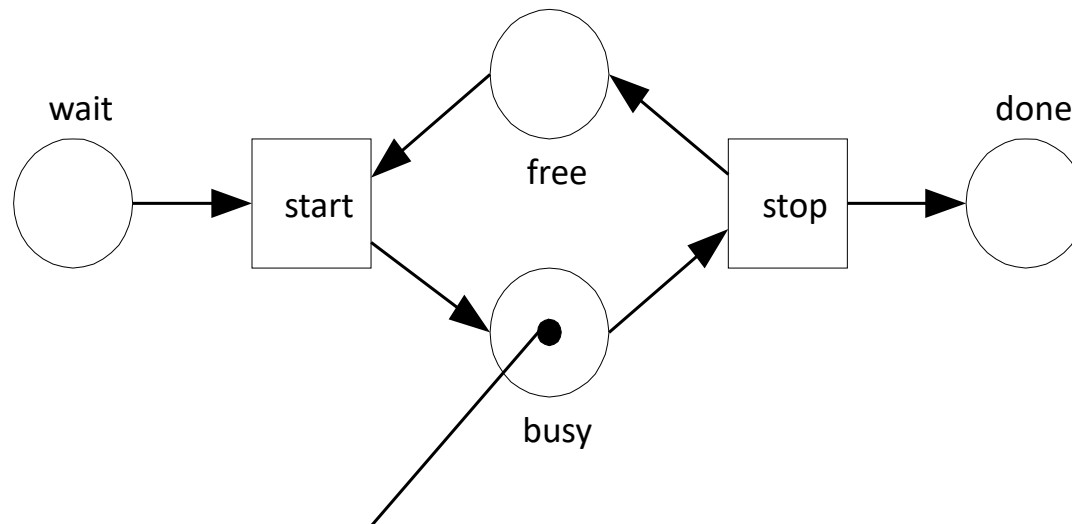
Initial State: Start is enabled



Colored Petri Nets

Transition start fired

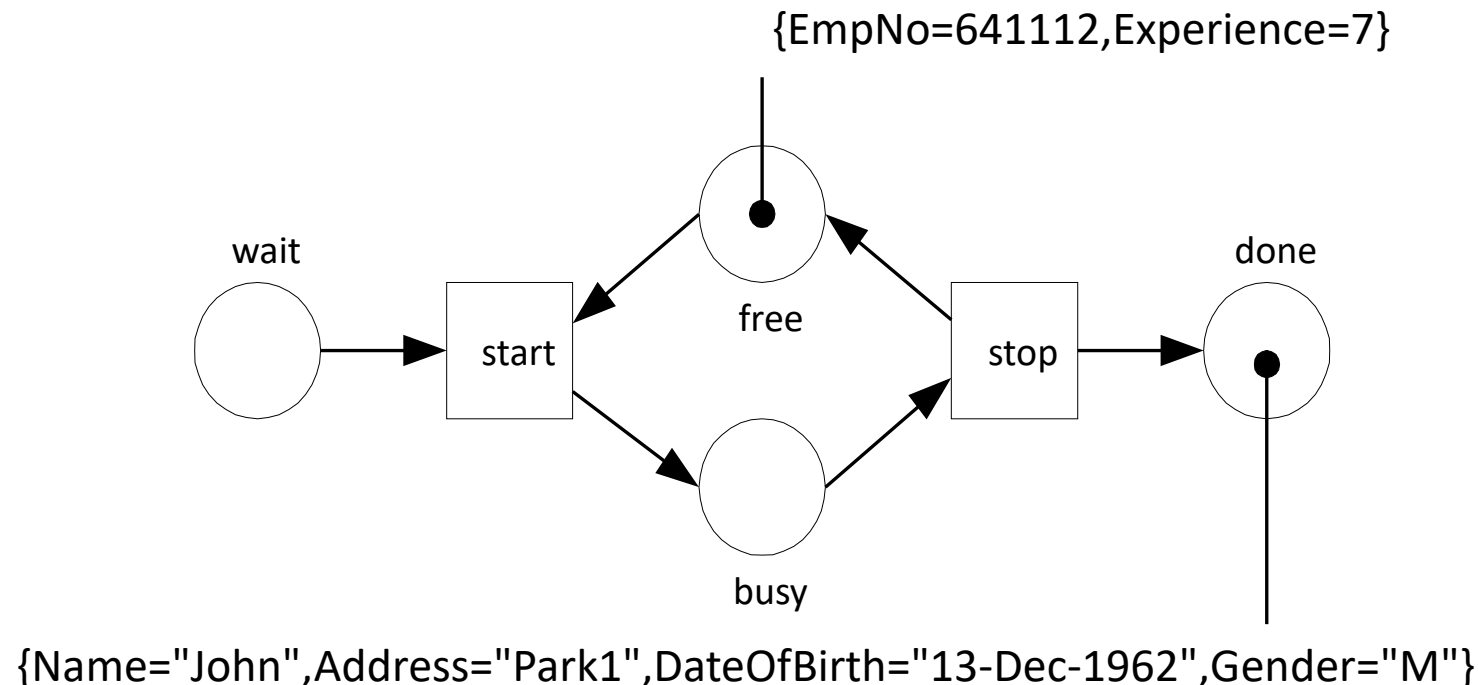
New value is created by simply merging the two records



`{Name="John",Address="Park1",DateOfBirth="13-Dec-1962",Gender="M",EmpNo=641112,Experience=7}`

Colored Petri Nets

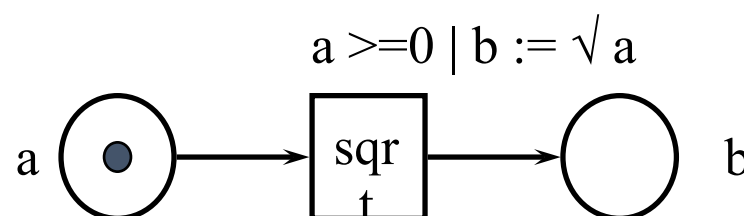
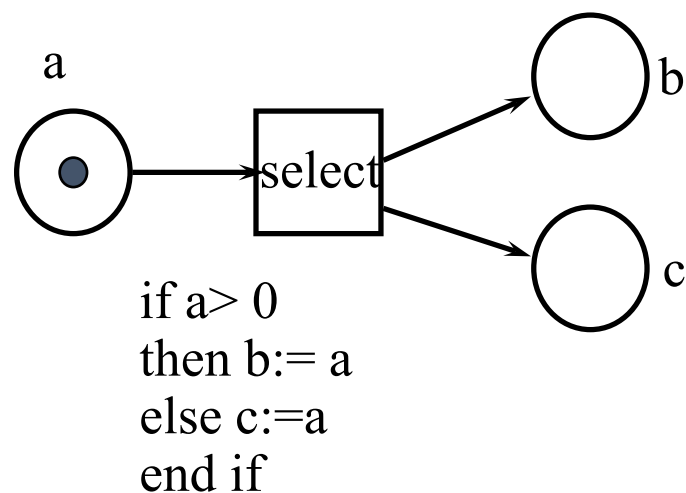
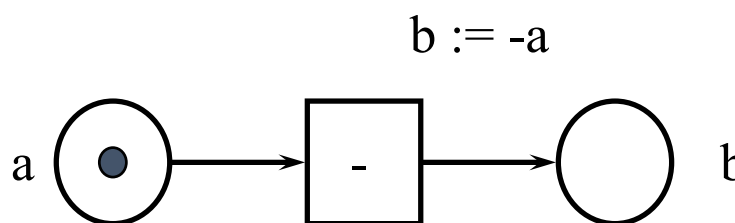
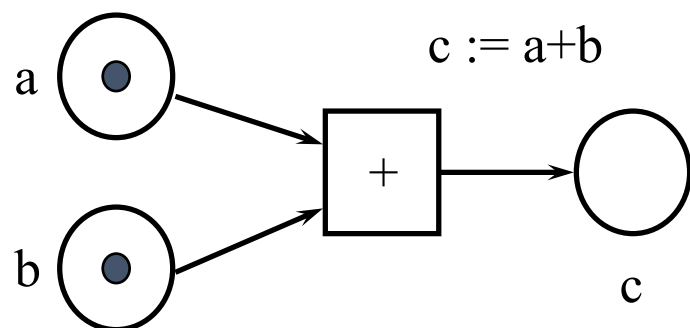
- Transition stop fired
- New values are created by simply splitting the two records in two parts



Colored Petri Nets

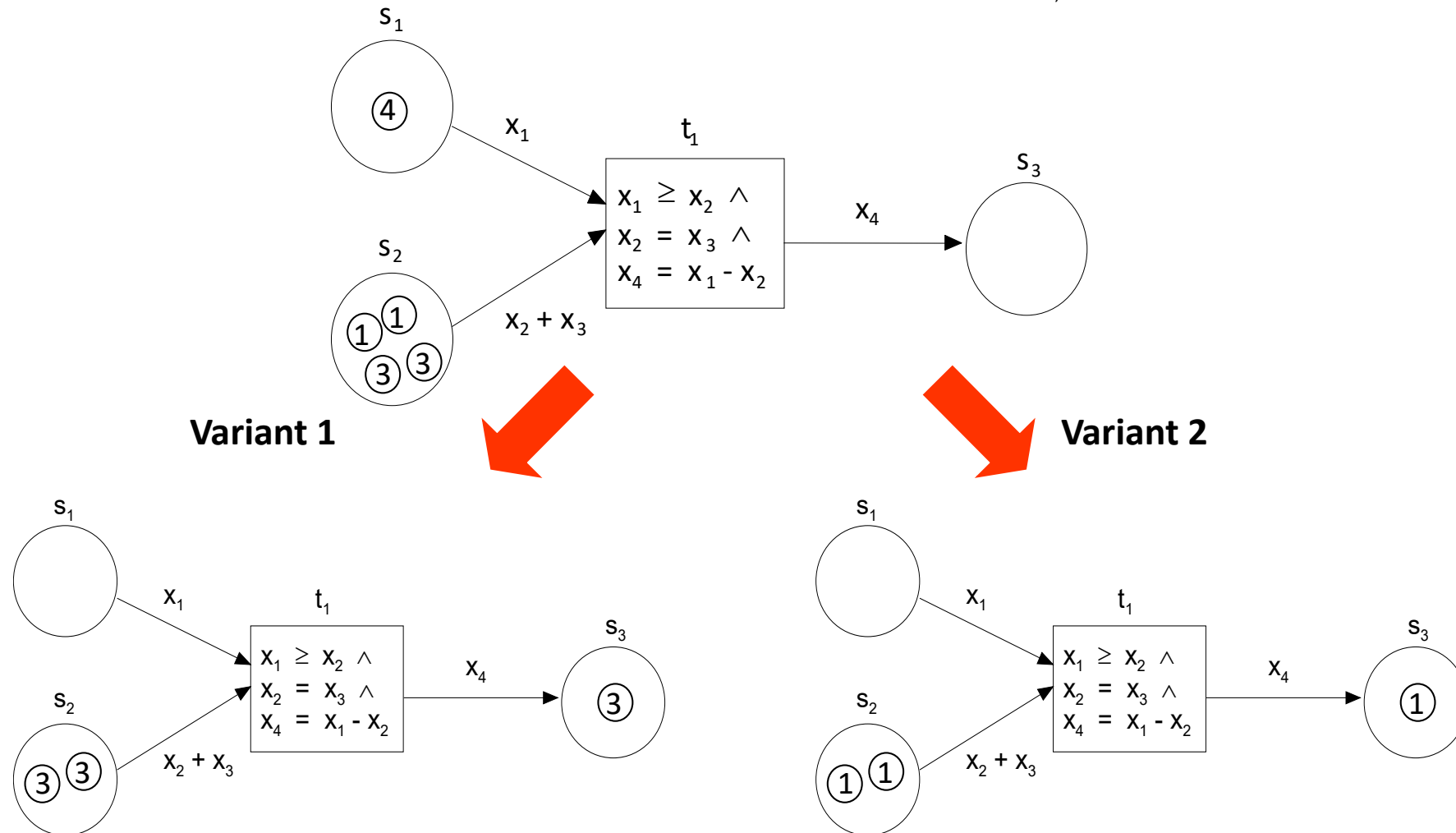
- Conditions for the values of the tokens to be consumed can be set
- A transition is then only enabled when there is at least one token at each input place and the preconditions have been met
- Each transition has an (in)formal specification which specifies:
 - the number of tokens to be produced,
 - the values of these tokens,
 - and (optionally) a precondition.

Colored Petri Nets



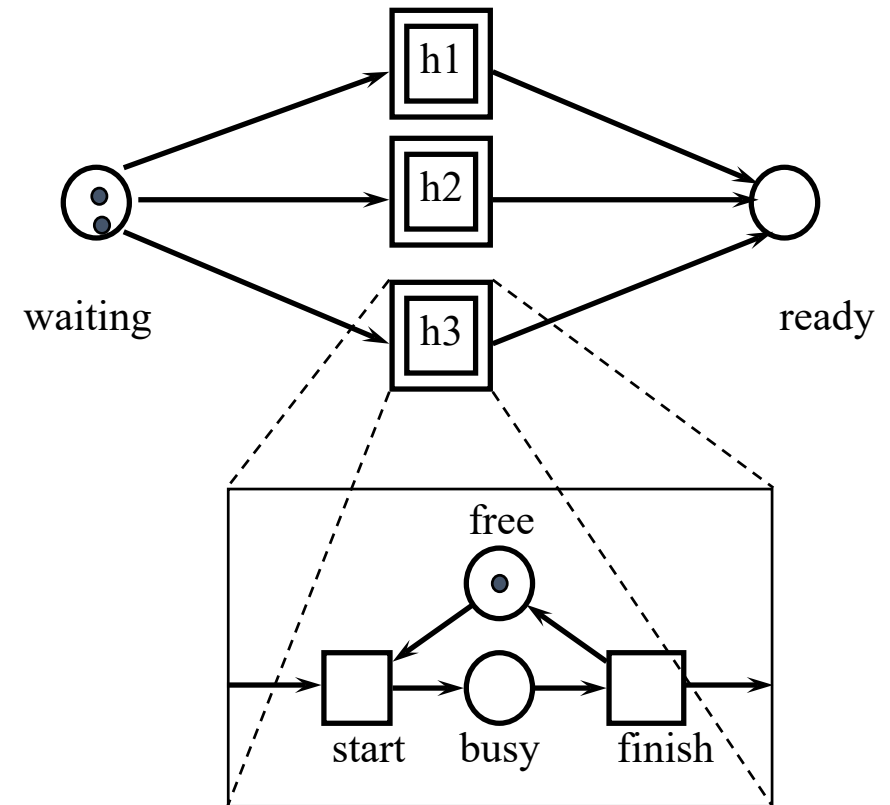
Color Extension exercise:
calculate $\sqrt{|a+b|}$ using
these building blocks

Predicate/Transition Nets

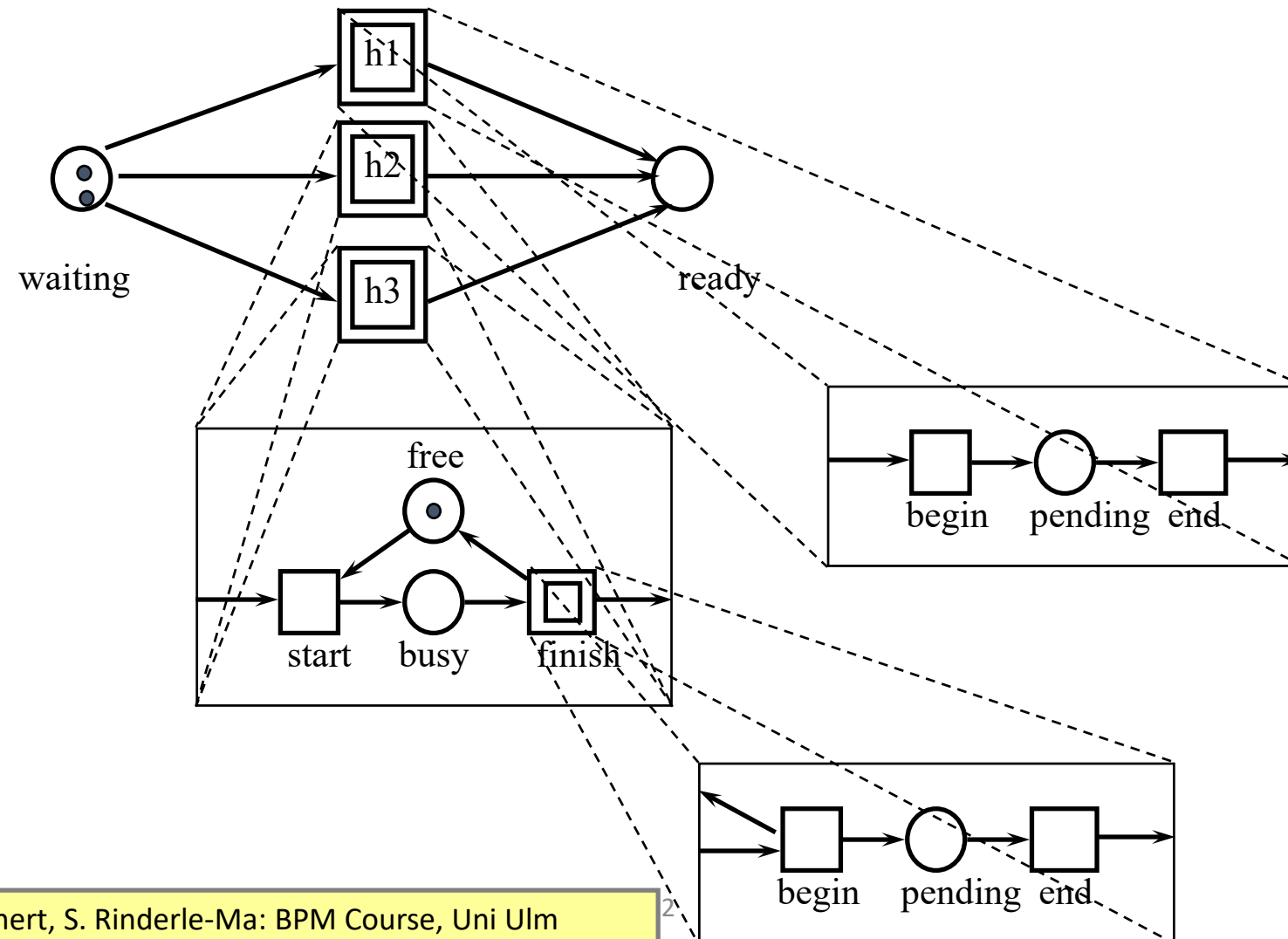


Hierarchy Extension

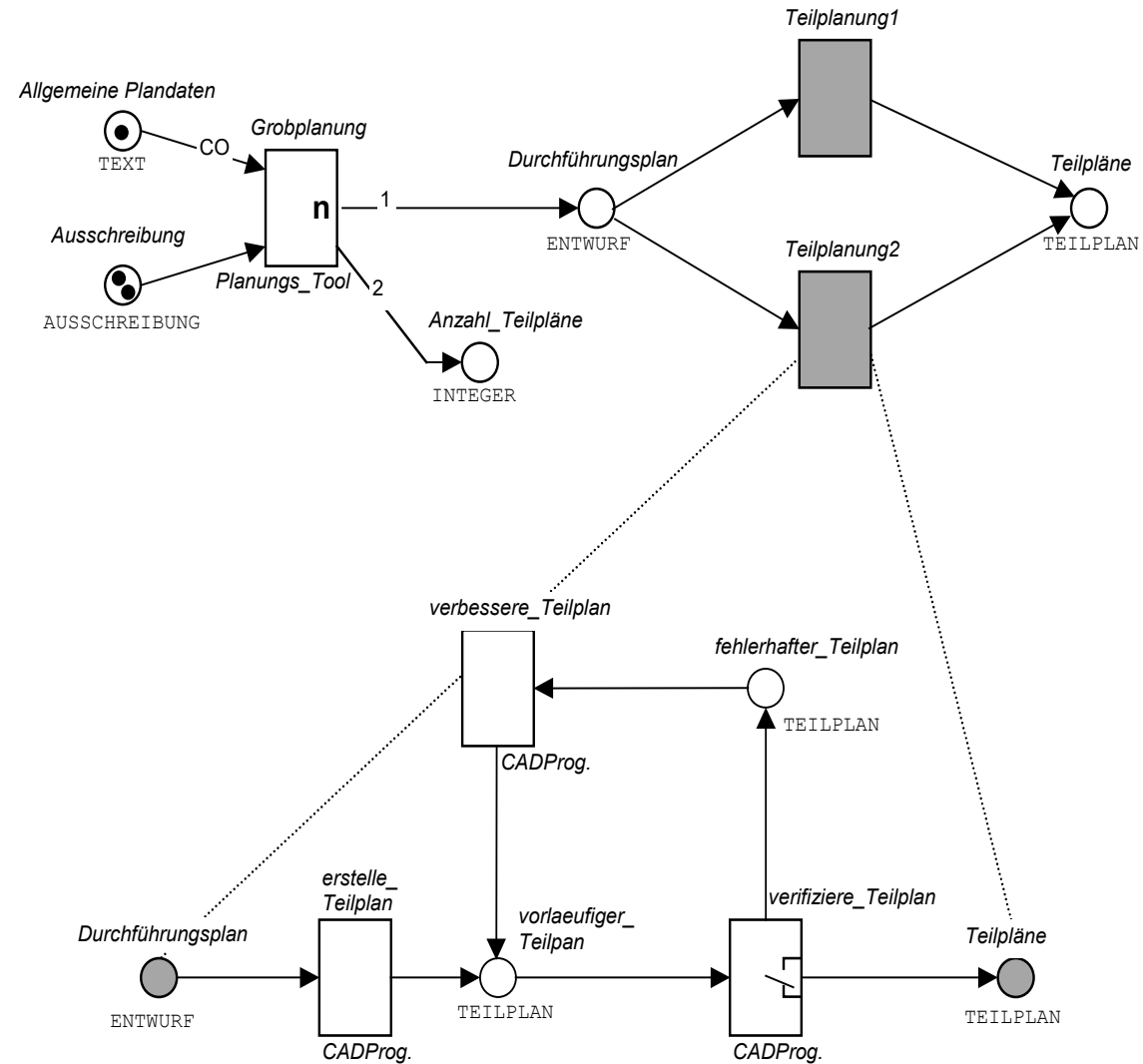
- A mechanism to structure complex Petri nets
- A subnet is a net composed out of places, transitions and subnets.



Hierarchy Extension



FunSoft Nets



Workflow Nets

- Why Workflow Nets?
- Our objective: Modelling processes / workflows
- Classical Petri Nets are
 - Not expressive enough on the one hand side: Tokens should carry information
 - Too expressive at the other hand side: Possibility to model incorrect workflows
- Plus need for
 - Formal semantics
 - Graphical representation
 - Analysis of process properties
 - Tool independence

Partly based on: Weske, M. (2019) Business Process Management - Concepts, Languages, Architectures. Springer 2019

Workflow Nets



DEFINITION 2.9 (Workflow Net): A Petri net $PN = (P, T, F)$ is called Workflow Net (WF Net for short) if and only if the following conditions holds:

- There is a distinguished place $i \in P$ (called initial place) that has no incoming edge, i.e., $\bullet i = \emptyset$
- There is a distinguished place $o \in P$ (called final place) that has no outgoing edge, i.e., $o \bullet = \emptyset$
- Every place and every transition is located on a path from the initial place to the final place

- *Consequences:*
 - i is the only place without incoming edges
 - o is the only place without outgoing edges
- WHY?

Partly based on: Weske, M. (2019) Business Process Management - Concepts, Languages, Architectures. Springer 2019

Workflow Nets

- Example: Sequence “First A, then B”

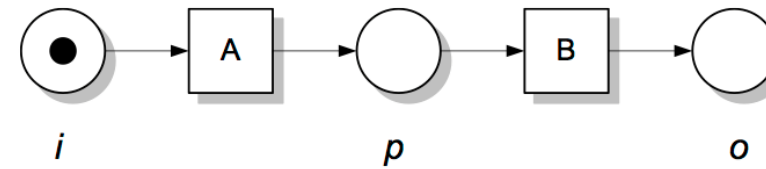


Fig. 4.42. Sequence pattern in workflow net

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2012, 2007

- Example: Parallelism “B and C can be processed in arbitrary order”

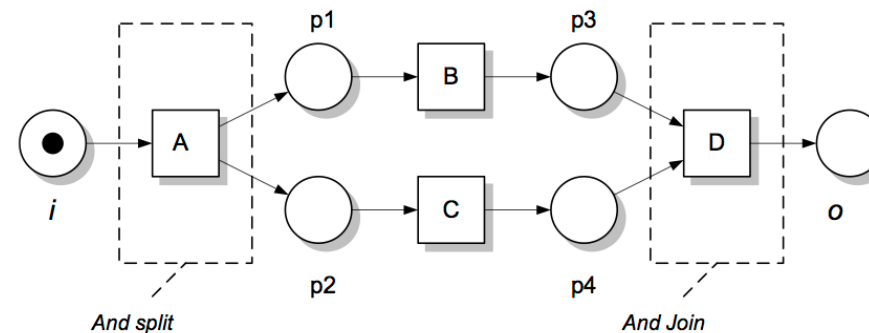


Fig. 4.43. And split and and join patterns in workflow nets

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2012, 2007

Workflow Nets

- Example: Conditional Paths “Either A or B” → implicit choice (depending on whether A or B fires first)

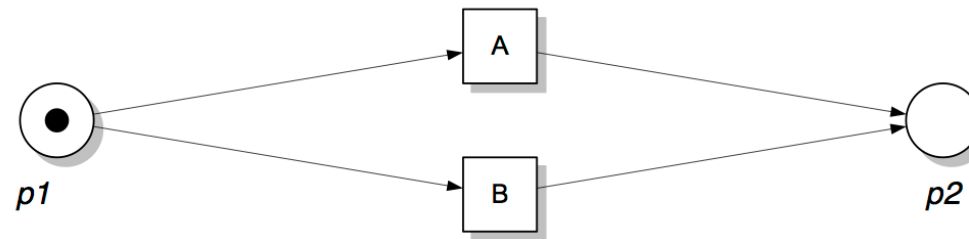


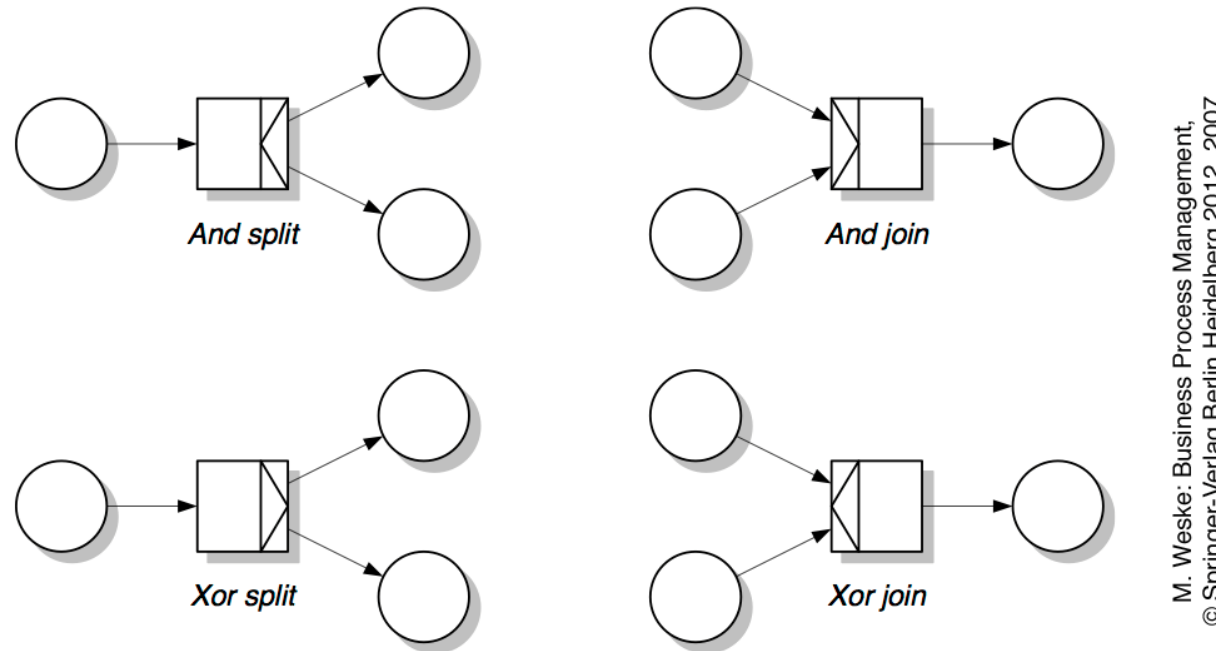
Fig. 4.44. *Implicit exclusive or split* also known as *deferred choice*

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2012, 2007

- Explicit choice by using a decision transition → Transition implements decision rule
- Exclusive Or: one and only one condition evaluates to True
- Use of default branches to avoid that workflow gets stuck

Workflow Nets

Workflow Nets [Aals07] provide syntactical and semantical extensions in order to allow for more compact models (syntactic sugaring)

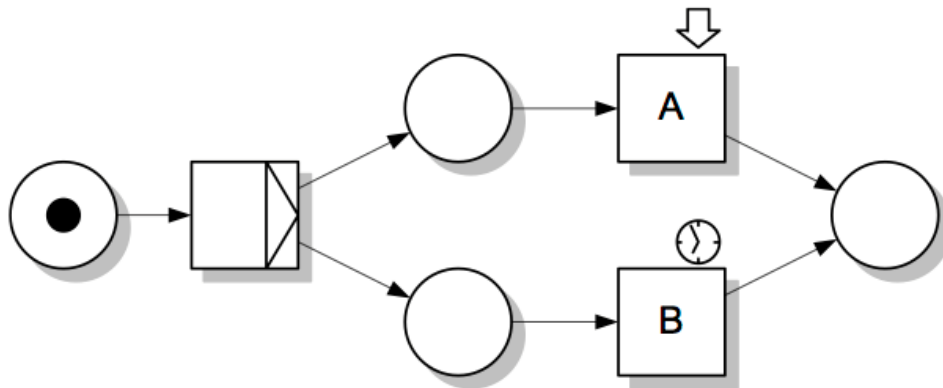


M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2012, 2007

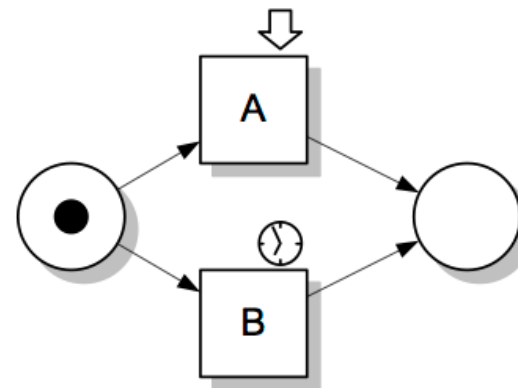
Fig. 4.46. Syntactic sugaring of transitions in workflow nets

Workflow Nets

Example: Explicit choice of conditional branch (by user or based on transition predicates)



(a) *Explicit xor split* does not enable A and B concurrently



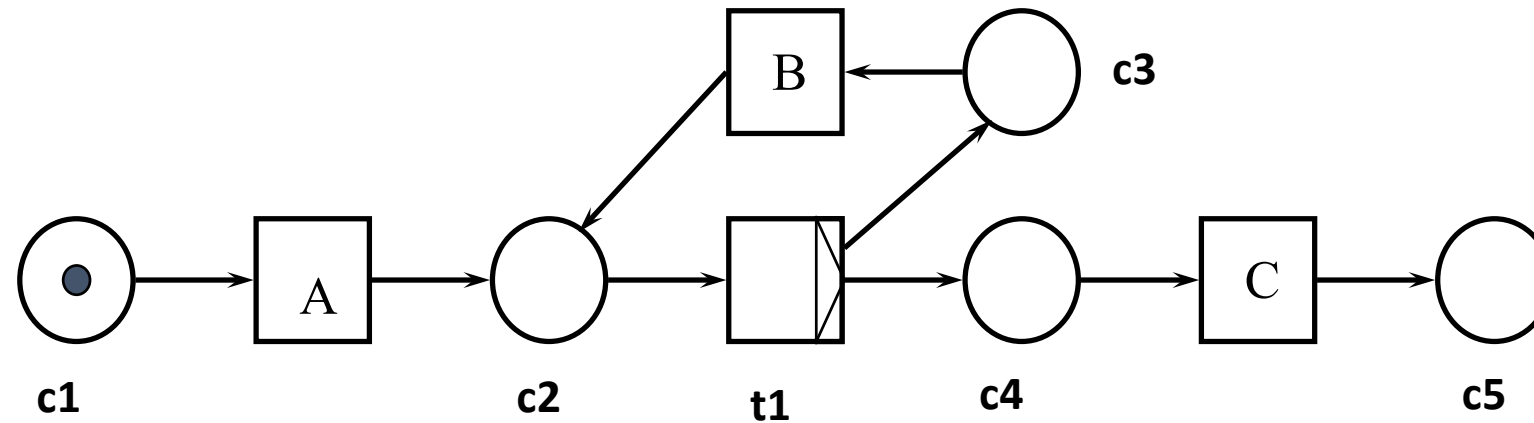
(b) *Implicit xor split* enables A and B concurrently

Fig. 4.49. Sample workflow nets illustrate the difference between *explicit xor split* (a) and *implicit xor split* (b)

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2012, 2007

Workflow Nets

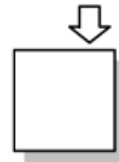
Example: Modeling loop backs: *B may be fired multiple times!*



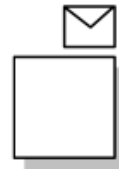
Workflow Nets



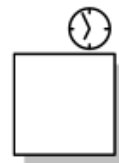
Automatic Trigger: Task enacted automatically



User Trigger: A human user takes initiative and starts activity



External Trigger: External event required to start activity



Time Trigger: Activity started when timer elapses

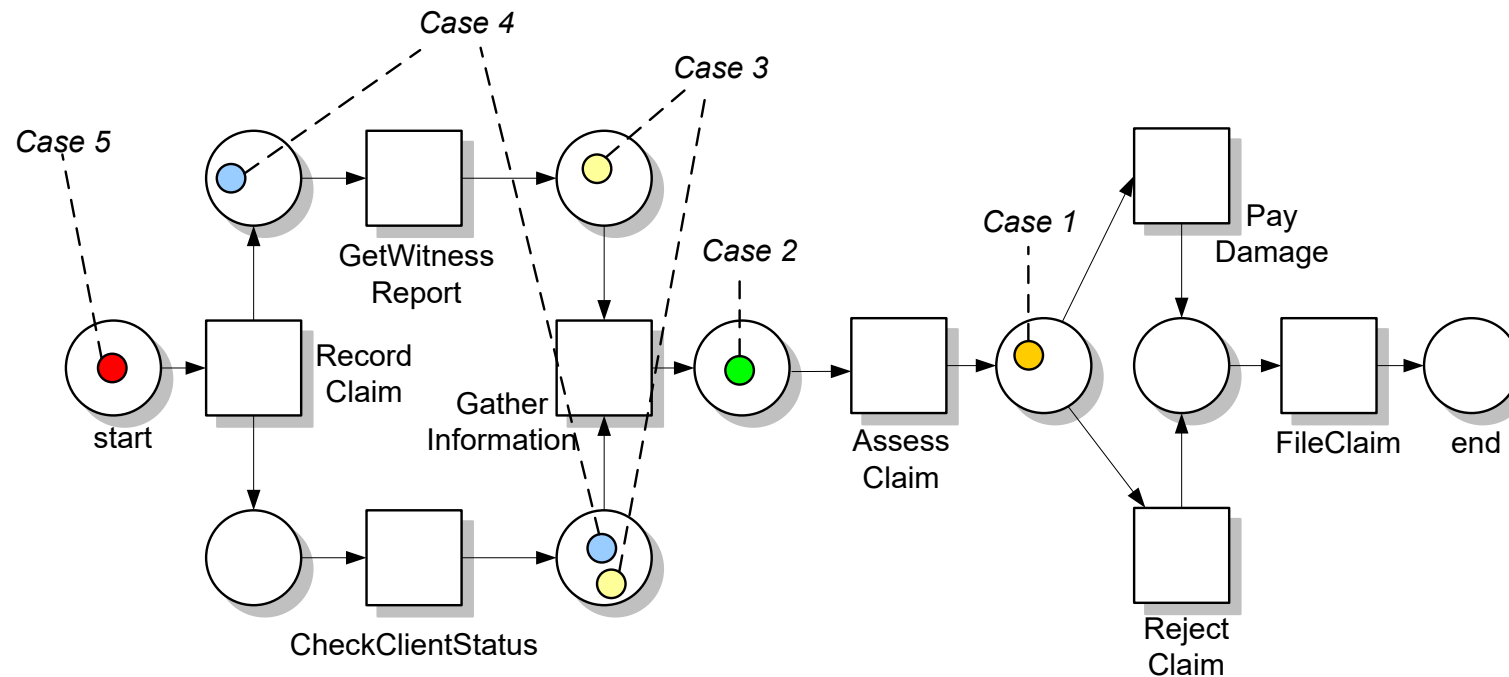
Fig. 4.47. Triggers in workflow nets



©Prof. Dr. Stefanie Rinderle-Ma (TUM) | Business Process Technologies and Management | Summer 2022

Workflow Nets

Representation of different workflow instances by **token colors**



M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Fig 4.53. Sample workflow net with coloured tokens representing process instances

Workflow Nets



- First of all note the following important distinctions:
 - **task**: A logical step which may be executed for many cases.
 - **work item** = task + case -> A logical step which may be executed for a specific case.
 - **activity** = task + case + (resource) + (trigger) -> The actual execution of a task for a specific case.
- Work items and activities are task instances.
- Workflow Nets abstract from data flow and temporal issues
- Extended Workflow Nets → YAWL (Yet Another Workflow Language): support of Workflow Patterns (later)

Soundness Notions and Analysis

- Problem: Workflow design is often erroneous
- What is the error in the following example workflow model?
- What is cheaper?
 - Detecting the error before deploying the workflow ...
 - ... or when it is already running?

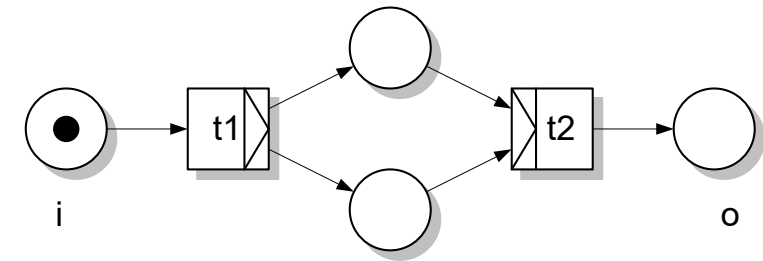


Fig 6.4. Workflow net with deadlock

Soundness Notions and Analysis



DEFINITION 2.10 (Structural Soundness) A process model is structurally sound if the following conditions hold:

- $\exists$ initial node (without incoming edges)
- $\exists$ final node (without outgoing edges)
- Each node is on a path from initial node to final node

- Criterion takes only structure of a process into account (no marking information yet)
- Independent of modeling notation / formalism
- Trivially fulfilled for WF Nets (fulfilled by definition)

Soundness Notions and Analysis

- The following Petri Net is obviously structurally not sound
- What are the consequences?

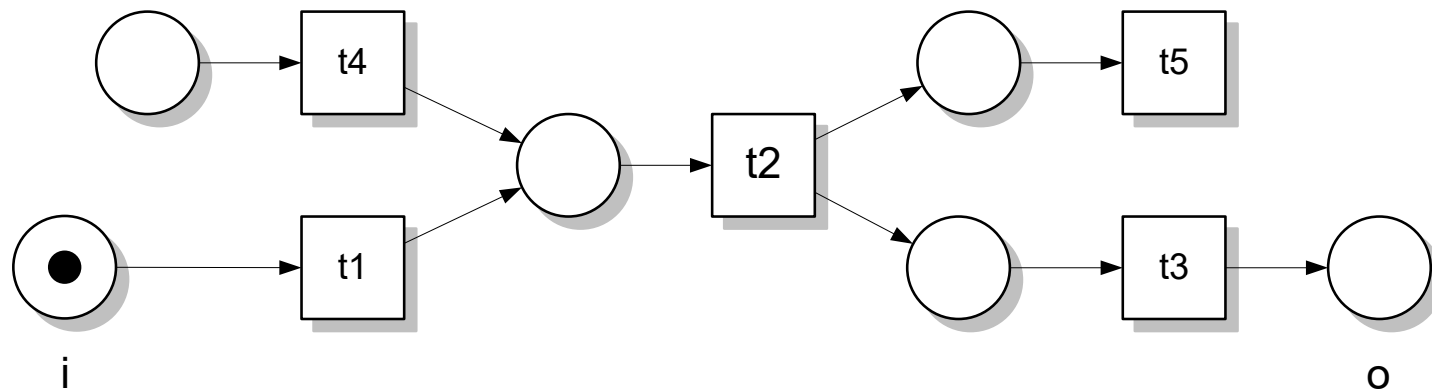


Fig 6.3. Petri net with dangling places and dangling transitions

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Soundness Notions and Analysis

- Let us go back to the motivating example
- The problem here is not a structural one, it is **behavioral** one
- Specifically, at runtime, the workflow instance will run into a **deadlock**
- A deadlock is a reachable state where no transition is activated
- How can we determine deadlocks?

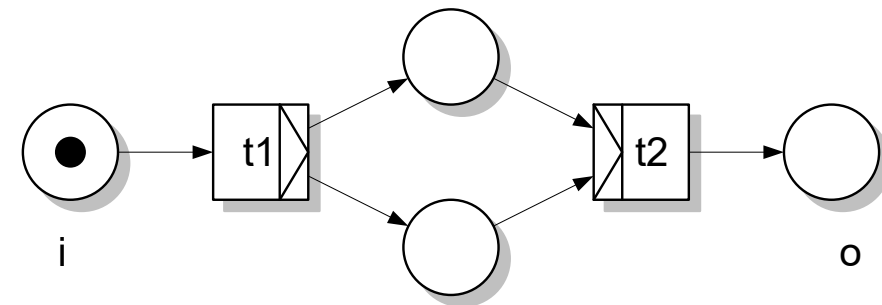


Fig 6.4. Workflow net with deadlock

Soundness Notions and Analysis

- In addition to deadlocks, the workflow might run into a **livelock**
- Livelock: a subset of the workflow activities is trapped in an infinite loop
- Possible reasons:
 - Loop backward condition always evaluates to true
 - Workflow model design as in the following example:

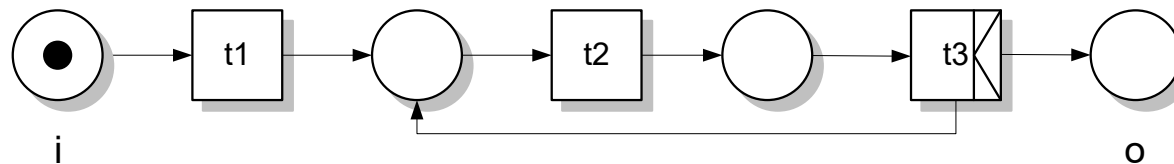


Fig 6.5. Workflow net with livelock

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Soundness Notions and Analysis

- What are the problems with the following WF Net?

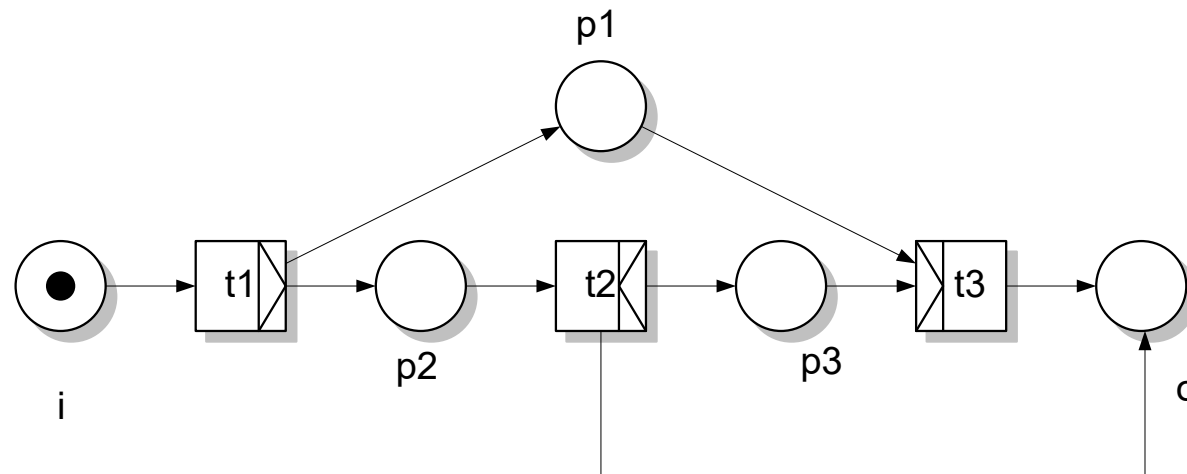


Fig 6.6. Workflow net with deadlock/remaining tokens

M. Weske: Business Process Management,
© Springer-Verlag Berlin Heidelberg 2007

Soundness Notions and Analysis



DEFINITION 2.11 (Useful Notions) Let $PN = (P, T, F)$ be a WF Net, $i \in P$ be its initial place, $o \in P$ its final place, and M, M' markings. Then we define the following notions:

- M_i is the state (marking) in which there is exactly one token in place i and no token in any other place $p \in P$
- M_f is the state (marking) in which there is exactly one token in place o and no token in any other place $p \in P$
- $M \geq M' \Leftrightarrow M(p) \geq M'(p) \forall p \in P$
- $M > M' \Leftrightarrow M \geq M' \wedge \exists p \in P \text{ with } M(p) > M'(p)$
- (PN, M_i) is called a workflow system based on WF Net PN with initial marking M_i

Soundness Notions and Analysis



DEFINITION 2.12 (Soundness) A workflow system (PN, M_i) with $PN = (P, T, F)$ is sound $\Leftrightarrow$ the following conditions hold:

- For every marking M reachable from M_i there exists a firing sequence leading from M to M_f , formally:

$$\forall M (M_i \xrightarrow{*} M) \Rightarrow (M \xrightarrow{*} M_f)$$

- M_f is the only marking reachable from M_i with at least one token in place o , formally:

$$\forall M (M_i \xrightarrow{*} M \wedge M \geq M_f) \Rightarrow (M = M_f)$$

- There are no dead transitions for PN in M_i , formally:

$$(\forall t \in T) \exists M, M' : i \xrightarrow{*} M \xrightarrow{t} M'$$

Soundness Notions and Analysis

- Soundness can be verified based on the reachability graph of the WF Net
- Is the following WF Net sound?

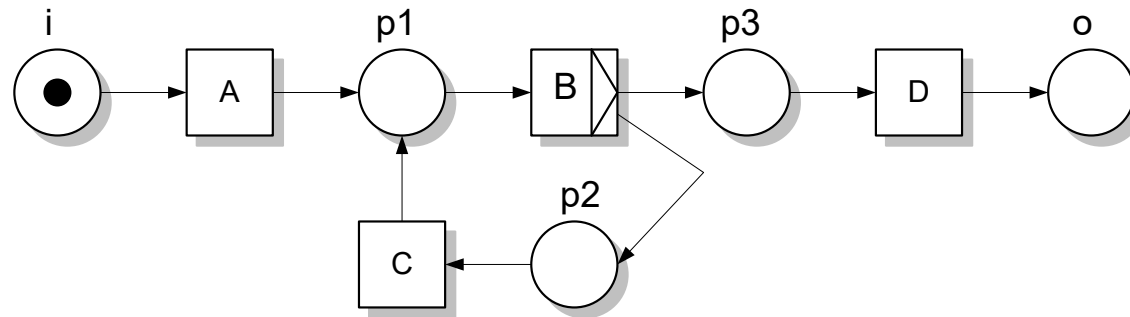


Fig 6.8. Workflow net with loop

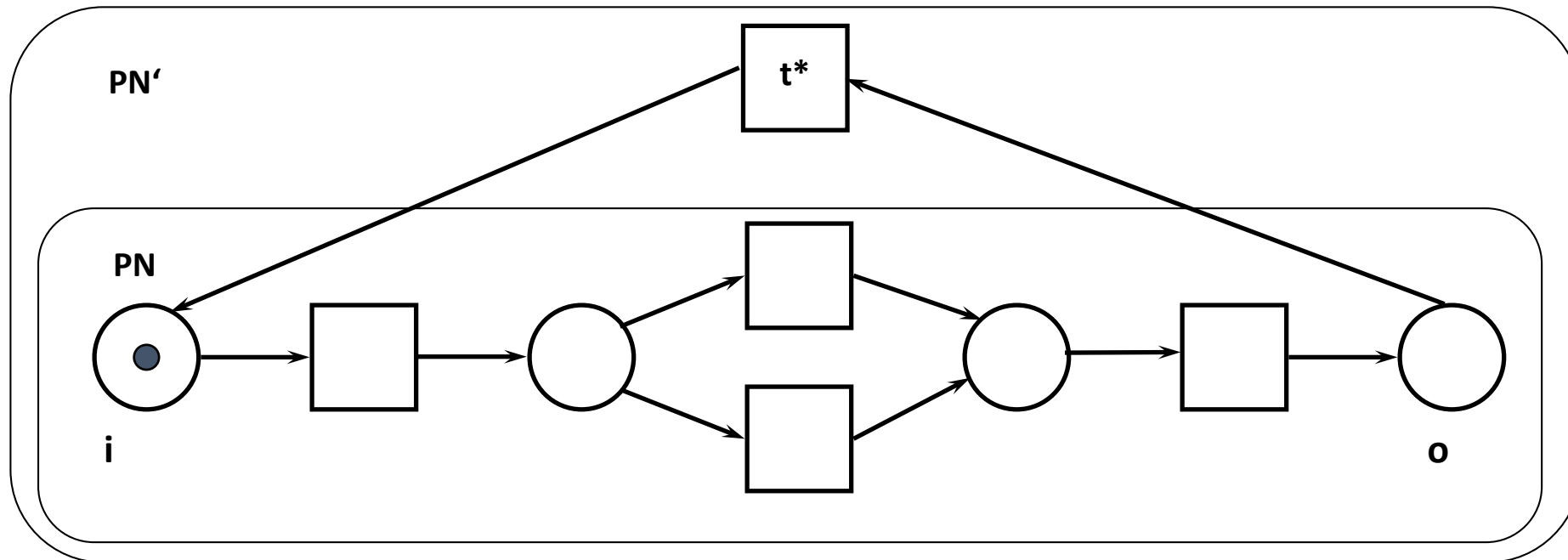
Soundness Notions and Analysis



- Computing the reachability graph works for simple models
- Support by analysis tools (e.g., Woflan
<http://www.win.tue.nl/woflan/doku.php>)
- Due to state explosion runtime behavior is exponential
- Idea: show that if certain properties hold for a WF Net, this WF Net is sound AND the properties can be checked efficiently
- The first and third soundness conditions imply some property such as liveness
- The second soundness condition implies some property such as boundedness
- However, these properties are not 100% fitting on WF Nets
- Trick: Use extension of WF Nets

Soundness Notions and Analysis

- Example WF Net PN is obviously sound, bounded, but not live
- How to extend PN such that the extended Petri Net becomes live?
- Introduce artificial transition t^* connecting places o and i



Soundness Notions and Analysis

- Liveness and boundedness can be checked in polynomial time for free-choice nets
- Within a free choice net the sets of input places of two transitions are either disjoint or identical

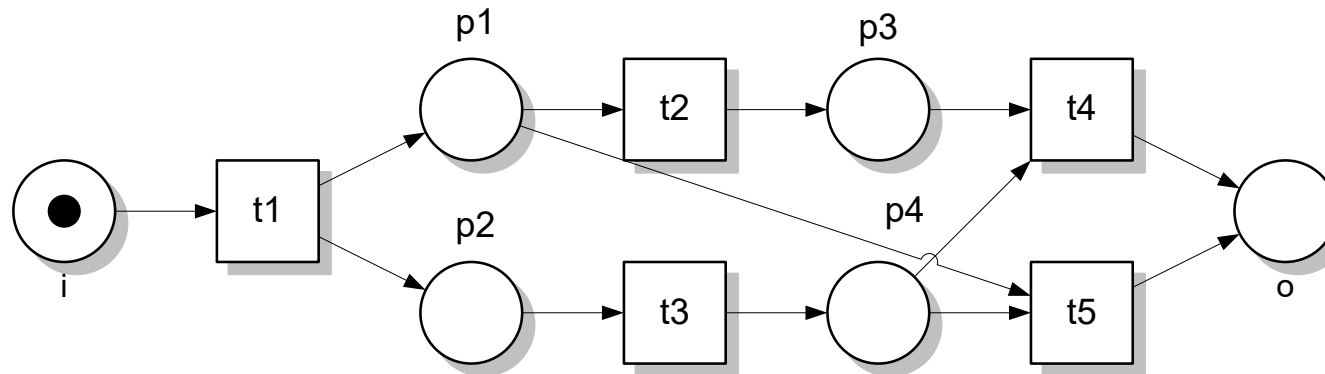


Fig 6.11. Non-free-choice workflow net

Soundness Notions and Analysis



Soundness is desirable for many workflow applications, because it guarantees the following properties

- **P1 (Termination):** every instance starting in the initial state finally reaches the final state → no instance „gets lost“ somewhere during the the workflow execution
- **P2 (Proper Termination):** the final state is the only state reachable from the intial state where there is a token in the final places → no tokens are somewhere left in the workflow when it is already finished
- **P3 (Do dead transitions):** Each transition can contribute to at least one workflow instances
- **P4 (Transition participation)** – (implied: $P1 \wedge P3 \Rightarrow P4$): for each transition t there exists a firing sequence from i to o in which t participates:

$$(\forall t \in T) \exists M, M': (M_i \xrightarrow{*} M \xrightarrow{t} M' \xrightarrow{*} M_f)$$

Soundness Notions and Analysis



- However, for some application scenarios soundness is too strict
 - Scenario 1: Semiformal business process modeling (e.g., using EPCs → Chapter 1)
 - Rather modeling of „best practices“ than of correct models
 - Goal: model should reflect necessary execution paths
 - However, it might also reflect non-correct ones
 - Scenario 2: In the context of **interorganizational workflows (workflow choreographies)**
 - Not executed within one enterprise (inhouse)
 - but executed among different partners, mostly based on message exchange (e.g., supplier → producer → customer)
 - Practical assumption: services are used which provide necessary functionality (and even more functionality than that) and the „excessive“ functionality does not have to be used

Soundness Notions and Analysis



- This leads to **relaxed** soundness notions (i.e., notions where we abstain from certain properties P1 – P4)
- For Scenario 1 the notion of **relaxed soundness** has been developed: *For each transition t there is a workflow instance execution which starts in the initial state, executes t , and terminates in the final state.*

DEFINITION 2.13 (Sound firing sequence) Let $S = (PN, M_i)$ be a workflow system. Let σ, σ' be firing sequences and let M, M' be markings. σ is a sound firing sequences if

$$M_i \xrightarrow{\sigma} M \wedge \exists \sigma': M \xrightarrow{\sigma'} M_f$$

DEFINITION 2.14 (Relaxed Soundness): A workflow system $S = (PN, M_i)$ is relaxed sound $\Leftrightarrow$

$$\forall t \in T \exists M, M': (M_i \xrightarrow{*} M \xrightarrow{t} M' \xrightarrow{*} M_f)$$

Soundness Notions and Analysis



- For Scenario 2 the **weak soundness** notion has been developed
- Weak soundness requires termination (P1) and proper termination (P2)
- However, it does not require that every transition participates in the workflow execution

DEFINITION 2.15 (Weak Soundness) A workflow system (PN, M_i) with $PN = (P, T, F)$ is weak sound $\Leftrightarrow$ the following conditions hold:

➤ For every marking M reachable from M_i there exists a firing sequence leading from M to M_f , formally: $\forall M (M_i \xrightarrow{*} M) \Rightarrow (M \xrightarrow{*} M_f)$

➤ M_f is the only marking reachable from M_i with at least one token in place o , formally:

$$\forall M (M_i \xrightarrow{*} M \wedge M \geq M_f) \Rightarrow (M = M_f)$$

Soundness Notions and Analysis



- *Soundness* is the strictest correctness notion requiring properties P1, P2, P3 (and implies P4)
- *Weak soundness* allows for dead transitions, but guarantees proper termination (P1, P2)
- *Relaxed Soundness* only guarantees that every transition participates in at least one workflow instance (P4), but allows for deadlocks
- *Lazy soundness* has been developed for certain workflow patterns which are neither sound nor weak sound nor relaxed sound
- Lazy soundness allows for tokens to be left within the workflow instance even if there is a token within the final place
- The transitions possibly activated by the left tokens are called lazy activities
- They might be fired even after the termination of the workflow instance
- Properties P1 – P4 cannot be clearly verified for or against lazy soundness → further investigations necessary

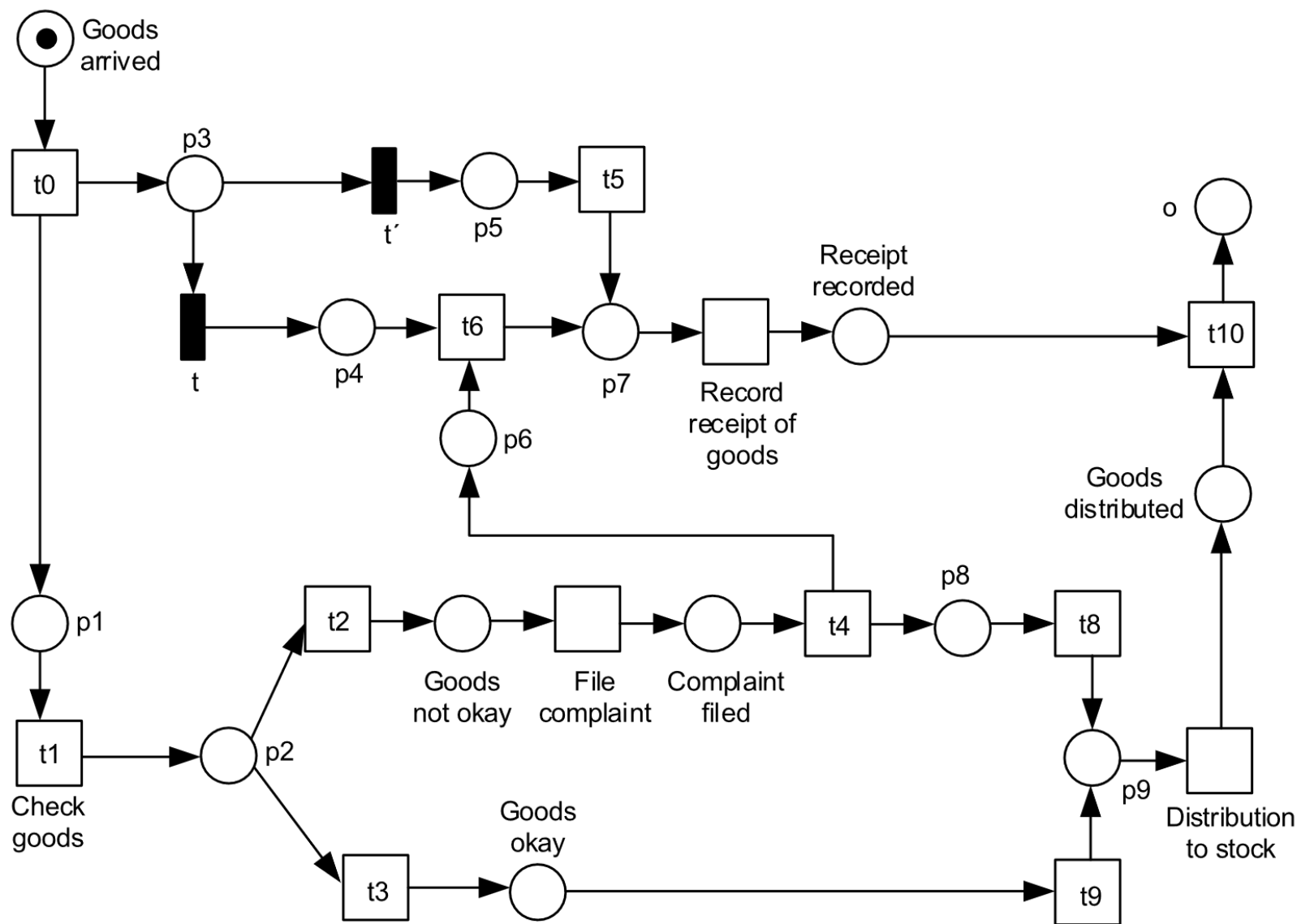
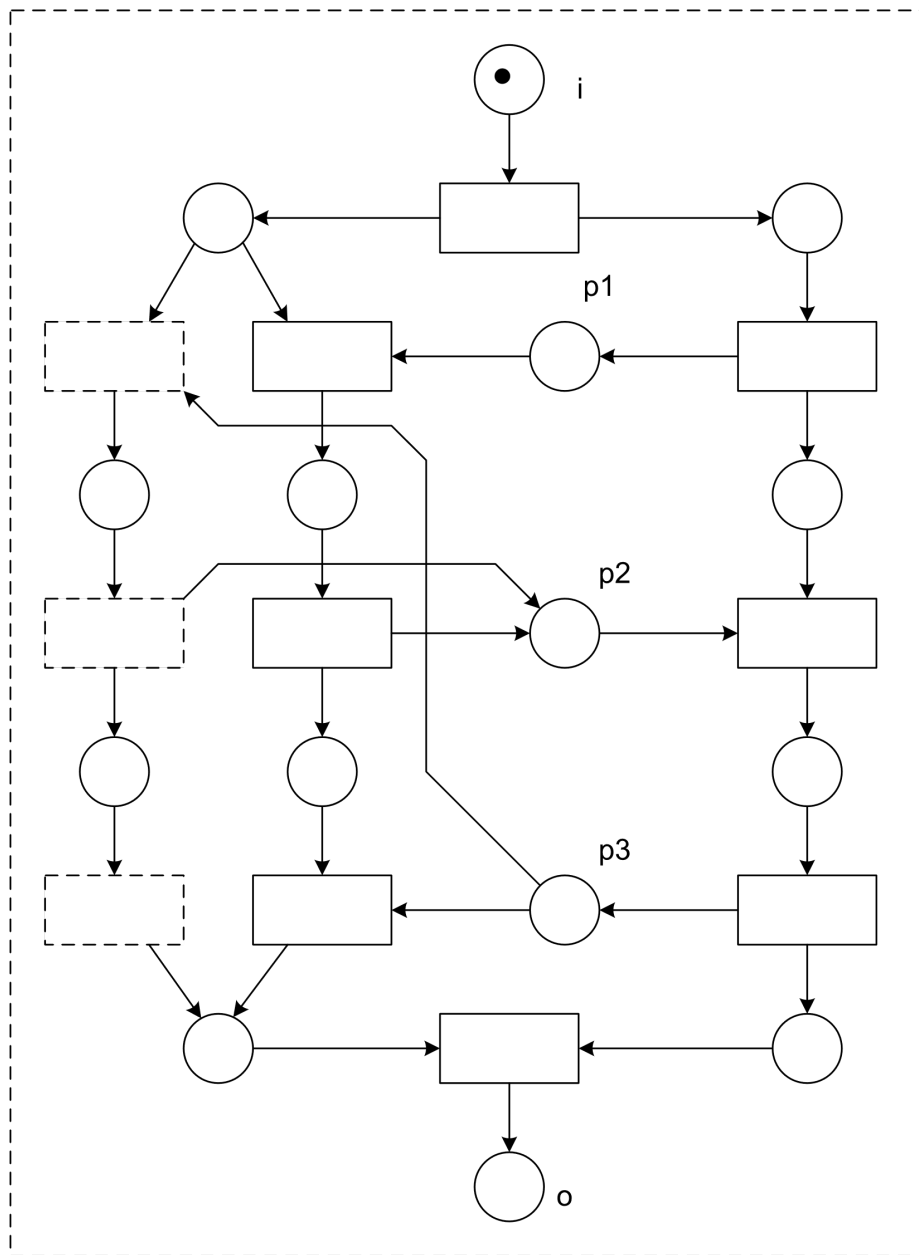


Fig. 7.19. Relaxed sound workflow net



M. Weske: Business Process Management, Third Edition
© Springer-Verlag 2019, 2012, 2007

Fig. 7.24. Example of weak sound workflow net

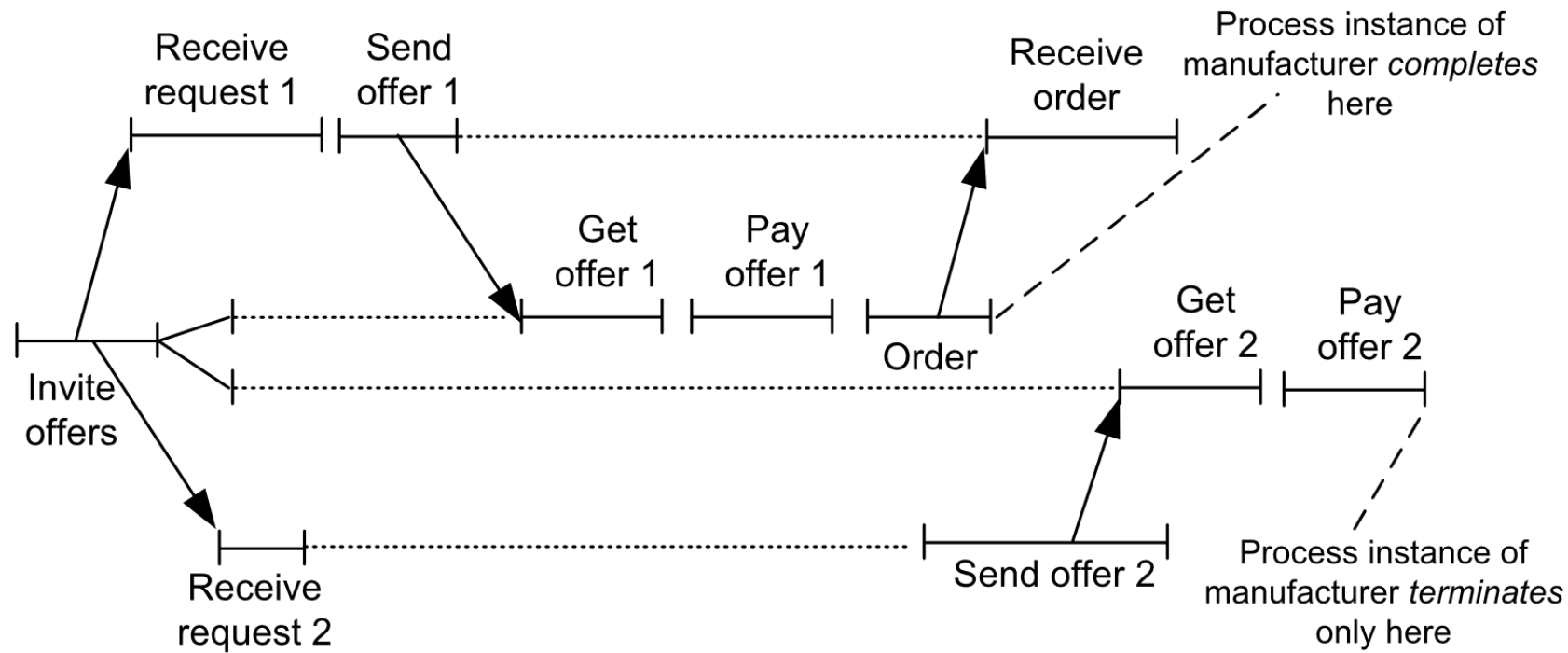
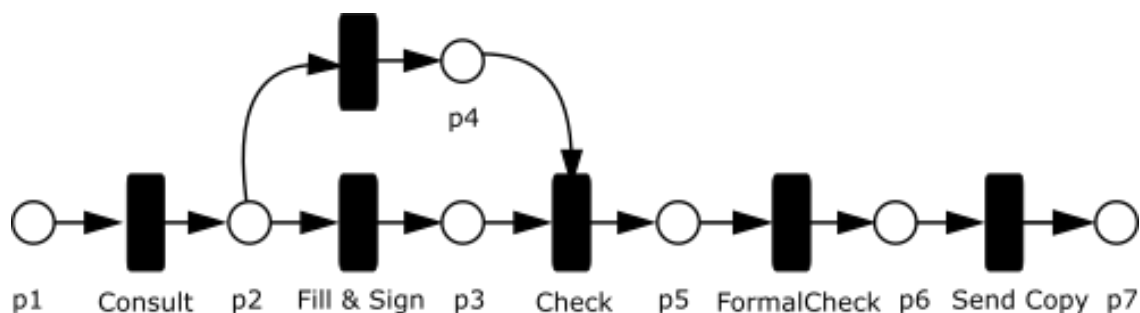


Fig. 7.28. Process instance, illustrating lazy soundness

Tool Support

- CPN-Tools: <https://cpntools.org/>
- Woflan: <http://is.tm.tue.nl/research/woflan/>
- YAWL (Open source)
 - YAWL: Yet Another Workflow Language
 - <https://yawlfoundation.github.io/>
 - Modeling and enactment of workflows based on high-level (Workflow) Nets
 - Omitting places allows for more compact visualization

Verification with Woflan



.tpn Format

```

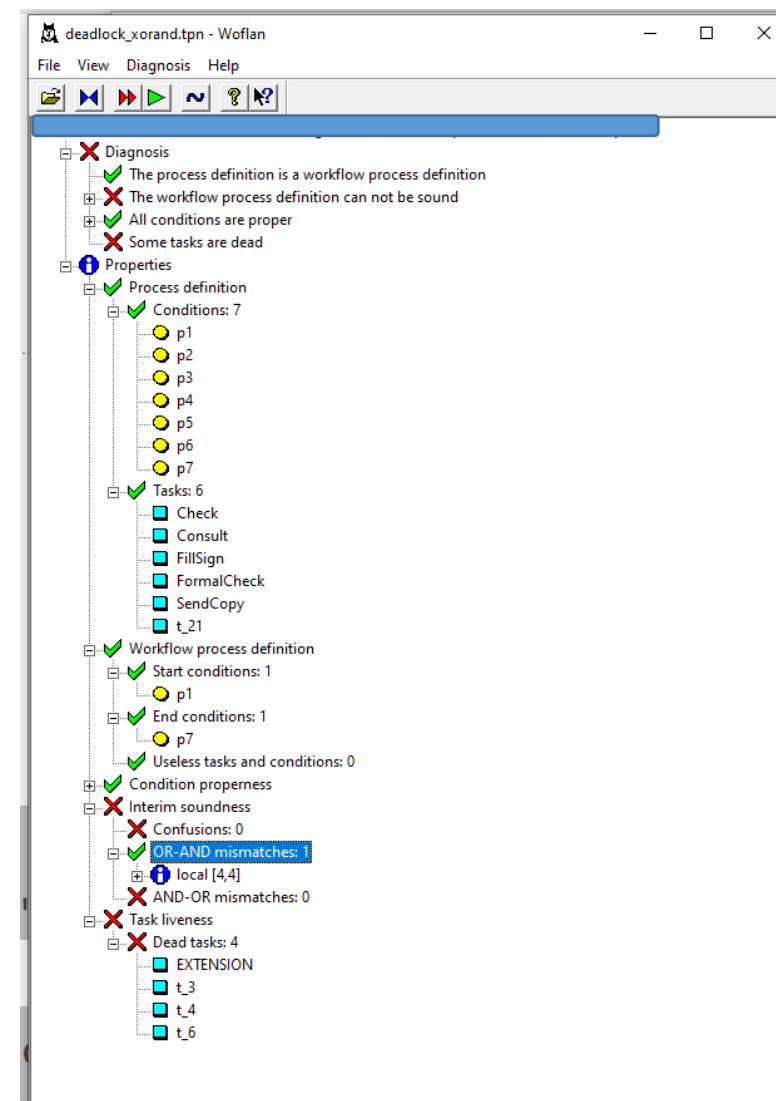
place "p1";
place "p2";
place "p3";
place "p4";
place "p5";
place "p6";
place "p7";
trans "t_1"~"Consult" in "p1" out "p2" ;
trans "t_2"~"FillSign" in "p2" out "p3" ;
trans "t_21" in "p2" out "p4" ;
trans "t_3"~"Check" in "p3" "p4" out "p5" ;
trans "t_4"~"FormalCheck" in "p5" out "p6" ;
trans "t_6"~"SendCopy" in "p6" out "p7" ;
  
```

Verification with Woflan

```

place "p1";
place "p2";
place "p3";
place "p4";
place "p5";
place "p6";
place "p7";
trans "t_1"~"Consult" in "p1" out "p2" ;
trans "t_2"~"FillSign" in "p2" out "p3" ;
trans "t_21" in "p2" out "p4" ;
trans "t_3"~"Check" in "p3" "p4" out "p5" ;
trans "t_4"~"FormalCheck" in "p5" out "p6" ;
trans "t_6"~"SendCopy" in "p6" out "p7" ;

```



Refined Process Structure Trees are a formal construct that helps to (1) verify that a workflow graph is executable, (2) can serve as the basis for executing (interpreting) the graph, or (3) can serve as the basis for easily transforming the graph to an imperative programming language (such as python).

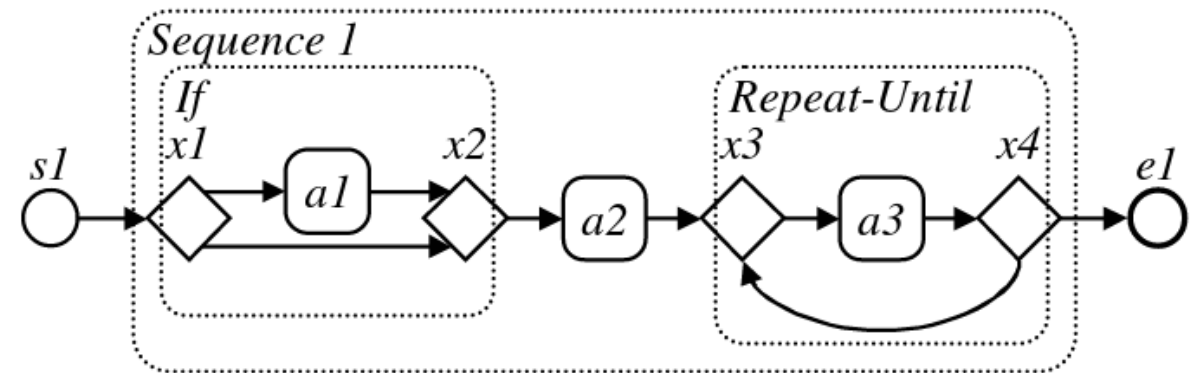
***All graphs and figures that explain the RPST and its concepts
have been taken from the literature below.***

Literature

- Vanhatalo, Jussi, Hagen Völzer, and Jana Koehler. "The refined process structure tree." International Conference on Business Process Management. Springer, Berlin, Heidelberg, 2008.
- Sadiq, M. E. Orlowska, Analyzing process models using graph reduction techniques., Inf. Syst. 25 (2) (2000) 117–134.

RPST

- Workflow Graphs: BPMN, EPKs, ... are saved as nodes (tasks) and edges that connect them.
- Process Execution Engines, like most compilers or virtual machines (e.g. java, gcc, python) internally rely on Abstract Syntax Trees (AST). A formal programming language, e.g. BPMN, is translated to a parse tree for analysis (find syntax errors) and execution.
- RPST is an AST, that is geared towards the rather minimal grammar workflow graphs often represent (many branches, no assignments, very simple conditions).
- RPSTs allow to decompose the graph into logical blocks (cmp. structural sound petri nets) with one single starting point, and one single end point.
- The leaves of the tree are always tasks, branches represent logical constructs such as XOR, OR, PARALLEL, ...



RPST



Workflow graphs - Two Terminal Graphs (TTG) - i.e. structural sound petri nets

- one source node (without incoming edges)
- one sink node (without outgoing edges)
- each node is on a path from source node to sink node

TTG: $G = (V, E, M)$

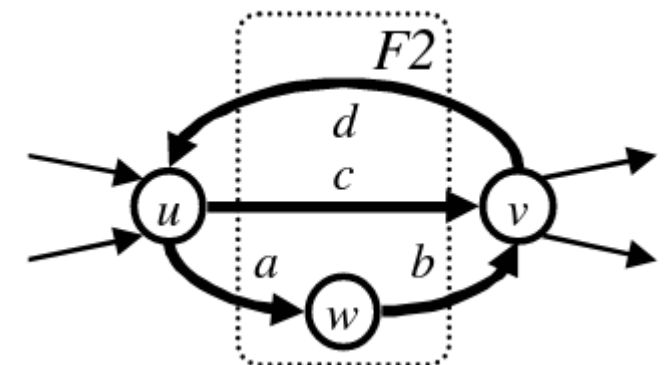
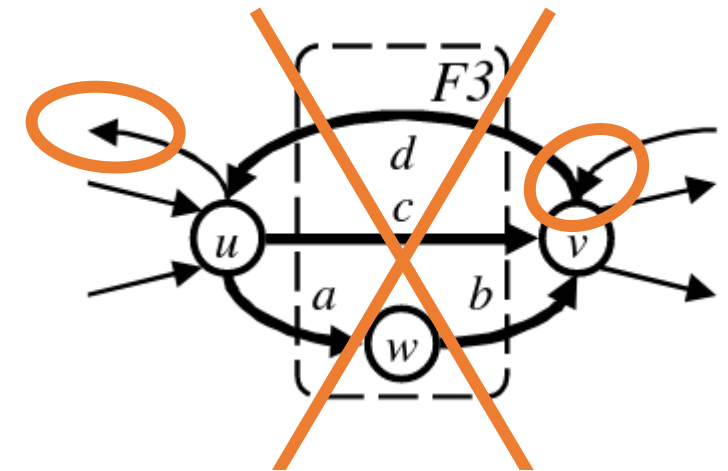
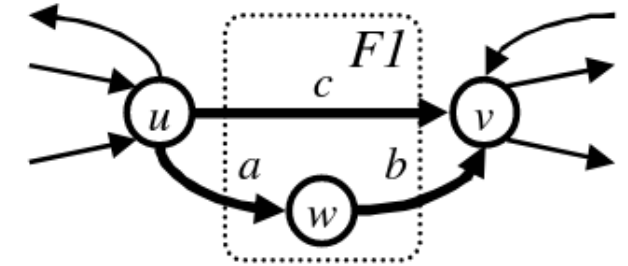
- V is the set of nodes (e.g. u, v, w)
- E the set of edges (e.g. a, b, c)
- M pairs of ordered/unordered nodes connected by edges

F = Fragment - a subset of edges E

- one source/entry boundary node s - all outgoing edges of s are in F **OR** no incoming edges of s are in F
- one sink/exit boundary node t - all incoming edges of t are in F **OR** no outgoing edges of t are in F
- t and s have to be connected through the edges in F .

All edges in F have to be in at least one path from s to t . For example:

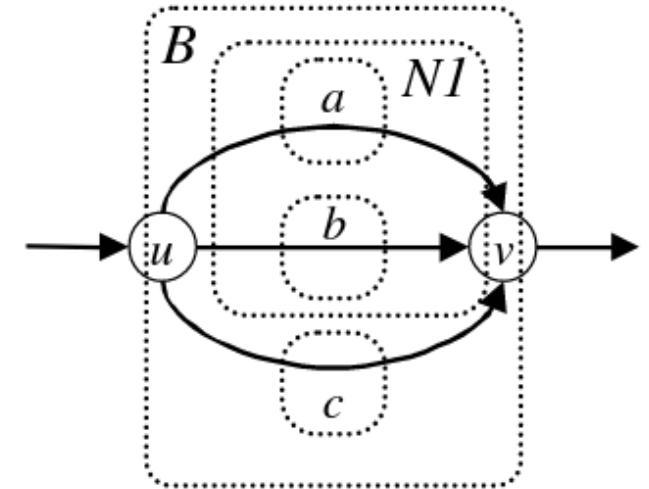
- $F1 = \{a, b, c\}$, boundary nodes u, v
- $F2 = \{a, b, c, d\}$, boundary nodes u, v ; ab is a path from s to t that connects through w .
- $F3 = \{a, b, c, d\}$ - WRONG



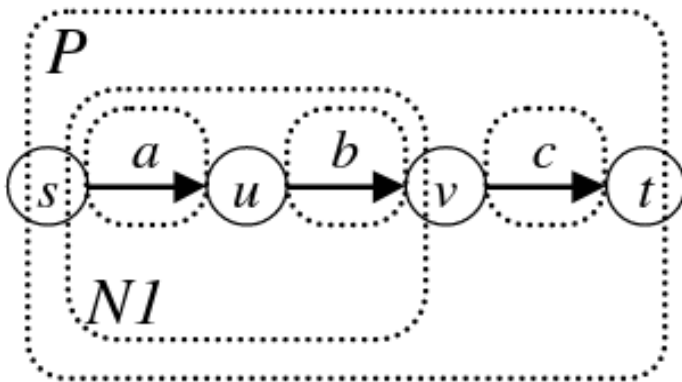
RPST

Decomposition of $G = (V, E, M)$ - build a set of fragments ε

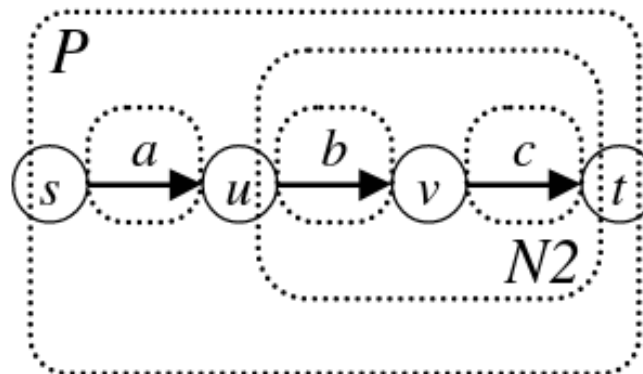
- Rule 1: $E \in \varepsilon$ - each edge is part of the fragment set
- Rule 2: $F1 \cap F2 = \emptyset$ or $F1 \subseteq F2$ or $F2 \subseteq F1$ - fragments are either disjoint or part of each other. They are not allowed to partially overlap.
- Decompositions should be maximal - i.e. Rule 1 must be fulfilled and no more fragments exist that fulfill Rule 2.
- Different decompositions can exist.



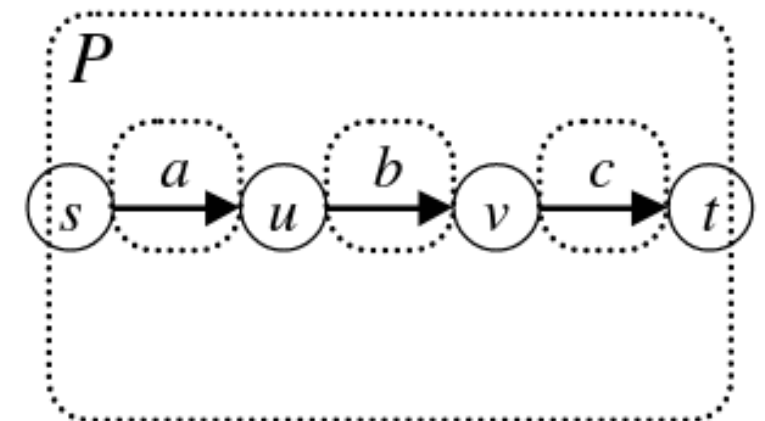
Maximal Decomposition



Maximal Decomposition



Alternative Maximal Decomposition

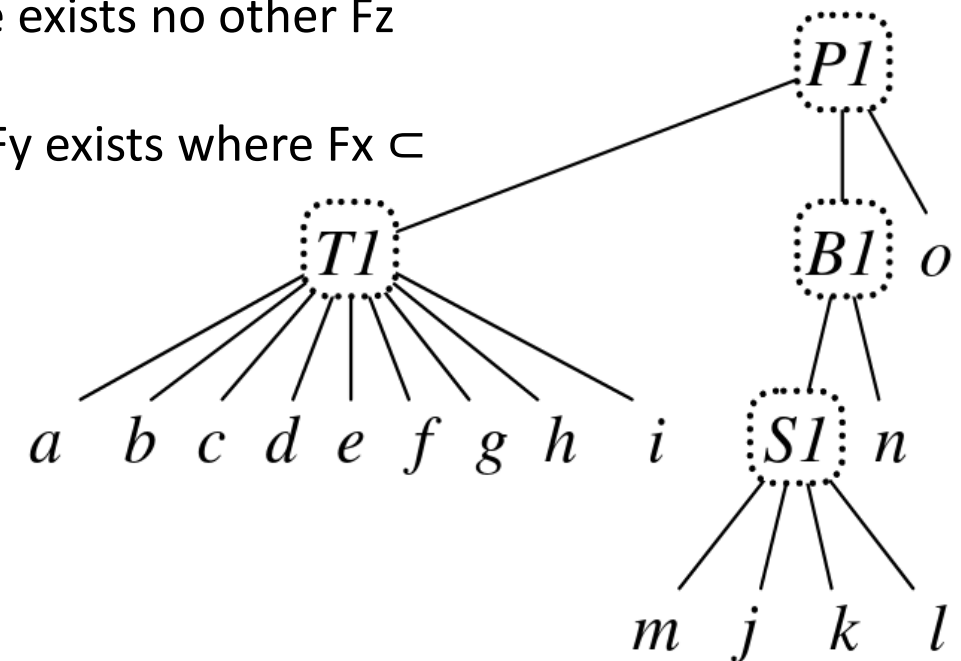
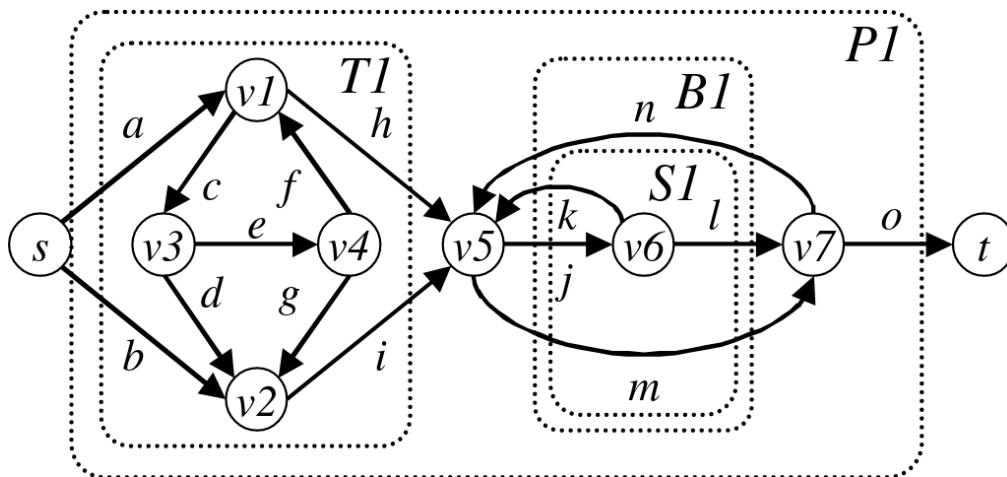


Not-yet-maximal Decomposition

RPST

Final Step - RPST

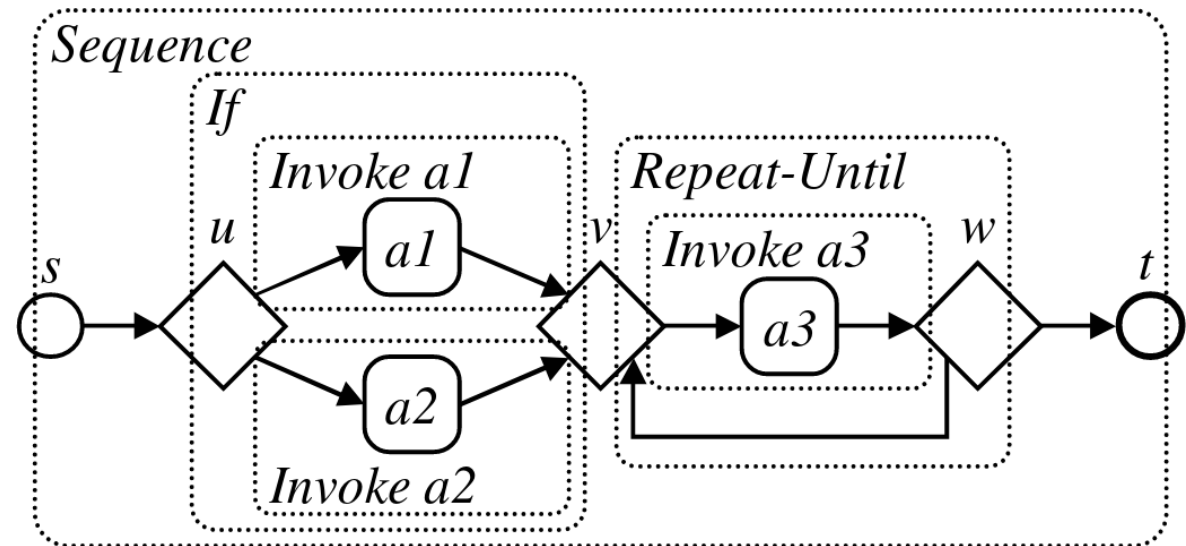
- One maximal decomposition.
- Nodes are leaves.
- Each fragment F_x has a parent F_y if $F_x \subset F_y$ and there exists no other F_z where $|F_z \setminus F_x| < |F_y \setminus F_x|$
- A fragment F_x is the root of the RPST if no fragment F_y exists where $F_x \subset F_y$



RPST



- Application - IBM WebSphere Business Process Suite: support different languages such as BPEL and BPMN. All formats are transformed to an RPST. RPST is used for control-flow analysis (different branches of the tree can be analysed in parallel) and refactoring (find identical branches).
- Application - cpee.org: an XML representation of the RPST is the native format of the BPMN modeler. When a BPMN process model is instantiated, the RPST is translated to executable code, compiled and executed on a target machine.
- Possible Applications: efficient difference and merge algorithms (see Theses page on <https://bpm.in.tum.de>).
- Possible Applications: every technique known from compiler theory, such as data-flow analysis or code optimization.



Further reading

- Please study [Vanhatalo, Jussi, Hagen Völzer, and Jana Koehler 2008] or [Sadiq, M. E. Orłowska 2000] for a linear time algorithm to automatically extract an RPST from a workflow graph.

Summary and Outlook

- Petri Nets provide the basis for many studies on workflow properties
 - Formal foundations
 - Formal semantics
 - Analysis functions
- For workflow verification often it is proposed to map other formalisms to Petri Nets → then use standard Petri Nets analysis tools
- Common workflow patterns are described by using Petri Nets (<http://www.workflowpatterns.com/>)
- However, as mentioned, important aspects are abstracted from
 - Data flow
 - Temporal restrictions
 - Organizational aspects
- Might be overcome by use of high-level Petri Nets

Business Process Technologies and Management

3. Process Implementation and Execution

Prof. Dr. Stefanie Rinderle-Ma

Dr. Jürgen Mangler

Technical University of Munich

Department of Informatics

Chair of Information Systems

and Business Process Management

stefanie.rinderle-ma@tum.de

juergen.mangler@tum.de



Literature and links



- Weske, M. (2019) Business Process Management - Concepts, Languages, Architectures, Third Edition. Springer 2019, ISBN 978-3-662-59431-5, pp. 1-417 2019
- Marlon Dumas, Marcello La Rosa, Jan Mendling, Hajo A. Reijers: Fundamentals of Business Process Management, Second Edition. Springer 2018, ISBN 978-3-662-56508-7, pp. 1-527
- <https://www.signavio.com/bpm-academic-initiative/>
- <https://cpee.org/>

Teaching Objectives

In this chapter we want to

- introduce the notion of execution semantics
- discuss process execution languages
- show the different aspects/perspectives to be considered to develop and implement executable processes and process-oriented applications
- give an overview of both buildtime and runtime components of a process management system (org modeler, work list, etc)
- discuss selected implementation issues related to process management systems

Developing Executable Processes...



... requires the definition and specification of the following aspect (selection):

- ➔ • Control Flow (Behavioral Aspect)
- Data Flow (Information Aspect)
- Activities (Functional Aspect)
- Deadlines (Temporal Aspect)
- Actor Assignments (Organizational Aspect)
- Application Services (Operational Aspect)

Short recap

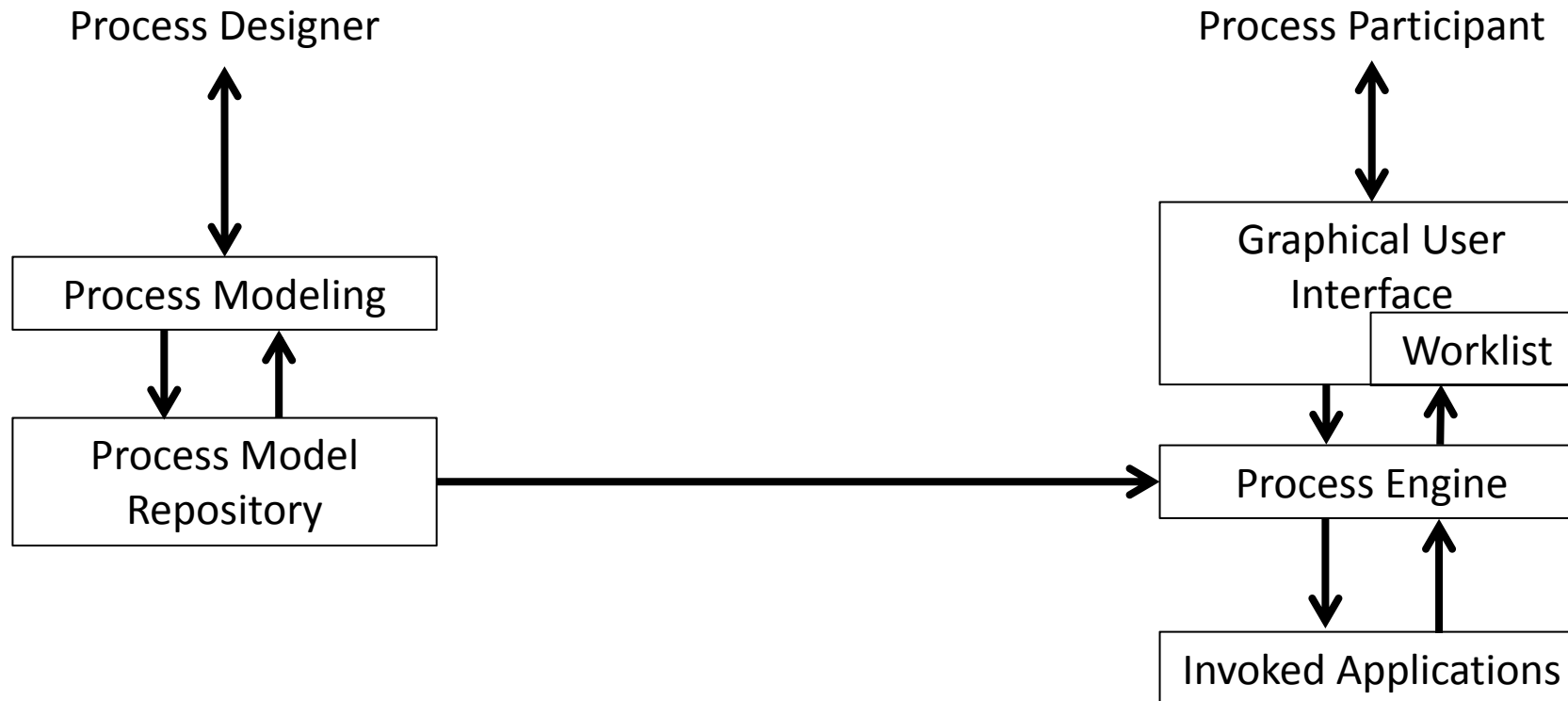
- *So far:*
 - Process modeling (on a semantically high level)
 - Process verification
- *Now:*
 - Process modeling in an execution language
 - Concepts for organizational modeling
 - Concepts for application invocation and integration

Process Model / Schema



- Formal representation of (parts of) a business process → Enacted by a process management system or process engine
- Comprises a set of process steps – **tasks** – which are logically related in terms of their contribution to the overall business process
- Describes the **control flow** as well as the **data flow** between activities
- May contain **references** to other model components like sub-processes, organisation model, resource model, and so forth
- Based on a **process modeling language** (= **process meta model**) like Workflow Nets, WSM Nets, or CPEE Trees

Architectural considerations



adapted from M. Weske: Business Process Management.
Springer 2019

Aspects of Process Execution Modeling



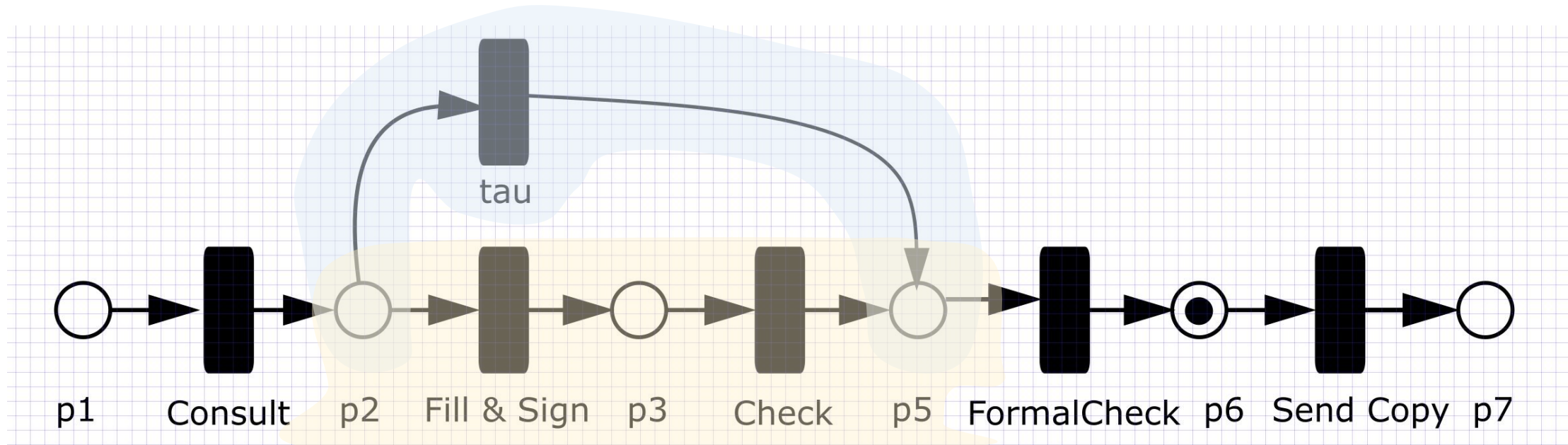
- Formal and operational semantics
- Graphical modeling model-driven coding
- Ease of use
- Expressiveness
- Tools support
- Standard support
- Extensibility

Workflow Nets



- Basic definition of Workflow Nets in Chapter 2
- Execution semantics is precisely defined by marking and firing rules
- Workflow Nets with colored tokens distinguishable process instances
- Modeling control flow
- Execution in YAWL <https://yawlfoundation.github.io/>

A closer look at execution semantics



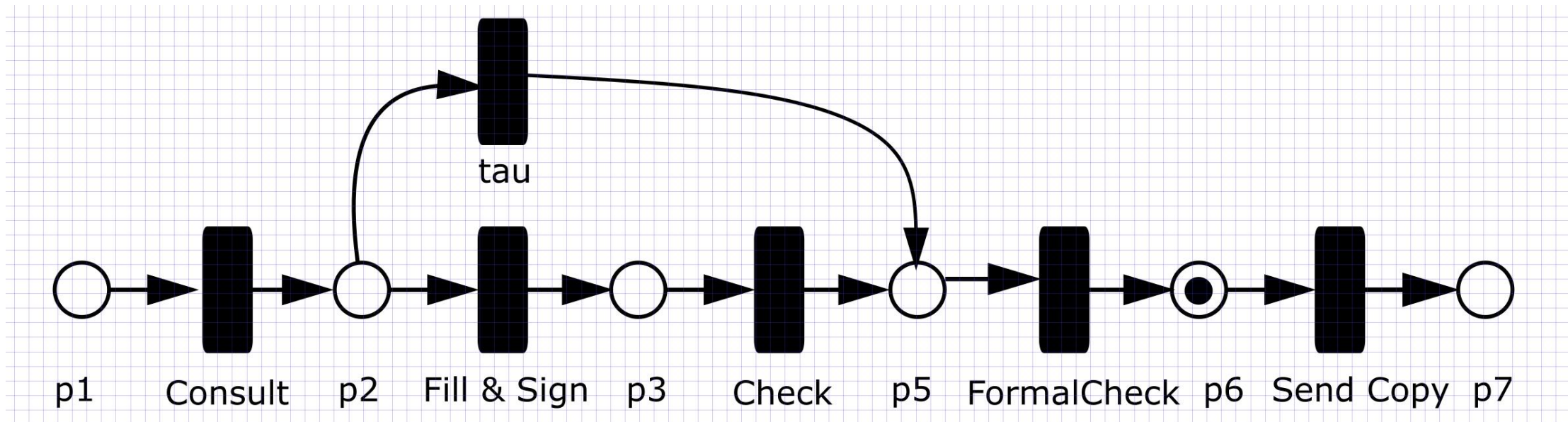
- If you look at the current state, how did we arrive at this state?
- In particular, which of the paths in the alternative branching were chosen?

TRUE Semantics



- Petri Nets and Workflow Nets (without any further information) offer a TRUE semantics
- TRUE semantics mean that we can only see the current state and the paths that have been definitely taken (TRUE), but not the paths that have not been taken (FALSE, SKIPPED)
- However, the information about skipped paths can be interesting for analysis, for example, for decision mining/analysis.
- Thus, a first approach here is to equip models with TRUE semantics with additional process history/log information.

TRUE Semantics with Process Log



trace1 = <Consult, Fill&Sign, Check, FormalCheck, SendCopy>

trace2 = <Consult, FormalCheck, SendCopy>

TRUE/FALSE Semantics



- There are also process execution languages with TRUE/FALSE semantics.
- They offer execution markings including markings for SKIPPED tasks and edges that are marked with FALSE.
- In the following:
 - Well-Structured Marking Nets (WSM Nets)
 - Cloud Process Execution Engine Trees (CPEE Trees)
- READ: [RRD04] S. Rinderle, M. Reichert, P. Dadam: Correctness criteria for dynamic changes in workflow systems - a survey. Data Knowl. Eng. 50(1): 9-34 (2004). DOI: 10.1016/j.datak.2004.01.002

Well-Structured Marking Nets



Definition 4.1 (Control Flow Schema (WSM-Net)). A tuple $S = (N, D, NT, CtrlE, SyncE, LoopE, EC)$ is called a control flow schema if the following holds:

- N is a set of activities and D a set of process data elements
- $NT: N \rightarrow \{\text{StartFlow, EndFlow, Activity, AndSplit, AndJoin, XOrSplit, XOrJoin, StartLoop, EndLoop}\}$
 NT assigns to each node of the WSM-Net a respective node type.
- $CtrlE \subseteq N \times N$ is a precedence relation representing “normal” control dependencies between sequential activities
- $SyncE \subseteq N \times N$ is a precedence relation between activities of parallel branches
- $LoopE \subseteq N \times N$ is a set of loop backward edges
- $EC: CtrlE \rightarrow \text{EdgeCode} \cup \{\text{UNDEFINED}\}$, i.e., $EC(e)$ assigns a selection code to the outgoing control edges of an XOR-Split. A global process data element indicates the branch to be selected.

Definition taken from [RRD04a] S. Rinderle, M. Reichert, P. Dadam: Flexible Support of Team Processes by Adaptive Workflow Systems. Distributed Parallel Databases 16(1): 91-116 (2004).
DOI: 10.1023/B:DAPD.0000026270.78463.77

Well-Structured Marking Nets



- Control flow modeling is based on a block-structured and graph-based approach
- Concept of regular block structuring:
 - Control structures (sequence, branchings, loops, ...) are mapped onto blocks with unique start and end activities (nodes)
 - Blocks must not overlap, but can be arbitrarily nested
 - Leads to „symmetric“ structuring of control flow graphs since splits and joins are done at specific nodes and in a pair-wise manner

Well-Structured Marking Nets



Structural correctness rules (from [RRD04a])

- $S_{\text{fwd}} = (N, \text{CtrlE}, \text{SyncE})$ is an acyclic graph, i.e., the use of control and sync edges must not cause undesired cycles leading to deadlocks
- For each split (loop start) node there is a unique join (loop end) node, and
- S is structured following a **block concept**; control blocks (sequences, branchings, loops) can be nested but must not overlap.

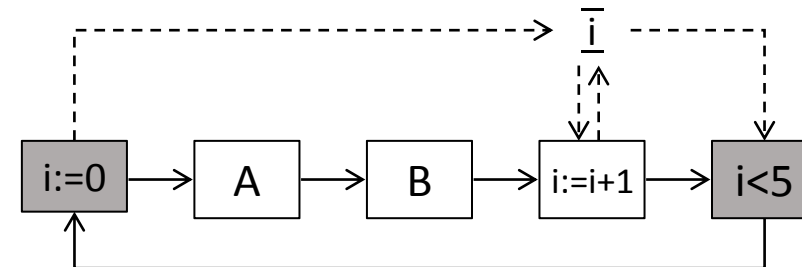
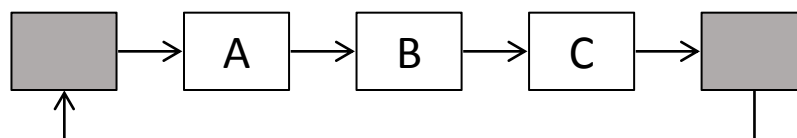
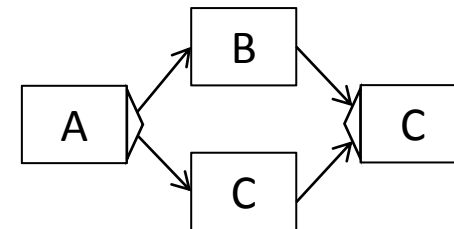
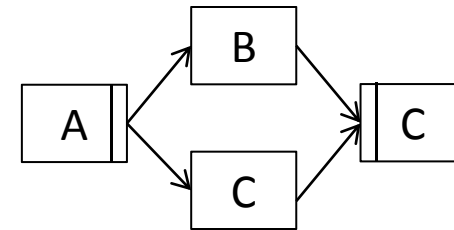
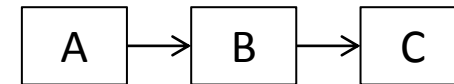
Block structure

Following a (regular) block structure bears several advantages

- Better user support in creating, analyzing, and changing workflow models
- Particularly: modeling is less error-prone
- Efficient correctness checks (control flow, data flow, dynamic behavior)
- Certain dynamic properties can be guaranteed „by construction“
- Support by syntax-driven editors
- Change operations can be implemented more easily
- Precise criteria for instance behavior (e.g., when is an instance finished)
- However: expressiveness of process models is restricted for strict block structuring → extensions

Well-Structured Marking Nets

- Sequence
- AND-Split / AND-Join
- XOR-Split / XOR-Join
- Loops



Well-Structured Marking Nets



Definition 4.2 (Process Instance Based on a WSM-Net - control flow perspective). A process instance I is defined by a tuple $(S, M^S, Val^S, \mathcal{H})$ where

- $S = (N, D, NT, CtrlE, SyncE, \dots)$ denotes the process model the execution of I is based on.
- $MS = (NS^S, ES^S)$ describes node and edge markings of I with:
 - $NS^S: N \rightarrow \{\text{NotActivated}, \text{Activated}, \text{Running}, \text{Completed}, \text{Skipped}\}$
 - $ES^S: (CtrlE \cup SyncE \cup LoopE) \rightarrow \{\text{NotSignaled}, \text{TrueSignaled}, \text{FalseSignaled}\}$
- Val^S is a function on D . It reflects for each data element $d \in D$ either its current value or the value UNDEFINED (if d has not been written yet).
- $\mathcal{H} = e_0, \dots, e_k$ is the execution history of I whereby $e_0, \dots, e_k$ denote the start and end events of activity executions. For each started activity X the values of data elements read by X and for each completed activity Y the values of data elements written by Y are logged.

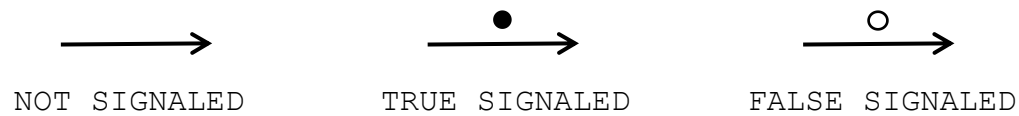
Well-Structured Marking Nets

Operational Semantics:

- Node markings (selection):



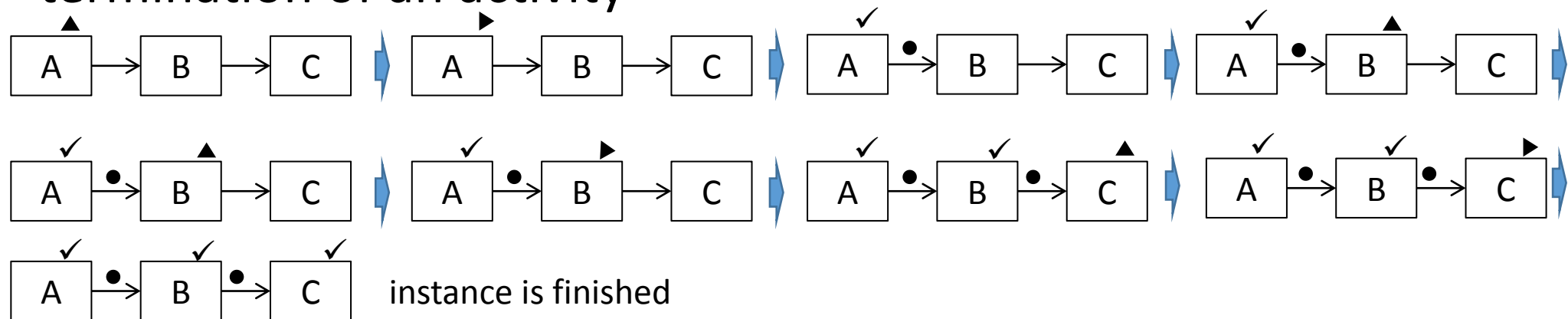
- Edge markings:



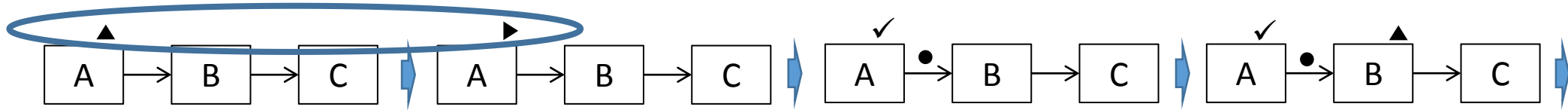
Well-Structured Marking Nets

Control flow is based on a set of well-defined execution and marking rules (cf. firing and marking rules for Petri Nets):

- **Execution rules** define under which graph markings an activity can be activated
- **Marking rules** define which node and edge markings result after termination of an activity



Well-Structured Marking Nets



- What can we see here in comparison to executing Petri Nets?
- Distinction between states `ACTIVATED` and `RUNNING`
- This means that we can express that an activity is doing something that takes some time.
- Without time extensions, transitions in Petri Nets just fire.

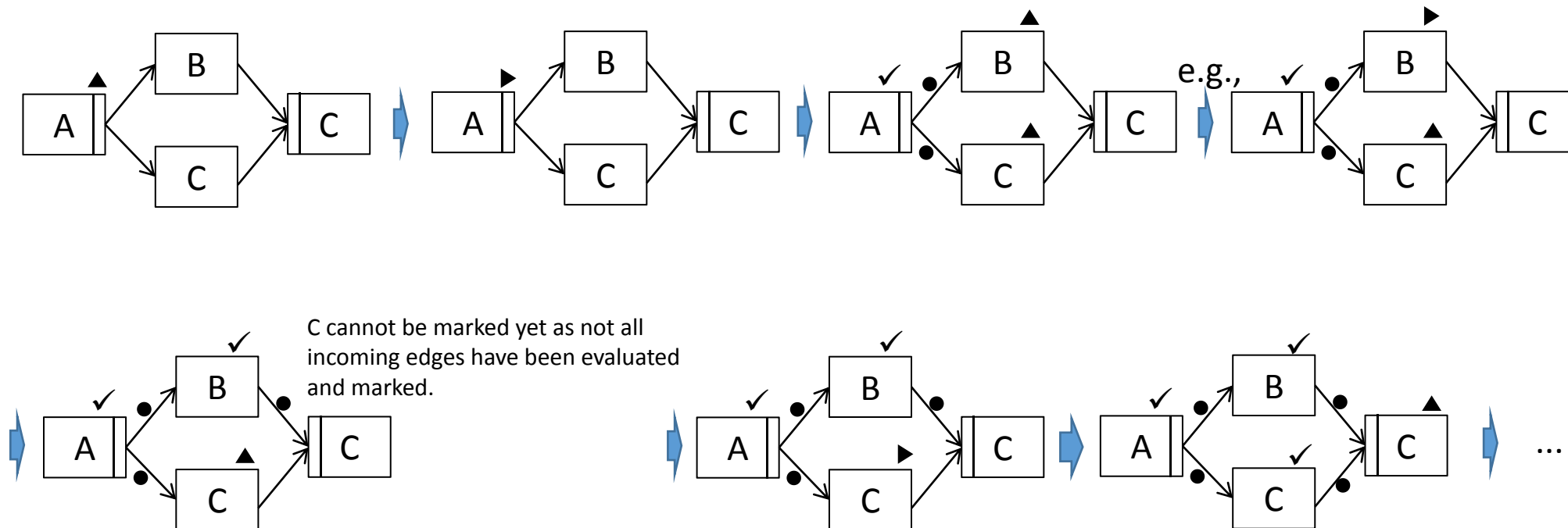
Well-Structured Marking Nets



- Marking and execution rules on parallel branches
- After completing an AND-Split, all outgoing edges are marked as `TRUE_SINGALED` and their target activities are marked as `ACTIVATED`.
- The activities in parallel branches can be then executed in arbitrary order (but have to follow the order within the branches).
- The AND-Join can only be `ACTIVATED` if all incoming edges have been evaluated and are marked as `TRUE_SINGALED`.

Well-Structured Marking Nets

- Marking and execution rules on parallel branches based on example



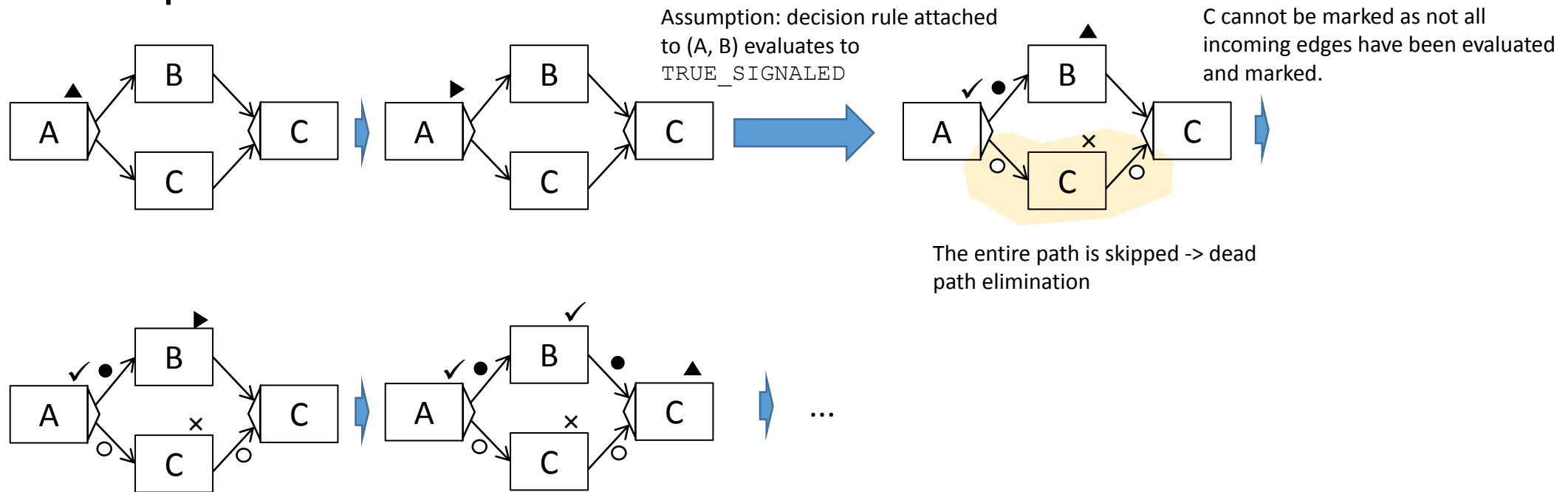
Well-Structured Marking Nets



- Marking and execution rules on alternative branches
- After completing the XOR-Split - based on a decision rule - it is determined which outgoing edge is marked as `TRUE_SINGALED` (one) and all the other outgoing edges are marked as `FALSE_SINGALED`.
- All activities with incoming edges that are marked as `FALSE` are marked as `SKIPPED`.
- If an activity is marked as `SKIPPED` all outgoing edges can be marked as `FALSE_SINGALED` again.
- Iteratively done until XOR-Join is reached dead path elimination.
- The XOR-Join is marked as `ACTIVATED` if all incoming edges are marked and one of them is marked as `TRUE_SINGALED` and all other are marked as `FALSE_SINGALED`.

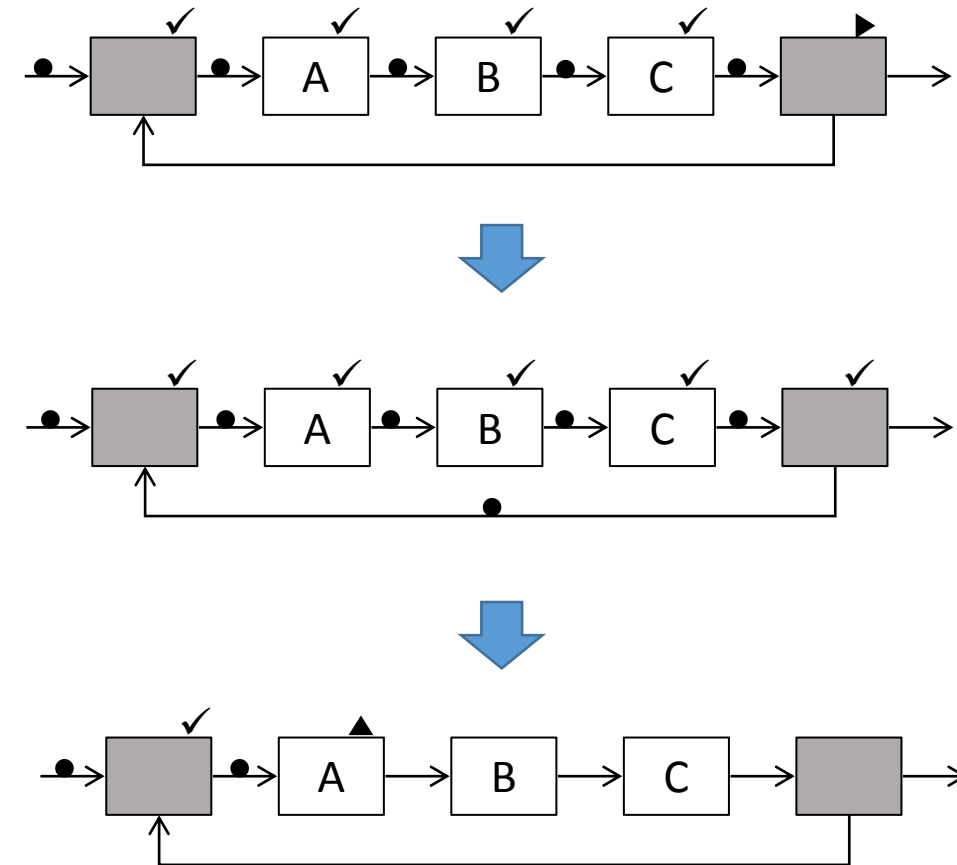
Well-Structured Marking Nets

- Marking and execution rules on alternative branches based on example



Well-Structured Marking Nets

- Marking and execution rules on alternative branches
- When the loop end node is reached, it is directly `COMPLETED` and the loop condition is evaluated.
- If the loop back edge is evaluated as `TRUE SIGNALLED`, the loop start node is directly marked as `COMPLETED`.
- All other activities in the loop are marked as `NOT_ACTIVATED`, except for the direct predecessor of the loop start activity. This node is marked as `ACTIVATED`.



Well-Structured Marking Nets

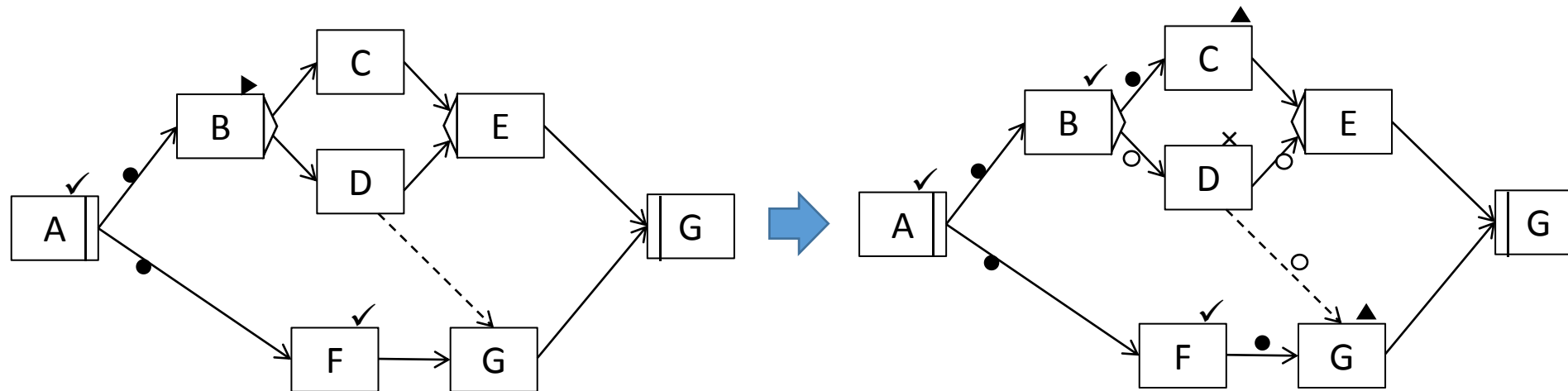


Strict block structuring is perceived as too restrictive.

- Therefore, synchronization edges are used:
- Construct for synchronizing the execution of activities in different parallel branches
- A (soft) sync edge $X \rightarrow Y$ describes a delay between parallel activities X and Y
- Semantics: Y can be activated only if X has been successfully finished or if X cannot be executed anymore (e.g., if X has been skipped)

Well-Structured Marking Nets

Marking and execution with synchronization edges, example:



Well-Structured Marking Nets

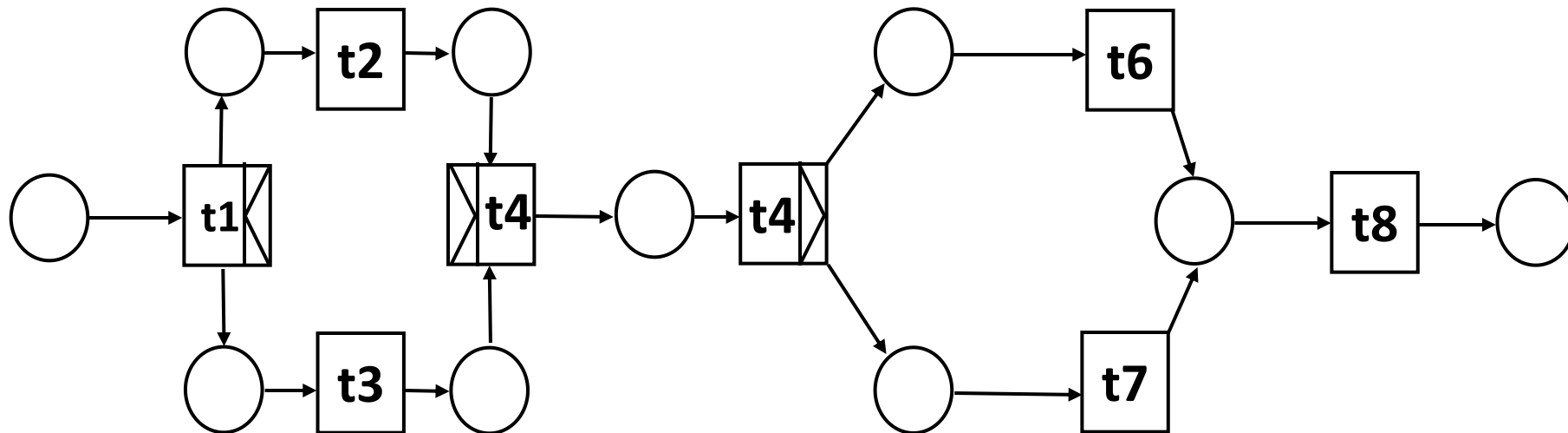


Based on the marking and execution rules we can prove that

- For each activity of a process instance marked as `COMPLETED` or `SKIPPED` holds that each of its predecessor nodes is marked as either `COMPLETED` or `SKIPPED`.
- Under each of the reachable end markings of a workflow instance, all nodes are marked as either `COMPLETED` or `SKIPPED`
- There will be no deadlocks at runtime if it is guaranteed that the using sync edges does not produce a cycle.
- Remark: Under certain fairness assumptions (e.g., excluding infinite loops) we can conclude that all workflows terminate correctly.

Well-Structured Marking Nets

- Transformation between different formalisms is often essential.
- For example, to transform for verification and execution.
- Exercise: transform the following Workflow Net into as WSM Net



Execution on BPMN

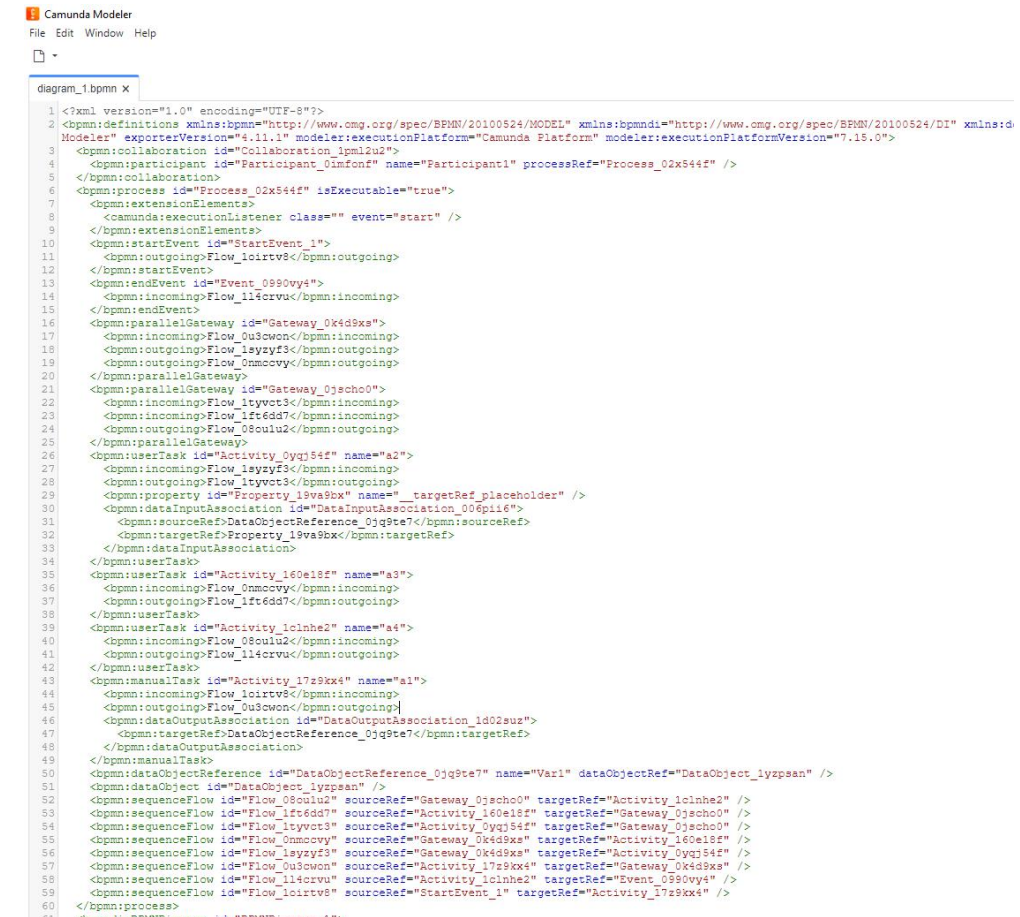
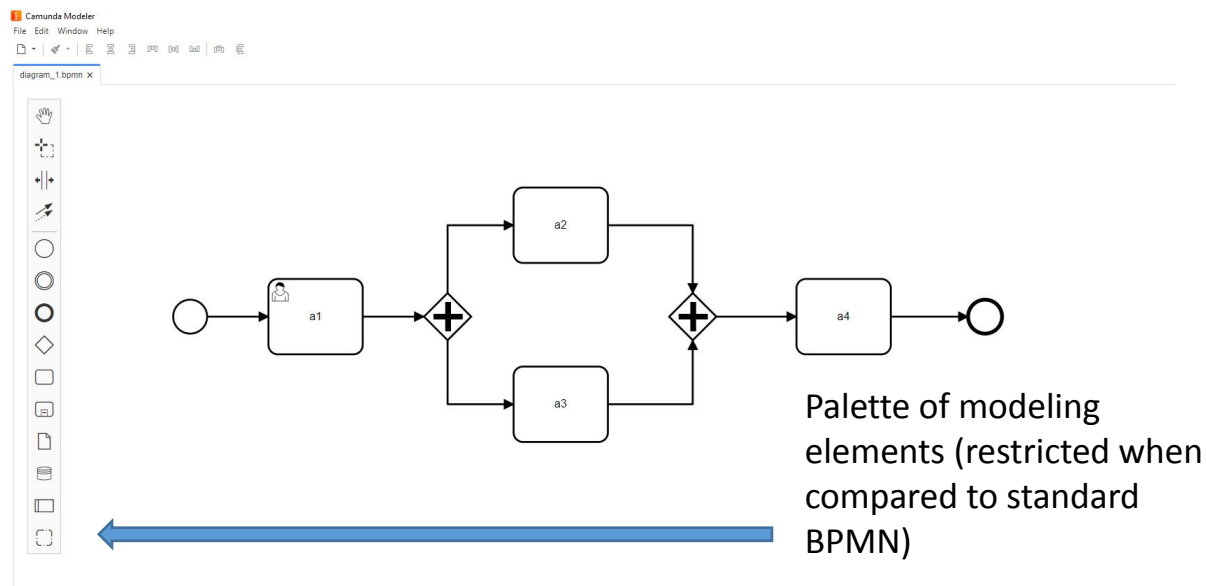


BPMN can be used for

- defining executable models under strict modeling guidelines/restrictions and refinement of data flow, authorization, and invocation of web services/applications.
- defining conceptual models that can be mapped to executable models in a straightforward way, again by employing modeling guidelines and of data flow, authorization, and invocation of web services/applications.

Execution in Camunda

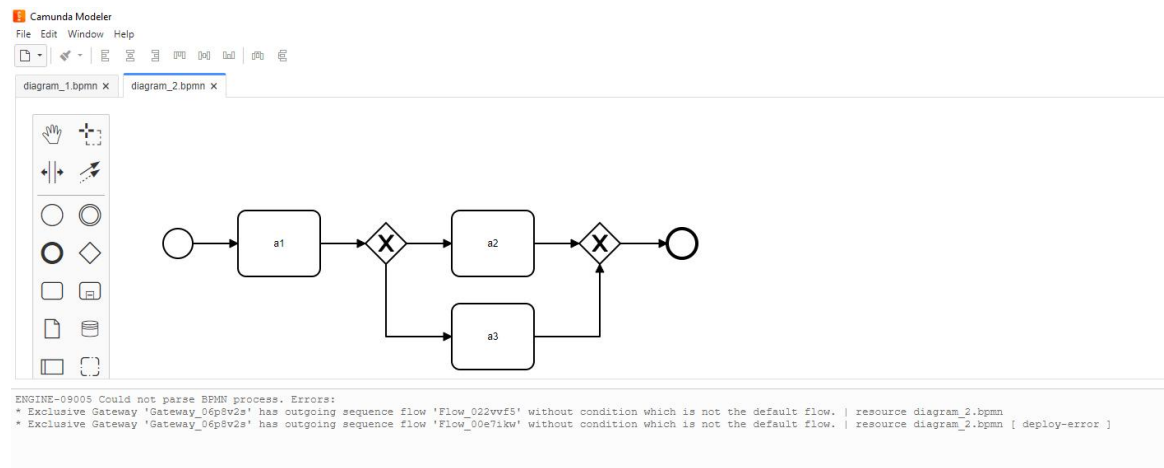
- <https://camunda.com/>
- commercial and open source
- Will be used in the exercises.



The image shows the Camunda Modeler interface with the XML code for the BPMN diagram displayed on the right. The code is a valid BPMN 2.0 XML snippet. It defines a process with a start event, an exclusive gateway, two parallel tasks (a2 and a3), another exclusive gateway, and a final task (a4) leading to an end event. The code includes various BPMN elements like `<startEvent id="StartEvent_1">`, `<exclusiveGateway id="Gateway_0k4d9xs">`, `<task id="Activity_0yq354f" name="a2">`, `<task id="Activity_1c0lnhe2" name="a4">`, and `<endEvent id="Event_0990vy4">`. The code is wrapped in a `<process id="Process_02x544f" isExecutable="true">` tag.

Execution in Camunda

- Modeling rules to guide users and correctness checks before deployment to the engine
- When starting exactly one start event exists.
- Before deployment several checks are conducted.



CPEE Trees



CPEE (Cloud Process Execution Engine - cpee.org) Trees are a RPST (Refined Process Structure Tree) inspired implementation for storing process models.

Properties:

- XML: a small set of elements and their potential children (XML NS <http://cpee.org/ns/description/1.0>).
- No edges. The edges are implicit, i.e. derived from the structure of the tree.
- Deleting nodes does not require adjustment of edges (see: no edges).
- Inserting nodes does not require insertion of edges (see: no edges).
- As long as the tree XML is correct and adheres to an xml schema, no structural deadlocks are possible (of course there might be hidden dependencies in the individual implementation of task, which might deadlock parallel branches).

Advantages:

- Easy to parse in comparison to Workflow Graphs.
- Deletion and Insertion are trivial.
- Transformation to different formats (including workflow graphs, programming languages such as python, ...) is trivial (cmp. how difficult the parsing of workflow graphs and transforming them into an RPST is).
- Good basis for analysis and execution.
- Enforces syntactical correctness of the process model (no structural deadlocks).

CPEE Trees - Elements 1



ROOT <description xmlns="http://cpee.org/ns/description/1.0"/>

task <call id="a1" endpoint="..." />

call to functionality, has input arguments, potentially modifies the instance context (data). A task might delegate work to humans, hardware, or arbitrary software - as long as they accept the input, and generate the expected output. This element can have no children.

parallel <parallel wait="-1" cancel="last" />

The children of this element will be evaluated. The result of this evaluation determines the number of parallel branches. The mode determines the join semantic: **-1** means that all parallel branches have to be finished. Any number **bigger than 0** means that after this number of branches (or all), all remaining branches will be canceled. The cancel attribute further refines the join semantic: if wait is 1 and cancel is **last**, then after 1 branch ended, all other branches are canceled, no matter at which task they currently are. If wait is 1 and cancel is **first**, after the first task of any branch finishes, all other branches are canceled, and the original branch continues normally. This is the equivalent of an event-based gateway.

CPEE Trees - Elements 2



loop *<loop mode="pre_test" condition="..." />*

The children of this element will be repeated over and over. The mode determines if the condition is evaluated before or after the first iteration. The loop continues until the condition is false.

- **escape** *<escape />*
jump out of the loop if execution is inside a loop, even if the loop is not the parent. If no ancestor is a loop, the escape is ignored.

choose *<choose mode="exclusive" />*

No children except the following two are allowed. The mode allows for differentiation between exclusive and inclusive or. Important note: the order of the children determines the order of the evaluation.

- **alternative** *<alternative condition="..." />*
if the condition evaluates to true, an alternatives' children are traversed.
- **otherwise** *<alternative condition="..." />*
*if no alternative has been found the children of the otherwise element are traversed. If no otherwise can be found, an empty otherwise is assumed. The otherwise does not have to be the last child of **choose** to work properly.*

CPEE Trees - Elements 3



script task `<manipulate id="a2" label="...">...</manipulate>`

can hold arbitrary code, it is up to execution to make use of this code. The code may modify instance context (data).

terminate `<terminate/>`

immediately terminates execution if encountered. Is not intended to have children.

stop `<stop id="a2"/>`

is intended to pause the execution of an instance. The difference between terminate and stop is, that termination is final - an instance is finished after execution. A stopped instance might continue executing at exactly the position it stopped. Is not intended to have children.

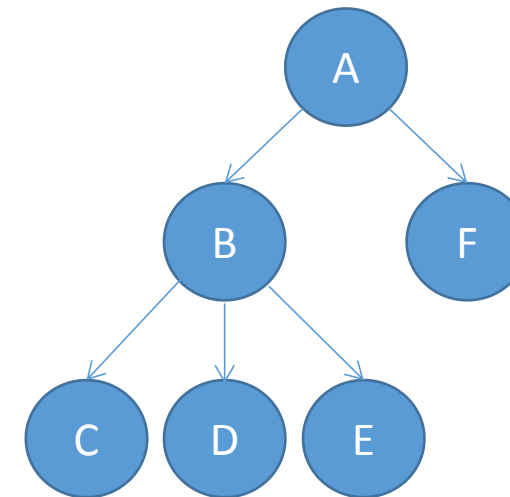
Generic note - only elements where a potential execution can stop have an id:

- task
- script task
- stop

All other elements can not be interrupted during execution.

CPEE Trees - CPEE Trees are ...

- Ordered Trees: the order of elements in each node is important.
- Depth first traversed (see exception: parallel branch).
- Pre-order traversed (ABCDEF).
- Leaves are either (if any other element is a leaf, it can be ignored):
 - Task
 - Script
 - Terminate
 - Stop
 - Escape
- Leaves have no other elements as children.
- Branches are either
 - Choose - Alternative and Otherwise are allowed children
 - Alternative - all elements are allowed children
 - Otherwise - all elements are allowed children
 - Parallel - all elements are allowed children (use only Parallel Branches for standard BPMN syntax)
 - Parallel Branch - all elements are allowed children



CPEE Trees - Traversal - Sequence

depth-first, pre-order traversal; only one possible traversal:

- ABCD

order of elements B, C, D is important and influences the execution

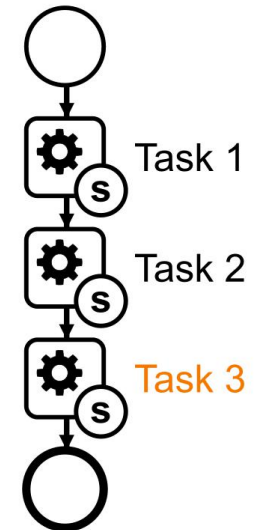
A `<description xmlns="http://cpee.org/ns/description/1.0">`

B `<call id="a1" endpoint="" />`

C `<call id="a2" endpoint="" />`

D `<call id="a3" endpoint="" />`

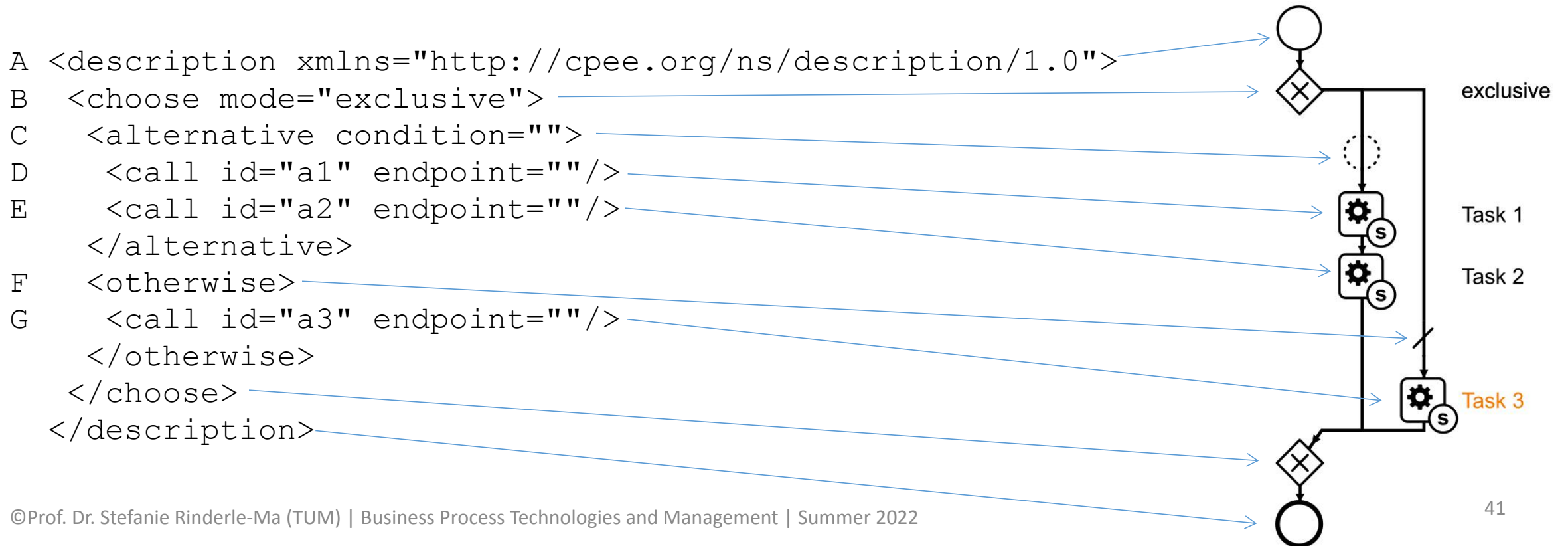
`</description>`



CPEE Trees - XOR (BPMN Exclusive Gateway)

depth-first, pre-order traversal; potential traversals:

- ABCDE
- ABCFG



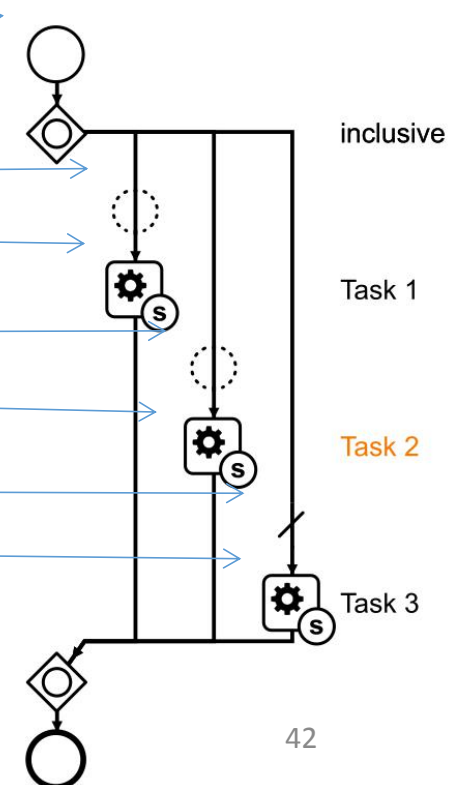
CPEE Trees - OR (BPMN Inclusive Gateway)



depth-first, pre-order traversal; potential traversals:

- ABCDE, ABCEF, ABCEGH only one branch or otherwise
- ABCDEF multiple branches

```
A <description xmlns="http://cpee.org/ns/description/1.0">
B   <choose mode="inclusive">
C     <alternative condition="">
D       <call id="a1" endpoint=""/>
E     </alternative>
F     <alternative condition="">
G       <call id="a2" endpoint=""/>
H     </alternative>
I   <otherwise>
J     <call id="a3" endpoint=""/>
K   </otherwise>
L </choose>
M </description>
```

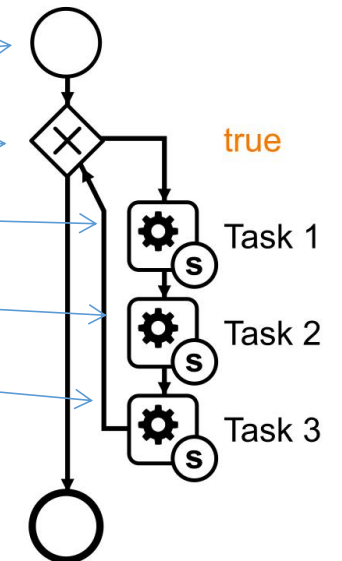


CPEE Trees - Pre-Test Loop

depth-first, pre-order traversal; potential traversals:

- A(BCDE)*B (condition from B is tested before each CDE)
- e.g. for 3 iterations: ABCDEBCDEBCDEB
- * means 0..n

```
A <description xmlns="http://cpee.org/ns/description/1.0">  
B   <loop mode="pre_test" condition="true">  
C     <call id="a1" endpoint="" />  
D     <call id="a2" endpoint="" />  
E     <call id="a3" endpoint="" />  
     </loop>  
  </description>
```

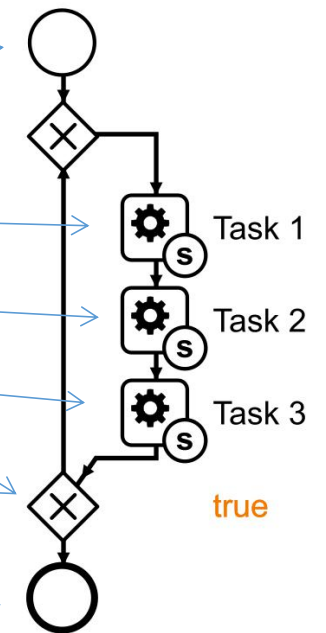


CPEE Trees - Post-Test Loop

depth-first, pre-order traversal; potential traversals:

- AB(CDEB)+ (condition from B is tested after each CDE)
- + means 1..n
- The grey B occurs, but is not tested

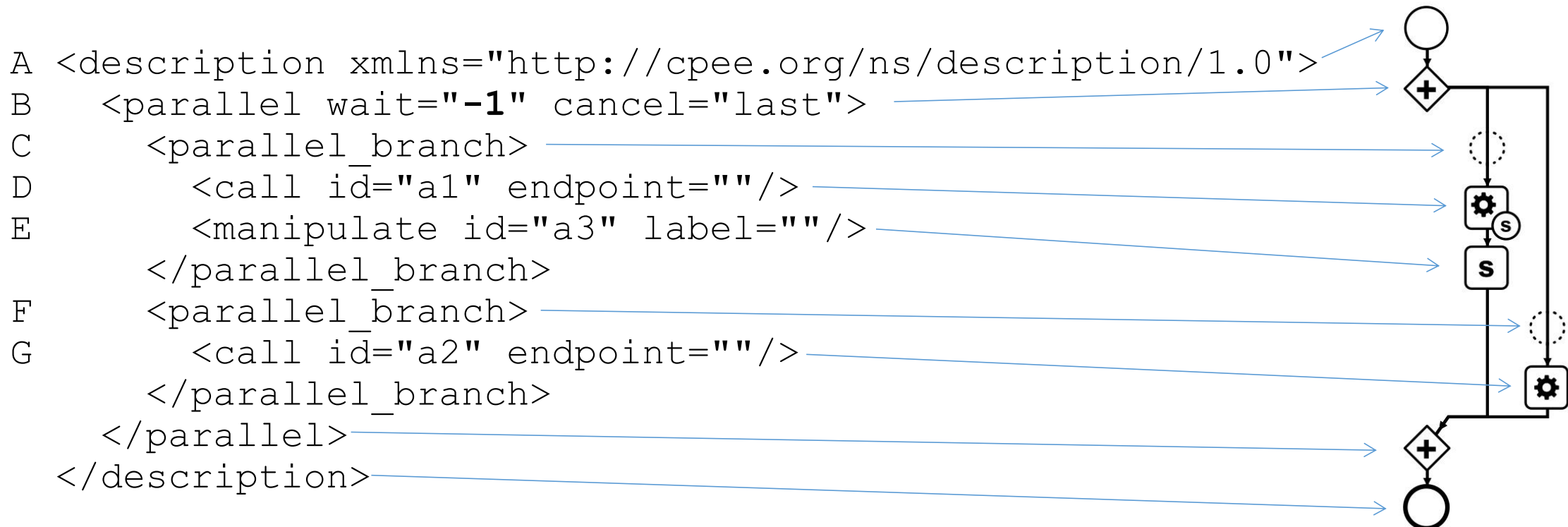
```
A <description xmlns="http://cpee.org/ns/description/1.0">  
B   <loop mode="post_test" condition="true">  
C     <call id="a1" endpoint="" />  
D     <call id="a2" endpoint="" />  
E     <call id="a3" endpoint="" />  
    </loop>  
</description>
```



CPEE Trees - Parallel

depth-first + breadth, pre-order traversal; potential traversals:

- ABCFDEG
- ABCFDGE
- ABCFGDE

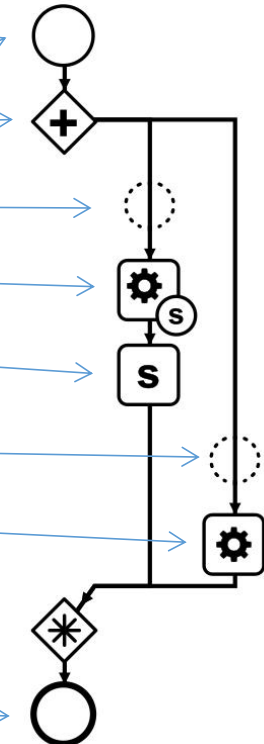


CPEE Trees - Parallel - Special Join

depth-first + breadth, pre-order traversal; potential traversals:

- ABCFDEG, ABCFDGE, ABCFGD
- ABCFDE, ABCFDG, ABCFG (the G,E,D have not yet been started)
- tasks may have irreversible consequences (i.e. sending an email); for ABCFG, ignoring E can be problematic.

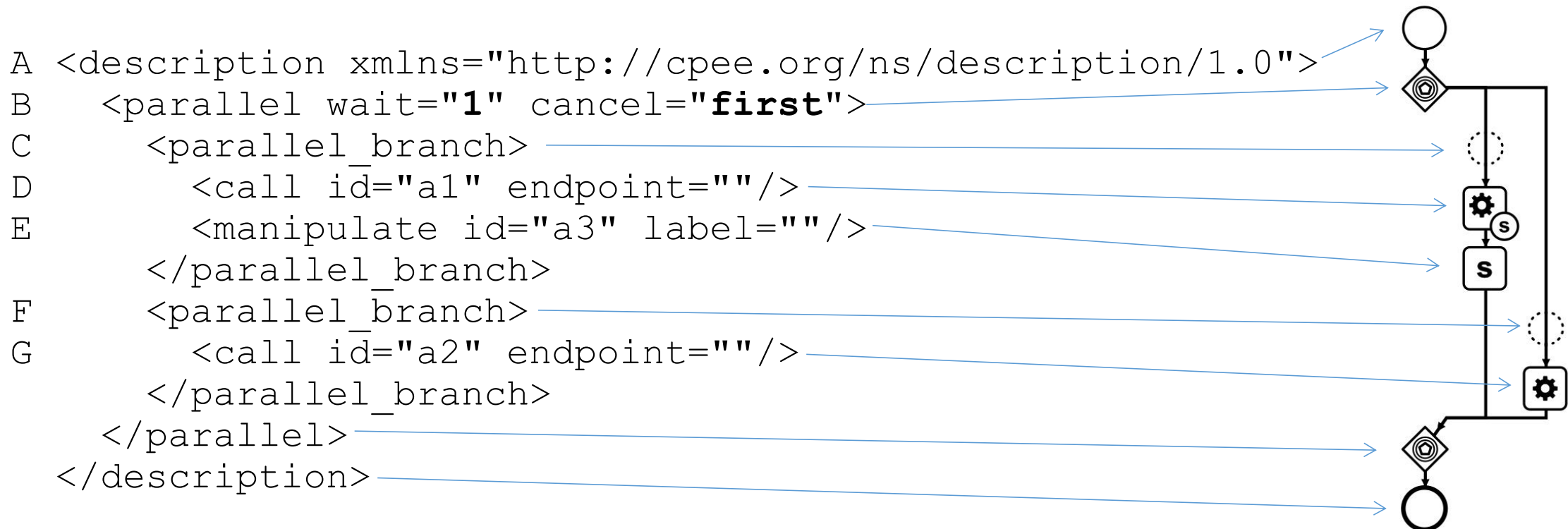
A <description xmlns="http://cpee.org/ns/description/1.0">
B <parallel wait="1" cancel="last">
C <parallel_branch>
D <call id="a1" endpoint=""/>
E <manipulate id="a3" label=""/>
F </parallel_branch>
G <parallel_branch>
 <call id="a2" endpoint=""/>
 </parallel_branch>
 </parallel>
 </description>



CPEE Trees - Parallel - Event-Based Gateway

depth-first + breadth, pre-order traversal; potential traversals:

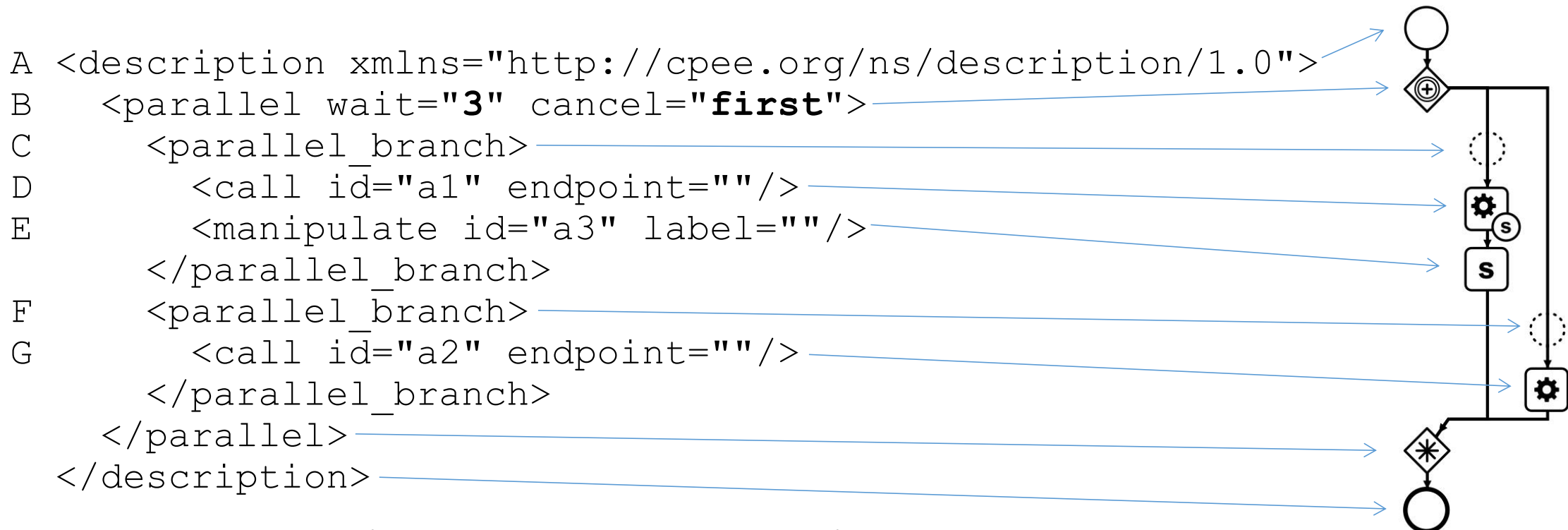
- ABCFG, ABCFDE
- if D and G are considered events that are waited for, these tasks are non-consequential. Ignoring either D or G will have no consequences.



CPEE Trees - Parallel - Special Cancelation

depth-first + breadth, pre-order traversal; potential traversals:

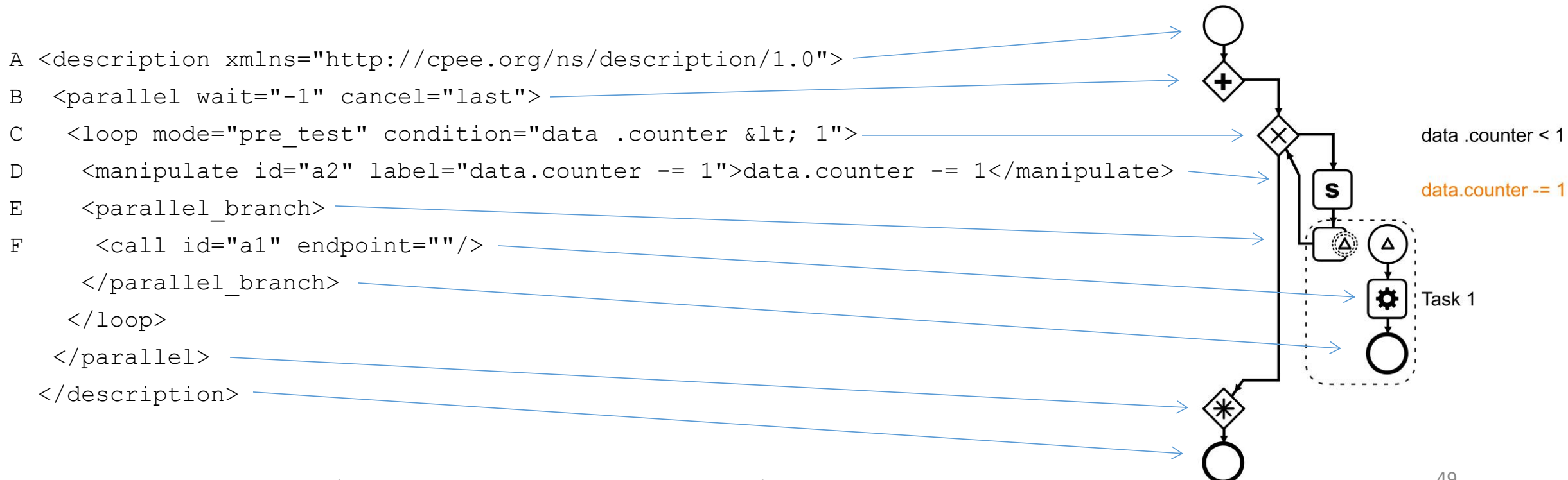
- ABCFDEG, ABCFDGE, ABCFGDE
- in this case behaves like a standard parallel. If more branches existed, this would realize a special case of the Event-Based Gateway



CPEE Trees - Variable Parallel

depth-first + breadth, pre-order traversal; potential traversals:

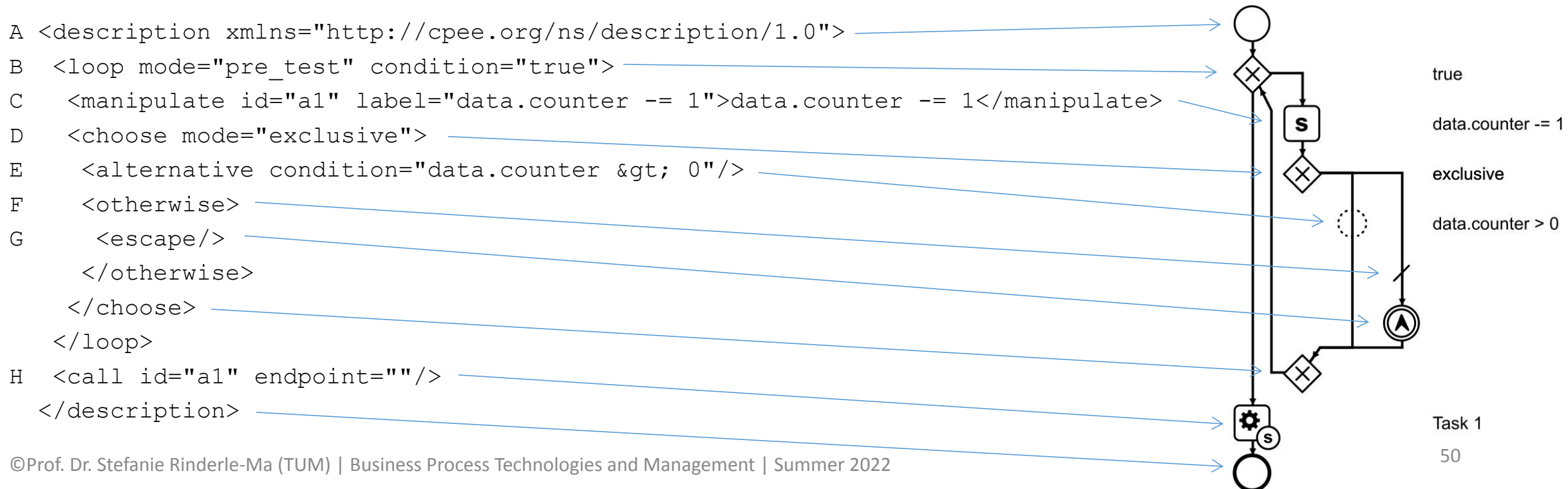
- $AB(CDE)*C(F)*$
- e.g. for counter = 3: ABCDECDECDECFFF
- arbitrary elements might exist as children of parallel; they determine the number of branches.



CPEE Trees - Escape (BPMN Escalate)

depth-first, pre-order traversal; only one possible traversal:

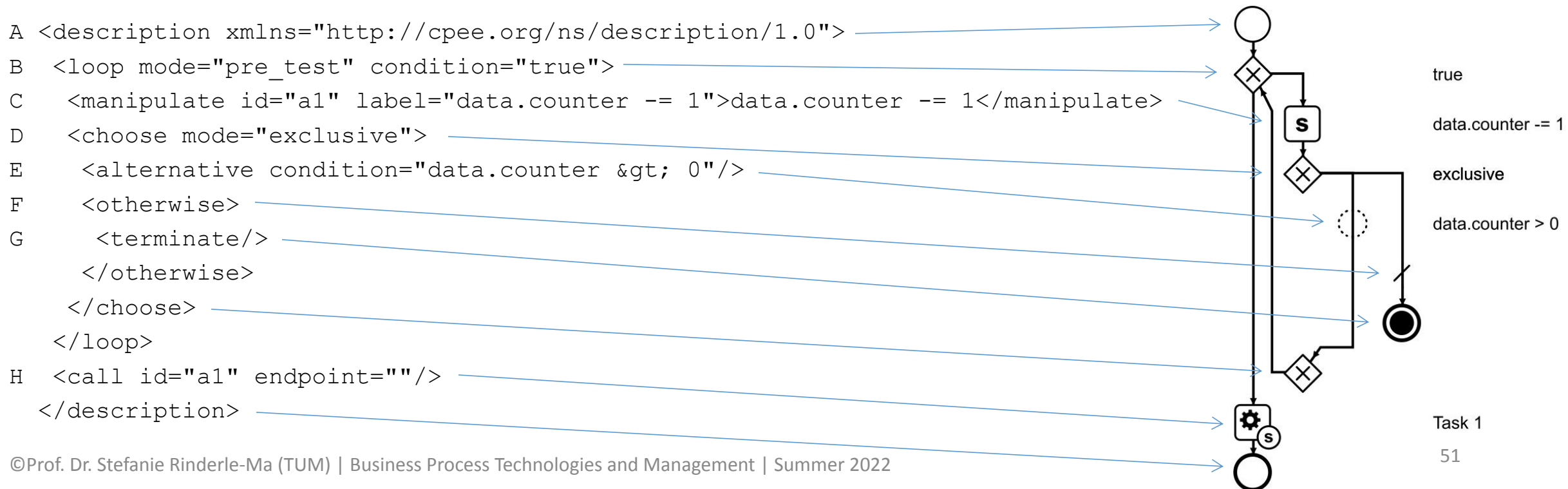
- A(BCDE)*FGH
- e.g. for counter = 3: ABCDEBCDEBCDEFGH



CPEE Trees - Terminate

depth-first, pre-order traversal; only one possible traversal:

- A(BCDE)*FG
- e.g. for counter = 3: ABCDEBCDEBCDEFG



Developing Executable Processes...

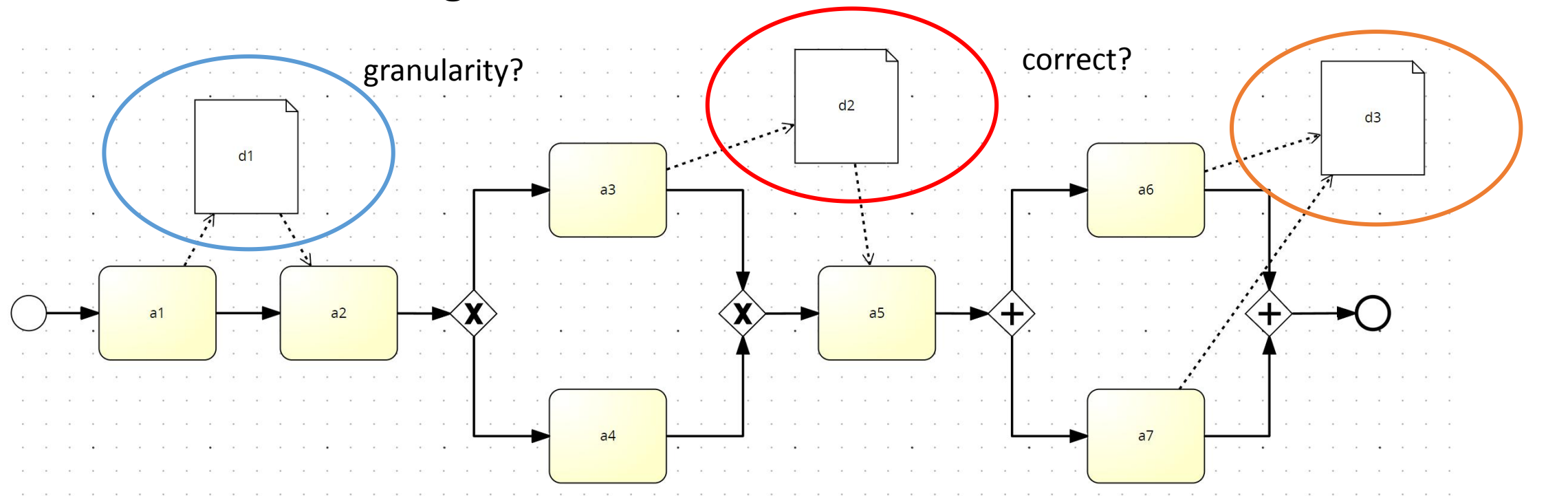


... requires the definition and specification of the following aspect (selection):

- Control Flow (Behavioral Aspect)
- ➔ • Data Flow (Information Aspect)
- Activities (Functional Aspect)
- Deadlines (Temporal Aspect)
- Actor Assignments (Organizational Aspect)
- Application Services (Operational Aspect)

Data Flow

- Data flow modeling in BPMN



Data flow in WSM Nets

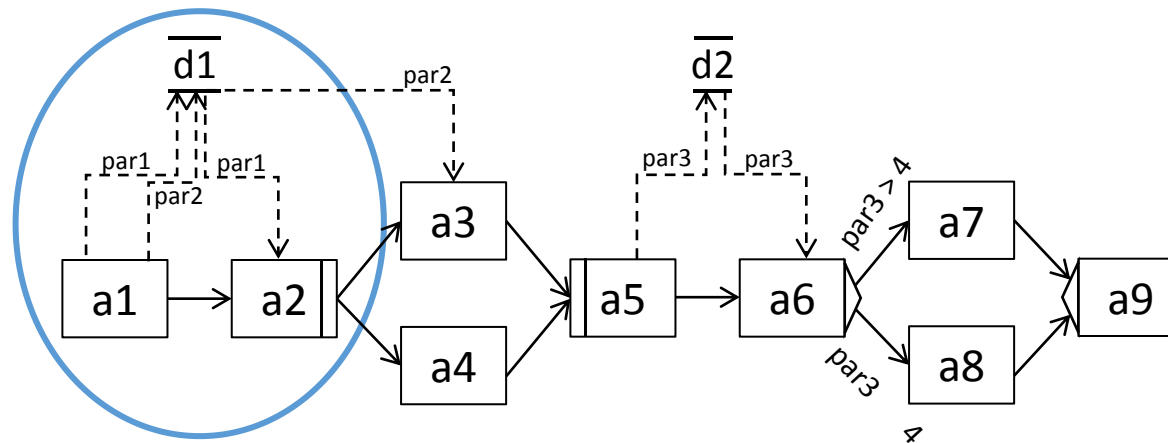


Definition 4.3 (Data flow schema, simplified from [ReDa98]). Let $S = (N, D, NT, \dots)$ be the control flow schema (based on WSM Nets) and let D denote a finite set of data elements associated with S . Let further $\text{PARS}(X)$ denote the set of parameters associated with the task X [$\text{PARS}(X) := \text{InPARS}(X) \cup \text{OutPARS}(X)$]. A data link df between a parameter par^{df} and a data element d^{df} is then described by the 4-tuple $df = (d^{df}, n^{df}, \text{par}^{df}, \text{access_mode}^{df})$ with

- $d^{df} \in D, n^{df} \in N, \text{par}^{df} \in \text{PARS}(n^{df}), \text{access_mode}^{df} \in \{\text{read}, \text{write}\}$

The set of all data links DF , connecting task or service parameters with global data elements from D , is called the data flow schema of S .

Data flow in WSM Nets



granularity at parameter level

- $DF = \{(d1, a1, par1, write), (d1, a1, par2, write), (d1, a2, par1, read), (d1, a3, par2, read), (d2, a5, par3, write), (d2, a6, par3, read)\}$
- Execution: read access when activity is started

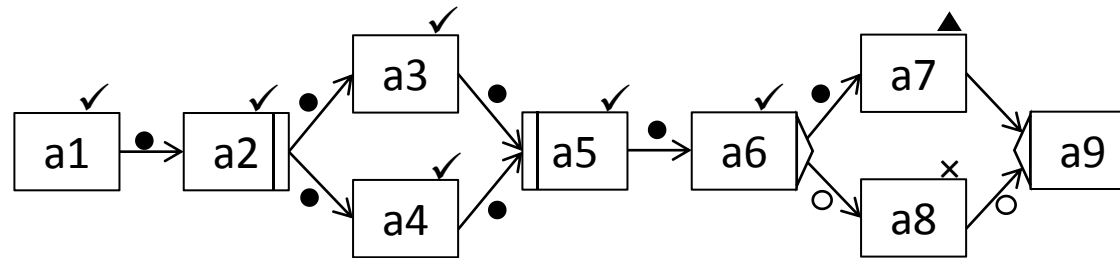
Data flow in WSM Nets



Definition 4.2 (Process Instance Based on a WSM-Net). A process instance I is defined by a tuple $(S, M^S, Val^S, trace^I)$ where

- $S = (N, D, NT, CtrlE, SyncE, \dots)$ denotes the process schema the execution of I is based on.
- $MS = (NS^S, ES^S)$ describes node and edge markings of I with:
 - $NS^S: N \rightarrow \{\text{NotActivated}, \text{Activated}, \text{Running}, \text{Completed}, \text{Skipped}\}$
 - $ES^S: (CtrlE \cup SyncE \cup LoopE) \rightarrow \{\text{NotSignaled}, \text{TrueSignaled}, \text{FalseSignaled}\}$
- Val^S is a function on D . It reflects for each data element $d \in D$ either its current value or the value UNDEFINED (if d has not been written yet).
- $trace^I = e_0, \dots, e_k$ is the execution history of I whereby $e_0, \dots, e_k$ denote the start and end events of activity executions. For each started activity X the values of data elements read by X and for each completed activity Y the values of data elements written by Y are logged.

Data flow in WSM Nets



- $DF = \{(d1, a1, par1, write), (d1, a2, par2, read), (d1, a3, par1, read), (d2, a5, par3, write), (d2, a6, par4, read)\}$
- $trace^I = \langle start(a1), end(a1, write\{(d1, par1, „Smith“), (d1, par2, 34)\}), start(a2, read(d1, par1, „Smith“)), end(a2), start(a3, read(d1, par2, 34)), start(a4), end(a4), end(a3), start(a5), end(a5, write(d2, par3, 5)), start(a6, read(d2, par3, 5)) \rangle$

Data flow Correctness



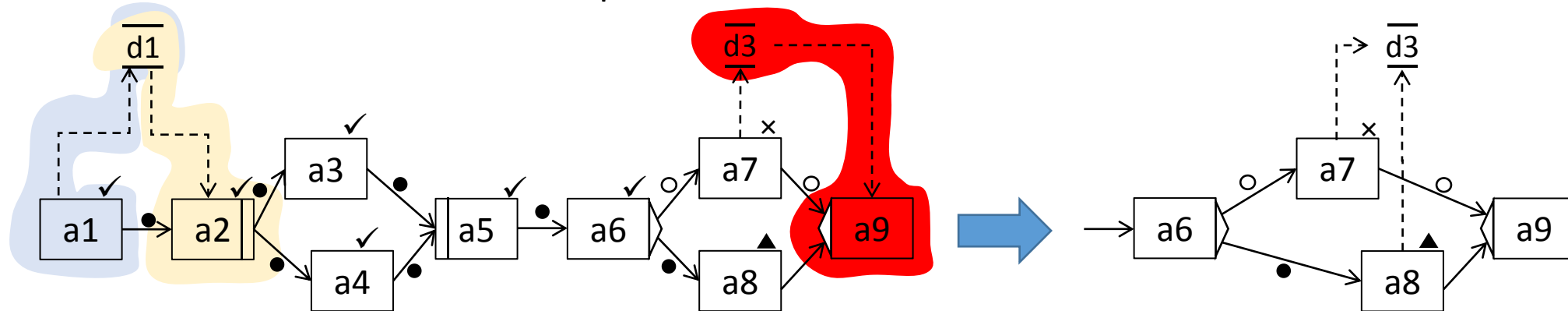
Rule 1: For every read access to a data element d , more specifically to a parameter par^d , by activity a , there must exist a mandatory write access to d , par^d by a predecessor activity a' of a .

Rule 1 guarantees the supply of data elements for read accesses and hence prevents that data elements are missing, resulting in, for example, failure of activity invocations during runtime.

Particularly „dangerous“ are read accesses from branches of alternative branches (see next slide). Requirement: the data element must be written from each branch.

Data flow Correctness

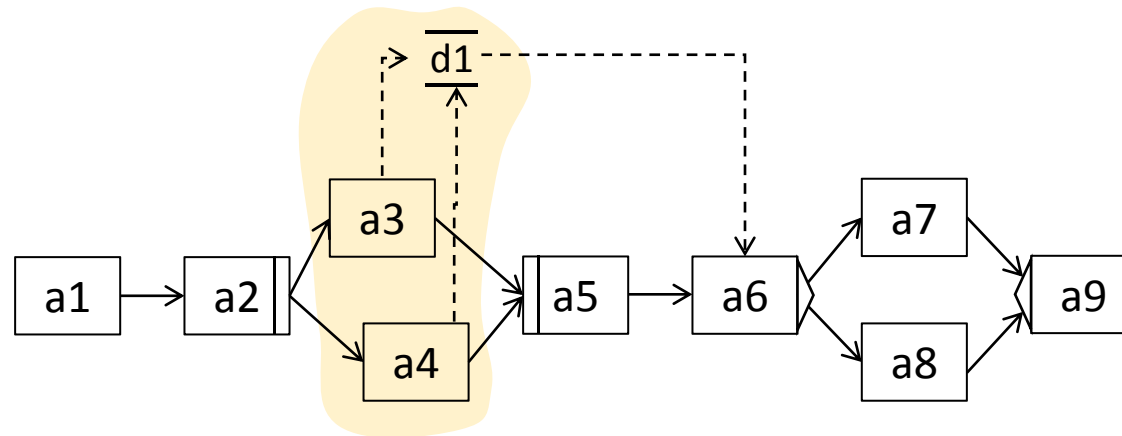
Rule 1 illustrated based on the example WSM Net



- a2 reads d1, par1; a1 is predecessor of a2 with mandatory write access
- a9 reads d3, but there is no mandatory write access, i.e., d3 might be written (if a7 is activated and executed), but could be also not written in case a7 is skipped.
- Hence, for alternative branchings it is required that the data element is written by activities from all branches.

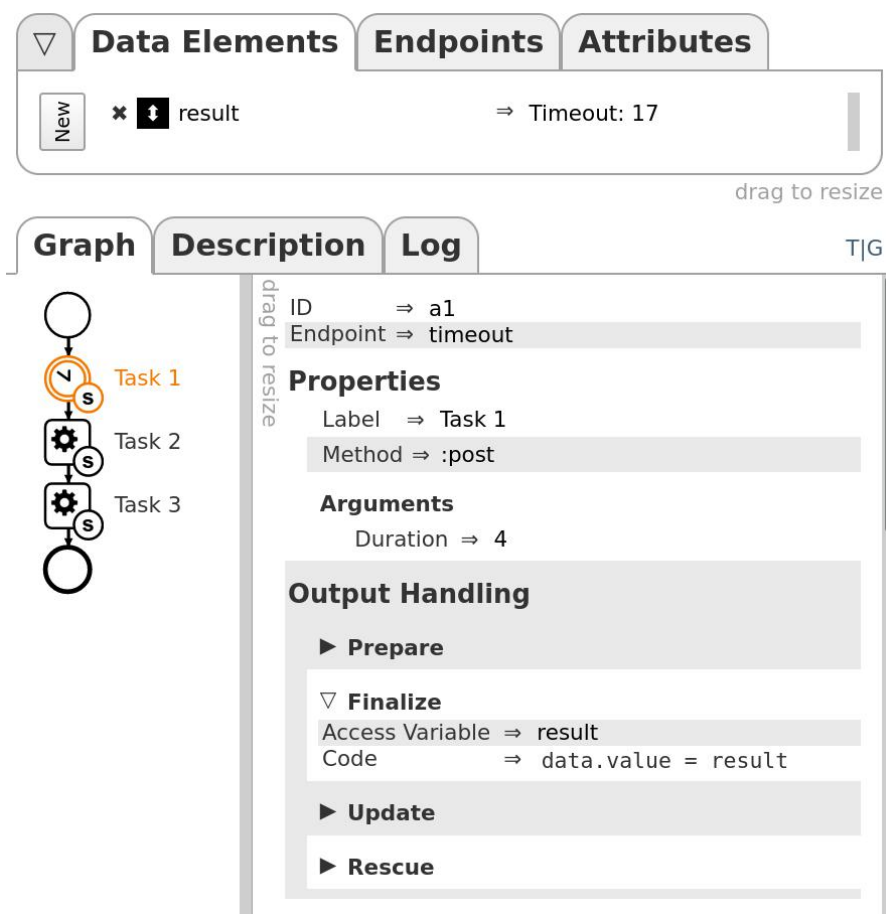
Data flow Correctness

Rule 2: no concurrent write accesses to data elements/parameters by activities in parallel branches.



- a3 and a4 write d1 (in arbitrary order); hence one write access is always overwritten by the other one.
- This could happen also to subsequent write accesses, if the data element is not read by an activity in between.

CPEE Trees - Data



All input to a task must come from the process context.

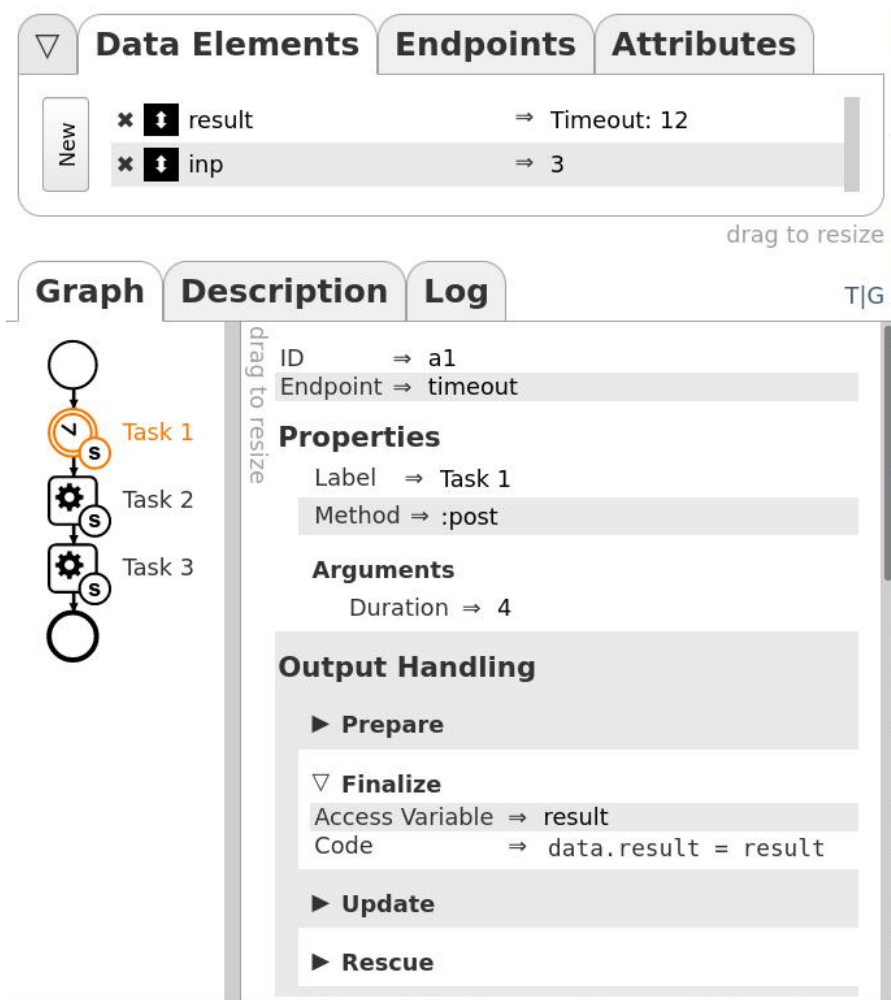
Output from a task can be stored in the process context.

The process context consists of a list of variables with a name and a value. For cpee.org the values can be:

- String
- Integer
- Float
- JSON

Everything else will be represented as an IETF RFC 2397 data-url.

CPEE Trees - Data



Example on the left:

- One data element named “result” exists
- The data element has the string value “Timeout: 12”
- Task 1 is an event which is implemented through the endpoint “timeout”
- Task 1 is implemented through a REST service
 - the call method is “post”
 - the static argument is “4”

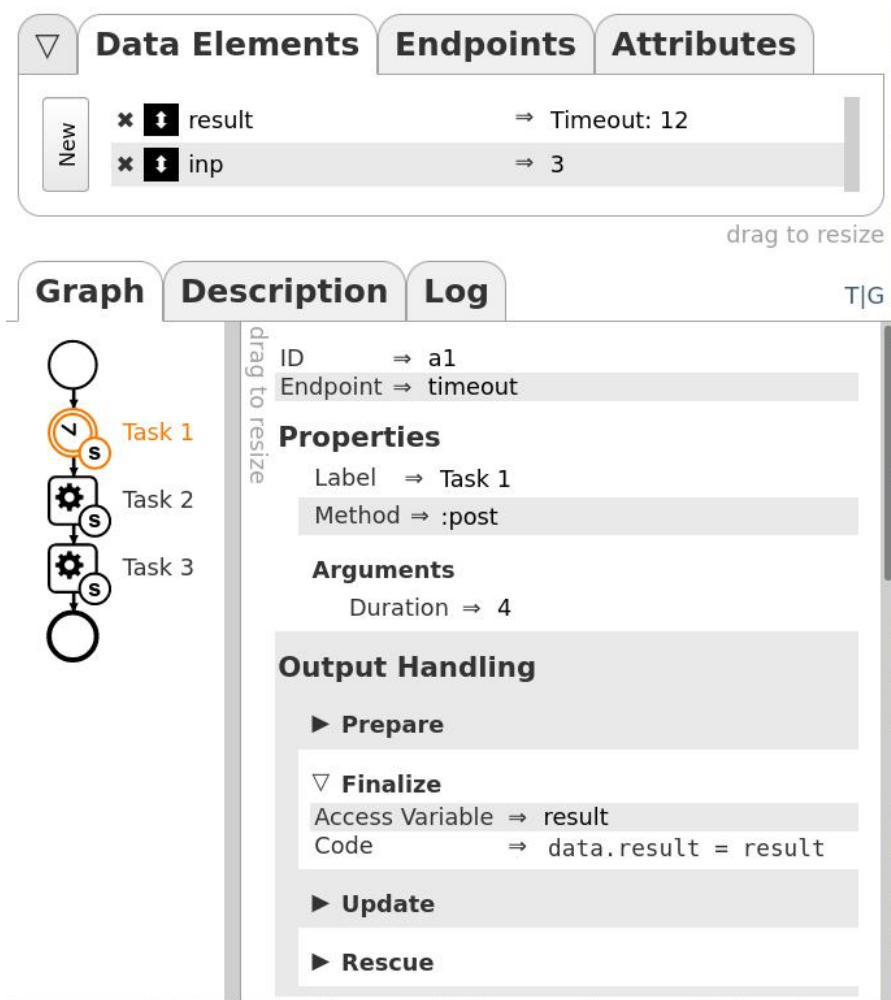
For a process to be dynamic, i.e. read data in one task, and pass it as an argument to the next task, it is necessary to:

- use data elements as argument values
- write data elements when a task returns data

Example on the left:

- To pass the data element “inp” as an argument
Duration => !data.inp
- Each argument value that starts with a ! can access all data elements, all endpoints and all arguments with **data.name, endpoints.name or arguments.name**
- The output of the task/event is handled through a code snippet in “Finalize”:
 - Everything that is returned is stored in “result”
 - Everything that should be kept in the process context can be assigned to a data element, i.e.:
data.name = result
 - If result is a json, keys can accessed, i.e.: data.name = result[“key”]

CPEE Trees - Data



Data flow is neither graphically represented nor represented by special elements in CPEE trees.

- Arguments describe input to the implementation
- Output handling dynamically filters the received data and moves it to the process context (data elements).

Developing Executable Processes...



... requires the definition and specification of the following aspect (selection):

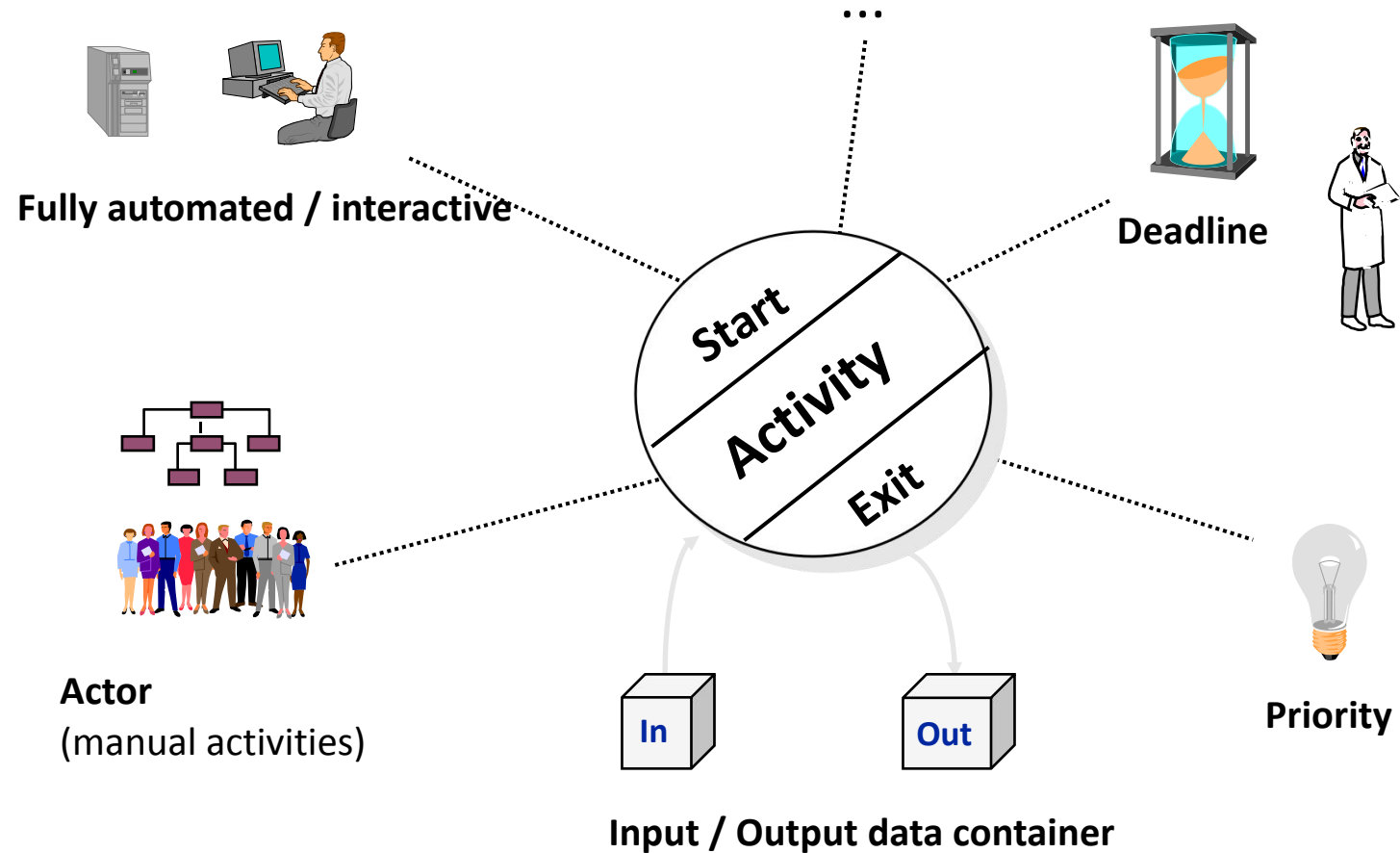
- Control Flow (Behavioral Aspect)
- Data Flow (Information Aspect)
- ➔ • Activities (Functional Aspect)
- Deadlines (Temporal Aspect)
- Actor Assignments (Organizational Aspect)
- Application Services (Operational Aspect)

Functional Aspect: Activities



- Activity: a description of a piece of work that forms one logical step in a process model. Activities are also called tasks.
- Requires human and/or machine resource(s) for its execution
- During runtime activity executions are controlled by the process management system/process engine. This may result in:
 - executed application services being automatically invoked by the process management system/process engine, i.e., with no process participant being involved (*fully automated*)
 - one or more work items being assigned to actors who may work on the respective activity (*interactive*)

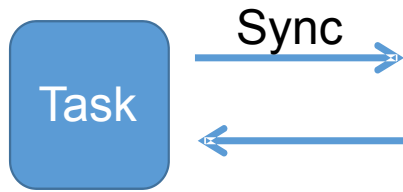
Functional Aspect: Activities



Functional Aspect: Interaction Patterns

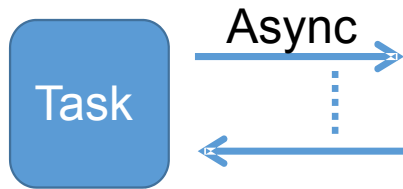


Tasks communicate with functionalities (built-in, webservices, ...) that implement them. Tasks send input (optional), and receive output that can be merged with the process context. The following basic interaction patterns between tasks and their implementations exist:



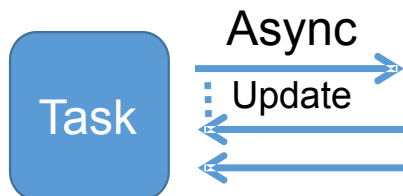
Answer a request immediately. It is not important for implementations to know details about the tasks.

- For requests where the implementation can return data immediately.
- A process can only be stopped when upon return.



Answer a request later. An implementation has to know instance and task id, in order to successfully address the task upon return.

- For long-running requests (can be months or years).
- A process engine can cancel such as task if it is e.g., stopping. Explanation: the implementation actively returns data, thus can get information if data is no longer need, and therefor can react accordingly.



Answer a request multiple times. Special form of Async.

- For long-running requests (can be months or years).
- For informing the process engine about progress, or sending partial results.

Functional Aspect: Interaction Patterns



Camunda: implementations are external services which utilize a library/API

- Java through `org.camunda.bpm.client.ExternalTaskClient`
- NodeJS `ycamunda-external-task-client-js`
- Python `camunda-external-task-client-python3`
- Each implementation utilizes a camunda/community provided API. The features of the API controls all interaction with the engine.

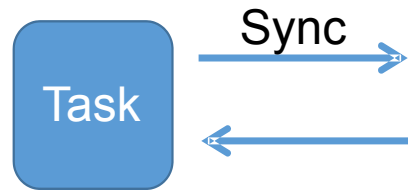
cpee.org

- Utilizes plain HTTP with special HTTP headers instead of a custom API.
- Fully language agnostic, all programming languages with support for HTTP can be used.
- Headers sent with each HTTP call to an implementation:
 - CPEE-BASE - base location of the engine where the instance is running on (e.g. `cpee.org/flow/engine/`)
 - CPEE-INSTANCE - instance number (e.g., 123)
 - CPEE-INSTANCE-URL - url pointing to the instance (e.g., `cpee.org/flow/engine/123`)
 - CPEE-INSTANCE-UUID - unique identifier of the instance
 - CPEE-CALLBACK - url to send any information to, should the implementation decide to answer asynchronously
 - CPEE-CALLBACK-ID - unique identifier for the answer
 - CPEE-ACTIVITY - id of the task calling the implementation (e.g. a1)
 - CPEE-LABEL - label of the task calling the implementation (e.g. Query Production Schedule)

Functional Aspect: Interaction Patterns



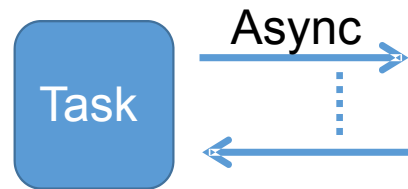
Each task implementation for cpee.org can utilize the following HTTP headers in their response.



CPEE-SALVAGE response header - implementation signals it internally failed but wants to retry.

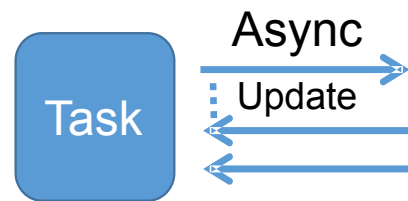
CPEE-INSTANTIATION response header - the implementation create a sub-process.

CPEE-EVENT - tell the engine to send a custom event. Used e.g. by worklists to communicate that a user has taken a task.



CPEE-CALLBACK: true response header and value - implementation signals that it wants to send data later, to the provided CPEE-CALLBACK request header.

All the headers above.



CPEE-UPDATE: true response header and value - implementation signals that it wants to send data more data after this response.

CPEE-UPDATE-STATUS: set the engine status with the response.

All the headers above.

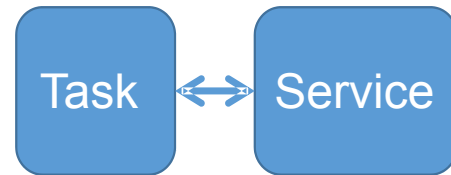
Functional Aspect: Interaction Patterns

For monolithic process engines, worklists or database query functionality might be built into the engine.

For modular process engines, everything is implemented as external services.



Engine communicates with an external service which in turn implements database access.



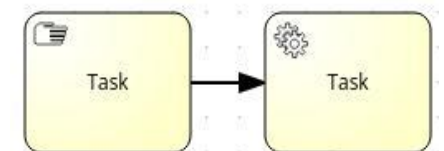
Engine communicates with an external service which receives input and generates output.



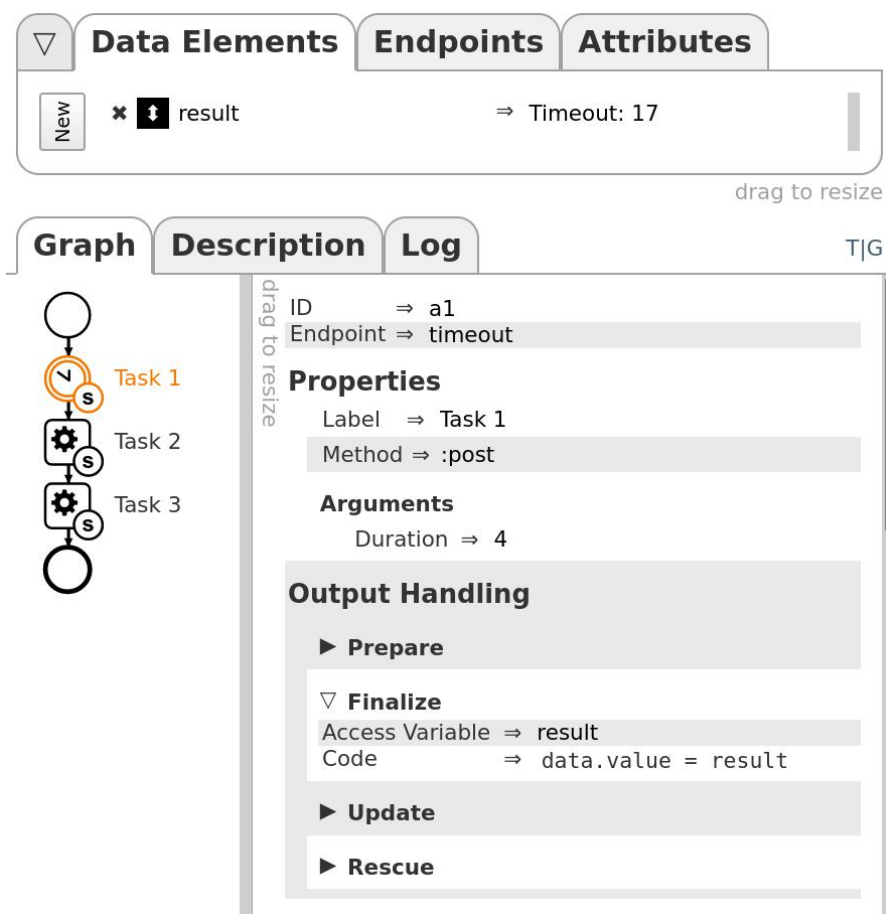
Engine communicates with a special external service which implements a UI for actors to interact with a process task.



Engine communicates with an external service which passively tracks the work done by humans



Functional Aspect: CPEE Trees



Tasks is the term that is used for an activity in BPMN

Tasks/Events have associated attributes:

- ID - uniquely identifies the task/event in a process
- Endpoint - link to the functionality that implements the task/event
- Label - a text that describes the task
- Method & Arguments - see slide Endpoints
- Output Handling - see slide Data

Important: from the POV of the process model, a task/event has an external implementation which is a **black box**. It is passed some input ("Arguments") and returns some output. The output is processed by using code snippets stored in "Output Handling".

Developing Executable Processes...



... requires the definition and specification of the following aspect (selection):

- Control Flow (Behavioral Aspect)
- Data Flow (Information Aspect)
- Activities (Functional Aspect)
- ➔ • Deadlines (Temporal Aspect)
- Actor Assignments (Organizational Aspect)
- Application Services (Operational Aspect)

Temporal Aspect



Deadline

- A time-based scheduling constraint which requires that a certain activity (or work item) shall be completed within a certain period of time → deadline)
- Activity scheduling by a WfMS attempts to meet deadline constraints set against particular tasks
- The deadline may be expressed as an attribute of the workflow model or within workflow relevant data
- Escalation / Notification procedures may be involved if deadlines are not met

Developing Executable Processes...



... requires the definition and specification of the following aspect (selection):

- Control Flow (Behavioral Aspect)
- Data Flow (Information Aspect)
- Activities (Functional Aspect)
- Deadlines (Temporal Aspect)
- ➔ • Actor Assignments (Organizational Aspect)
- Application Services (Operational Aspect)

Task authorization

- Definition of organizational models (organigram)
- Defines organizational units, roles, and actors
- Established: role-based access control (RBAC) → abstraction from actors who might change, i.e., authorization is defined based on roles and during runtime the roles are resolved to actors that are currently present in the organizational model
- Example: `access_rule (task) ← Role='clerk' AND OrgUnit= 'Loan'`

Task authorization

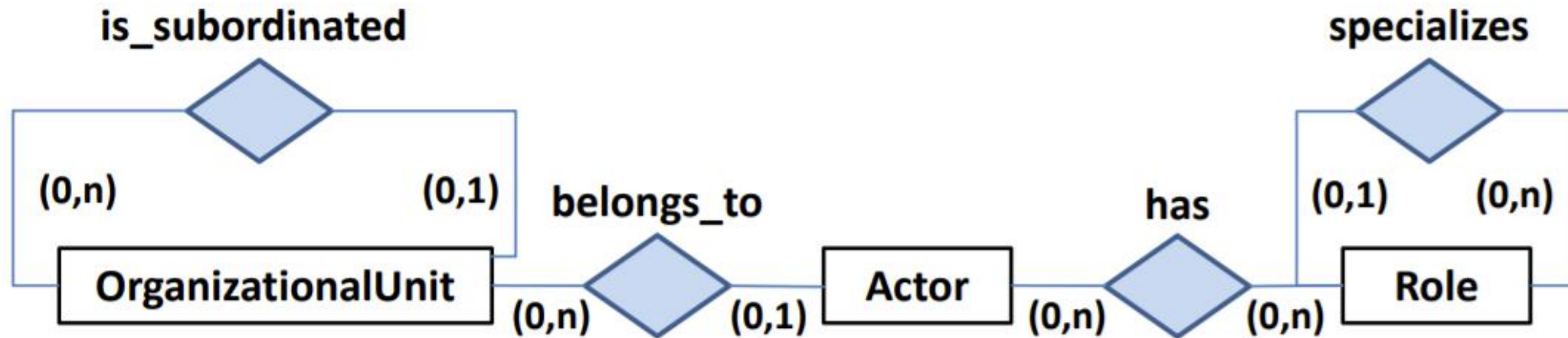


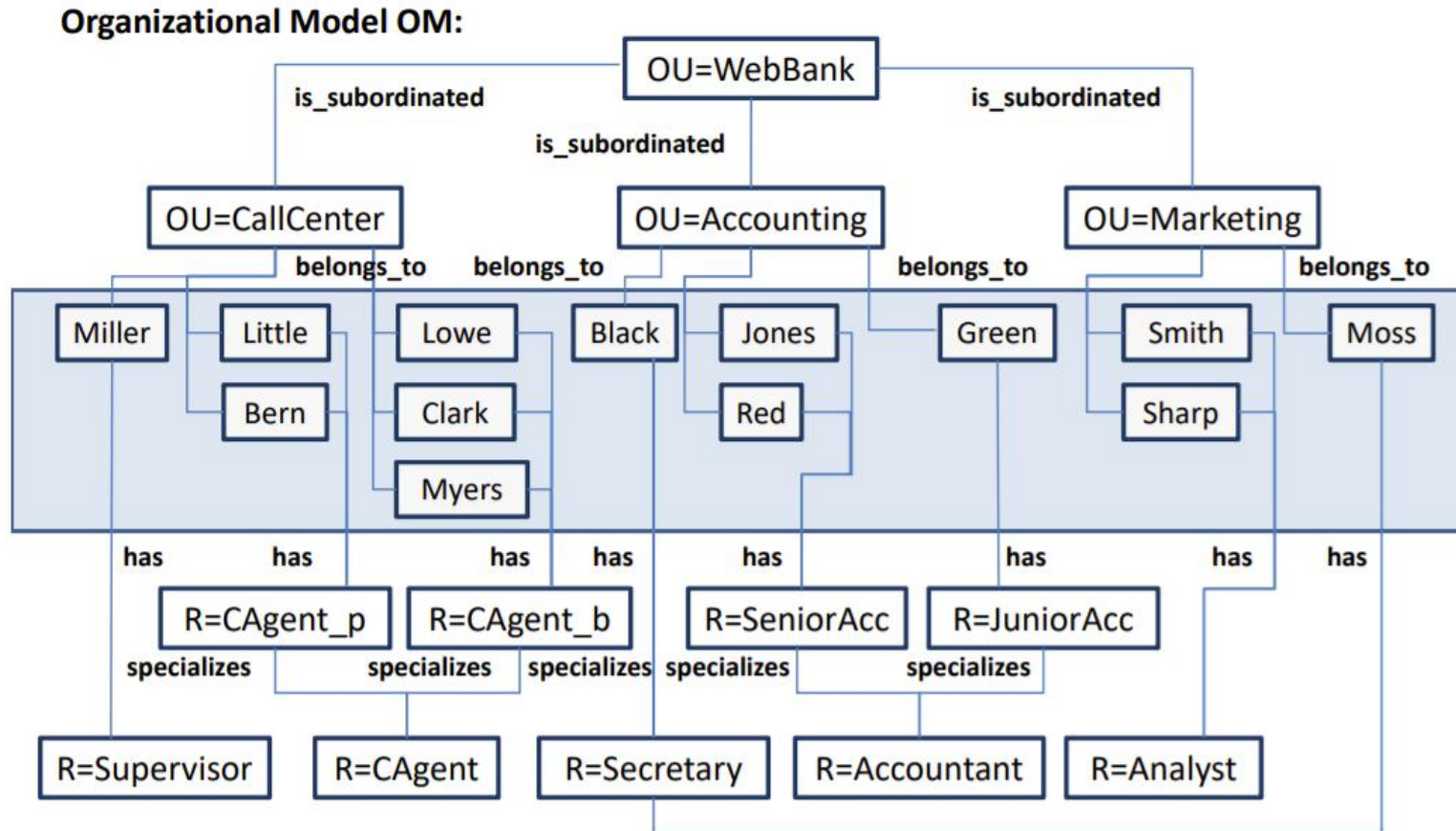
Figure 3. Organizational meta model (in ER notation)

© Taylor and Francis

Stefanie Rinderle-Ma, Manfred Reichert:

Comprehensive life cycle support for access rules in information systems: the CEOSIS project. *Enterp. Inf. Syst.* 3(3): 219-251 (2009)

Task authorization

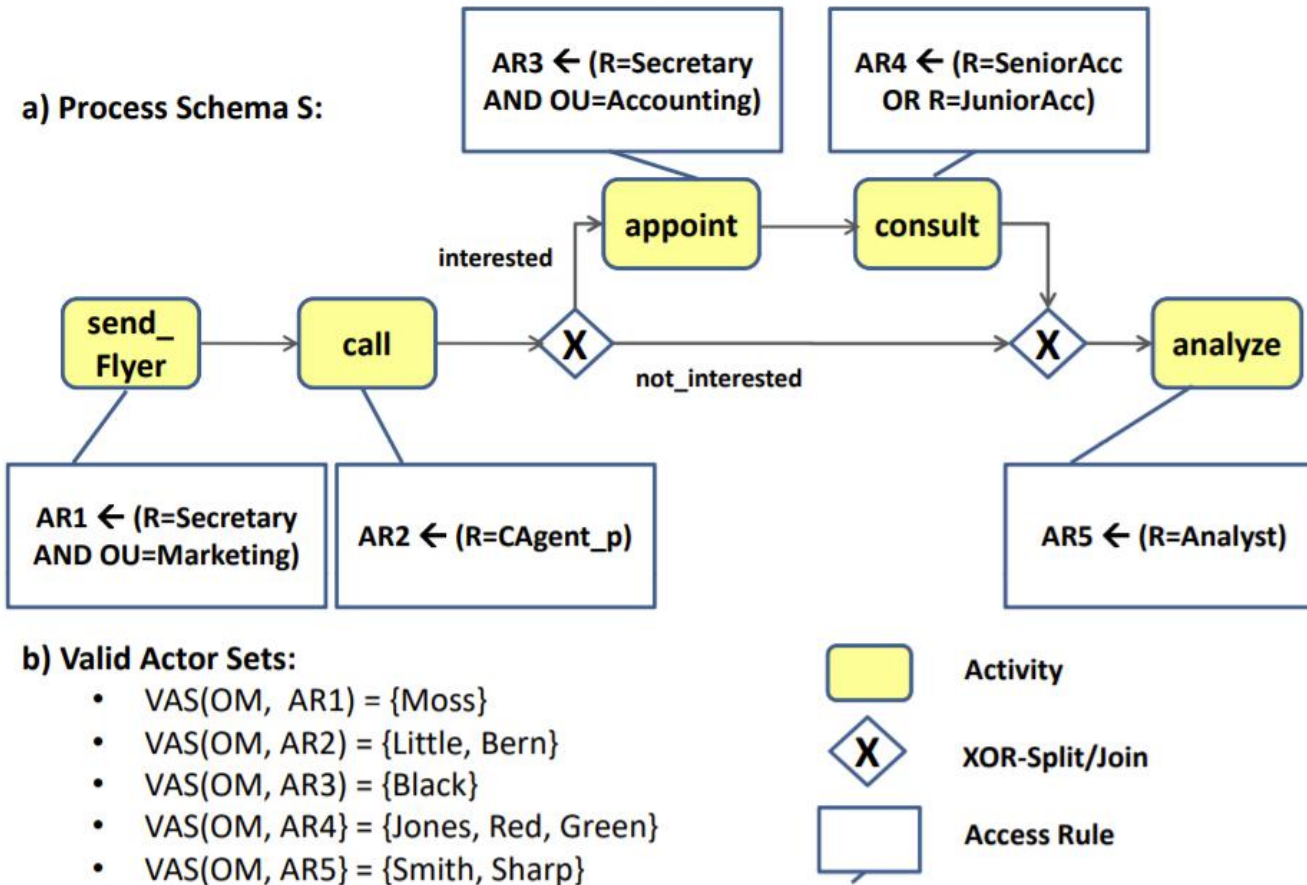


© Taylor and Francis

Stefanie Rinderle-Ma, Manfred Reichert:

Comprehensive life cycle support for access rules in information systems: the CEOSIS project. *Enterp. Inf. Syst.* 3(3): 219-251 (2009)

Task authorization



© Taylor and Francis

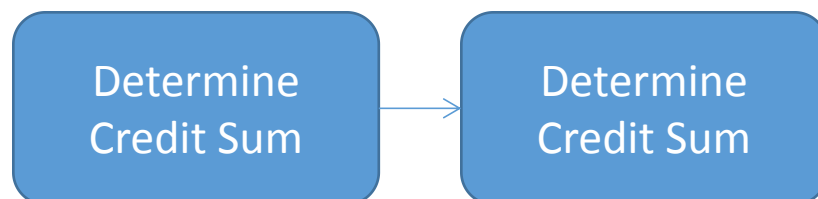
Stefanie Rinderle-Ma, Manfred Reichert:

Comprehensive life cycle support for access rules in information systems: the CEOSIS project. Enterp. Inf. Syst. 3(3): 219-251 (2009)

Task authorization

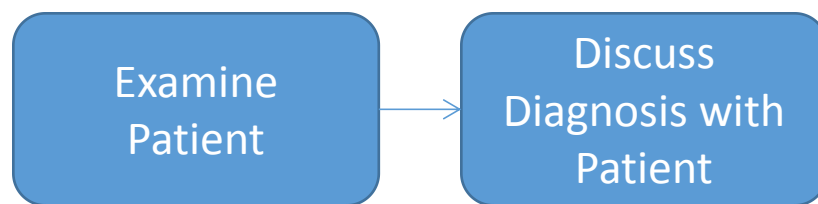
Separation of Duty

- A task is only authorized for an actor that was not involved in a different task
- Example: Controlling, Four Eyes Principle



Binding of Duty

- A task is only authorizes for an actor that was also authorized for a different task
- Example: Confidentiality, Privacy



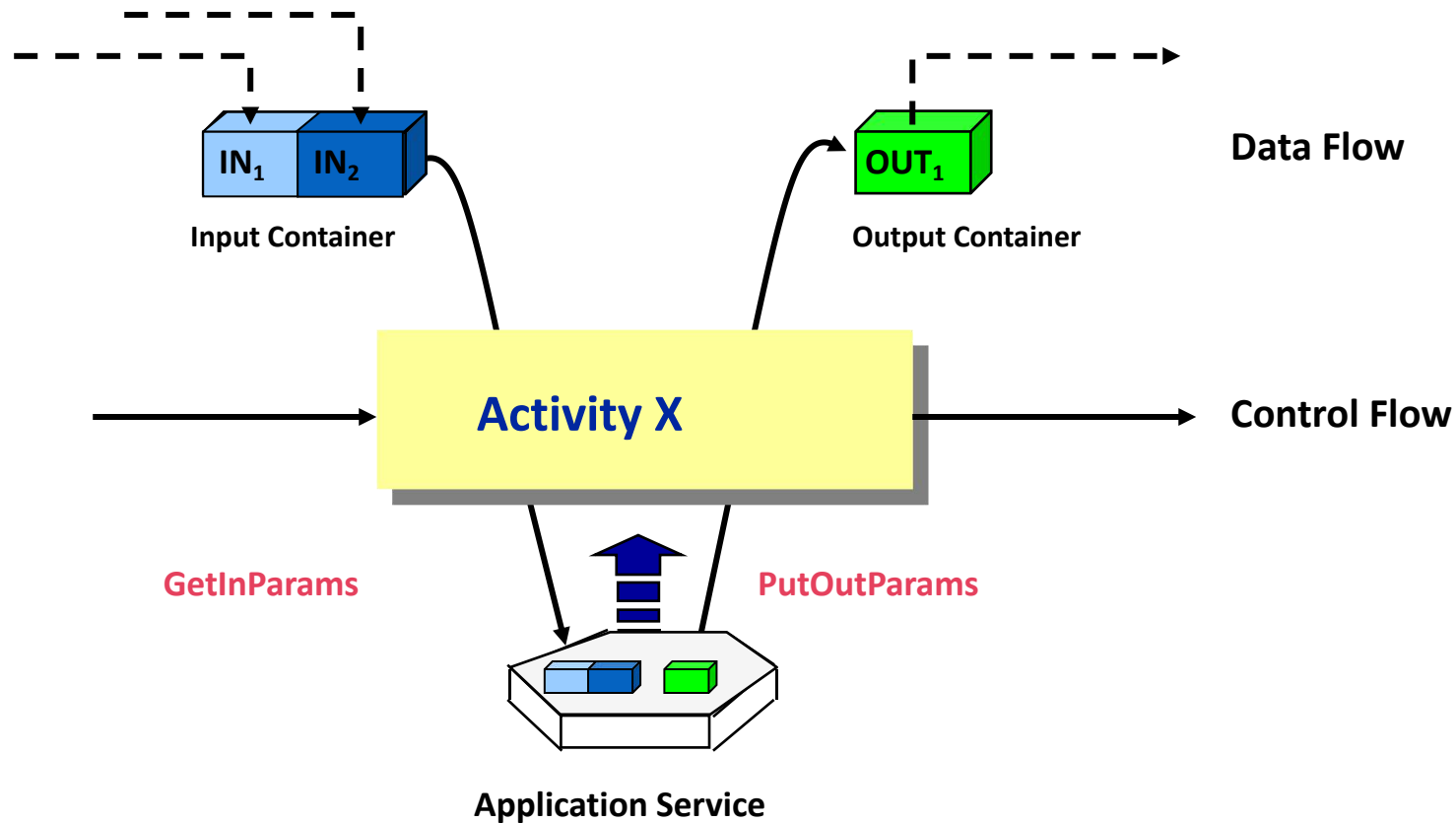
Developing Executable Processes...



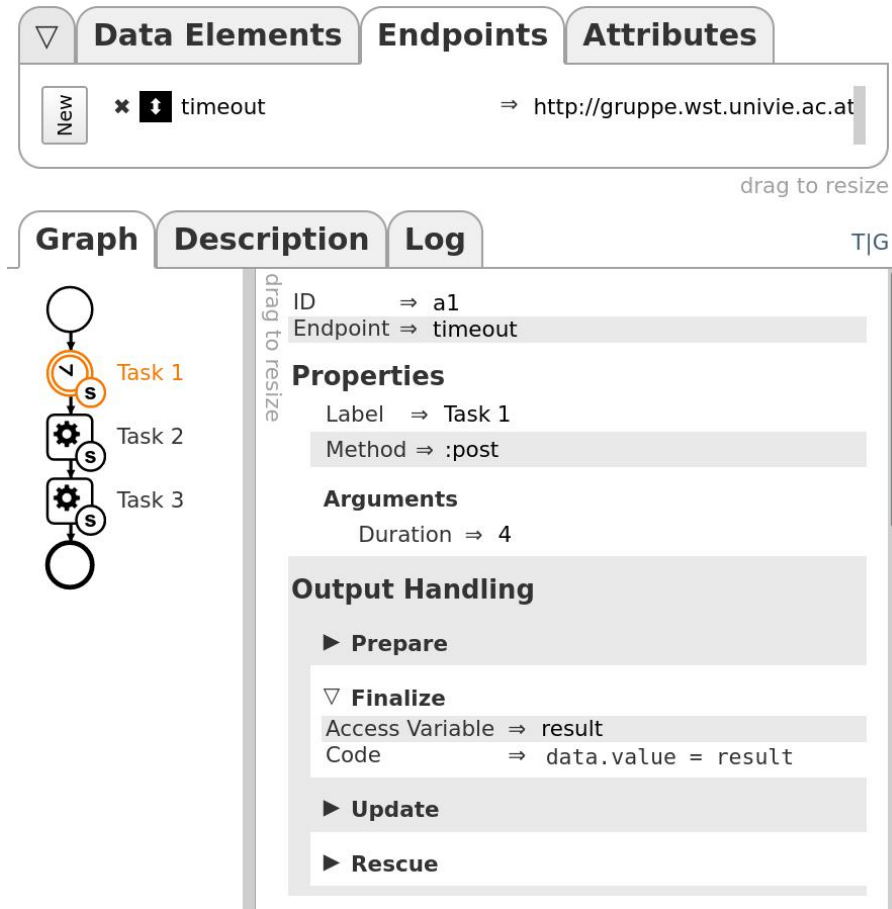
... requires the definition and specification of the following aspect (selection):

- Control Flow (Behavioral Aspect)
- Data Flow (Information Aspect)
- Activities (Functional Aspect)
- Deadlines (Temporal Aspect)
- Actor Assignments (Organizational Aspect)
- ➔ • Application Services (Operational Aspect)

Application Integration



CPEE Trees - Endpoints



Endpoints are links to the implementation of a task/event.

Each task/event has to point to **at least one endpoint** that implements it.

Endpoints can be addressed through a variety of different protocols, such as SOAP, REST, OPC-UA, ...

Standard endpoints for CPEE trees are assumed to point to REST services:

- They have a method: GET, POST, PUT, DELETE
- They have 1..n arguments. Arguments are always name/value pairs. The same name may occur more than once.

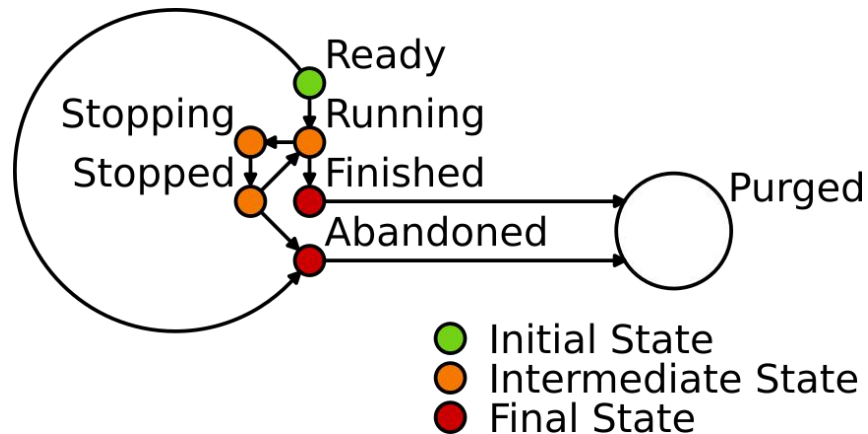
By default all endpoints are considered to be automatic tasks. If special annotations for a rest service exist, the visual representation of the task changes to represent its purpose. Examples:

- Sub-process
- Send message
- Receive message
- Wait

Nonetheless - even the creation of a sub-process is implemented as a REST service with specific parameters:

- The path to process model (which represents the sub-process)
- Data-elements passed to the sub-process upon creation (BPMN start-event with envelope).

CPEE Trees - Instance Lifecycle



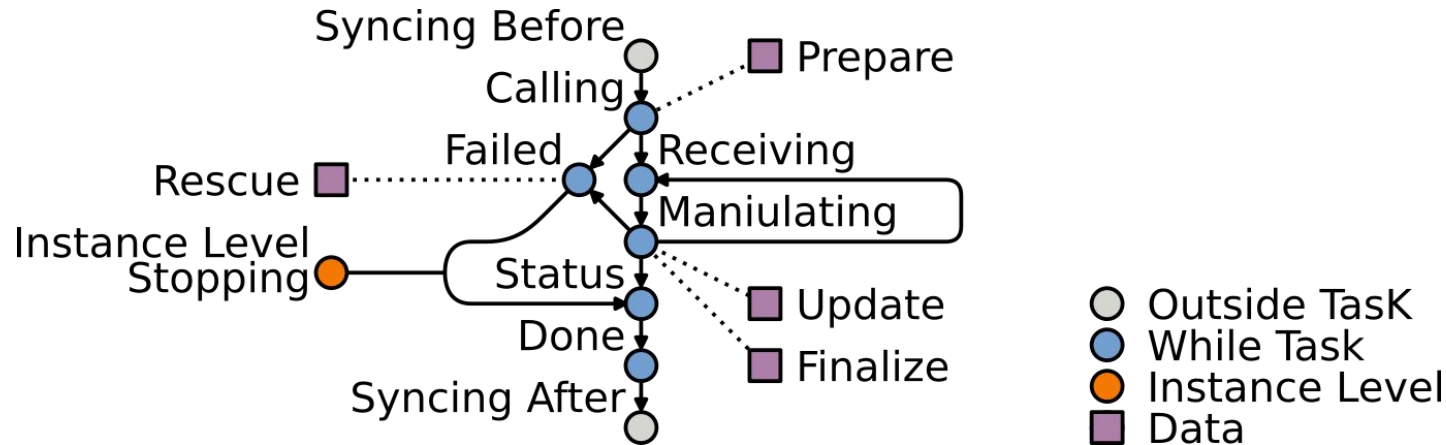
Each process instance goes through a series of states while it is executed:

- Upon creation it is ready
- When started it is running
- If an error occurs it is stopping
- When all parallel running tasks (even the ones that had no errors) are finished, it is stopped
- A user can set the instance back to running (after potentially changing the data elements or the control flow)
- After an end event is found it is finished
- A finished instance can **NOT** be restarted again.
- If an instance is ready or stopped, it can be set to state abandoned by a user.

The data-elements of all instances, even finished and abandoned ones can be inspected in the process engine.

- A user can decide to fully remove the instance from the process engine, in which case it is purged.
- Purged instances can only be inspected in a log.

CPEE Trees - Task Lifecycle



Each task goes through a series of states when it is executed:

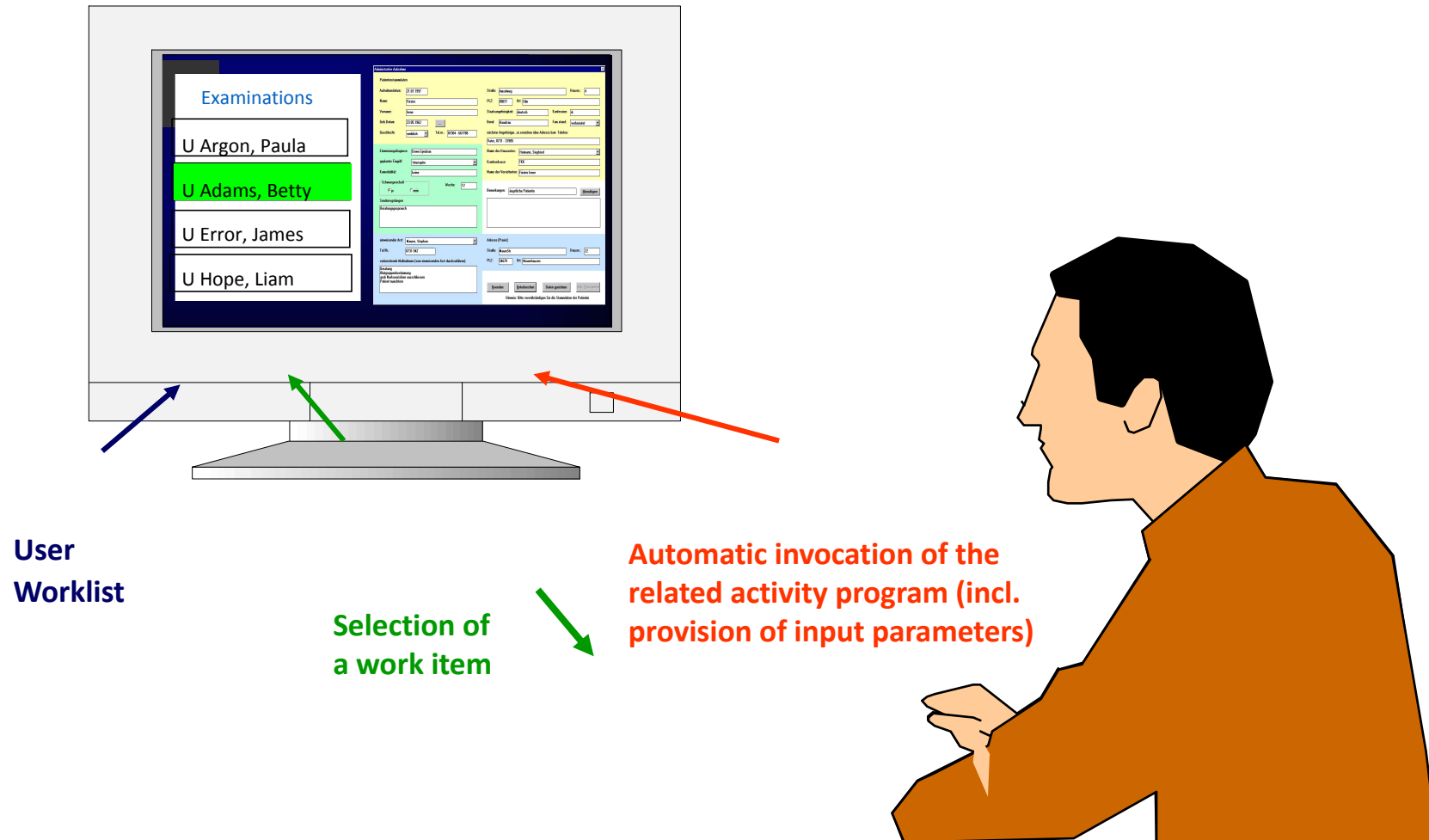
- **Calling:** the REST service is called with a specific method and the arguments.
- **Receiving:** the REST service sends back raw data.
- **Maniulating:** the “Finalized” code snippet is executed and is changing the process context.
- **Status:** the status of the process instance is set, e.g. to the string “task x sent back 12 bytes”. Setting the status is optional, and is determined through code in “Finalize”.
- **Done**

- **Failed:** if an error occurs when calling the REST service, the process goes to state failed. In failed the optional “Rescue” code snippet is called. This code can determine, if the instance is stopped or the instance can continue normally. This code snippet can change the process context (i.e. write data elements) as well as the status).
- **Maniulating ctd:** REST services can send back data in multiple small packages. If this is detected, “Receiving” might occur again. As long as additional data packages are expected, the “Update” code snippet is called for each package.
- **Calling ctd:** the “Prepare” code snippet can be utilized for changing the process context before a particular task is called.

Refined lifecycles might exists for particular task implementations: e.g. the XES IEEE 1849-2016 standard defines states which are especially suitable for humans carrying out individual tasks such as schedule, assign, reassign, start, suspend, resume, complete, withdraw, ... **Important:** from the POV of the process engine, a task carried out by a human is still a black box:

- The task has input - information which is passed on to the human.
- The task has output - some artefacts (e.g. documents) created by the human.
- The task waits until the output is revealed.

Worklists

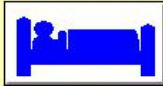


Worklists

End User View (1)

Stationsübersicht

Zimmer 57



Fr. Schiebe
frakt. Abrasio
Aus OP zurück



Fr. Mayer
Interruptio
Im OP

Zimmer 58



Fr. Apfel
frakt. Abrasio





Fr. Heinrich
IVF
In OP bringen





Maßnahme anordnen

OP-Plan

Umschalten zu Ambulanz

Notfallaufnahme

Anstehende Aufgaben:

Patientin	Tätigkeit	Bemerkung	Ausführen	Aktualisieren
Fr. Schiebe	OP-Bericht erstellen			
Fr. Schiebe	Klinische Entlassung			
Fr. Förster	Überprüfung der Befunde	ängstliche Patientin		
Fr. Hansen	Überprüfung der Befunde			

Freitag, 18. Juli 1997

Einbestellungsplan GYN IV

Patientin	Diagnose-OP	Tel.Nr.
Fr. Mayer (Abruf)	Interruptio	07551-22456
Fr. Dampf (Ueberpruefung der Befunde)	dg/op Hysteroskopie	
Fr. Heinrich (Abruf in Ausfuehrung)	IVF	
Fr. Apfel (Stationaere Aufnahme)	frakt. Abrasio	07551-22456
Fr. Frentze (Ueberpruefung der Befunde)	IVF	0731-3051
Fr. Schiebe (Rueckmeldung)	frakt. Abrasio	

<< Info >>

Tag

Woche

Monat

←

→

neuer Termin

Termin verschieben

Termin sperren

Termin löschen

Patientenstammdaten

Worklist

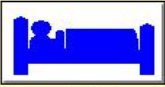
Work Item

Worklists

End User View (2)

Stationsübersicht

Zimmer 57



Fr. Schiebe
frakt. Abrasio
Aus OP zurück



Fr. Mayer
Interruptio
Im OP

Zimmer 58



Fr. Apfel
frakt. Abrasio





Fr. Heinrich
IVF
In OP bringen





Maßnahme anordnen

OP-Plan

Umschalten zu Ambulanz

Notfallaufnahme

Anstehende Aufgaben:

Patientin	Tätigkeit	Bemerkung	Ausführen	Aktualisieren
Fr. Schiebe	OP-Bericht erstellen			
Fr. Schiebe	Klinische Entlassung			
Fr. Förster	Überprüfung der Befunde	ängstliche Patientin		
Fr. Hansen	Überprüfung der Befunde			

Invoked Activity Program

Befunde prüfen

Name	Förster	Vorname	Irene
Geb.Datum	23-05-1962	Tel.Nr.:	07304 - 667788
Schwangerschaft	ja		
Komorbidität	keine		
Diagnose	Down-Syndrom		
einweisender Arzt	Dr. Stephan Maurer		
Tel.Nr.:	0731-542		
Eingriff	Interruptio		
OP-Termin:	21.07.1997		

Bemerkungen :

Hinzufügen

ängstliche Patientin

Befunde

[zur Einsicht bitte auf Dokument doppelklicken]

EKG	Eselsberg Kardiologie	Befund fehlt
Lungenfunktion	Michelsberg Pulmonologie	Befund erhalten

Markierten Befund einsehen

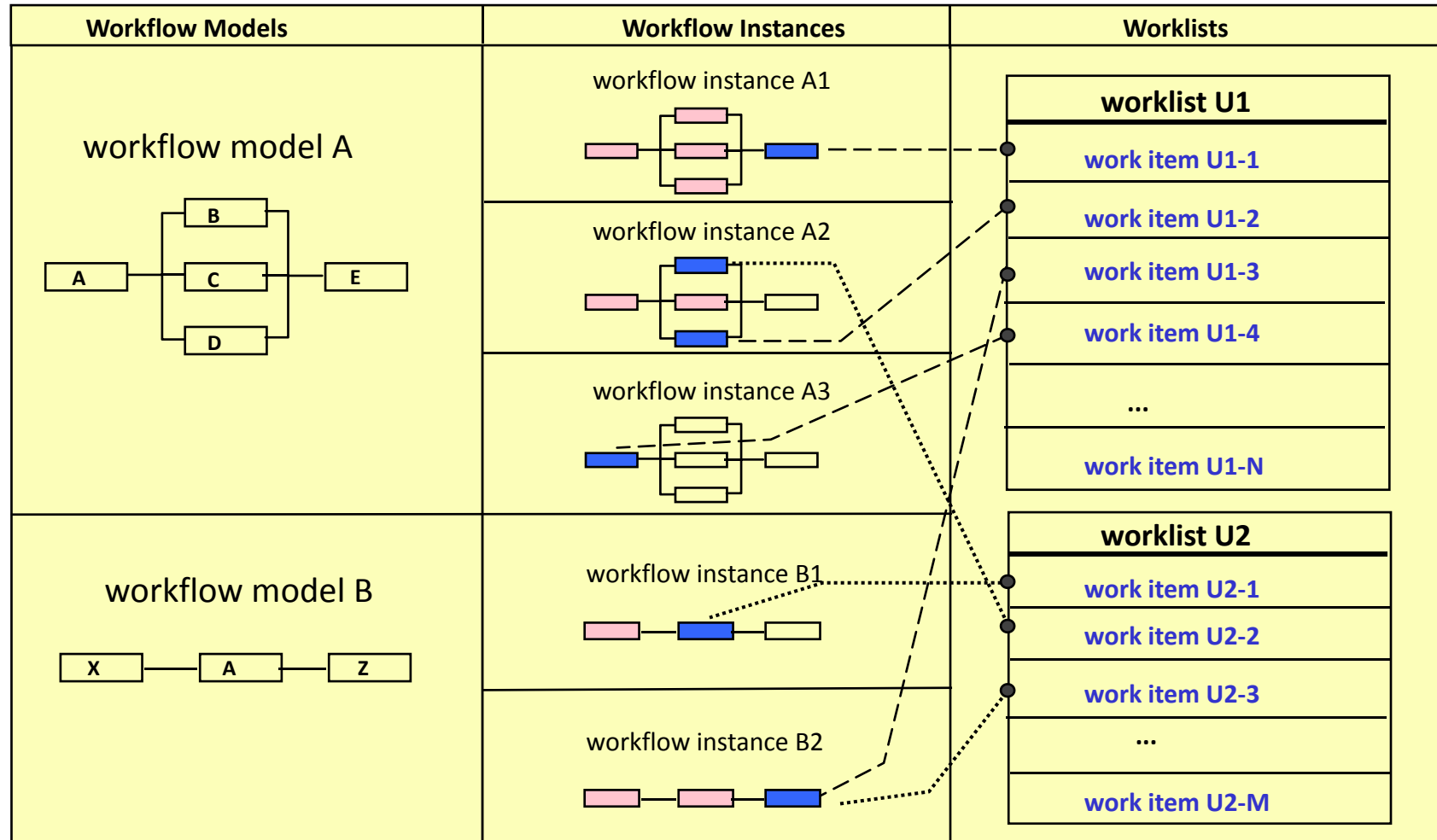
Op-Termin verschieben

Akte anzeigen

Unterbrechen

Beenden

Worklists



Worklists



Vendors often provide simple generic worklist generic clients.

Purpose: presentation of generic, application-independent workflow / activity attributes:

- Activity status
- Activity name
- Related application program / service
- Activation time

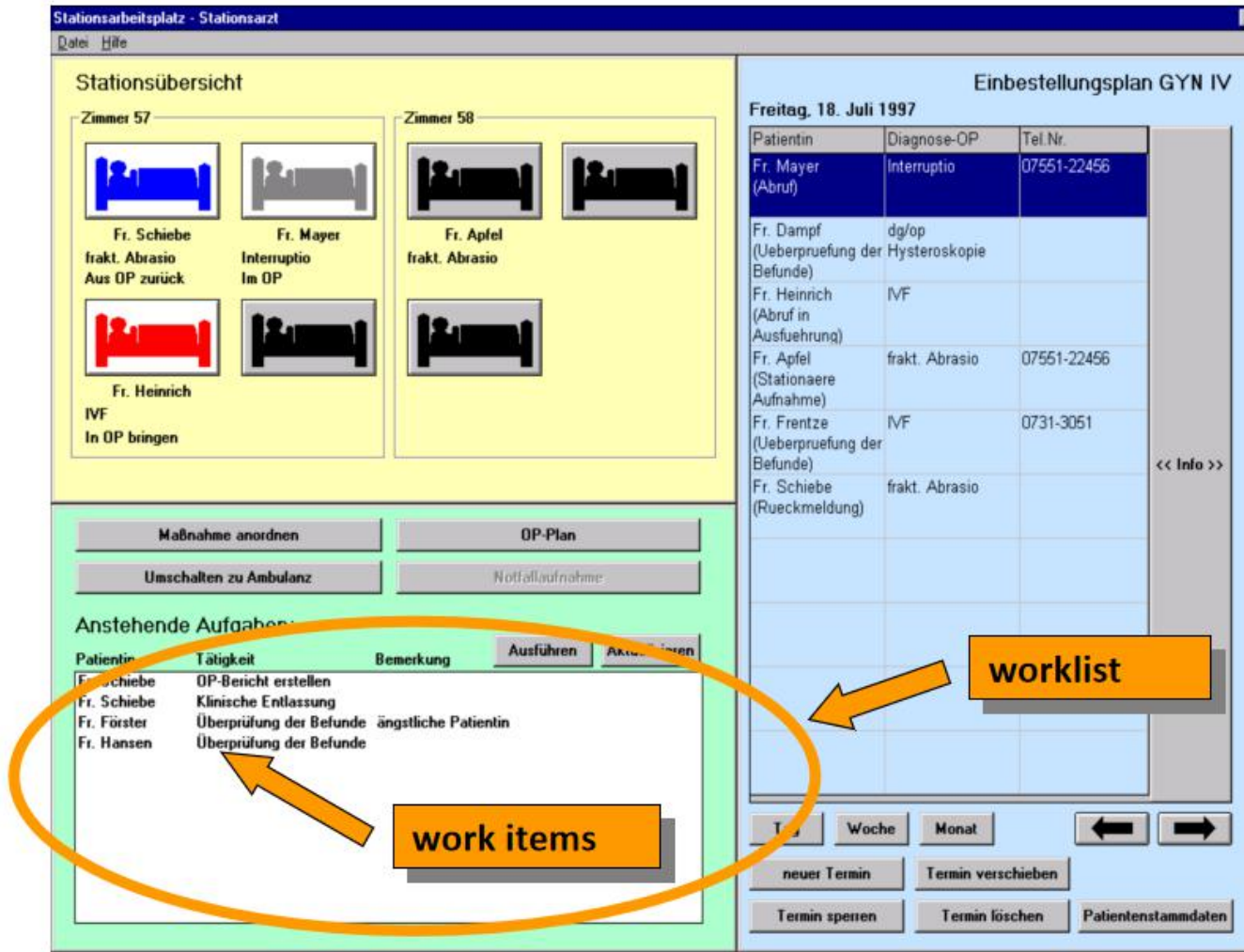
Worklists



Characteristic properties of a worklist client:

- Parallel connection to multiple process management systems, process engines (→ transparent for the end user)
- Control and monitoring of **process instances**
 - Querying the status of process instances
 - Creating, starting, terminating, suspending, ... process instances
- Control and monitoring of **activity instances** and **work items** respectively
 - Aggregating *work items* in **worklists** (→ application-specific criteria: priority, kind of task, process type, ...)
 - Delegating, starting, suspending, terminating ... tasks
 - Monitoring deadlines (→ escalation & notification mechanisms)
- Access to application data.

Domain-specific Worklists and UIs



Stationsübersicht

Zimmer 57

- Fr. Schiebe: frakt. Abrasio, Aus OP zurück
- Fr. Mayer: Interruptio, Im OP
- Fr. Heinrich: IVF, In OP bringen

Zimmer 58

- Fr. Apfel: frakt. Abrasio

Einbestellungsplan GYN IV
Freitag, 18. Juli 1997

Patientin	Diagnose-OP	Tel. Nr.
Fr. Mayer (Abruf)	Interruptio	07551-22456
Fr. Dampf (Ueberpruefung der Hysteroskopie Befunde)	dg/op	
Fr. Heinnich (Abruf in Ausfuehrung)	IVF	
Fr. Apfel (Stationaere Aufnahme)	frakt. Abrasio	07551-22456
Fr. Frentze (Ueberpruefung der Befunde)	IVF	0731-3051
Fr. Schiebe (Rueckmeldung)	frakt. Abrasio	

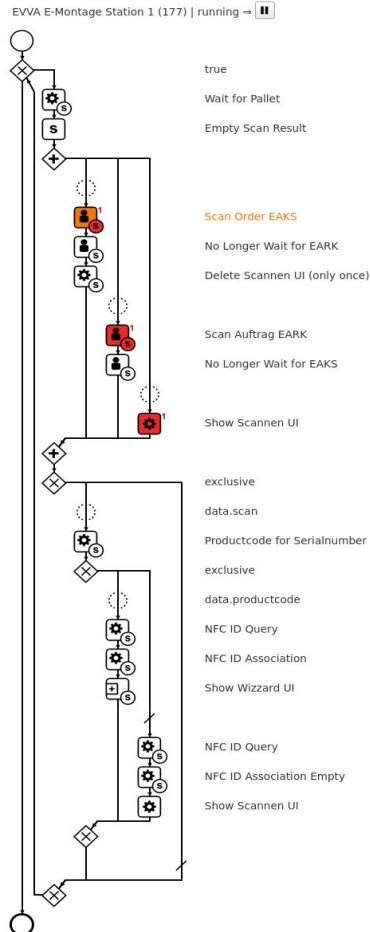
Anstehende Aufgaben:

Patientin	Tätigkeit	Bemerkung	Ausführen	Aktuelle Zeiten
Fr. Schiebe	OP-Bericht erstellen			
Fr. Schiebe	Klinische Entlassung			
Fr. Förster	Überprüfung der Befunde	ängstliche Patientin		
Fr. Hansen	Überprüfung der Befunde			

worklist (points to Einbestellungsplan GYN IV)

work items (points to Anstehende Aufgaben)

Station 1: Scan Packing Label & Equipping Work Piece Carriers



Station 1: Scan Packing Label & Equipping Work Piece Carriers



Follow the Instructions



Graph **Description** **Log**

true		
Wait for Pallet	a7	
Empty Scan Result	a11	
Scan Order EAKS	a20	
No Longer Wait for EARK	a5	
Delete Scannen UI (only once)	a10	
Scan Auftrag EARK	a3	
No Longer Wait for EAKS	a4	
Show Scannen UI	a9	
exclusive		
data.scan		
Productcode for Serialnumber	a8	
exclusive		
data.productcode		
NFC ID Query	a2	
NFC ID Associa		
Show Wizard		
NFC ID Query		
NFC ID Associa		
Show Scannen		

Graph **Description** **Log**

ID	⇒ a2	
Endpoint	⇒ put	
Properties		
Label	⇒ Set Image	
Method	⇒ :put	
Arguments		
num	⇒ !data.num	
total	⇒ !data.images.length	
style_url	⇒ https://centurio.work/customers/evva/wiz/style.css	
image_url	⇒ !File.join(endpoints.get.data.station_to_s,data.productcode.gsub(/\/:/,'_'),data.images[data.num-1])	
errors_url	⇒ https://centurio.work/customers/evva/wiz/error-reasons.xml	

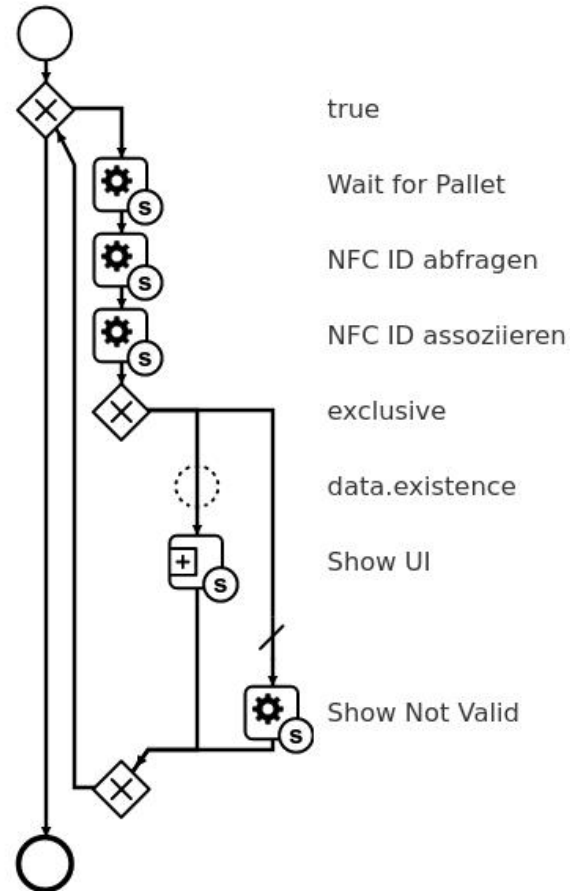


Station 2: Detect Work Piece Carrier, Show Associated Instructions

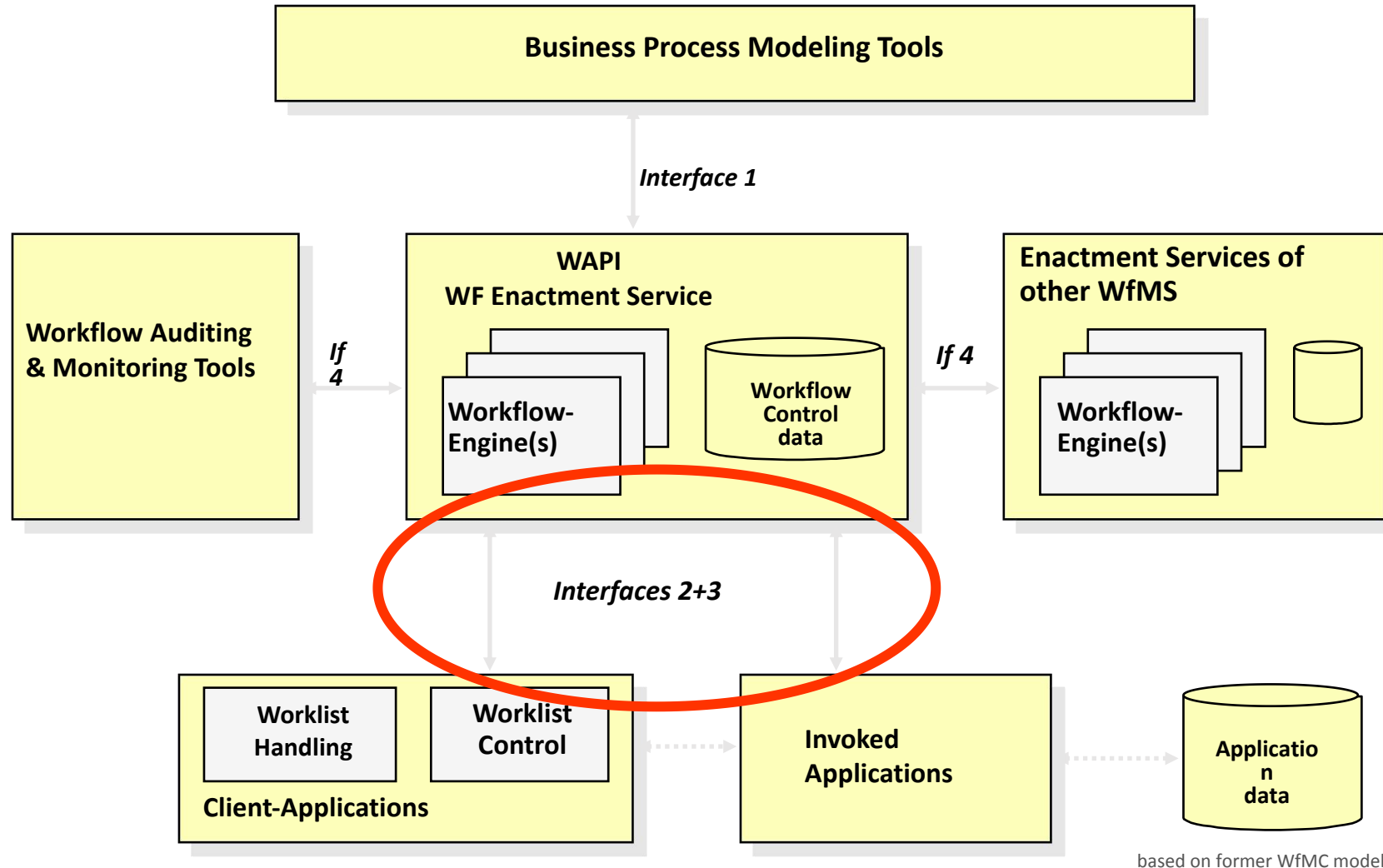


EVVA E-Montage Station 2 (960) | ready ⇒ ▶ / R / 🗑

Follow the Instructions



Workflow Reference Model



based on former WfMC model

Worklist Implementation



A Worklist consists of two parts

- Worklist control: manages available tasks for all process instances, knows.
- Worklist handling: user interface for actors to interact with information about available tasks, and for working on individual tasks.

Worklist control can be

- built into a process engine - monolithic process engines.
- be independent of a process engine - modular process engines.

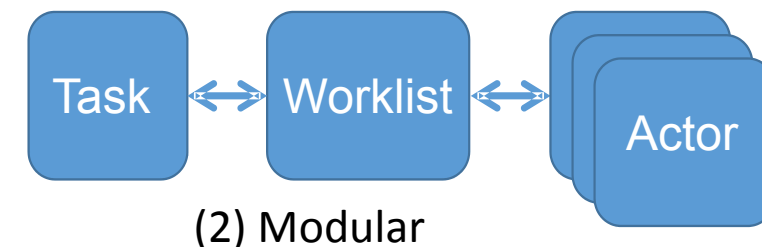
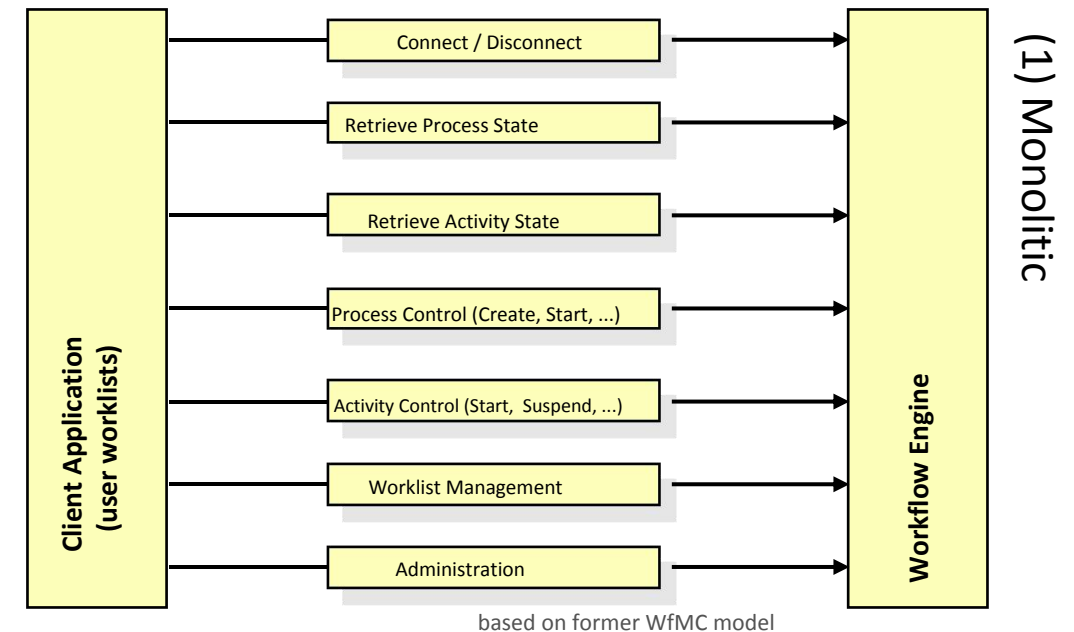
Worklist Implementation

Monolithic Process Engines with integrated worklist functionality:

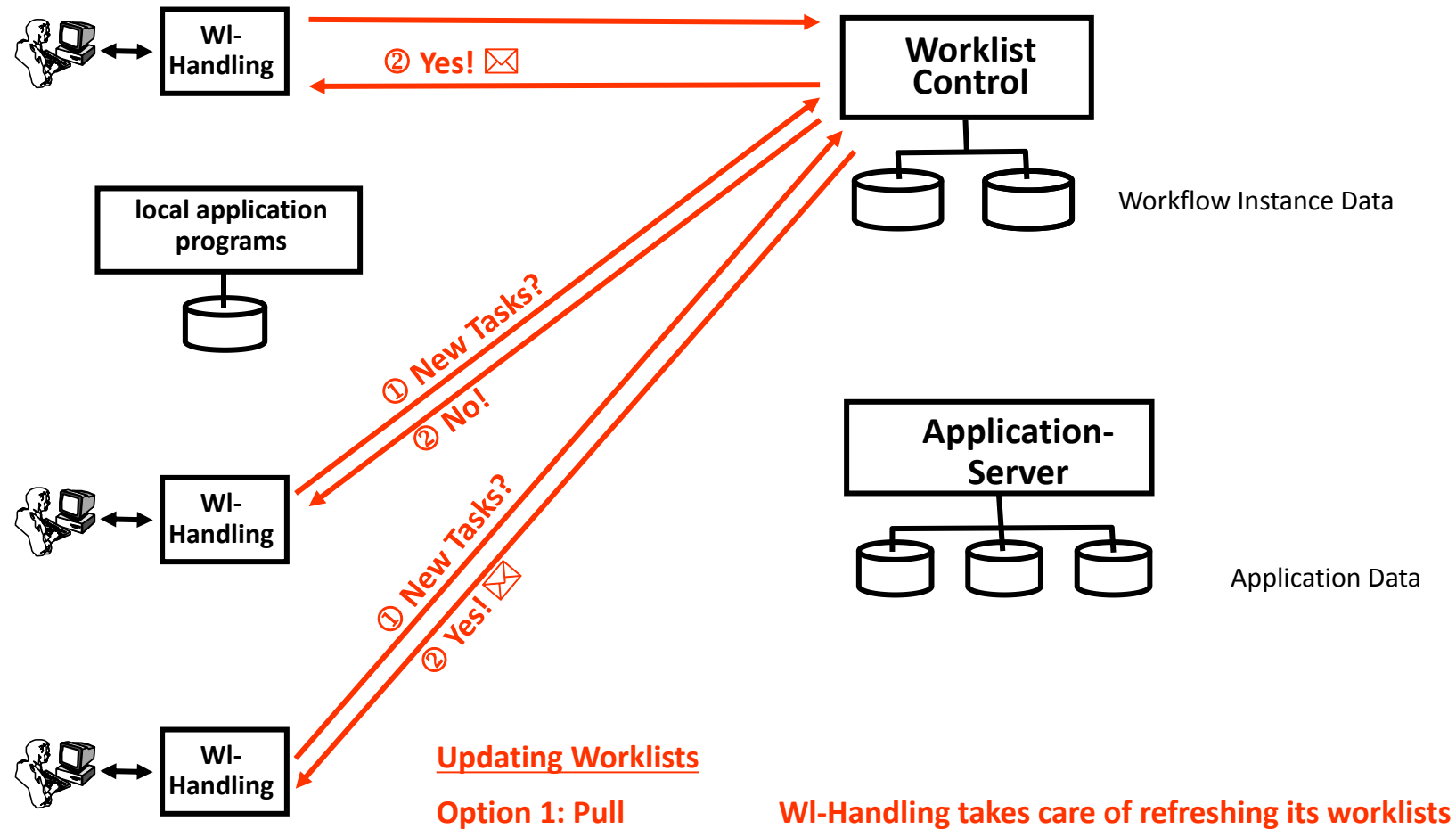
- Vendors provide **Application Programming Interface (API)** for implementing worklist clients (i.e., for accessing worklists)
- In most cases communication between client and server is **unilateral**; i.e., clients themselves are responsible for updating their worklists implementation efforts.
- Available APIs differ from vendor to vendor

Modular Process Engines:

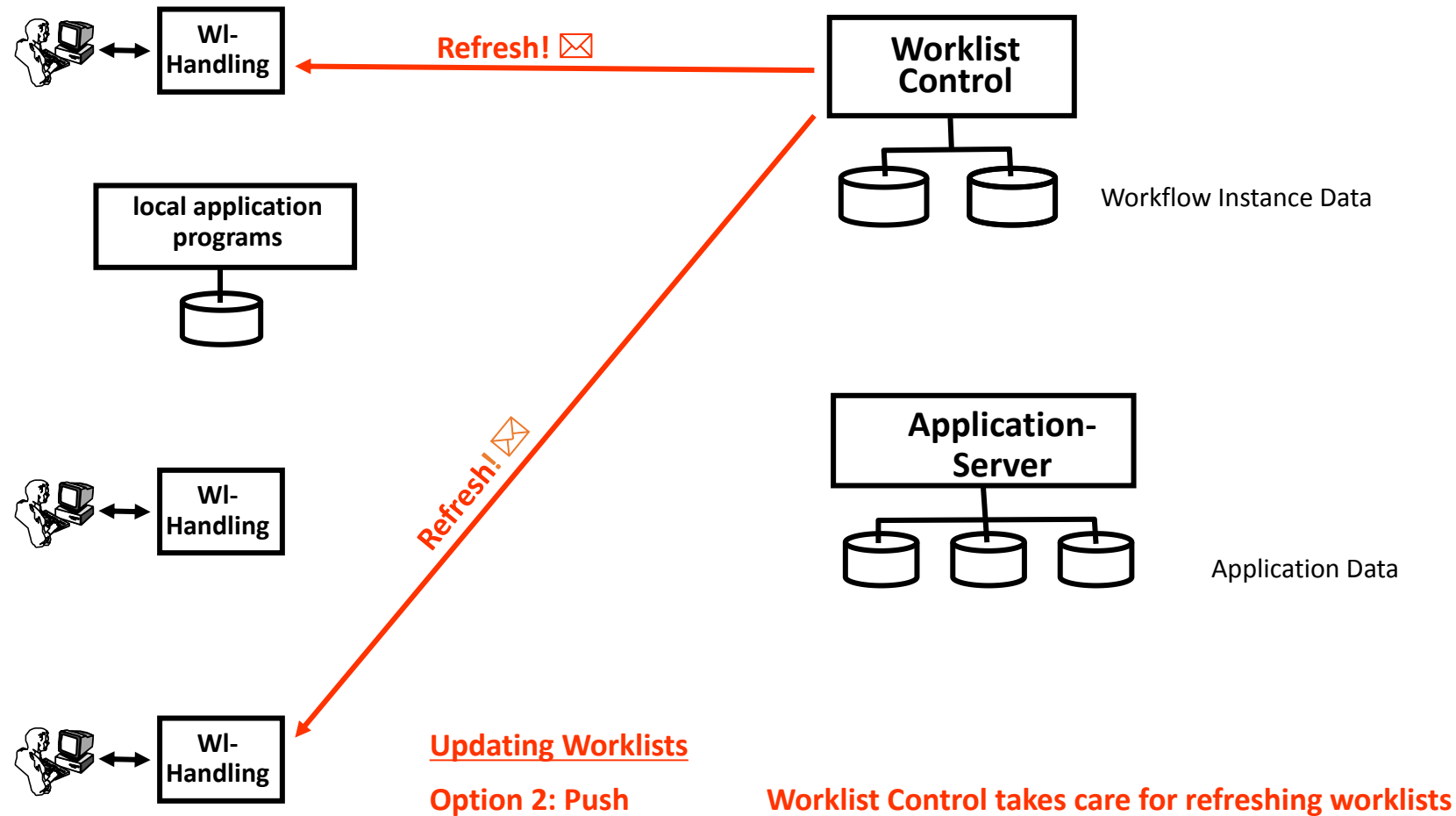
- Worklist is an external service like any other.
- Form the POV of the Process Engine, input data is sent, output data is received.
- All communication topics depicted in (1) are handled internally in the worklist.
- The worklist becomes vendor independent.



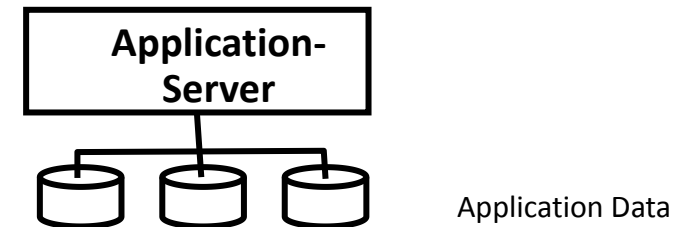
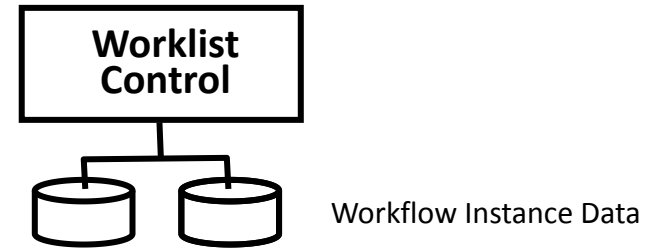
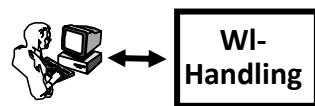
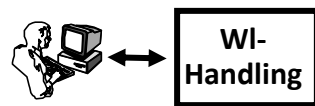
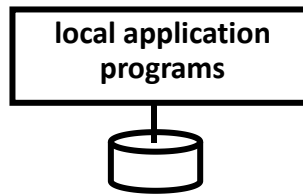
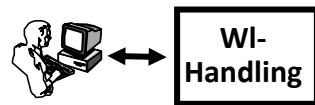
Worklist Implementation Variants



Worklist Implementation Variants



Worklist Implementation Variants



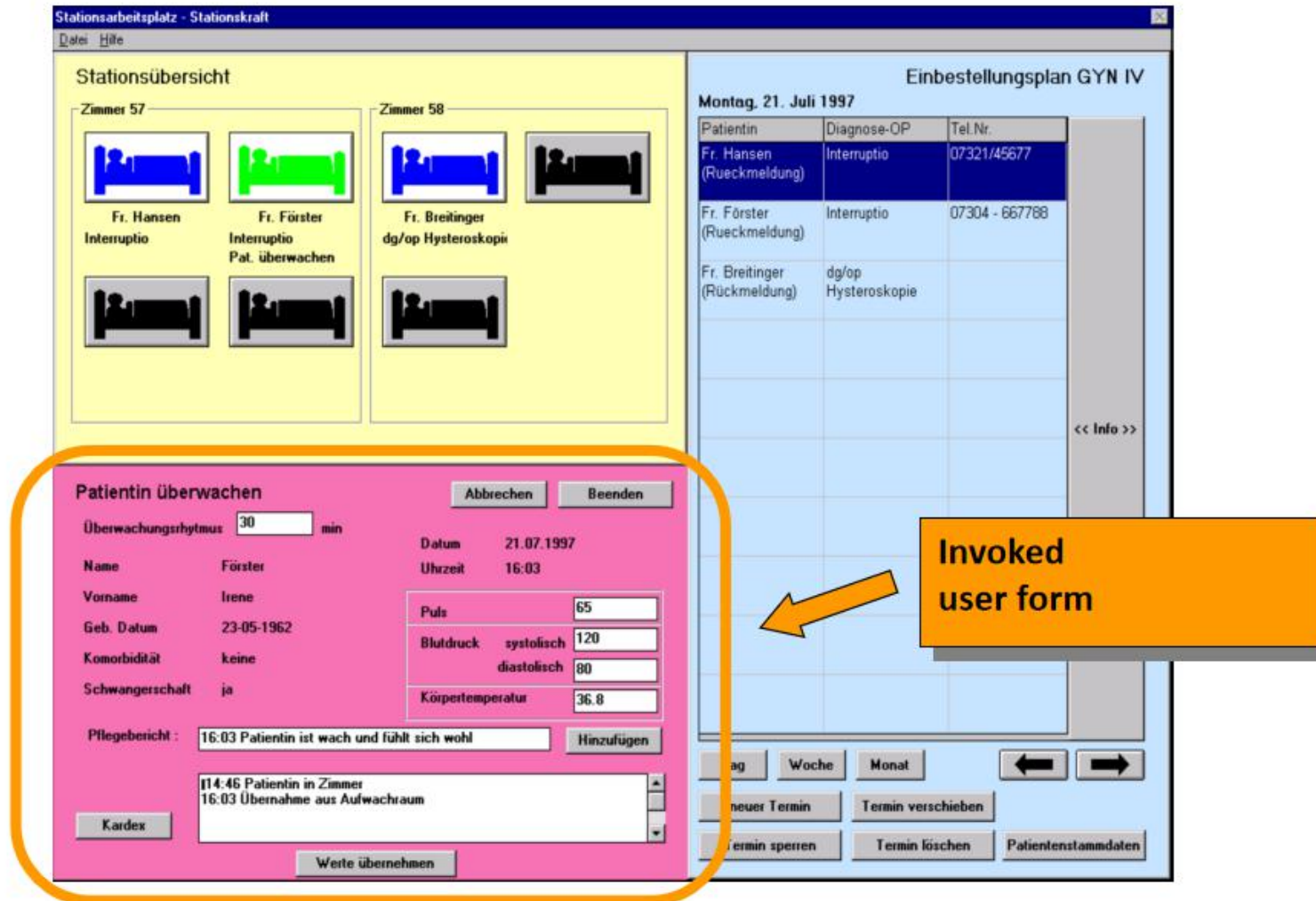
Updating Worklists (Evaluation)

Classical Pull: outdated worklists + unnecessary communication

Classical Push: potentially too much communication

Good compromise: time-driven push

Selection and Execution of a Work Item



The screenshot displays a medical station software interface titled "Stationsarbeitsplatz - Stationskraft". It is divided into two main sections: "Stationsübersicht" (Station Overview) and "Einbestellungsplan GYN IV" (Appointment Plan GYN IV).

Stationsübersicht: This section shows two rooms, Zimmer 57 and Zimmer 58, each with four patient beds. In Zimmer 57, the first two beds are occupied by "Fr. Hansen" (Interruptio) and "Fr. Förster" (Interruptio Pat. überwachen). In Zimmer 58, the first bed is occupied by "Fr. Breiting" (dg/op Hysteroskopie). The other beds in both rooms are empty.

Einbestellungsplan GYN IV: This section shows a table of appointments for Monday, 21. Juli 1997. The table has columns for Patientin, Diagnose-OP, and Tel. Nr.

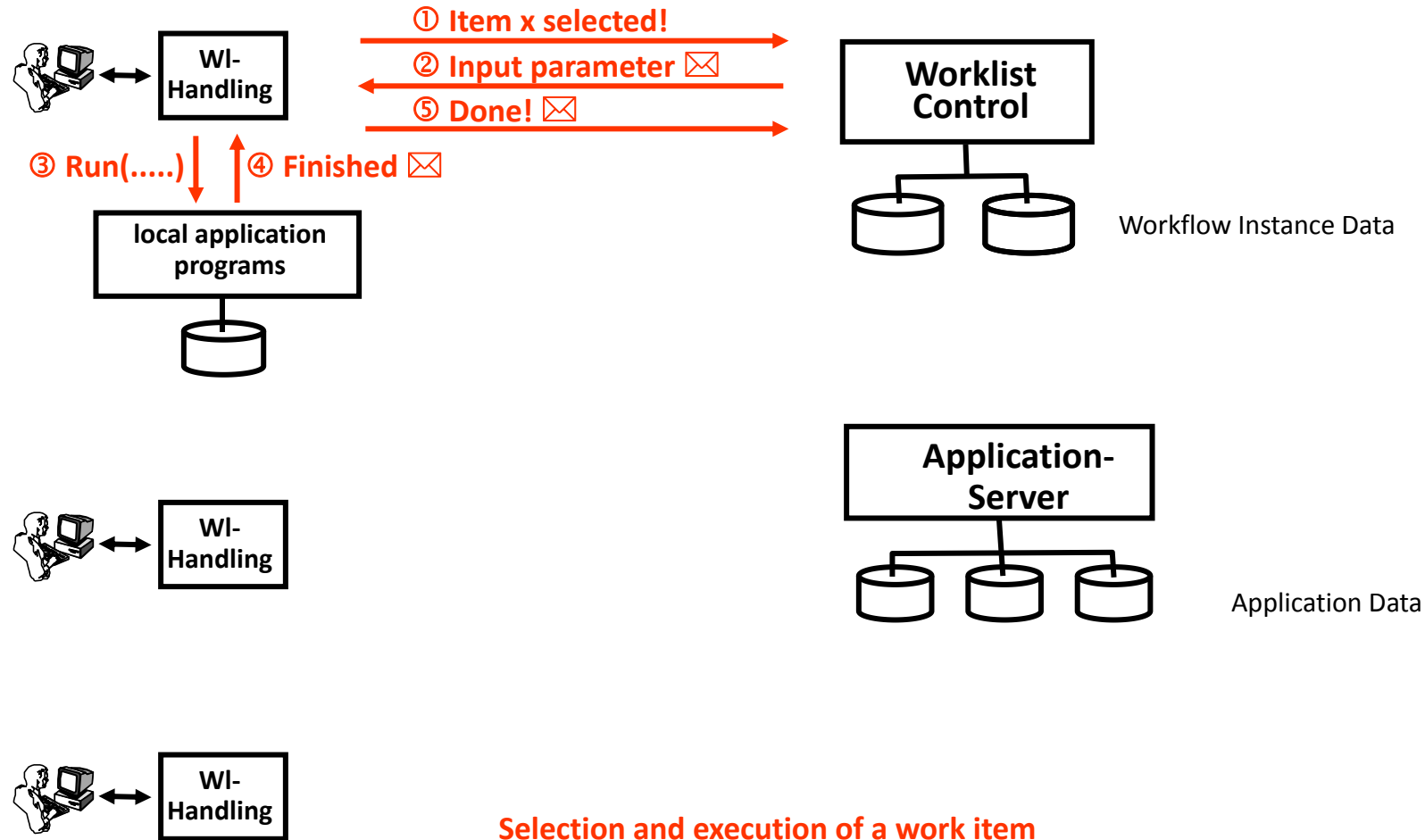
Patientin	Diagnose-OP	Tel. Nr.
Fr. Hansen (Rueckmeldung)	Interruptio	07321/45677
Fr. Förster (Rueckmeldung)	Interruptio	07304 - 667788
Fr. Breiting (Rückmeldung)	dg/op Hysteroskopie	

Patentin überwachen (Patient Monitoring): This form is highlighted with an orange border and an orange arrow pointing to it from a box labeled "Invoked user form". It contains the following fields:

- Überwachungsrhythmus: 30 min
- Datum: 21.07.1997
- Uhrzeit: 16.03
- Name: Förster
- Vorname: Irene
- Geb. Datum: 23-05-1962
- Komorbidität: keine
- Schwangerschaft: ja
- Puls: 65
- Blutdruck: systolisch 120, diastolisch 80
- Körpertemperatur: 36.8
- Pflegebericht: 16:03 Patientin ist wach und fühlt sich wohl
- Hinzufügen button
- Kardex button
- Werte übernehmen button

Buttons: The interface includes several buttons: "Abbrechen", "Beenden", "Hinzufügen", "Kardex", "Werte übernehmen", "Tag", "Woche", "Monat", "neuer Termin", "Termin verschieben", "Termin sperren", "Termin löschen", and "Patientenstammdaten".

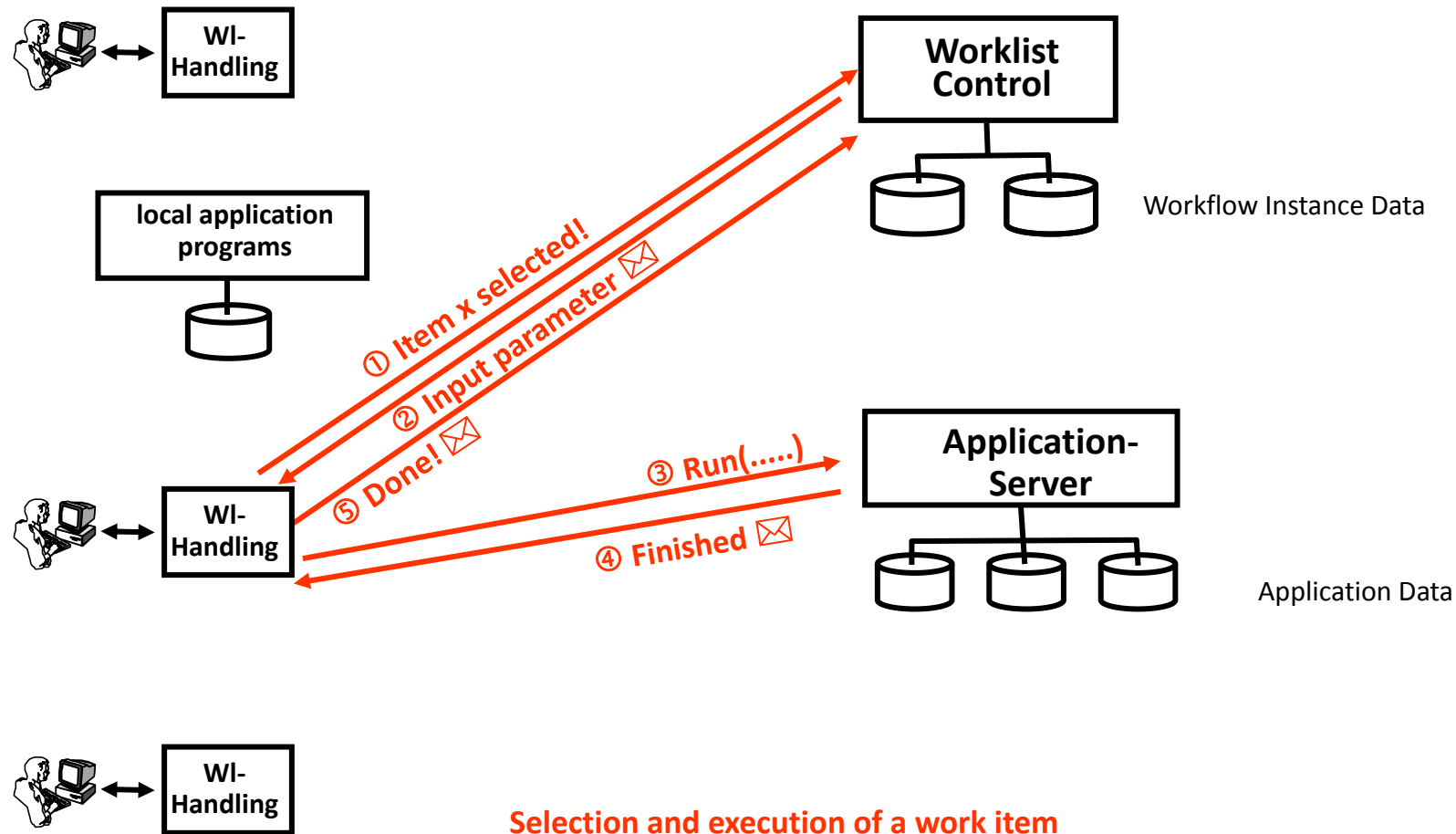
Selection and Execution of a Work Item



Selection and execution of a work item

Option 1: A local application component is invoked and executed!

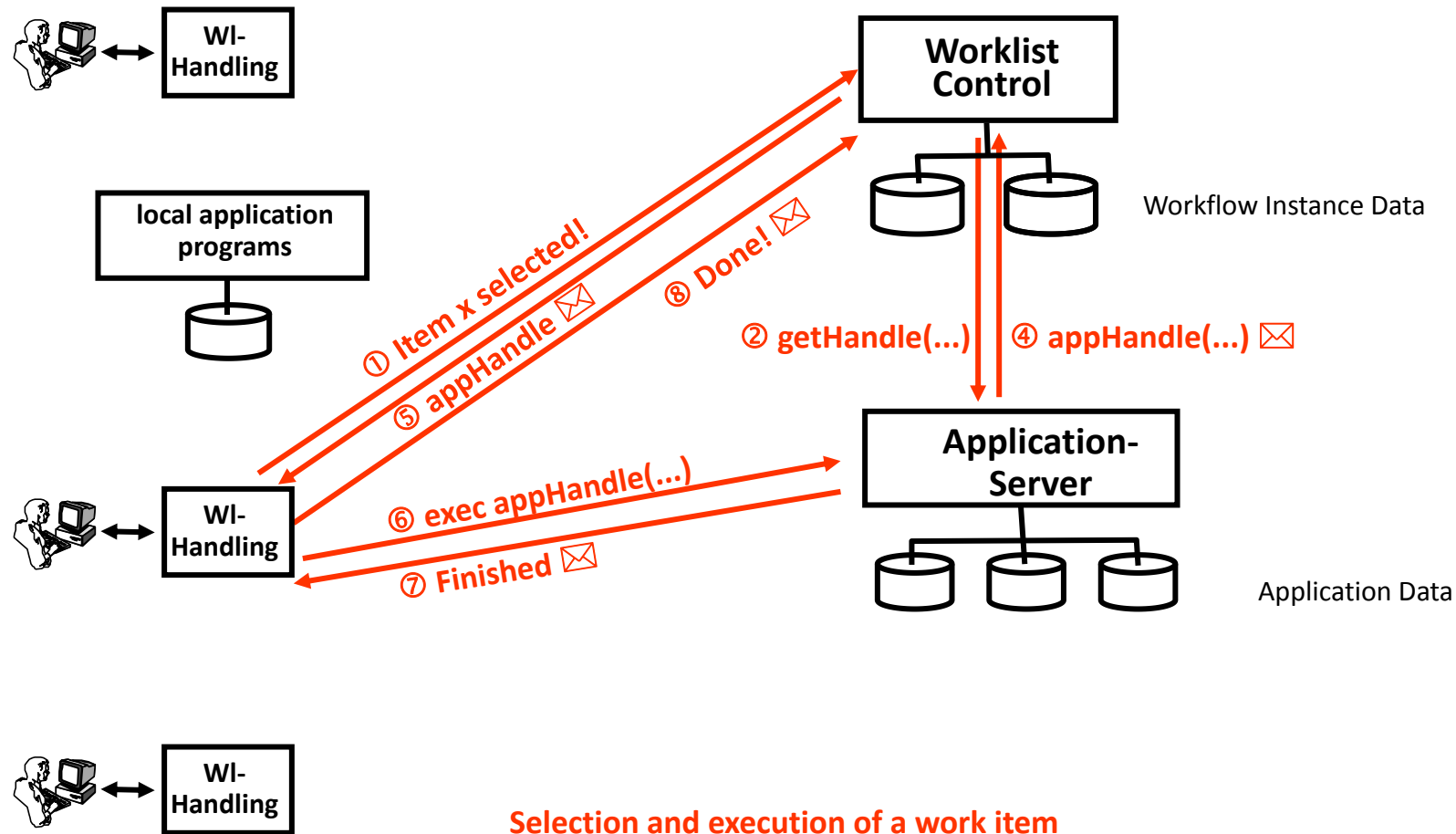
Selection and Execution of a Work Item



Selection and execution of a work item

Option 2: App.-Server – direct invocation by WI-Handling

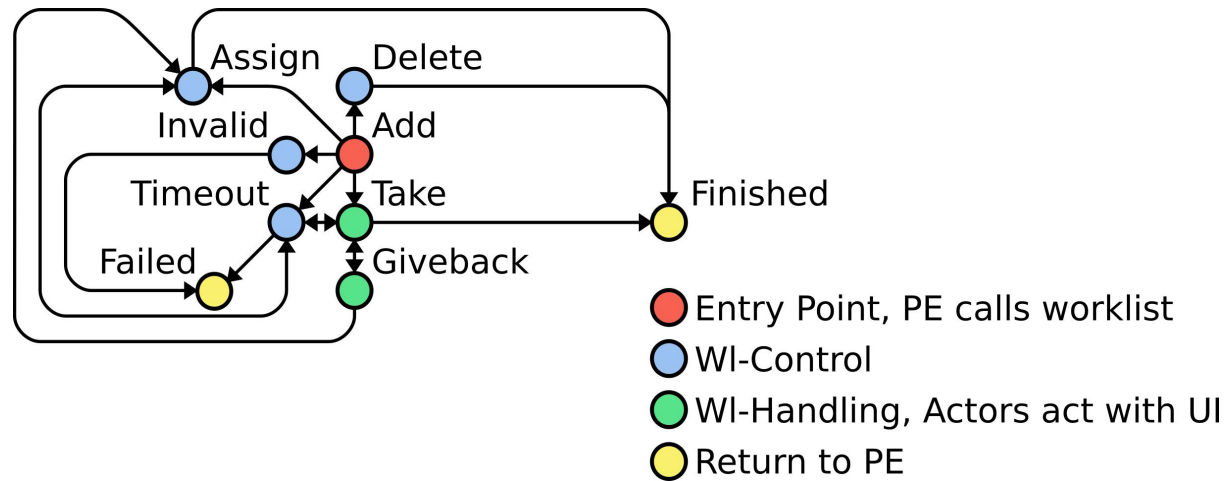
Selection and Execution of a Work Item



Selection and execution of a work item

Option 3: App.-Server – invocation prepared by WL-Handling

CPEE Worklist Lifecycle



- **Invalid:** the task is missing parameters. This leads to a failed state.
- **Delete:** the task is no longer necessary and thus deleted. Nonetheless a regular finish.
- **Failed:** control is returned to the PE.
- **Finished:** control is returned to the PE. Actor-generated data might be returned.

- **Add:** a process engine (PE) delegates work on a task - typical parameters include:
 - Organization model
 - Role
 - Subject - HTML Form, Word Document, Application to call,
 - Data elements (process context)
- **Take:** an actor can actively request the task
- **Giveback:** an actor can return the task to be available for other actors to be taken
- **Timeout:** a deadline was violated (see temporal aspect)
- **Assign:** the task is automatically assigned to an actor. Assign might use the following algorithms:
 - Round Robin
 - Lowest workload
 - Skill based
 - Customer based
 - ...

CPEE Worklist



The screenshot shows the CPEE Worklist interface in a web browser. The browser tab is 'Worklist (2035)' and the URL is 'https://cpee.org/flow/?monitor=https://cpee.org/flow/engine/2035/'. The interface has a top navigation bar with 'New', 'Instance', and 'Execution' tabs. Below this is a sub-navigation bar with 'Data Elements', 'Endpoints', and 'Attributes' tabs. The main area is divided into three sections: 'Graph', 'Description', and 'Log'. The 'Graph' section shows a task node labeled 'OK OR NOT OK' with ID 'a1'. The 'Description' section shows the task's properties and arguments. The 'Log' section is currently empty.

Task Configuration:

- ID ⇒ a1
- Endpoint ⇒ worklist
- Label ⇒ OK OR NOT OK
- Arguments:
 - Organisation Model ⇒ https://cpee.org/~demo/orgviz/organisation_int
 - Domain ⇒ VirtualBusiness1
 - Form Link ⇒ https://cpee.org/~demo/form/form-f.html
 - Role ⇒ Assistant
- Data Elements:
 - schaden ⇒ 10000
 - text ⇒ destroyed

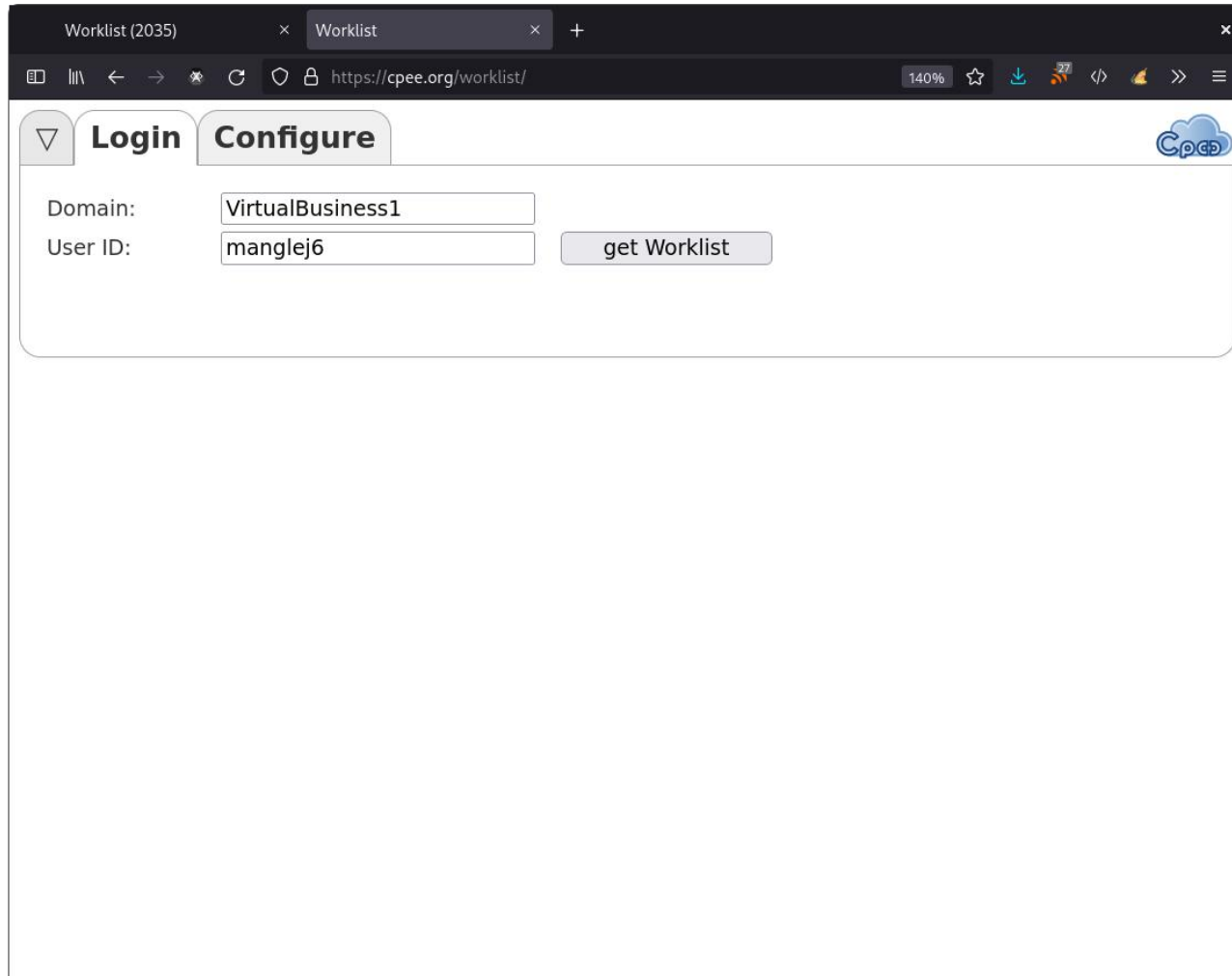
Simple sample process.

Task has parameters (see above):

- Endpoint: location of the worklist
- Organisation model
- Form Link: HTML form to be shown
- Role: group of actors that are authorized to work on the task
- Data elements to be passed to the worklist

The data elements should all information required by actors to work on the task. The actors will get not other information to work on the task (although they can use backend systems that are not known to the process engine).

CPEE Worklist



The screenshot shows a web browser window with the address bar displaying `https://cpee.org/worklist/`. The page has two tabs: "Worklist (2035)" and "Worklist". The main content area features a "Login" tab and a "Configure" tab. Under the "Login" tab, there are two input fields: "Domain:" with the value "VirtualBusiness1" and "User ID:" with the value "manglej6". To the right of these fields is a button labeled "get Worklist". A small CPEE logo is visible in the top right corner of the page.

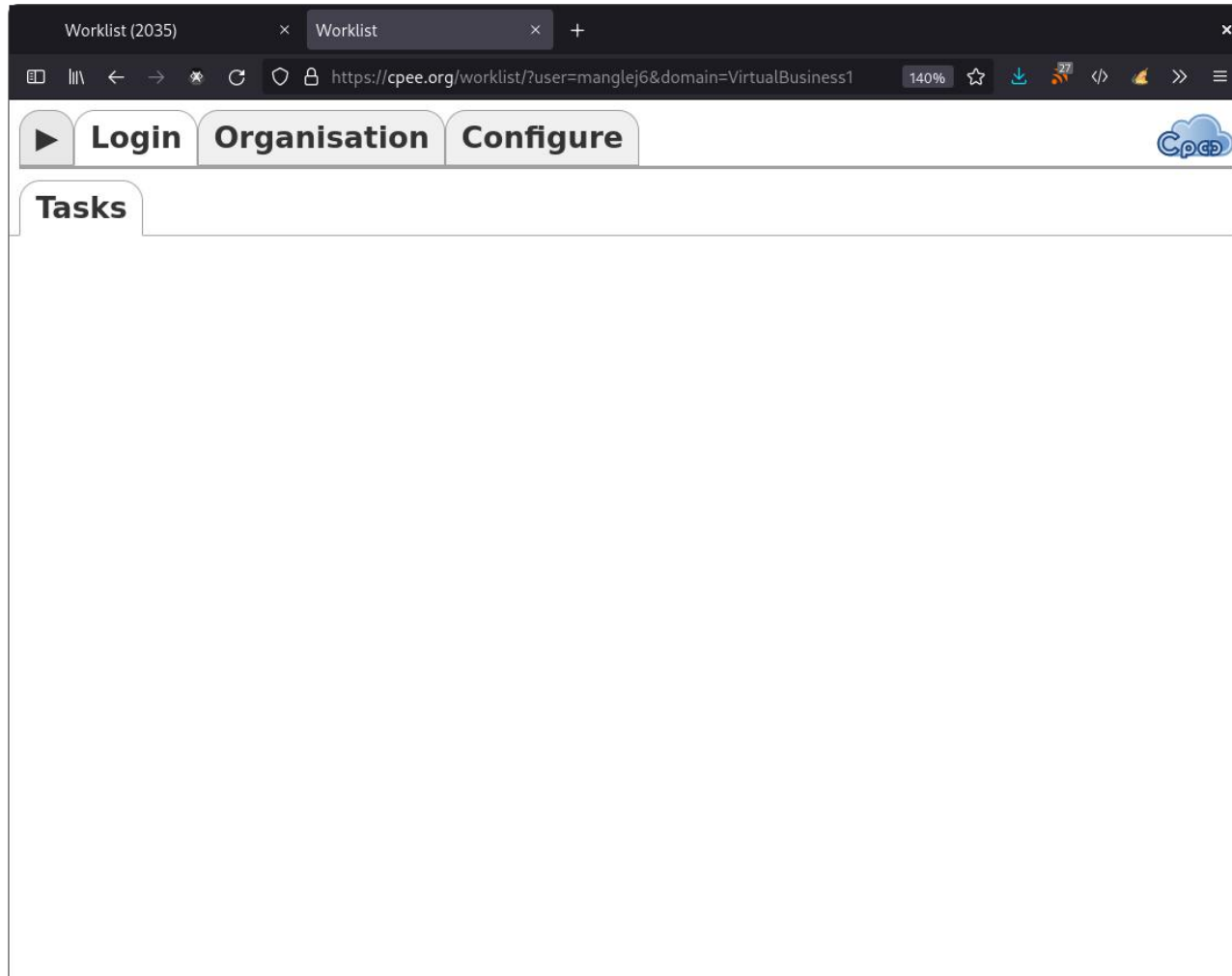
Actors have a special UI. They never see process engine UI.

The have to log in (user manglej6).

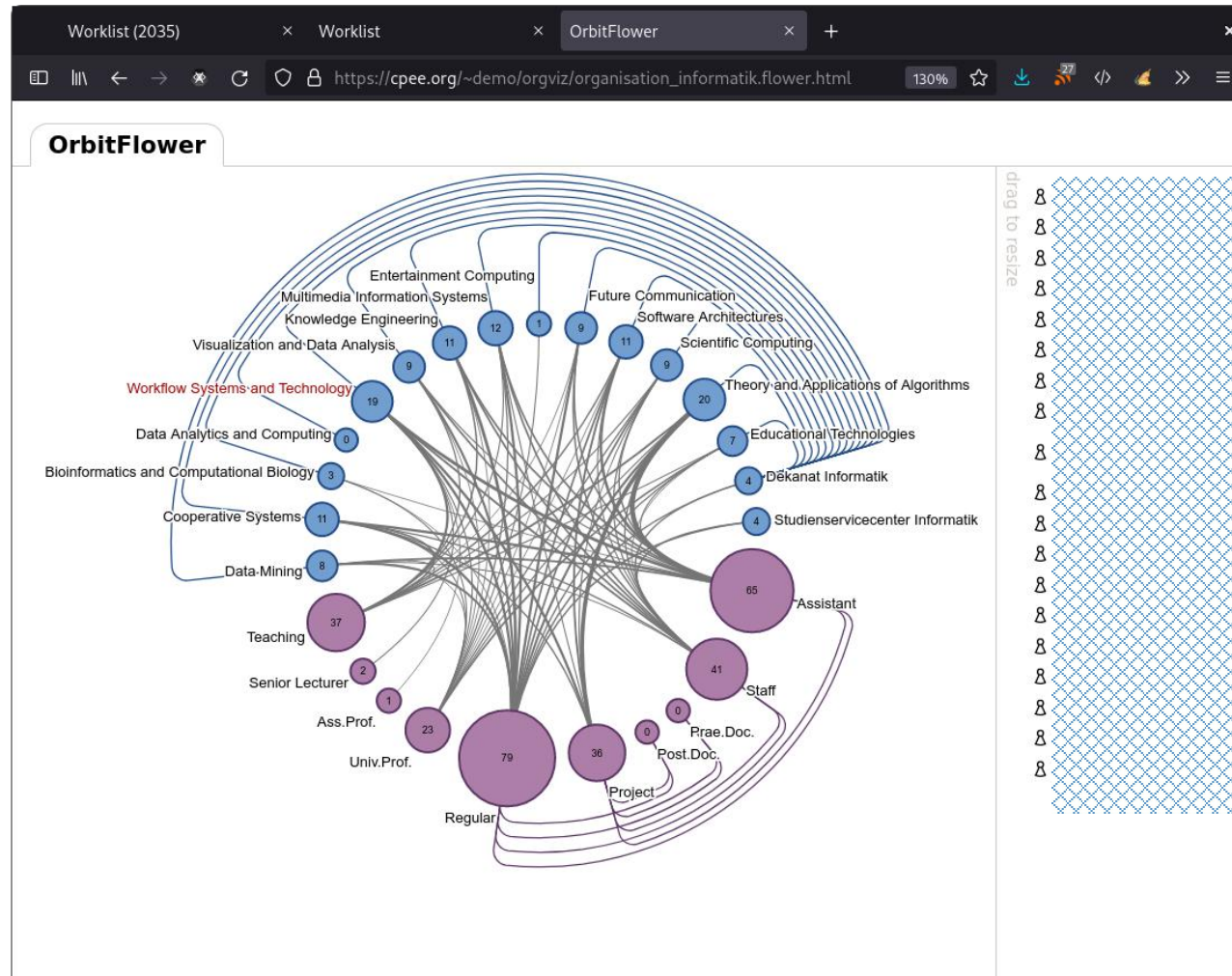
CPEE Worklist



The worklist is empty.



CPEE Worklist



The actor is part of an organisation, is assigned to a unit and has a role.

further reading OrbitFlower:
Simone Kriglstein, Juergen Mangler, Stefanie Rinderle-Ma: Who is who: On visualizing organizational models in Collaborative Systems. CollaborateCom 2012: 279-288

CPEE Worklist

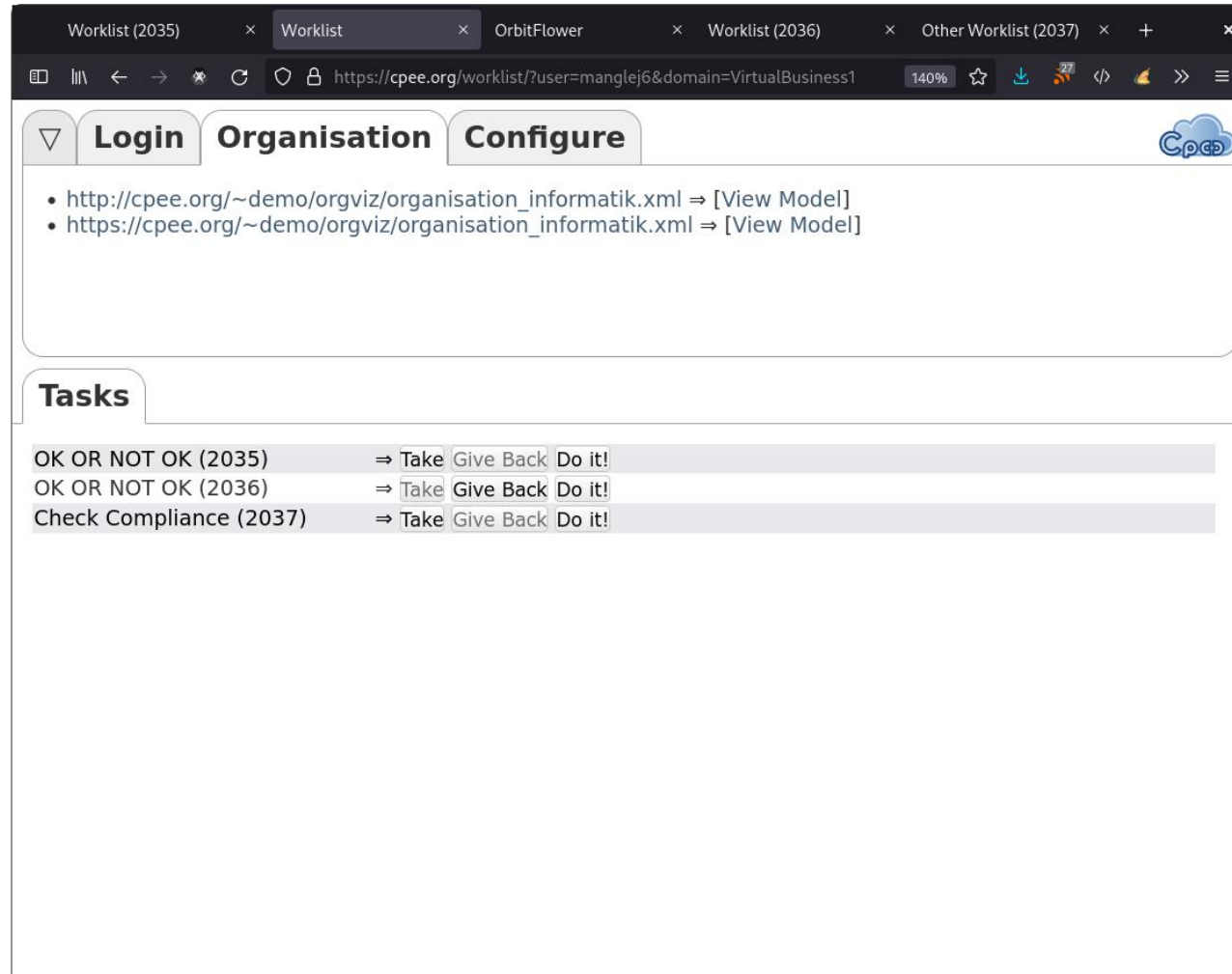


The screenshot displays the CPEE Worklist web application. The browser tabs include 'Worklist (2035)', 'Worklist', and 'OrbitFlower'. The address bar shows the URL: <https://cpee.org/flow/?monitor=https://cpee.org/flow/engine/2035/>. The interface has three main tabs: 'New', 'Instance', and 'Execution'. The 'Instance' tab is active, showing a 'State: running' with a 'Stop' button. Below this are tabs for 'Data Elements', 'Endpoints', and 'Attributes'. The 'Graph' tab is selected, showing a process diagram with a task labeled 'OK OR NOT OK' with ID 'a1'. The 'Log' tab is also visible. The 'Properties' section on the right lists the following details:

- ID ⇒ a1
- Endpoint ⇒ worklist
- Label ⇒ OK OR NOT OK
- Arguments:
 - Organisation Model ⇒ https://cpee.org/~demo/orgviz/organisation_int
 - Domain ⇒ VirtualBusiness1
 - Form Link ⇒ <https://cpee.org/~demo/form/form-f.html>
 - Role ⇒ Assistant

The process instance is running.

CPEE Worklist



The task shows up in the actors UI.

Also:

- A second instance has been started.
- A task from a very different process also shows up.

All tasks have in common: the actor role is authorized to work on them.

Tasks can:

- Taken
- Given back
- Done (which automatically takes them).

If an actor takes a task, it becomes invisible for all other actors. If it is given back it becomes visible again.

CPEE Worklist



The screenshot shows a web browser with multiple tabs. The active tab is 'Worklist' at the URL `https://cpee.org/worklist/?user=manglej6&domain=VirtualBusiness1`. The page has a navigation bar with 'Login', 'Organisation', and 'Configure' tabs. Below 'Organisation', there are two links: `http://cpee.org/~demo/orgviz/organisation_informatik.xml` and `https://cpee.org/~demo/orgviz/organisation_informatik.xml`, both pointing to '[View Model]'. The 'Tasks' tab is active, showing a task titled 'OK OR NOT OK (2036)'. The task description states: 'The overall cost of repair and retrieval of the vehicle is estimated 10000€.' and 'The current state of the car can be summed up as: destroyed.' Below this, there are radio buttons for 'OK' (selected) and 'Not OK'. A 'Submit' button is at the bottom.

The actor works on a taken task.

The UI passed by the PE is shown inline.

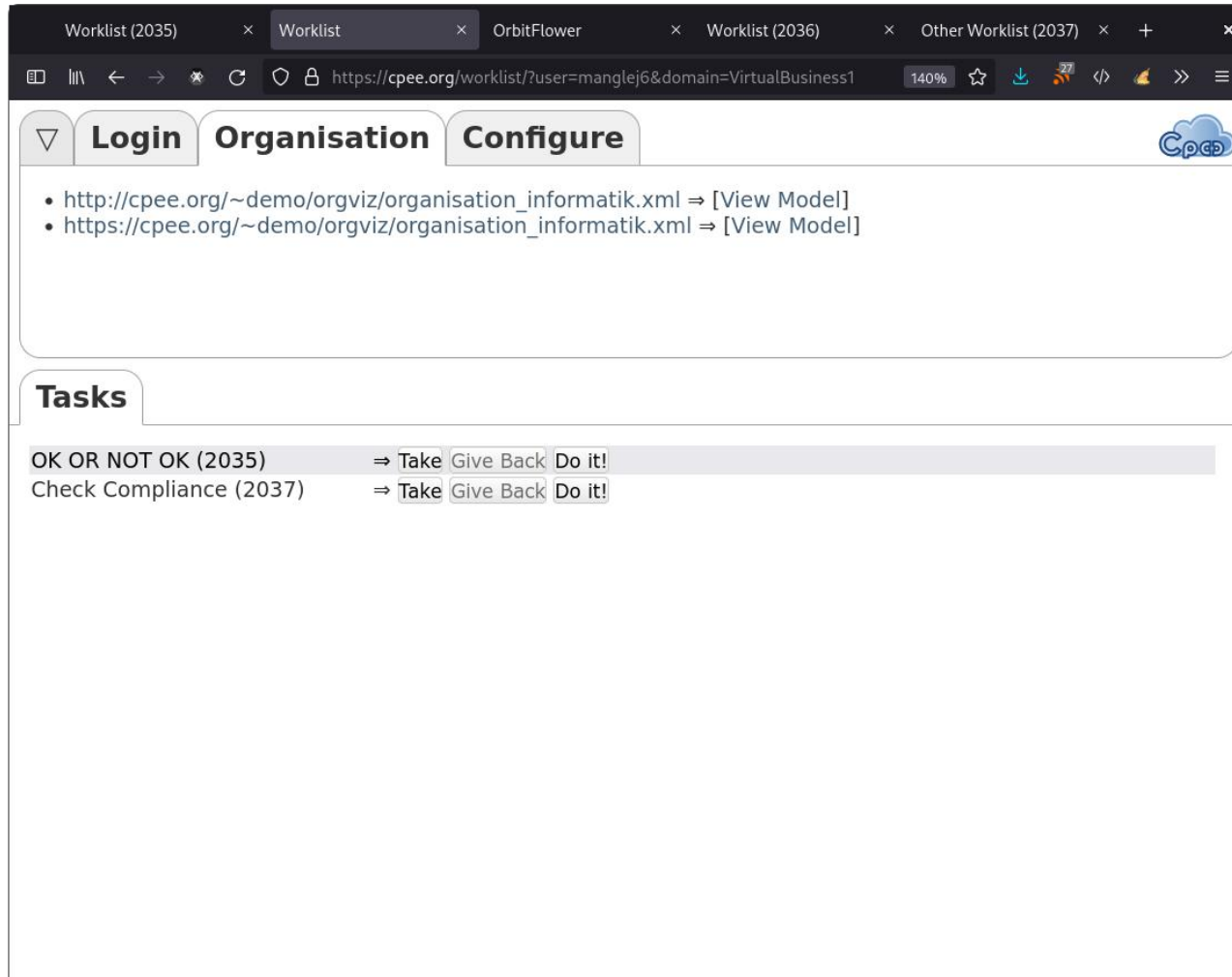
The data elements passed by the PE are used to customize the UI:

- The value in the data element “schaden” is copied into the UI.
- The value in the data element “text” is copied into the UI.

CPEE Worklist



The actor finished work on the task, it vanishes from the UI.



CPEE Worklist



The screenshot displays the CPEE Worklist interface in a web browser. The browser's address bar shows the URL `https://cpee.org/flow/?monitor=https://cpee.org/flow/engine/2036/`. The interface has a dark-themed header with tabs for 'Worklist (2035)', 'Worklist', 'OrbitFlow', 'Worklist (2036)', and 'Other Worklist (2037)'. The 'Worklist (2036)' tab is active, showing a process instance '2036|P|S|C' with a CPEE logo. Below the header, there are three main sections: 'New', 'Instance', and 'Execution'. The 'Instance' section is selected, showing a 'Debugging' checkbox (unchecked), a 'State' of 'finished', and a 'Continue' button. Below this, there are three tabs: 'Data Elements', 'Endpoints', and 'Attributes'. The 'Data Elements' tab is selected, showing a list of data elements with a 'New' button, a 'bla' element, and a '⇒ ok' button. Below the data elements, there are three tabs: 'Graph', 'Description', and 'Log'. The 'Graph' tab is selected, showing a process flow diagram with a task 'OK OR NOT OK' and a data element 'a1'. The 'Description' and 'Log' tabs are empty. The interface also includes a 'drag to resize' label and a 'T|G' button.

The corresponding process instance is finished. The task has received the data from the worklist, and saved it into a data element “bla”.

Summary



- Process implementation means to transform your process model into executable code.
- Hence, all execution aspects have to be taken care of.
- More aspects:
 - Process evolution and change
 - Interorganizational process execution

Business Process Technologies and Management

4. Interorganizational Processes

Prof. Dr. Stefanie Rinderle-Ma

Dr. Jürgen Mangler

Technical University of Munich

Department of Informatics

Chair of Information Systems

and Business Process Management

stefanie.rinderle-ma@tum.de

juergen.mangler@tum.de



Literature and links

- Weske, M. (2019) Business Process Management - Concepts, Languages, Architectures, Third Edition. Springer 2019, ISBN 978-3-662-59431-5, pp. 1-417 2019
- <https://www.signavio.com/bpm-academic-initiative/>
- <https://cpee.org/>

Teaching Objectives

In this chapter we want to

- introduce the notion of process choreographies
- distinguish process choreographies from process orchestrations
- show how to model process choreographies
- introduce notions of correctness for process choreographies
- introduce verification techniques for process choreographies

Introduction

- *Process Orchestration*: One (executable) business process from the perspective and under the control of one particular partner (endpoint)
- *Process Choreography*: Exchange of messages between partners according to defined interaction rules between two or more partners; also called global process.
- Examples for process choreographies:
 - healthcare
 - blockchain-based processes
 - logistics scenarios
 - supply chains

Process Choreography Design

1. High-level Structure Design

- Identifying participant roles and their communication structure
- Conducted during the *participant identification phase*

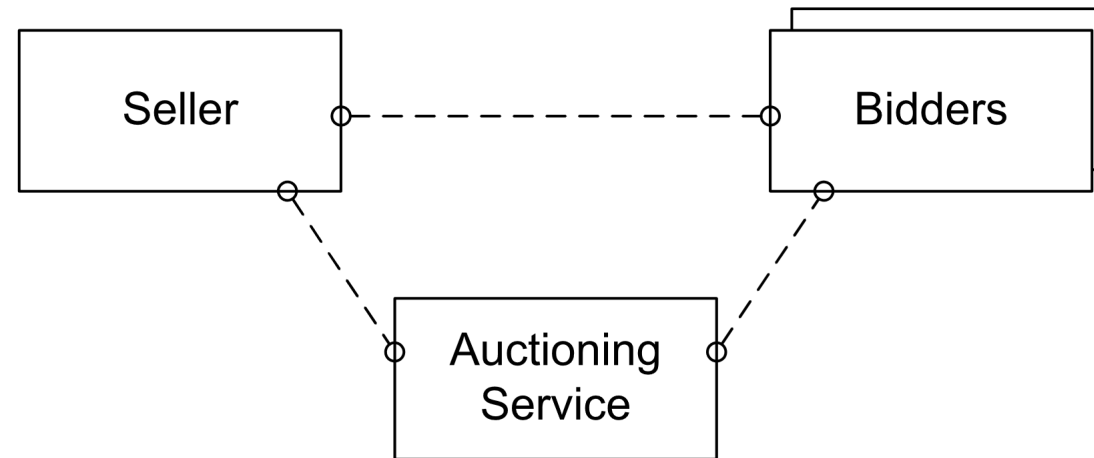


Fig. 6.5. High-level structural model of participants in bidding scenario

M. Weske: Business Process Management, Third Edition
© Springer-Verlag 2019, 2012, 2007

Process Choreography Design

2. High-level Behavioral Design

- Specifying the milestones of the collaboration and the order in which they are reached
- Conducted during the *milestone definition phase*

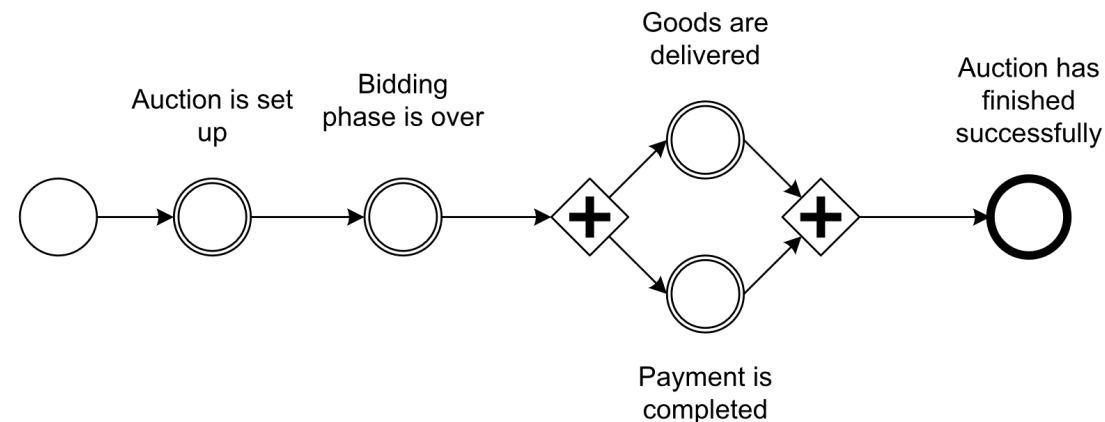


Fig. 6.6. High-level behavioural model for bidding scenario, represented by milestones

Process Choreography Design

- A certain milestone might be not reached in a certain conversation
- This situation occurs in the bidding scenario, for example, if no single bid is placed during the acution!
- High-level behavioral model for *bidding scenario* with different outcomes

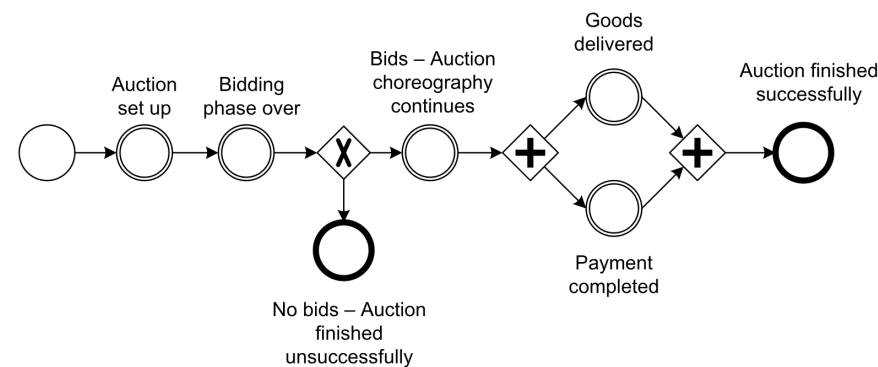


Fig. 6.7. High-level behavioural model for bidding scenario, with different outcomes

3. Collaboration Scenarios

- Refining high-level choreographies by introducing dedicated collaboration scenarios that relate the reaching of milestones to the communication between participating roles
- Developed in the *choreography definition phase*, based on scenarios informally specified during *scenario modelling*

Process Choreography Design

- **Collaboration scenario:** reaching milestones through interactions (i.e., message exchanges)

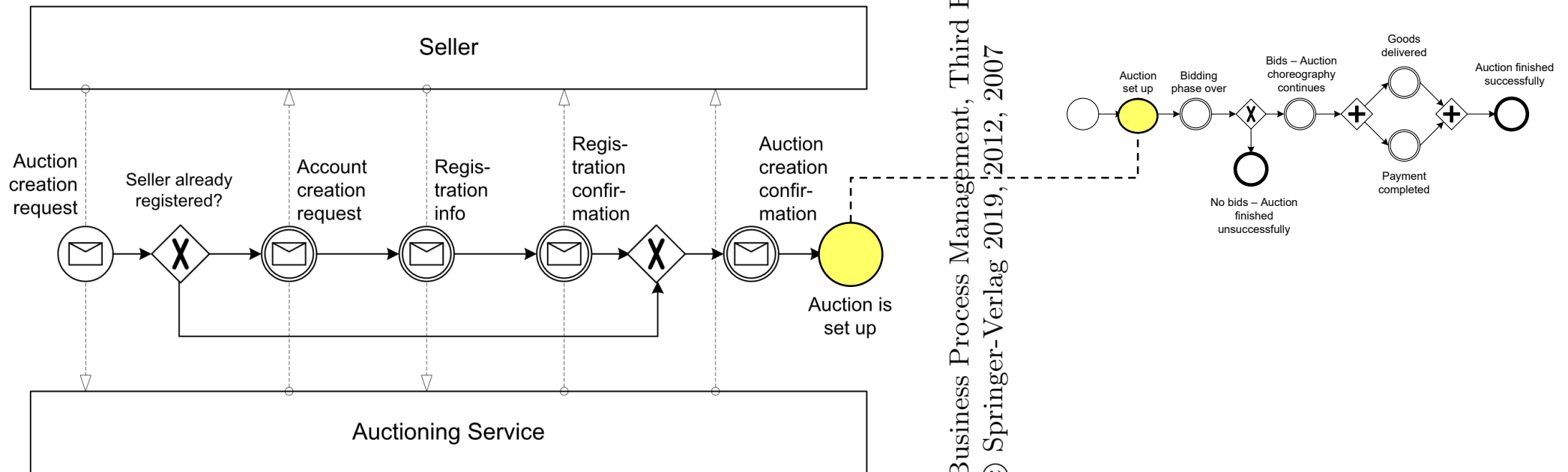


Fig. 6.8. Collaboration scenario: reaching milestones through interactions

Process Choreography Design

4. Behavioral Interfaces

- Deriving a behavioral interface for each participant role from the collaboration scenarios

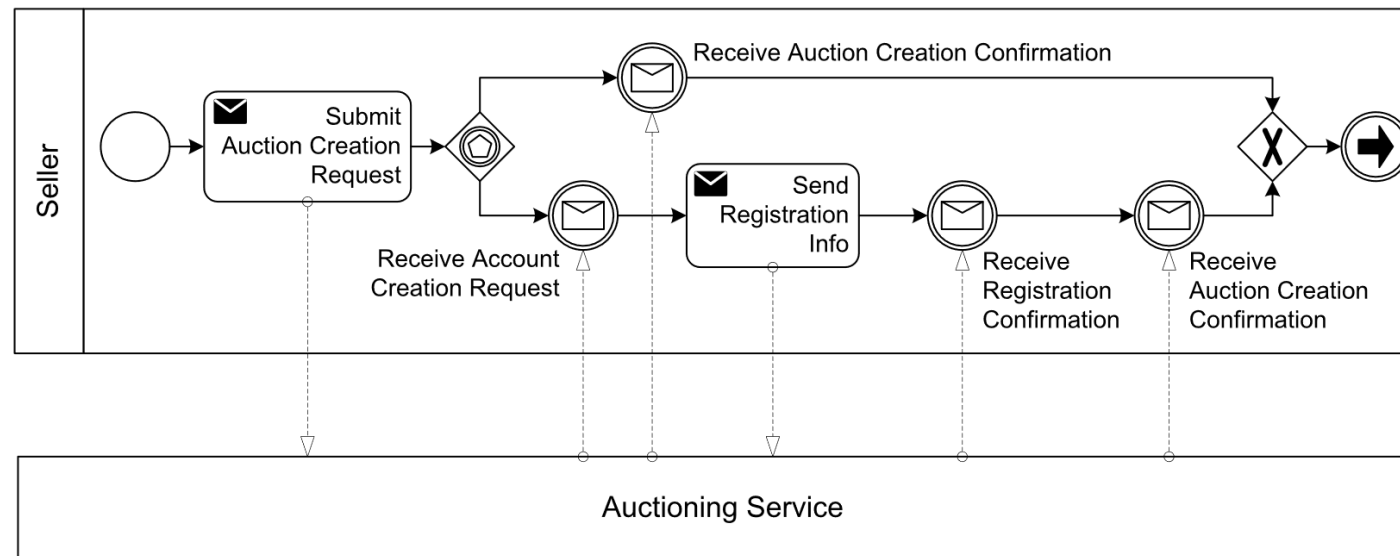


Fig. 6.9. Behavioural interface for seller

Process Choreography Design

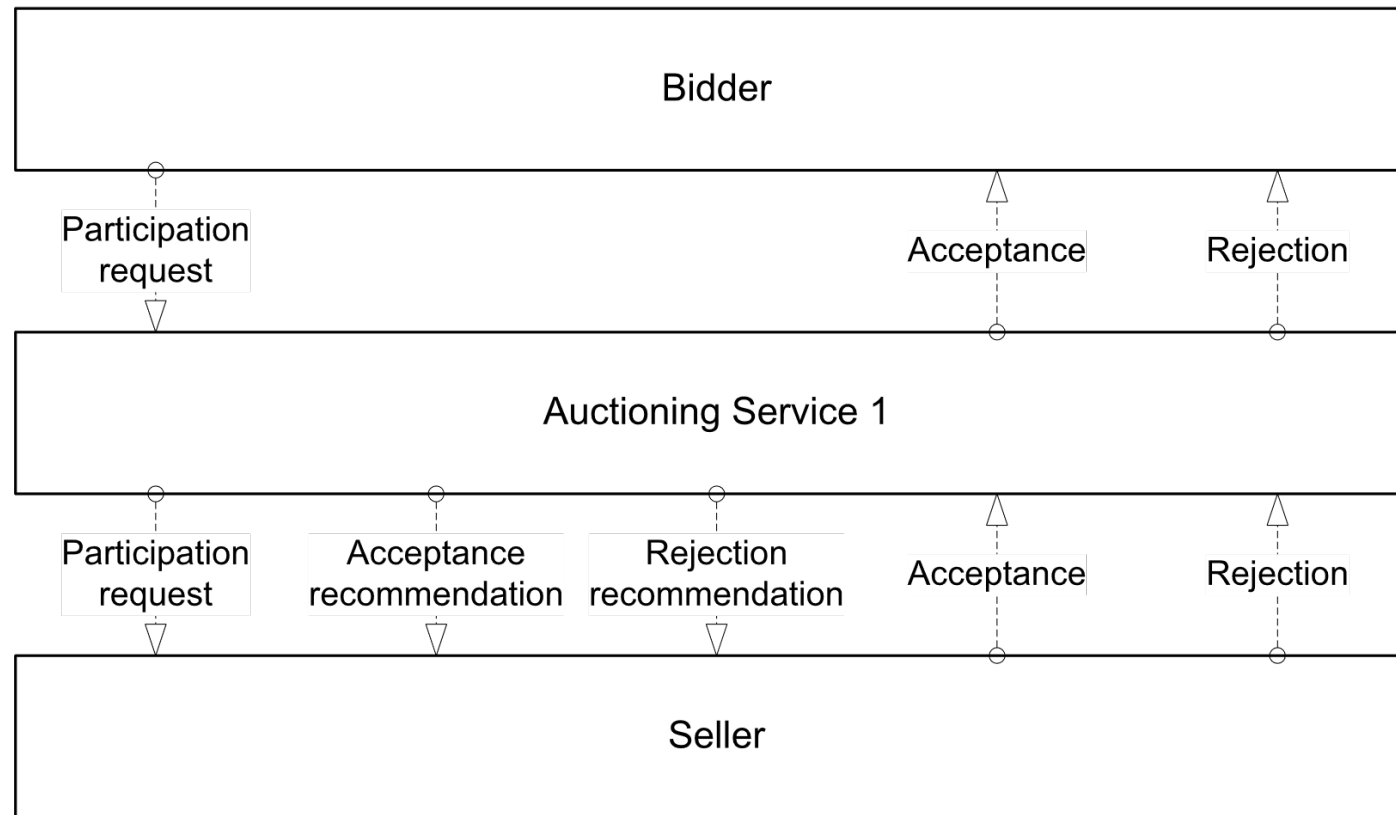
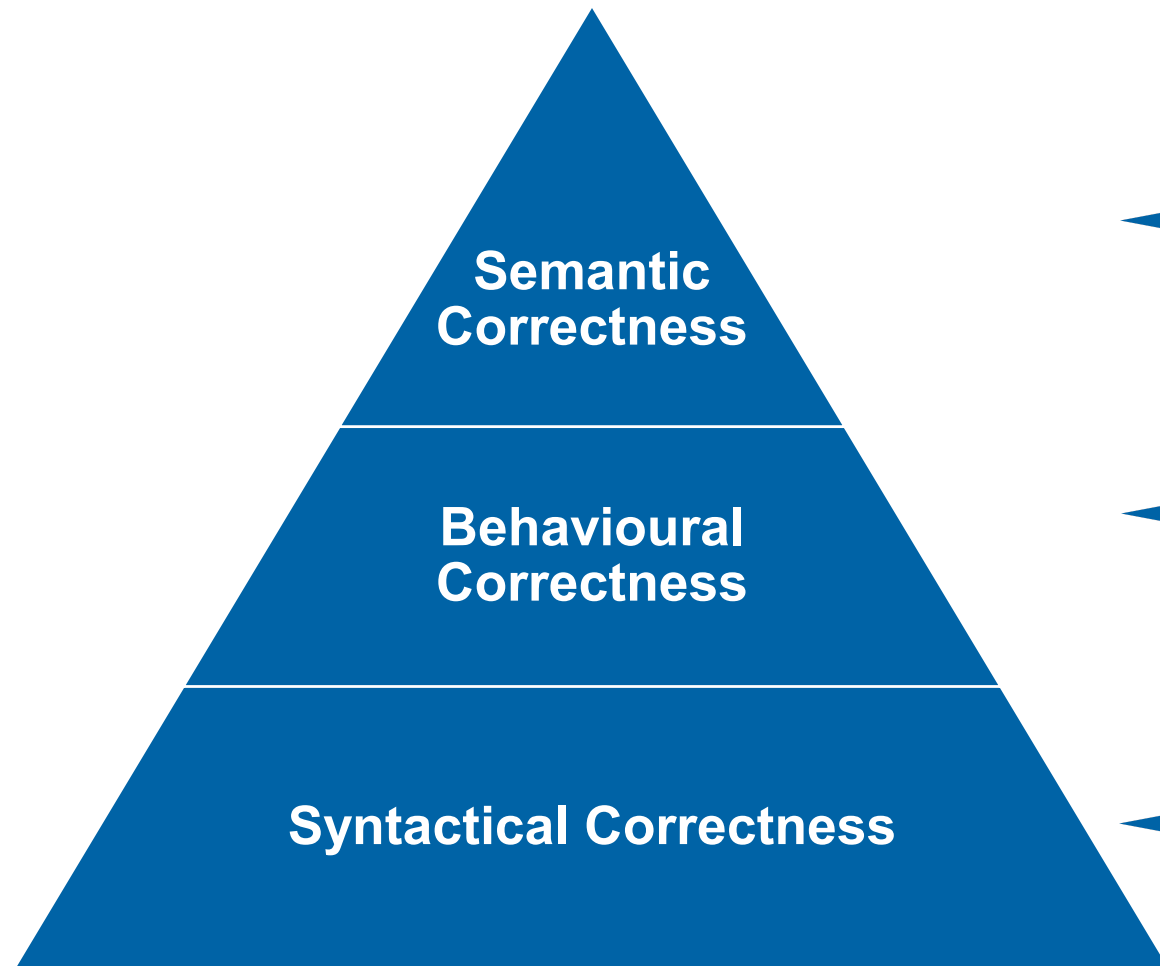


Fig. 6.10. Interactions of participants in auctioning scenario

Process Choreography Correctness



The process models...

...must **comply** with imposed local/global **compliance rules**.

...must be **executable** and the composition of the **public models** is **compatible** as well as that the **private models** are **consistent** with the **corresponding public model**.

...must ensures the **correct use** and **composition** of the corresponding **model elements**. This is defined by the underlying **BPMN meta model**.

Compatibility



- The design of a process choreography needs to ensure that the process orchestrations of the participants play together well in the overall collaboration
- **Compatibility** → Ability of a set of participants to interact successfully according to a given process choreography!
- Sources of incompatibility:
 - Different messages are used in a collaboration and one participant does not understand the content of a message sent by another participant
 - Wrong or misaligned interactions; e.g., if a participant expects a notification at some point in its process before it can proceed and none of the other participants sends such notification message → **DEADLOCK**
- Compatibility of interacting processes aims at avoiding such undesired behavior; i.e., to exclude erroneous interactions between orchestrations

Example



- Interactions between participants in auctioning scenario
- A potential bidder must be accepted for participation before she can place her bid.
- The bidder needs to send a Participation request to the auctioning service
- As response the latter can send an Acceptance notification or a Rejection notification
- In some cases the seller is requested to make the final decision on whether or not a bidder shall be accepted.
- To perform this interaction, the auctioning service forwards the request of the bidder to the seller
- It might also give a recommendation for accepting / rejecting the bidder
- The seller can send a notification about his decision back to the auctioning service

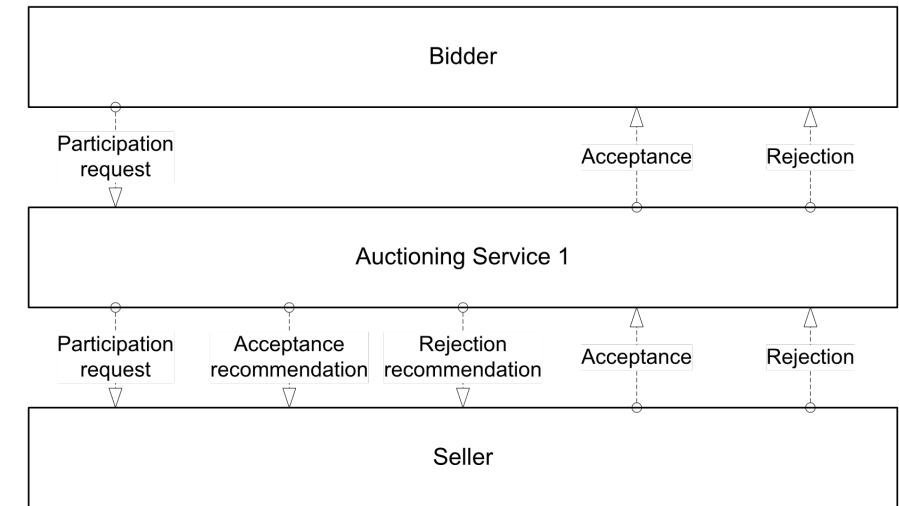


Fig. 6.10. Interactions of participants in auctioning scenario

- *Participants represented by pools that interact by sending and receiving messages*
- *Above figure does not show any behavioral dependencies between the different message exchanges*

Structural Compatibility



- *Weak structural compatibility*

- Messages that can be sent by a participant correspond to messages that other participants can receive
- Ensures that all messages sent can actually be received by participants
- Does not forbid that participants may receive additional messages not sent by any of the other participants (in the given choreography)

- *Strong structural compatibility*

- For every message that can be sent there is a participant that can receive it, and
- for every message that can be received there is a participant that can send it

Behavioral Compatibility



- *Behavioral compatibility*

- Considers behavioral dependencies (i.e., control flow) between interaction instances of a conversation as well
- The process orchestrations of the interacting partners are interconnected, and the resulting process structure is analyzed
- Such analysis of the dynamic behavior requires a formal, unambiguous representation

Behavioral Compatibility



Checking behavioral compatibility

- Representing process orchestrations by a specific class of Petri Nets, namely workflow modules
- Workflow modules: WF Nets with additional communication places that are used to represent message flow between participants
- Whenever a participant sends a message, the process orchestration of that partner features a transition with an output communication place that can hold messages sent
- At the receiver side, the workflow module requires a matching input communication place → input place of the transition that receives the message

Behavioral Compatibility



- Each process orchestration is represented by a workflow module that defines its internal behavior and its external communication behavior.

Definition (Workflow Module): A Petri Net $PN = (P, T, F)$ is a **workflow module** if and only if the following conditions hold

- P is the set of places that is partitioned into sets P^N of internal places, P^I of incoming places, and P^O of outgoing places
- T is a non-empty set of transitions
- The flow relation F is partitioned into an internal flow relation $F^N \subseteq (P^N \times T) \cup (T \times P^N)$ and a communication flow relation $F^C \subseteq (P^I \times T) \cup (T \times P^O)$
- (P^N, T, F^N) is a workflow net (i.e. a Petri Net with single start / end place)
- There is no transition t connected to both an incoming place and an outgoing place

Behavioral Compatibility

- The following figure shows workflow modules for participants

Auctioning Service 1 and Seller

- For the sake of readability, the workflow modules only represent a small part of the auctioning and seller process orchestrations

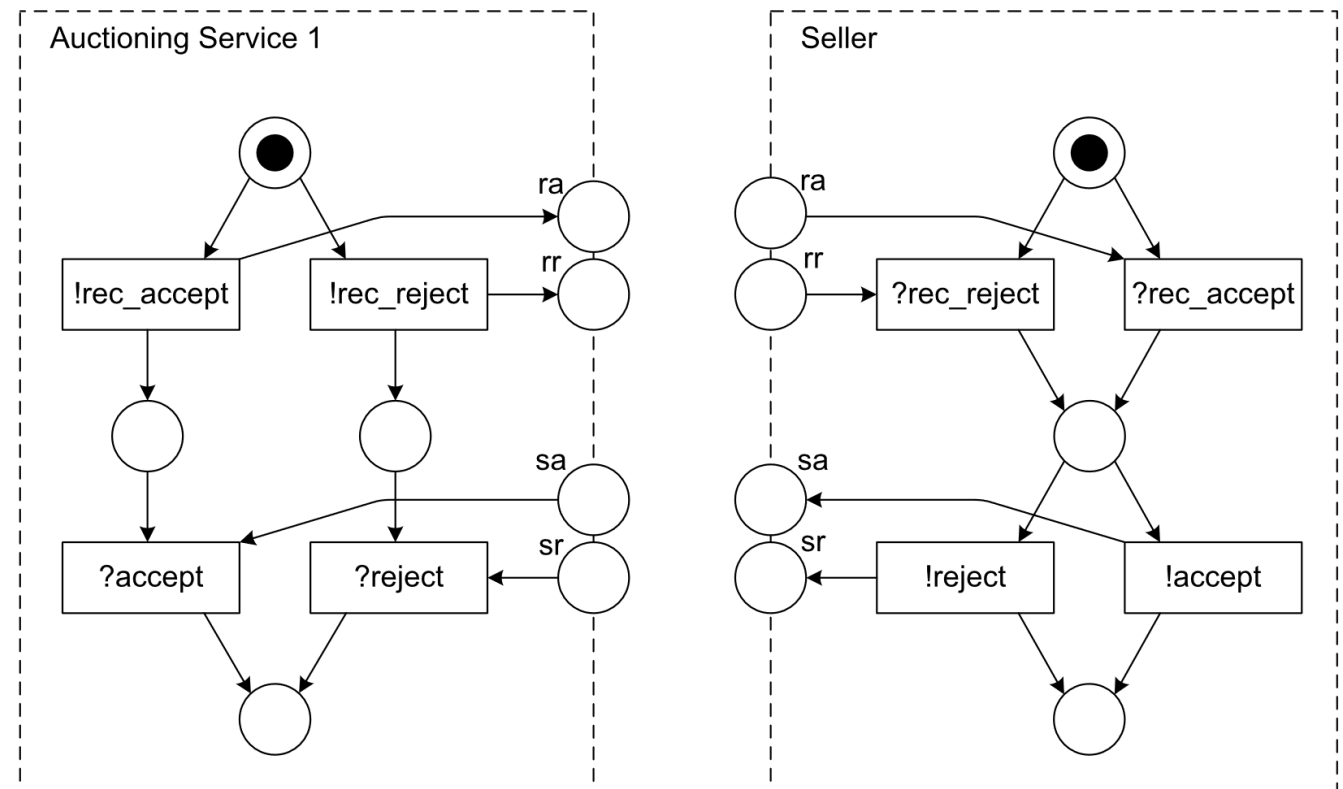
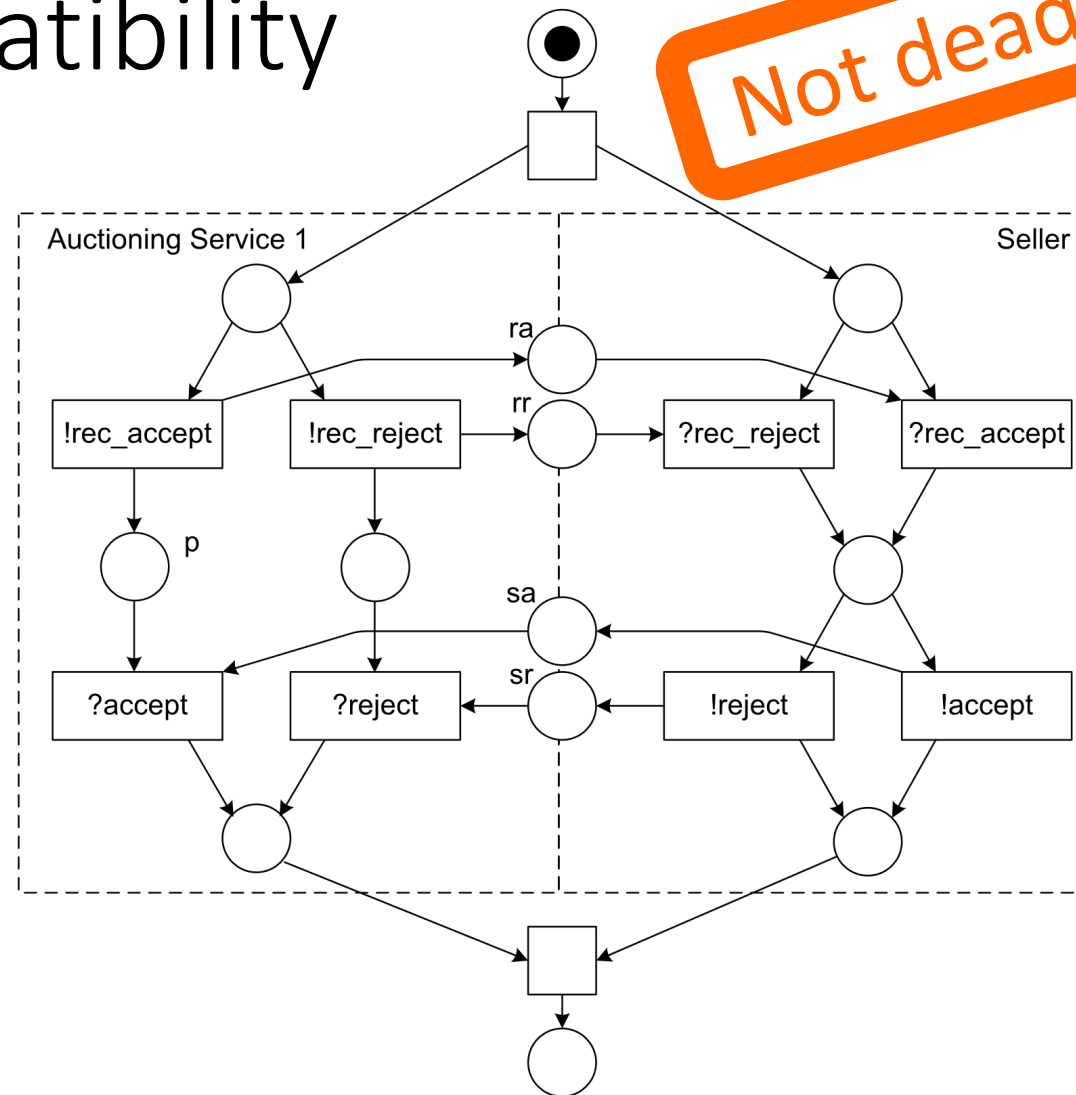


Fig. 6.12. Workflow modules as basis for checking compatibility

Behavioral Compatibility

- Workflow net as composition of the workflow modules (requires strong structural compatibility of the workflow modules)

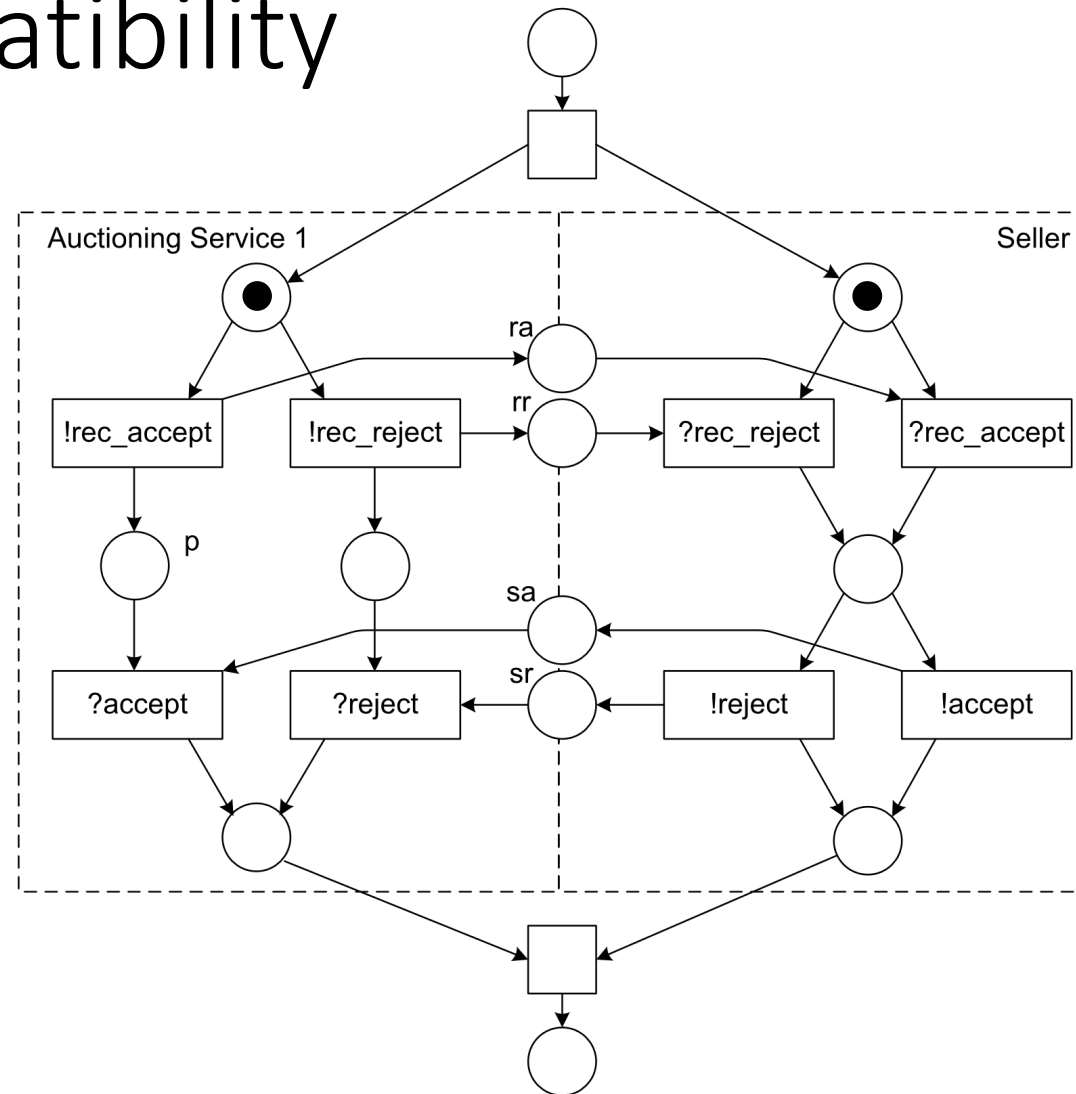


M. Weske: Business Process Management, Third Edition
© Springer-Verlag 2019, 2012, 2007

Fig. 6.13. Workflow net as composition of workflow modules

Behavioral Compatibility

- Workflow net as composition of the workflow modules (requires strong structural compatibility of the workflow modules)

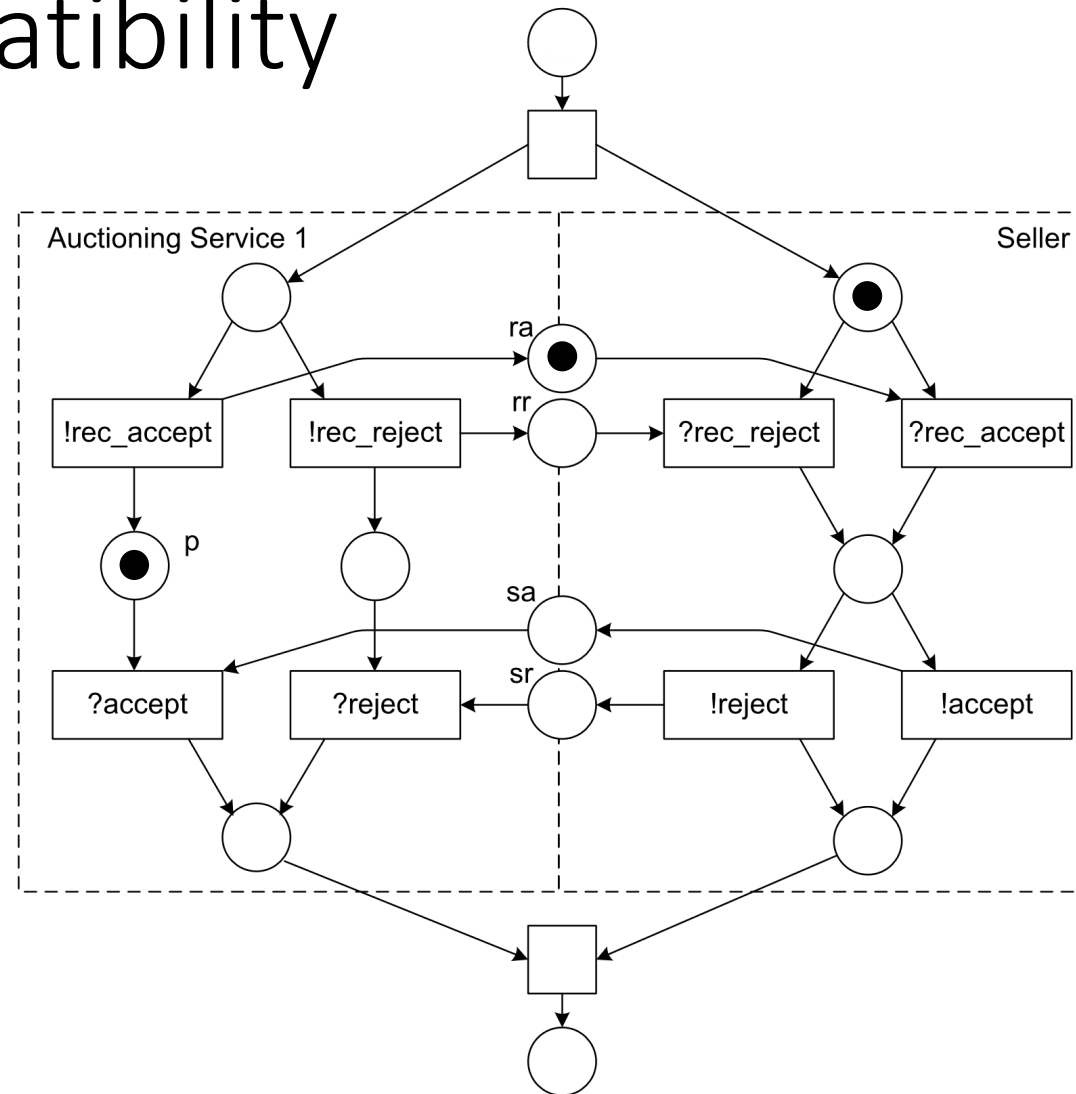


M. Weske: Business Process Management, Third Edition
© Springer-Verlag 2019, 2012, 2007

Fig. 6.13. Workflow net as composition of workflow modules

Behavioral Compatibility

- Workflow net as composition of the workflow modules (requires strong structural compatibility of the workflow modules)

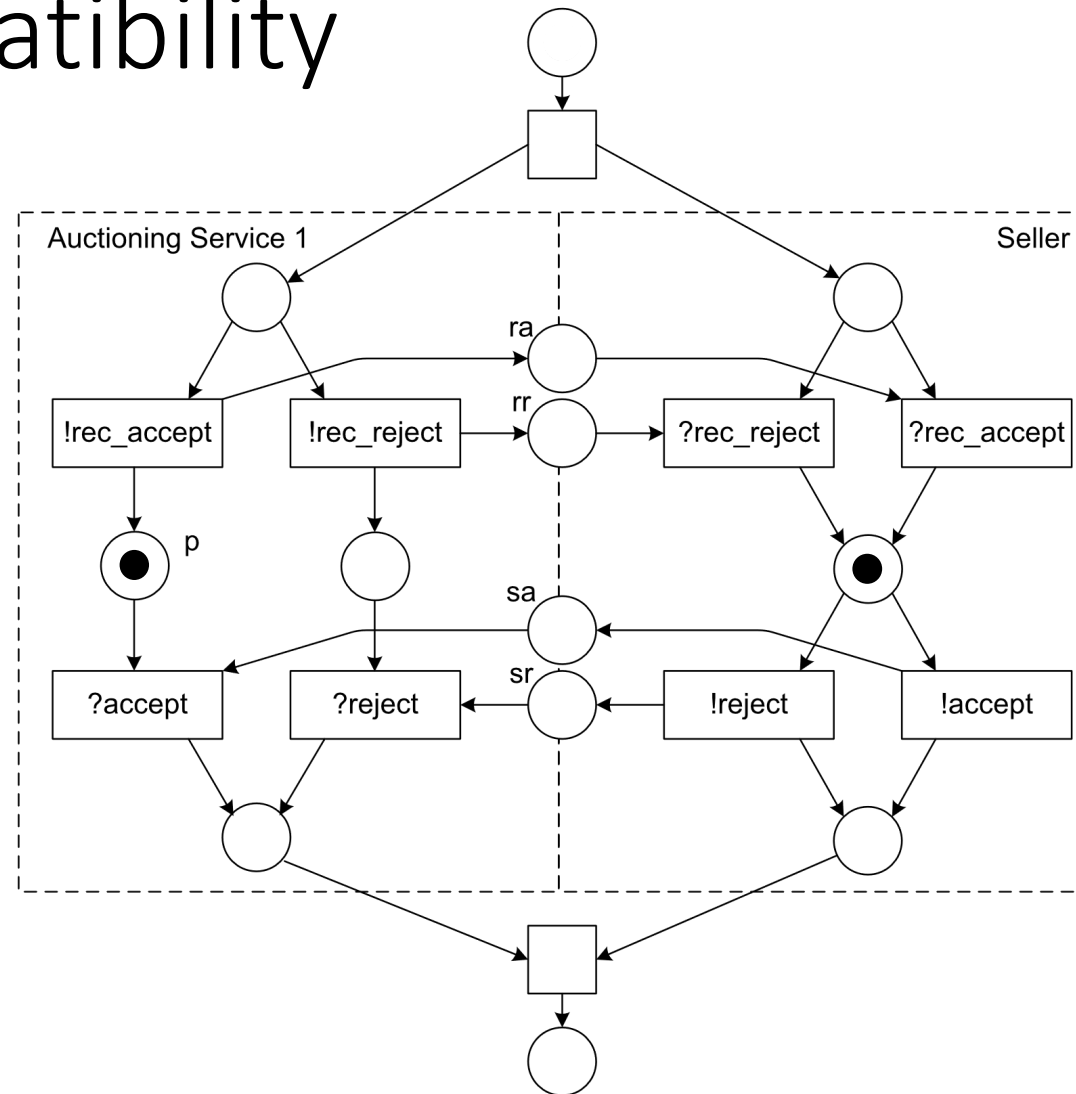


M. Weske: Business Process Management, Third Edition
© Springer-Verlag 2019, 2012, 2007

Fig. 6.13. Workflow net as composition of workflow modules

Behavioral Compatibility

- Workflow net as composition of the workflow modules (requires strong structural compatibility of the workflow modules)

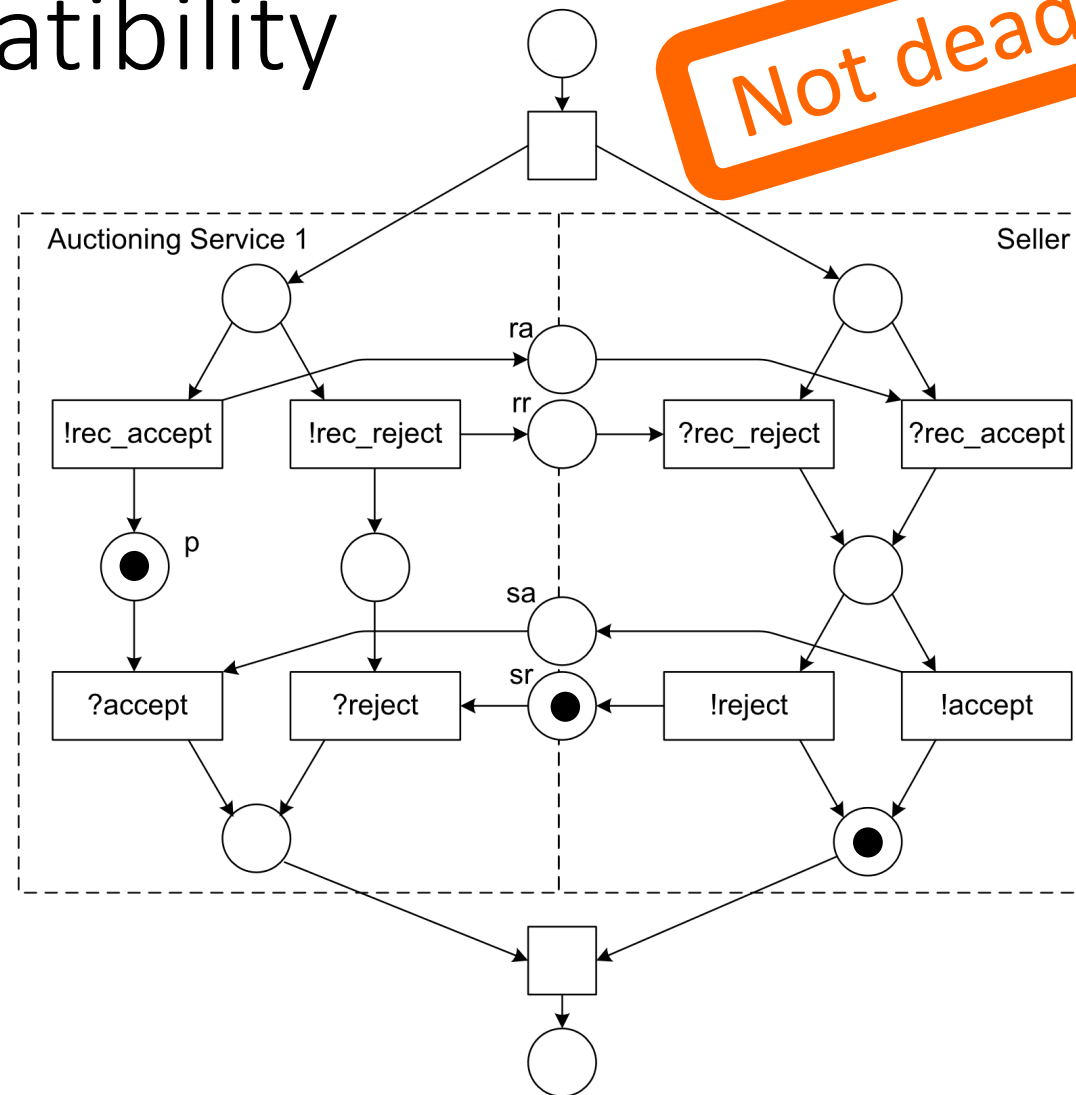


M. Weske: Business Process Management, Third Edition
© Springer-Verlag 2019, 2012, 2007

Fig. 6.13. Workflow net as composition of workflow modules

Behavioral Compatibility

- Workflow net as composition of the workflow modules (requires strong structural compatibility of the workflow modules)



M. Weske: Business Process Management, Third Edition
© Springer-Verlag 2019, 2012, 2007

Fig. 6.13. Workflow net as composition of workflow modules

Behavioral Compatibility

- Deadlock free?
- Reachability analysis on the „combined“ workflow modules

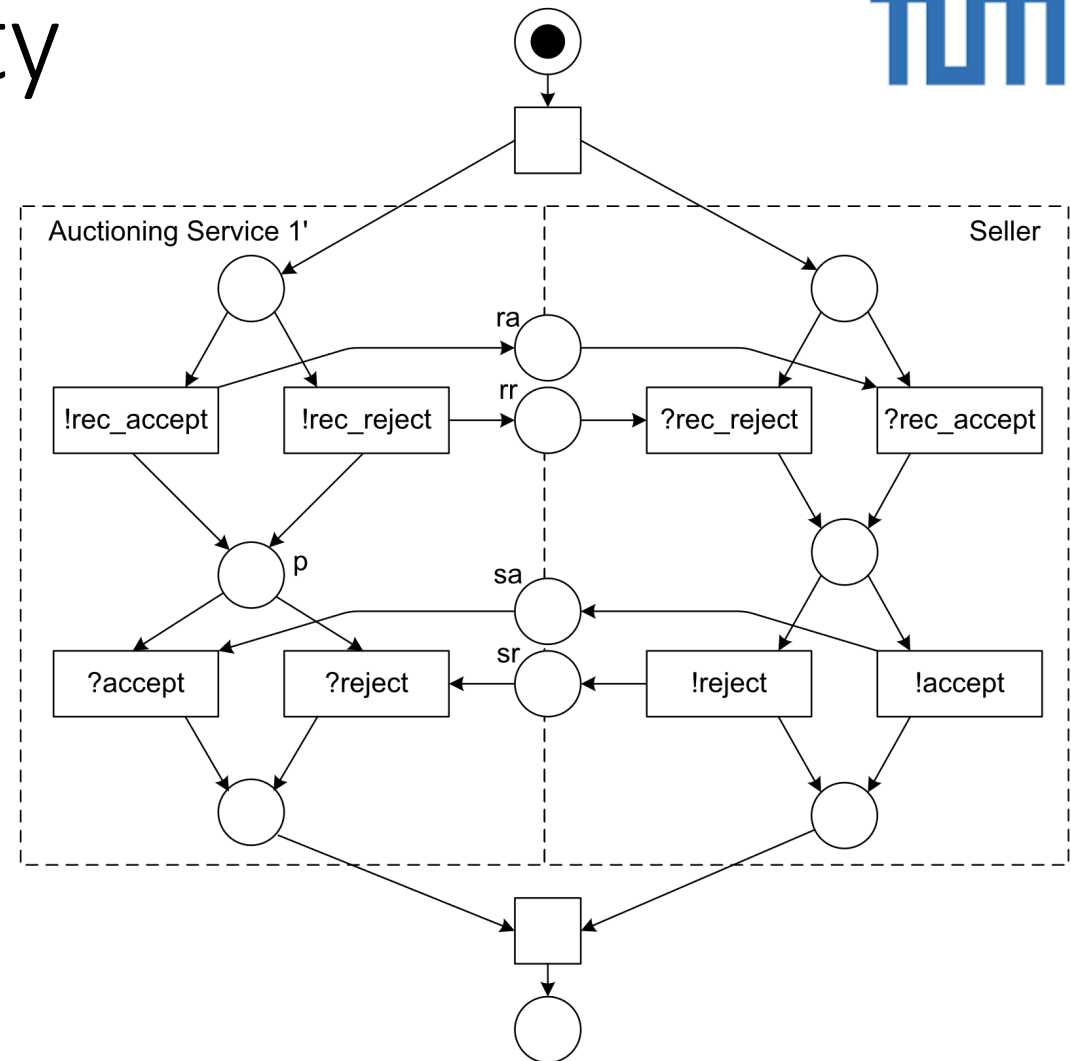


Fig. 6.14. Workflow modules that are compatible

Behavioral Compatibility

- A larger part of the collaboration is depicted in the figure on the right
- Deadlock?

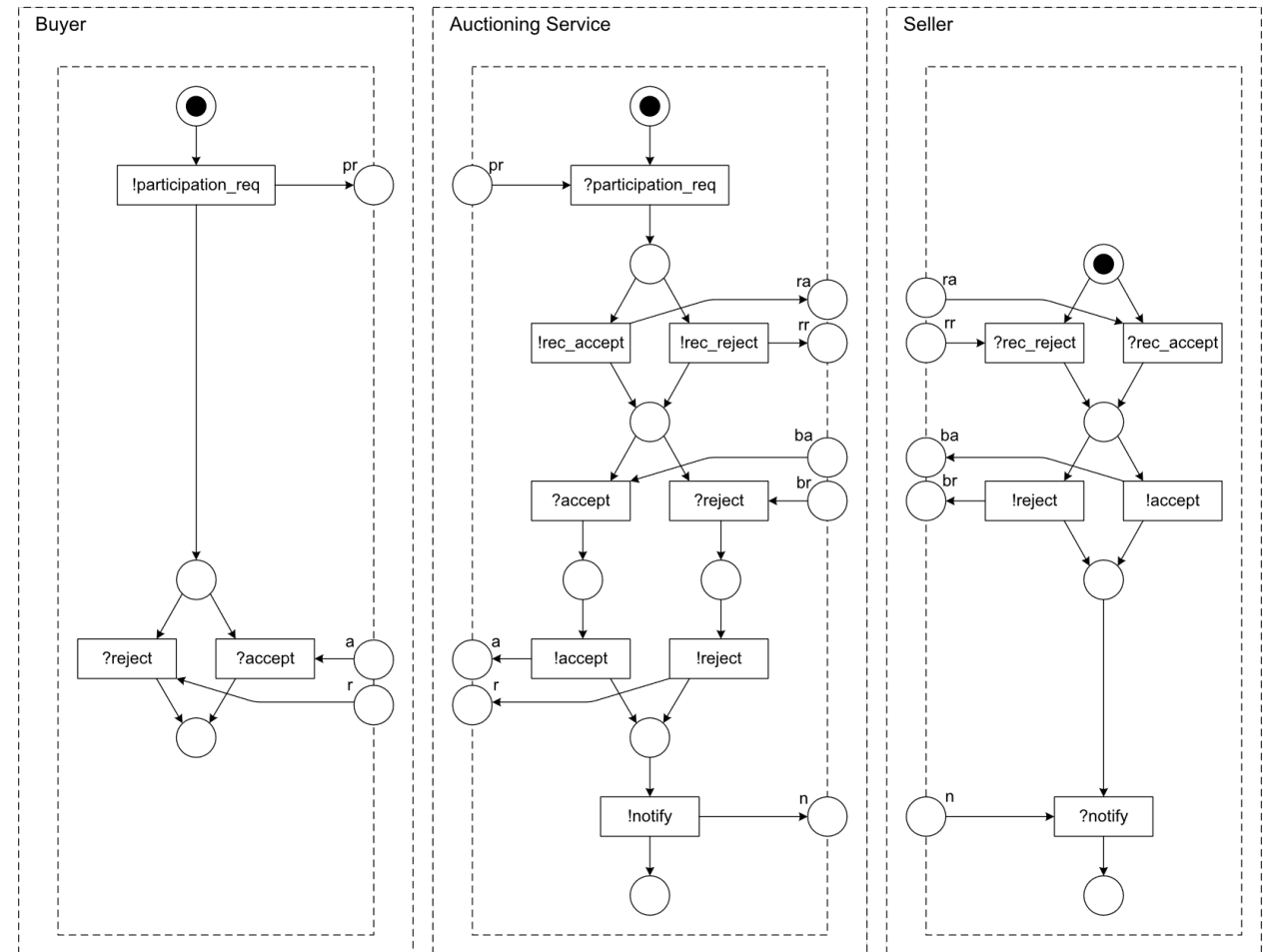


Fig. 6.15. Behavioural interfaces: getting a participation permission

Choreography Patterns

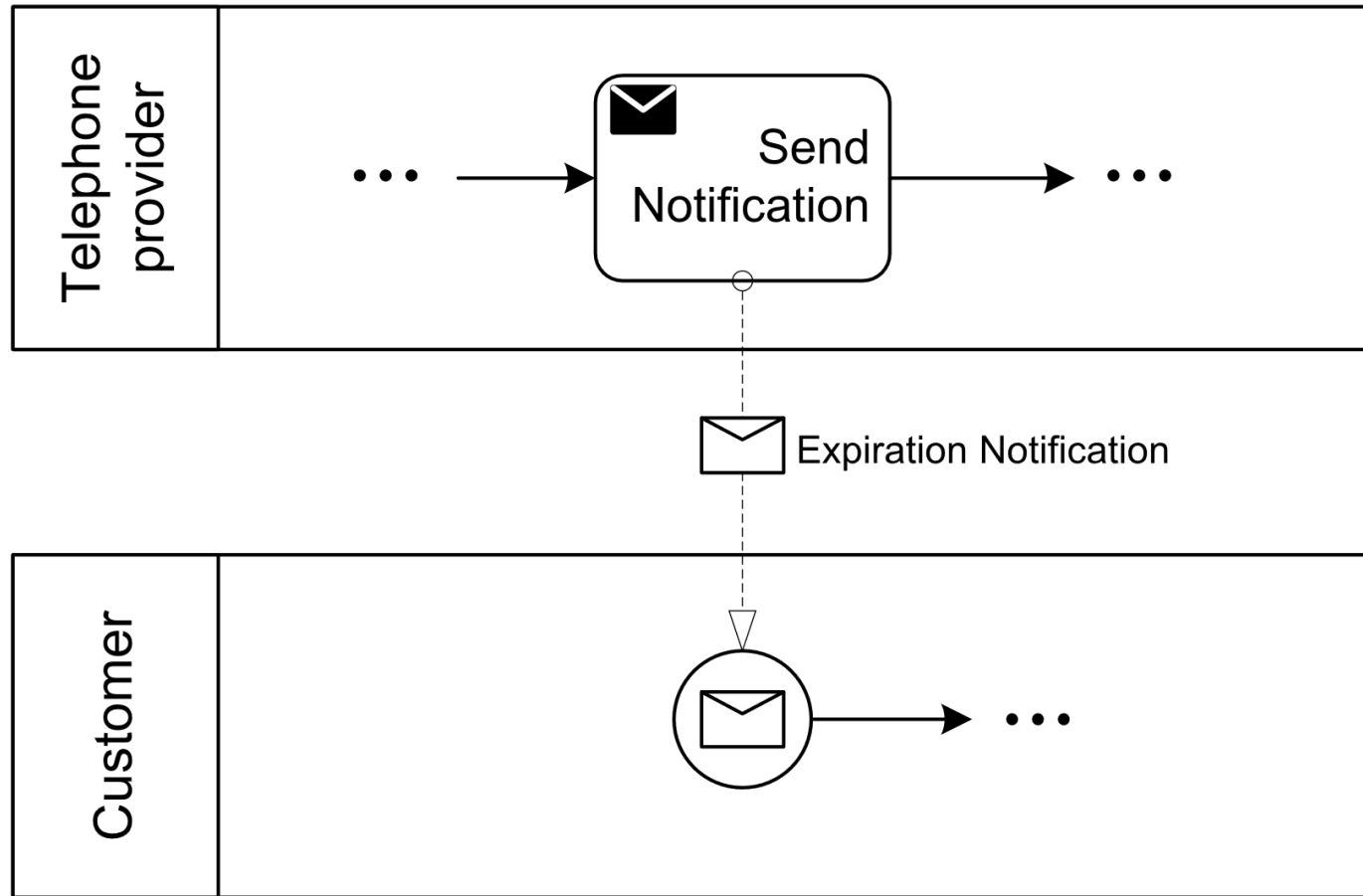


Fig. 6.22. Send pattern

M. Weske: Business Process Management, Third Edition
© Springer-Verlag 2019, 2012, 2007

Choreography Patterns

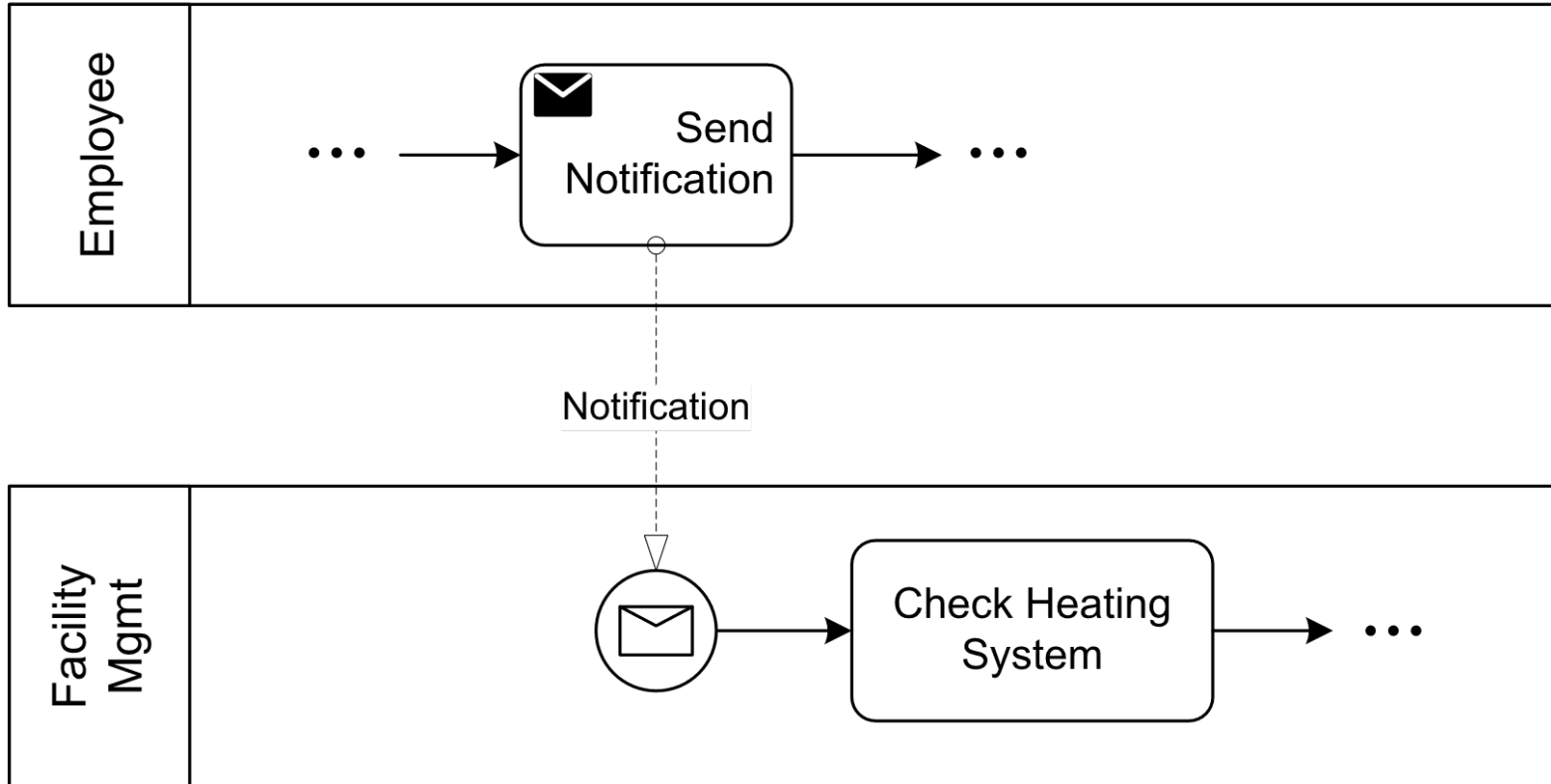


Fig. 6.23. Receive pattern

Choreography Patterns

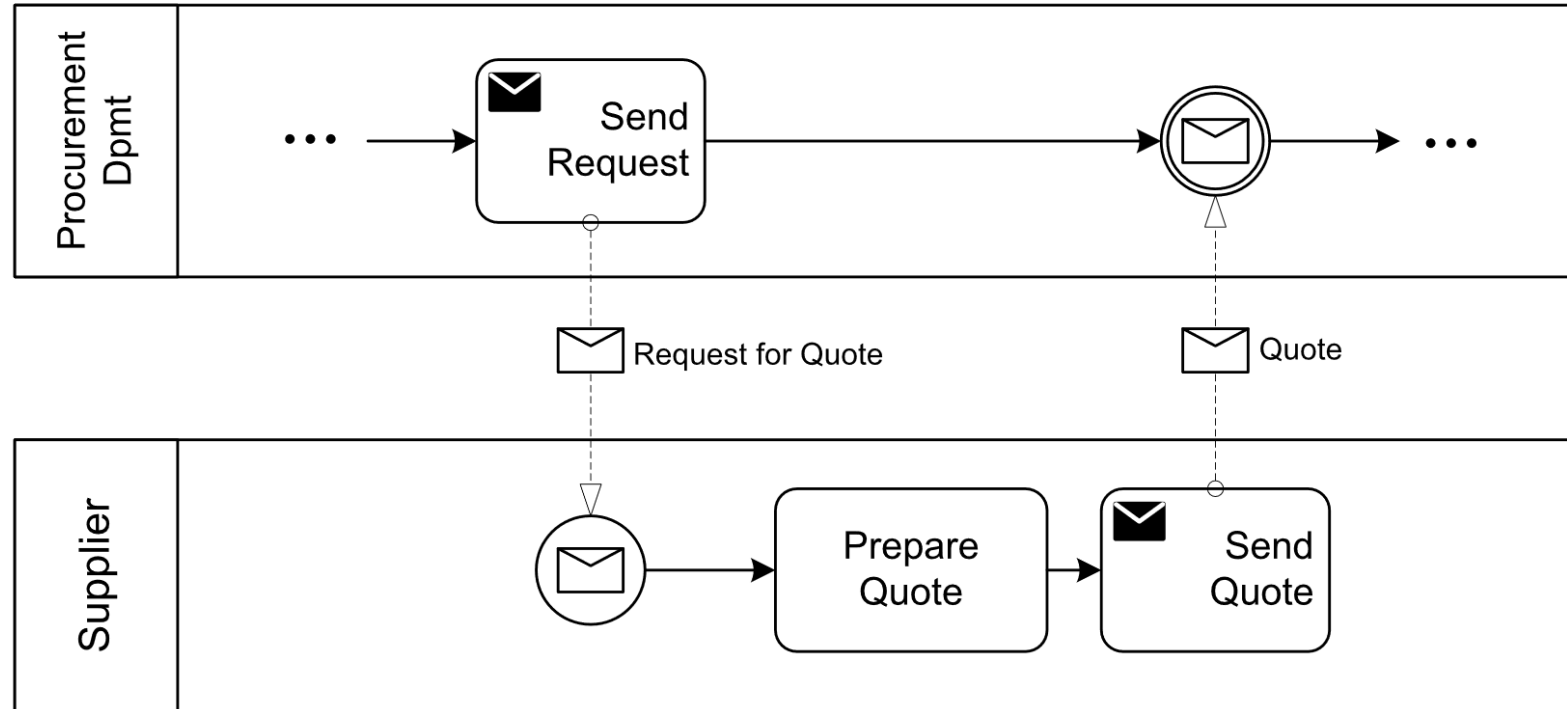


Fig. 6.24. Send/receive pattern

Choreography Patterns

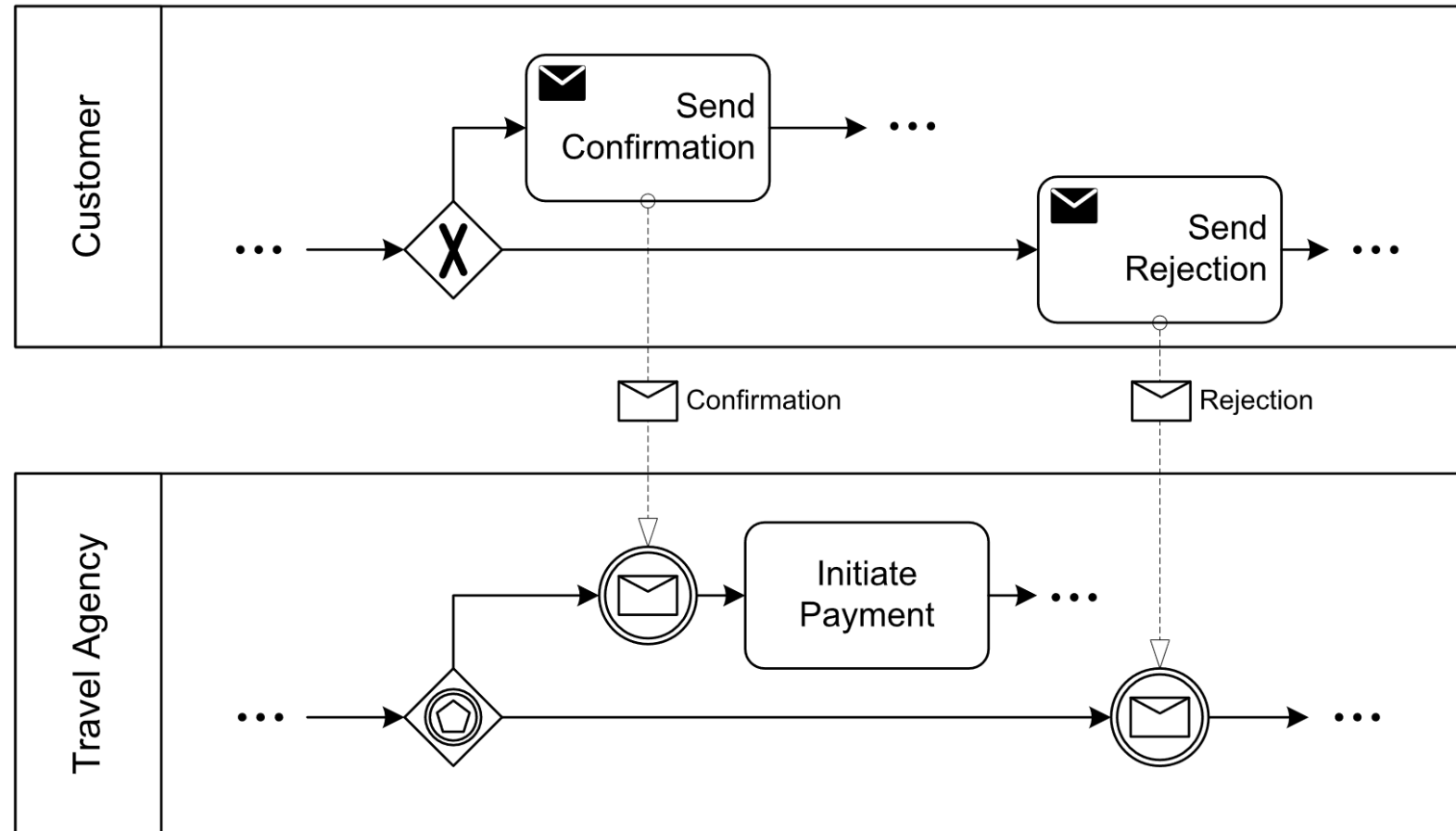


Fig. 6.25. Racing incoming messages pattern

Choreography Patterns

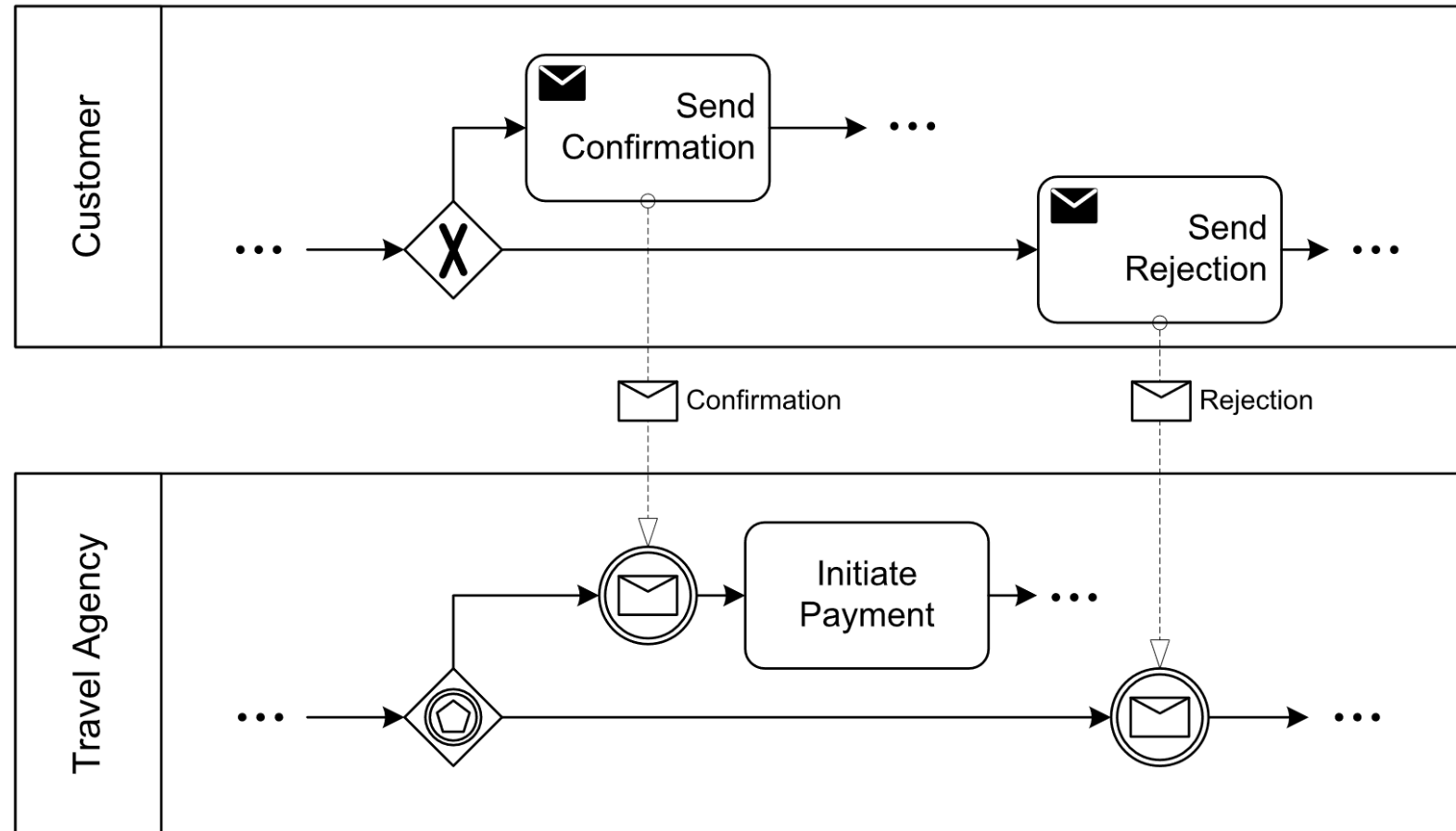


Fig. 6.25. Racing incoming messages pattern

Process Choreographies

- Represented by four model types.
- They differ in terms of perspectives and level of detail:
 - Private Model
 - Public Model
 - Collaboration Model
 - Choreography Model

Choreography Diagrams

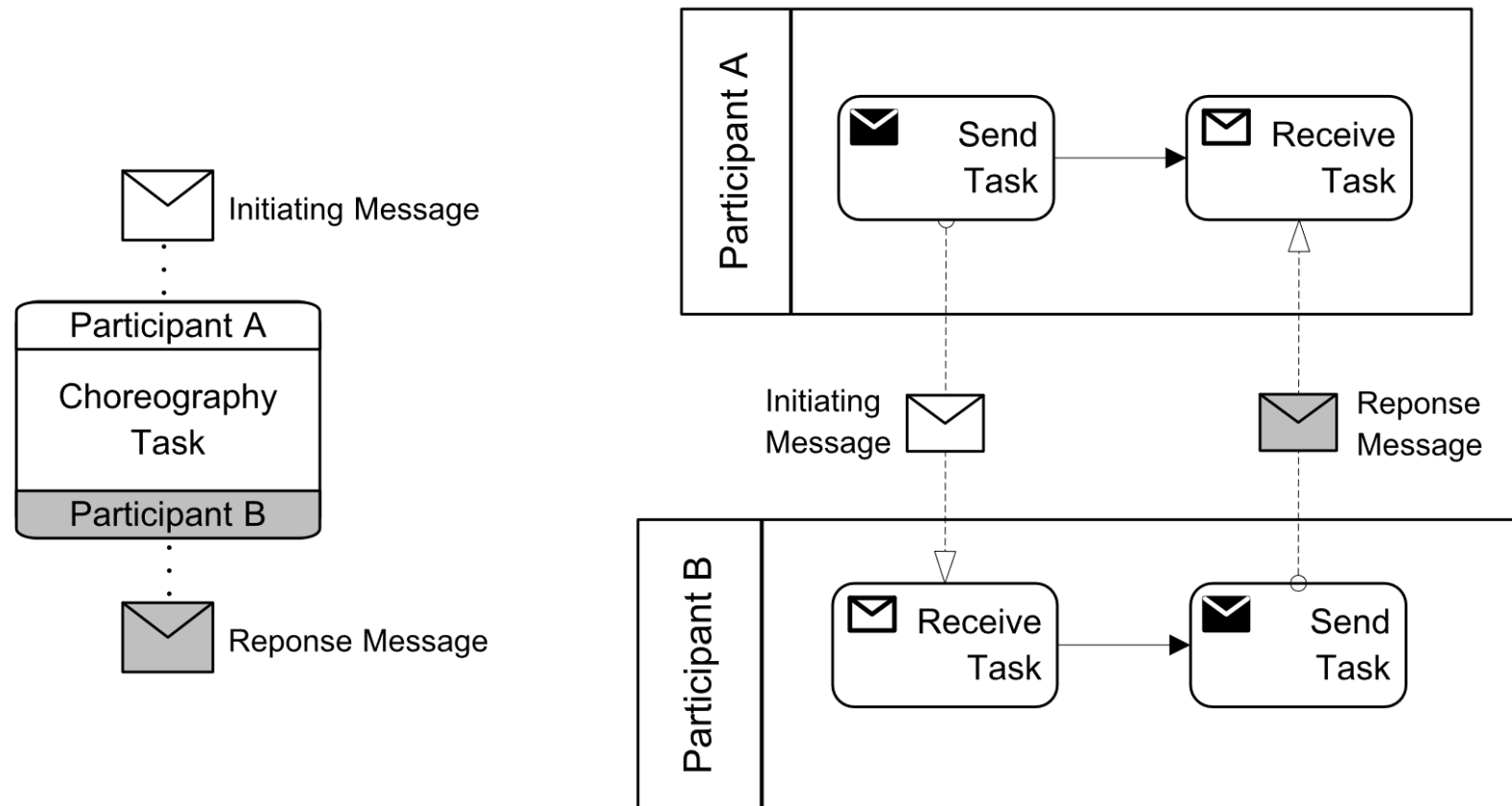
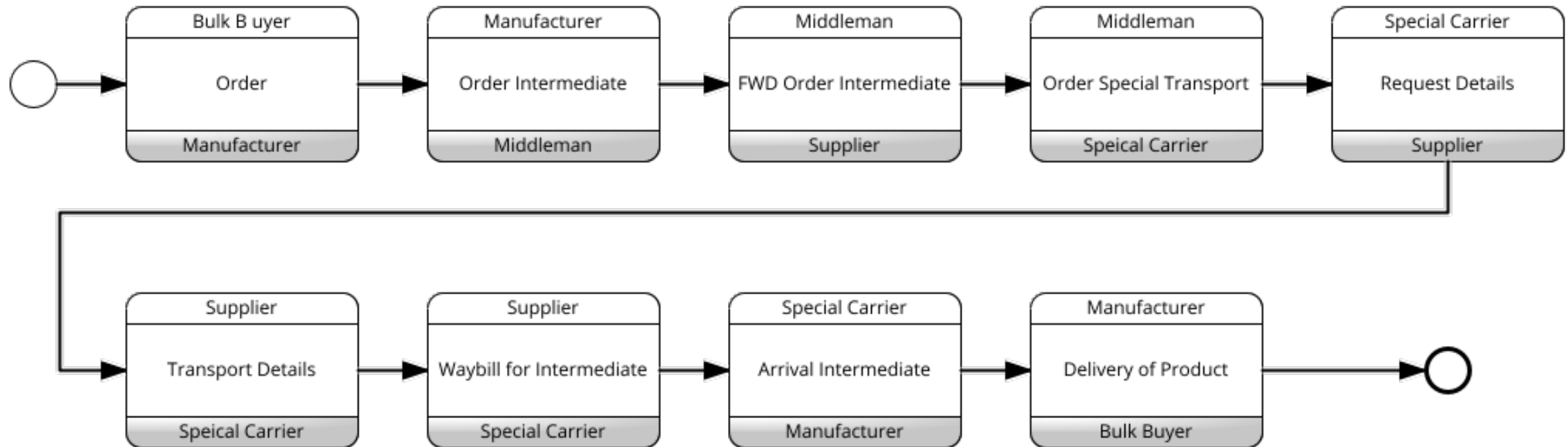
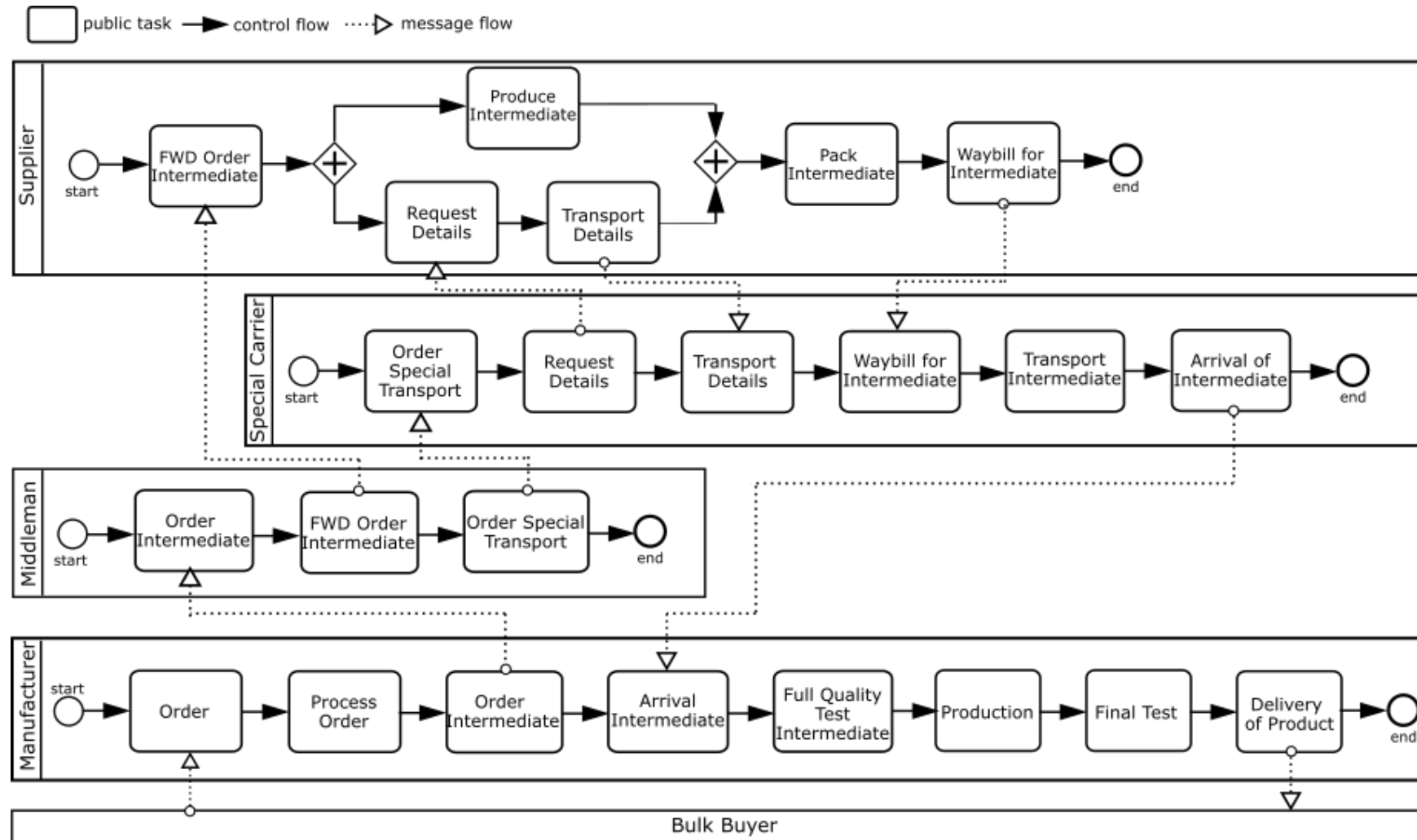


Fig. 6.37. Choreography task and corresponding collaboration diagram

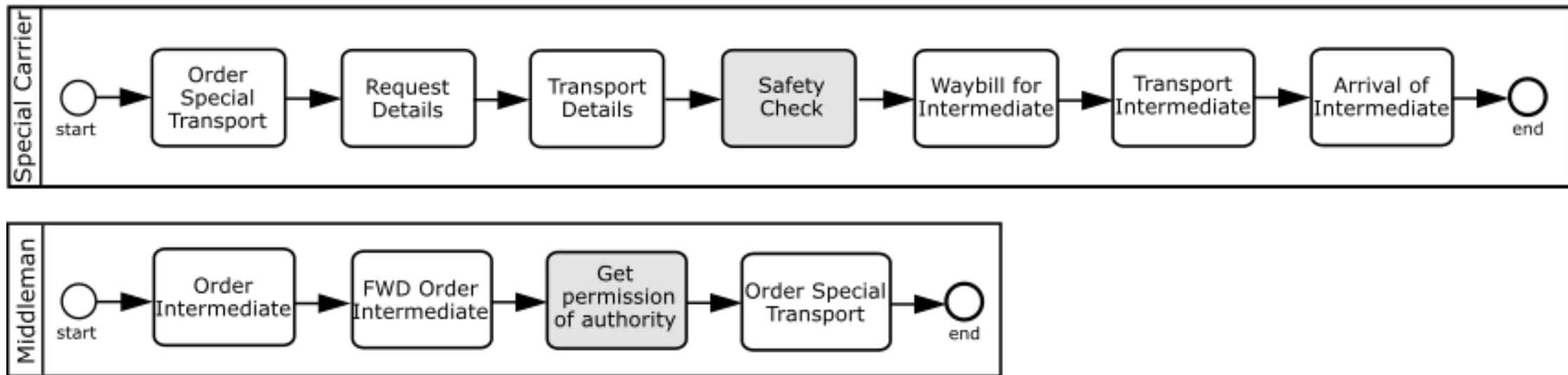
Supply Chain: Choreography

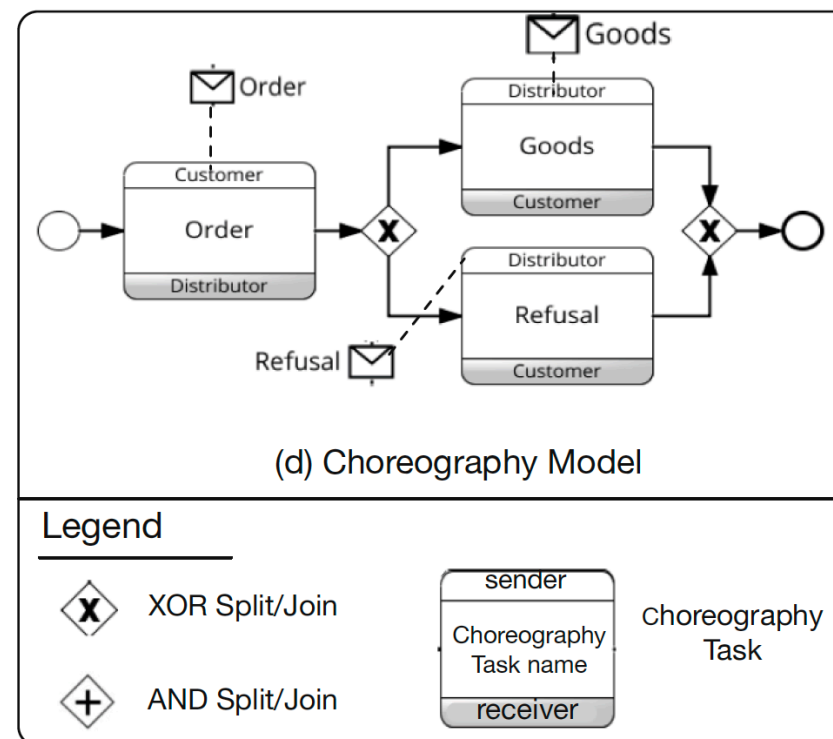
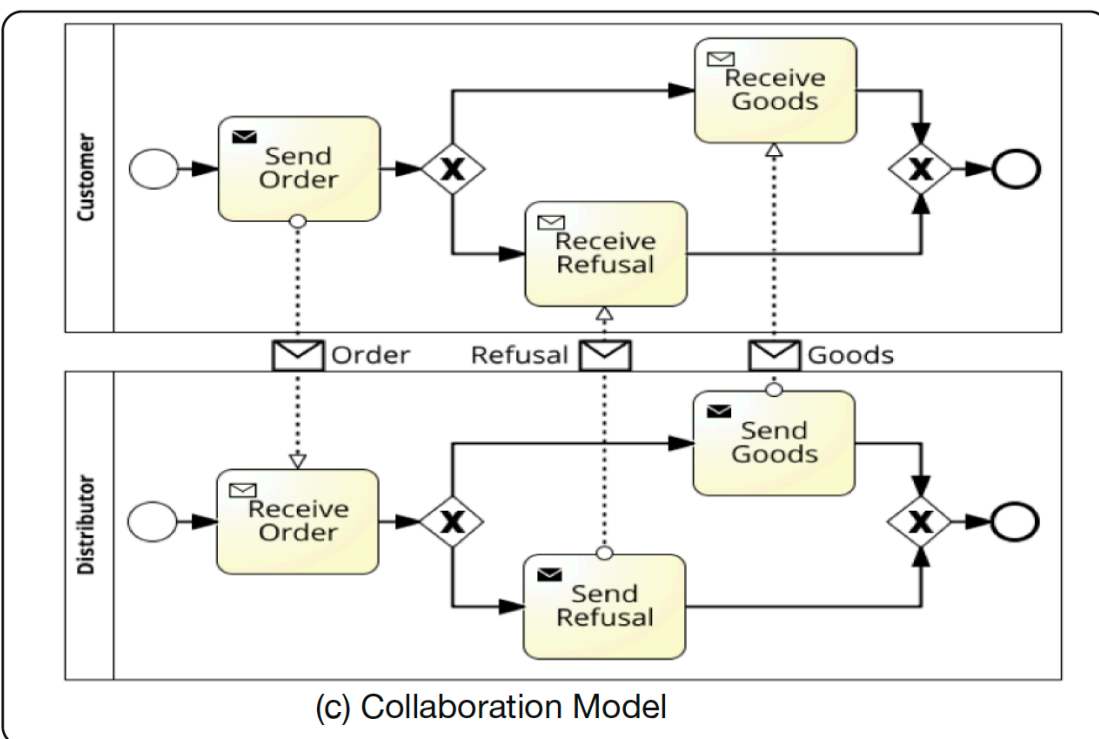
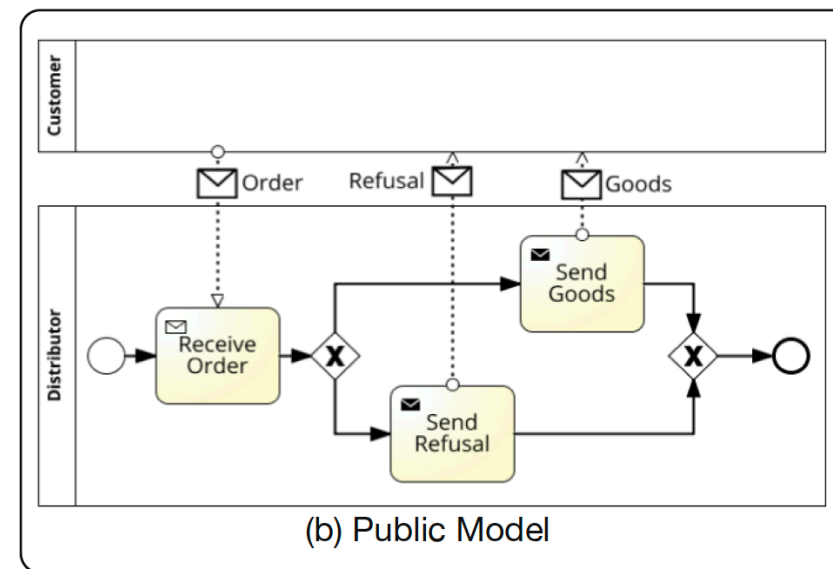
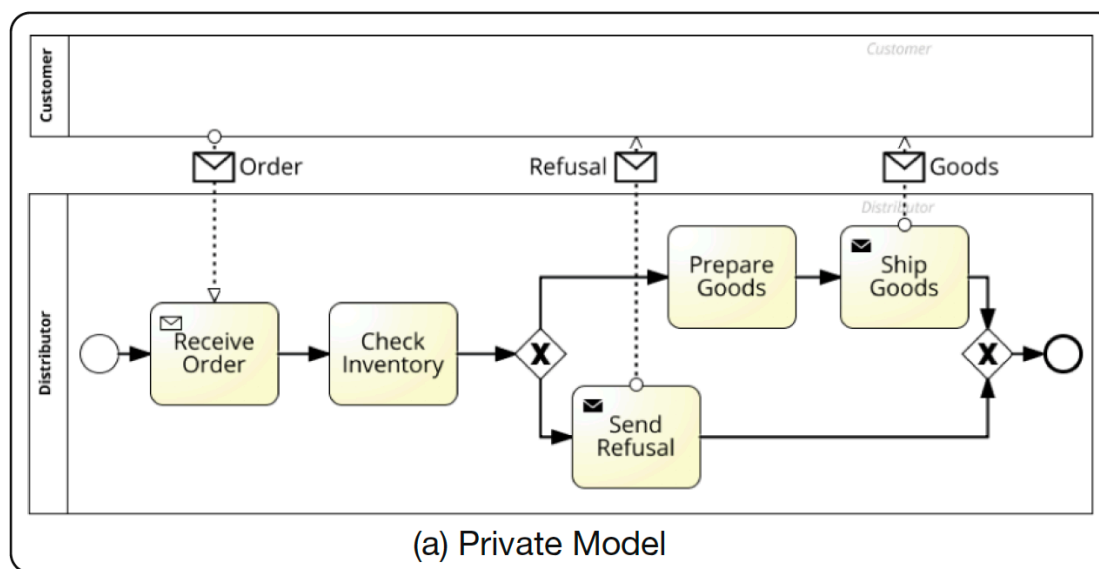


Supply Chain: Collaboration



Supply Chain: Selected Private Processes





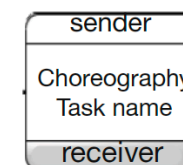
Legend



XOR Split/Join

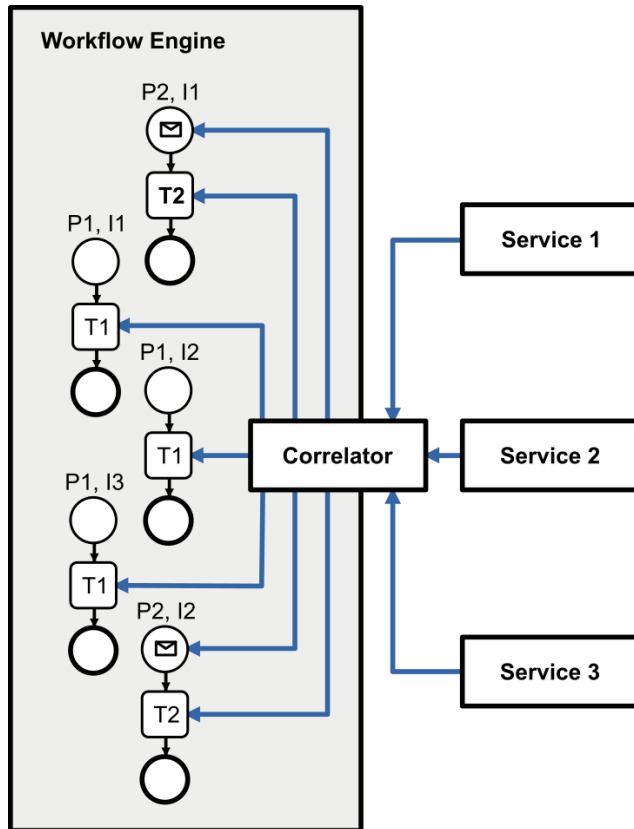


AND Split/Join



choreography Task

Correlation

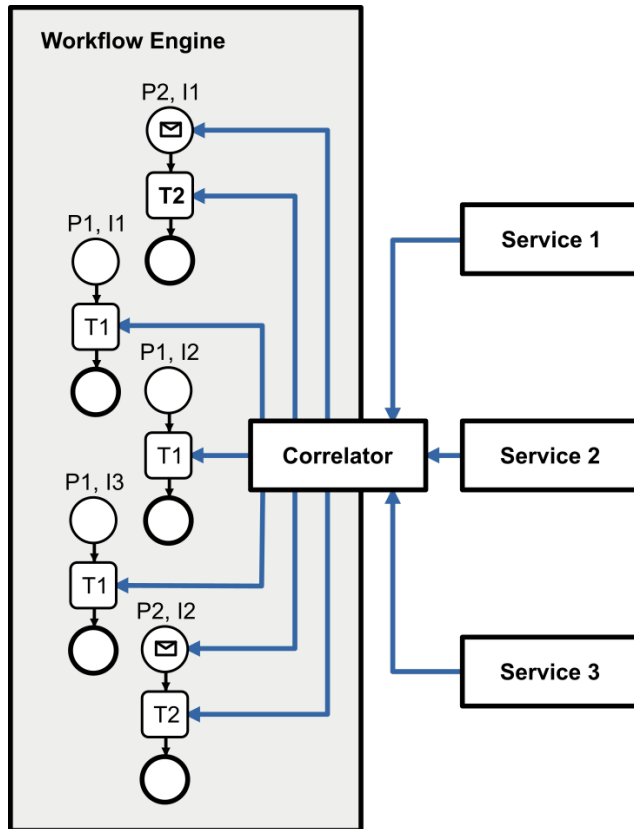


Px ... Process
Ix ... Instance
Tx ... Task
Service ... Implementation or other instance/engine
Correlator ... Match and distribute based on message content

Patterns - Point of View:

- Ask for information: the process engine asks an external service for some results.
- React on some external event:
 - A new process instance has to be started.
 - Waiting for some external message.
 - External source may be unknown.
 - External source may send the message long before an instance is needing the value.
 - External source may send the value long after an instance is needing the value.
 - The contents of a message contain the key to decide what to do

Correlation - How



Px ... Process
Ix ... Instance
Tx ... Task
Service ... Implementation or other instance/engine
Correlator ... Match and distribute based on message content

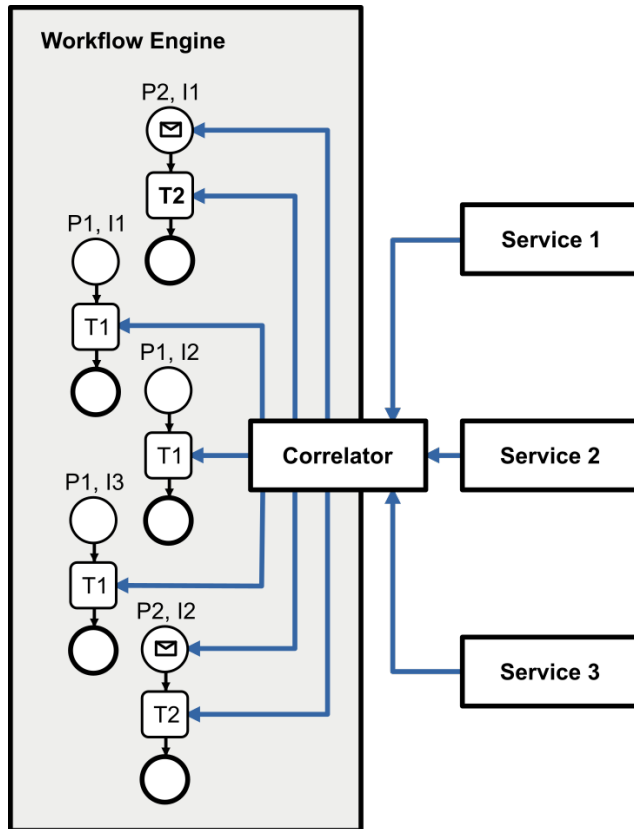
Correlator has to:

- Receive messages from arbitray sources
- Analyse the contents of the message. A message might be a mail received via IMAP, a word document received via HTML Form upload, or mor generic: any fileformat submitted through any means/protocol.
- Forward the message to a potential instance that is waiting for the message, OR
- Store the message for later use OR
- Create a new instance of a process, with the message as input

Correlator requires:

- A set of rules to analyse incoming messages
 - Message type, e.g. email
 - Where to look in message, e.g. subject contains ticket number
 - What to do with the message: forward or instantiate

Correlation - Message Analysis



Px ... Process
Ix ... Instance
Tx ... Task
Service ... Implementation or other instance/engine
Correlator ... Match and distribute based on message content

A correlator will support a set of sources: Mailbox, HTTP/REST file upload, ...

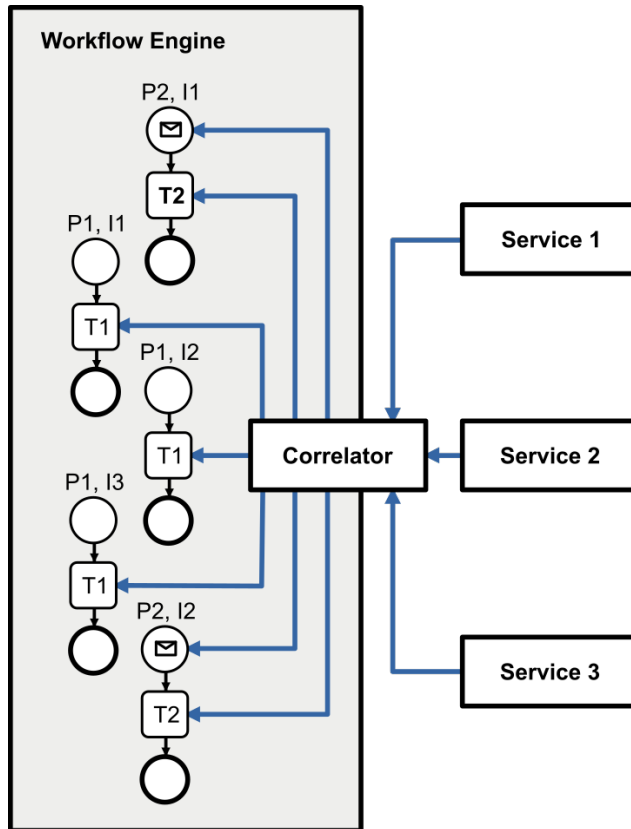
A correlator will support a set of tile types that can be analysed:

- A means to read and analyse the contents of a word file
- A means to read and analyse the contents of a excel file
- A means to read and analyse the contents of a CAD file
- ...

A correlator will support a rule language to describe how to extract correlation information

- A word files contains the signature of a particular person at the end of page 7, contains a ticket number in the form `/#\w{12}/` on page 2, and has a heading "Customer Contract" on page 1
- An excel sheet has three worksheets, and on the first worksheet cell A1 contains the identifier of the customer
- ...

Correlation - Target Identification



Processes that are to be instantiated have to contain **correlation rules** when to do so:

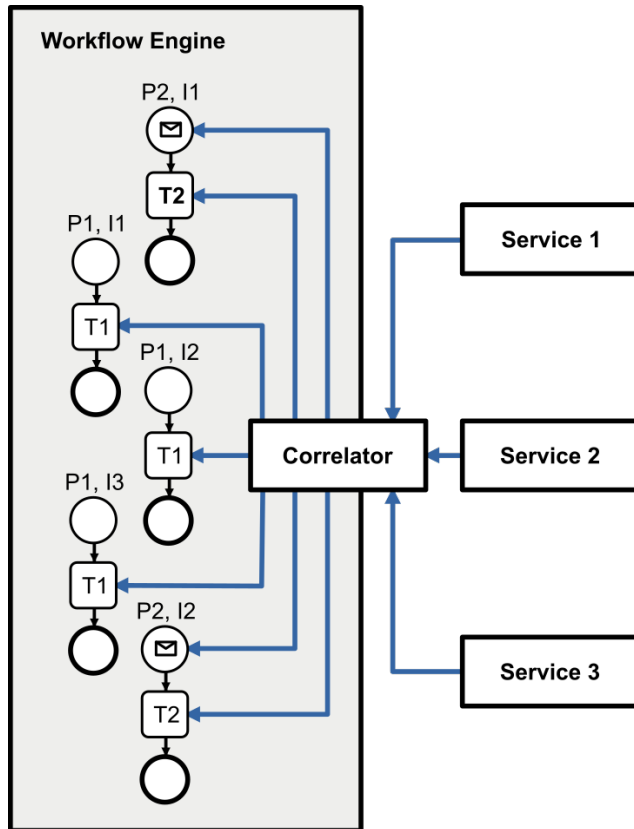
- A word files contains the signature of a particular person at the end of page 7, contains a ticket number in the form `/#\w{12}/` on page 2, and has a heading "Customer Contract" on page 1
- The rule is generic, any match leads to an instantiation
- The message is an input to the resulting process instance, see P2 on the left
- The correlator has to analyse each process model to extract the correlation rules

Instances that require certain message (but no direct means to contact a source) have to make a **correlation request** to get the required information from the correlator:

- I need a word files contains the signature of John Boss at the end of page 7, contains a ticket number in the form `#QWER12tzui34` on page 2, and has a heading "Customer Contract" on page 1
- The request is very special, this one message matches
- The correlator can store a list of requests on the fly, and delete entries whenever a request has fulfilled

Px ... Process
Ix ... Instance
Tx ... Task
Service ... Implementation or other instance/engine
Correlator ... Match and distribute based on message content

Correlation - Message Received



A matching correlation rule exists:

- instantiate process

A matching correlation request exists:

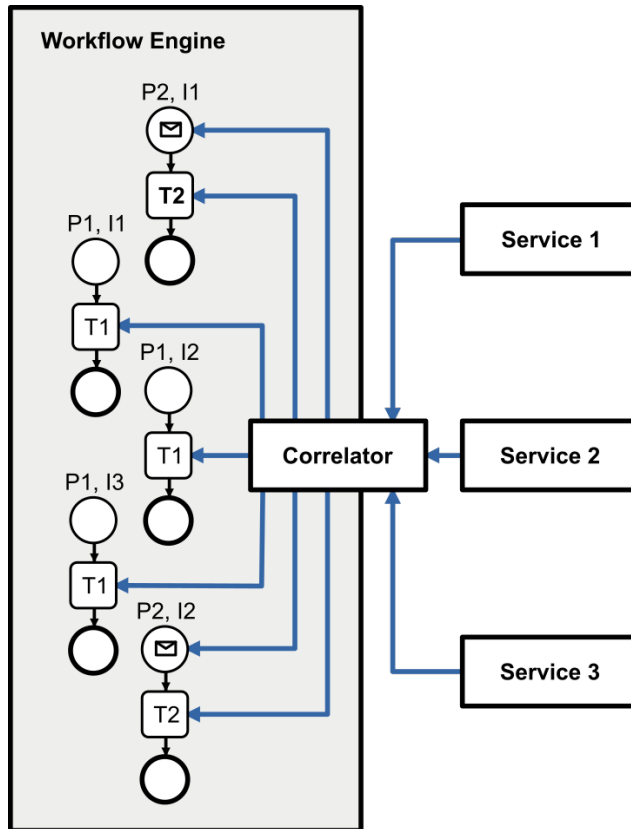
- forward message
- Data Retention (next slide)

No matches exists:

- Data Retention (next slide)

Px ... Process
Ix ... Instance
Tx ... Task
Service ... Implementation or other instance/engine
Correlator ... Match and distribute based on message content

Correlation - Data Retention



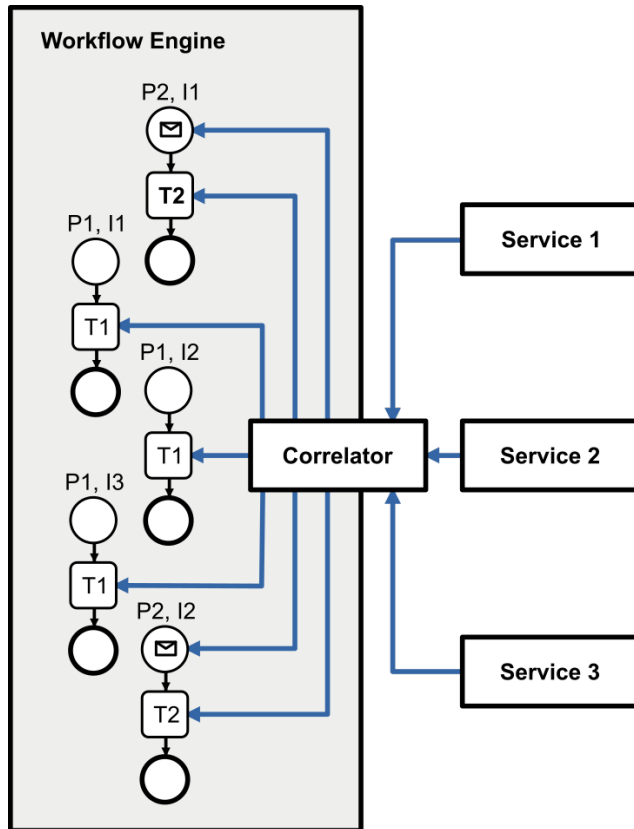
Px ... Process
Ix ... Instance
Tx ... Task
Service ... Implementation or other instance/engine
Correlator ... Match and distribute based on message content

A message that instantiates a process: drop message after instantiation

A message that matches a correlation request - a retention policy has to exist:

- No policy: drop the message after forward
- Store indefinitely
 - Other requests can be answered
- Store for a limited time
 - Other requests can be answered
- Store with additional request rules
 - Additional parameters have to be fulfilled in order to reuse the message to fulfill request
 - E.g. a requestor has to submit a code to get the info
- Queue vs. Slot
 - Slot - Messages can replace other existing messages - only the newest message fitting a correlation request is stored
 - Queue - All messages are stored - the newest message fitting a correlation request is delivered

Correlation - Application 1



Px ... Process
Ix ... Instance
Tx ... Task
Service ... Implementation or other instance/engine
Correlator ... Match and distribute based on message content

Security: correlators connect external systems to internal process instances without exposing the internal structure.

Loose Coupling: external systems have a single point where to send information.

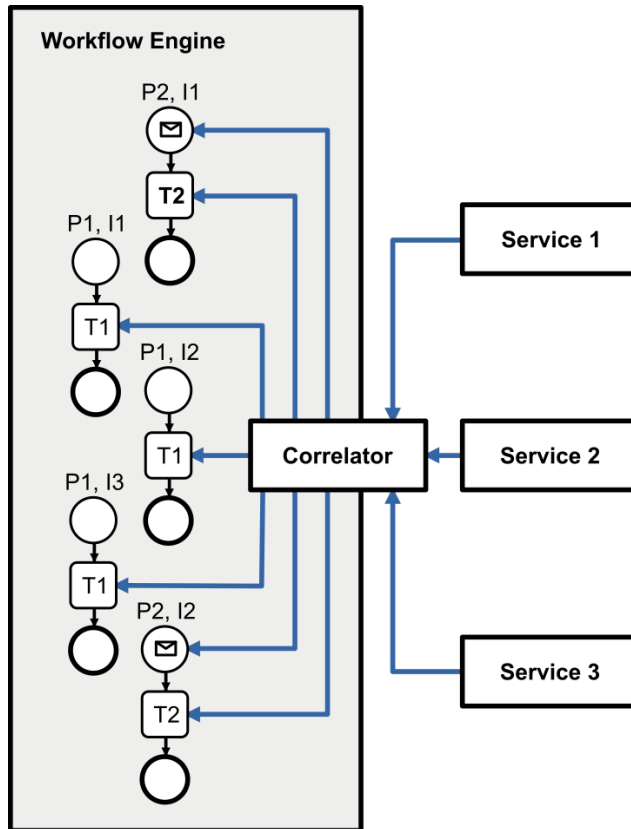
Policy Enforcing: how to deal with message retention - loose coupling & maintainability.

Compliance Checking: incoming data can be checked before it enters internal infrastructure

Maintainability: single place to deal with incoming messages. No need to implement anything in the engine.

The alternative would be much more complicated and relying on tightly coupled system.

Correlation - Application 2



Services (see left) can be other Workflow Engines or BPM systems:

Correlation is imperative for Choreographies

Px ... Process
Ix ... Instance
Tx ... Task
Service ... Implementation or other instance/engine
Correlator ... Match and distribute based on message content

Summary



- Interorganizational processes are prevalent in many application domains, e.g., logistics, health care.
- In this chapter, we looked at top-down design of process choreographies, i.e., designing them on from scratch.
- In reality, often there are already processes running at the partners' side, requiring often lengthy and costly negotiations.
- Further challenges:
 - Change → W. Fdhila, C. Indiono, S. Rinderle-Ma, M. Reichert: Dealing with change in process choreographies: Design and implementation of propagation algorithms. Inf. Syst. 49: 1-24 (2015), <https://doi.org/10.1016/j.is.2014.10.004>
 - Compliance → W. Fdhila, D. Knuplesch, S. Rinderle-Ma, M. Reichert: Verifying Compliance in Process Choreographies: Foundations, Algorithms, and Implementation. Information Systems 2022, <https://doi.org/10.1016/j.is.2022.101983>

Business Process Technologies and Management

5. Logging and Monitoring

Prof. Dr. Stefanie Rinderle-Ma

Dr. Jürgen Mangler

Technical University of Munich

Department of Informatics

Chair of Information Systems

and Business Process Management

stefanie.rinderle-ma@tum.de

juergen.mangler@tum.de



Differences



Logging:

- Available ex-post
- Data stored in files or databases
- Integrated into individual pieces of your architecture - i.e., each web-service or each software component logs individually.

Monitoring:

- For observing run-time.
- Data-streams; Potentially aggregated from individual pieces of your architecture.
- Structured as events:
 - Basics: timestamp, event-type, event name, life-cycle information, event content
 - Each component might deliver different events: in a different granularity, a different velocity, and with a different event content format.

Commonalities - Observability

Health checks - is the service working well?

- ping, heartbeat, resource utilization

Trace exchanged messages

- each message exchange has timestamp
- each message exchange has a unique id

Metrics - convey performance info, or inner state

- timing, memory consumption, storage consumption

Exception tracking

- errors in code

Auditing - strictly about business logic

- KPIs
- Exception tracking - exceptions in business logic

Logs 1



- Logs have a specific format:
 - NCSA Common Log Format https://en.wikipedia.org/wiki/Common_Log_Format
 - IEEE 1849-2016 XES Standard <https://xes-standard.org/>
- Logs have a file type:
 - XML (extendible markup language), JSON (javascript object notation):
 - not very well suited
 - each event has to be inserted at a certain point of the file (in contrast to beginning/end)
 - </end> and } prevent append to file
 - Bad IO performance (load, seek, insert, save), or memory usage (load once, insert, save, insert, save, ...)
 - CSV (character separated vector), YAML (yet another markup language), plain text
 - well suited
 - each event can be inserted at the end of a file
 - direct support by the OS/file system; highest possible performance (faster than database)

Logs 2



Logs have to be collected, unified, connected, and enriched

- collection: from different places/sources, e.g. databases, files
- unification: transformation into a common format
- connection: through timestamp, or better through semantic knowledge of data artefacts; i.e. an order id, ...
- enrichment: with order data, customer data, ...

Logs have to be securely stored/archived - Security

- economic espionage
- data leaks
- falsification; i.e. falsified production logs may lead to voided warranty and financial penalties

Logs contain sensitive data - Privacy

Unified, connected and enriched logs are the basis for Business Intelligence.

Logs 3 - Collect, Unify, Connect



ELK stack:

- Elasticsearch as a database for storing the logs.
- Logstash to read different file formats and deliver them to Elasticsearch.
- Kibana to do simple visualization and exploration of data.

Celonis: same idea, different visualization capabilities, more integrated.

Apache Flume: same idea, less polished data analysis and visualization capabilities.

Siemens Mindsphere: same idea, more adapters for industrial hardware and formats.

GE Predix: same idea, used in the US.

Microsoft PowerBI: same idea, works best when you have a certain infrastructure.

Logs 4



Security:

- Depends on software stack
- Fingerprinting individual messages
- Store in blockchain

Privacy:

- Depending on domain; i.e. medical domain (patients) may have higher requirements than manufacturing (workers).
- Data transformation: anonymisation, pseudonymisation (pseudonym translation table available)
- Specialized storage:
 - k-anonymity removes identifiers and pre-groups data
 - differential privacy adds additional artificial data, so that the data analysis is not skewed, but it becomes hard to identify individual subjects

Logs 5 - Analysis & Enrichment



All software stacks allow for standardized analysis:

- Timing analysis is simple
- Analysing semantic coherences always requires custom:
 - extraction of semantic information, e.g. data about quality of a product
 - configuration/code that interpretes/visualizes the information

Enrichment - the more information, the better the analysis:

- No free lunch - specialized code
- Examples:
 - independent sensor streams like temperature, location updates, ...
 - information about orders which have to be extracted from CRM (customer relationship management), ERP (enterprise resource planning), or MES (manufacturing execution system). All this information is probably highly domain specific, stems from different sources (SAP, ...) and has custom formats and information granularity

Logs 6 - Business Logic



How do the stacks handle the underlying business logic/process?

- The process is something that is independent from the log - many components or hardware contribute to the log.
- Mindsphere, PowerBI and Predix force you to recreate the process, i.e. configure how everything is connected together.
- Celonis gives you tools to simplify this recreation, and detect changes.
- All the others are more data-centric (process agnostic); manual work / custom code is necessary.

MAINTENANCE NIGHTMARE: changes to the underlying business logic/process require changes in logging/analysis stack.

Monitoring 1



Works on all information that is contained in a log, **BUT at runtime.**

Relies on data-streams - nothing is ever stored, it is sent through a network connection.

Data streams:

- PubSub
- Push/Pull
- No common / standardized format for the delivered data
- Common protocols: HTTP, MQTT (popular in IoT space).
- Common message types for individual events: Bson (Binary Json) & MsgPack (high performance), XML, YAML, ...
- Connected to enterprise service busses or message routing frameworks (MRF) like Apache Camel, Apache Kafka

Monitoring 2



Data streams ctd:

- Collection, Unification, Connection and Enrichment of individual data streams into one data stream has the same challenges as for logs: custom configuration and code.
- Security: secure protocols.
- Privacy: same challenges as for logs.
- Analysis:
 - Pipeline: e.g. Apache Kafka
 - Visualization - graphs that change at runtime to reflect:
 - the current state of a variable (e.g. stock of raw material)
 - a sliding window (e.g. quality of the last 100 produced products)
 - data drifts (e.g. difference in quality inside a batch of produced products)

Monitoring 3 - Software Stacks



Many of the before mentioned products are bad at monitoring, because they introduced delays and are not particularly well suited for high velocity (UI problem). Celonis, Mindsphere try to tackle the problem (please revisit the SOTA from time to time :-)

- Option 1: ELK - best of the rest for now. Kibana can not be tailored very well, so only for simple things. You probably have to write your own UI.
- Option 2: NASA OpenMCT - you have to bring your own stream analysis code/pipeline, but it helps you with UI.

Use your own message framework, such as ZeroMQ (simple and fast) or Apache Kafka (full featured and complicated) for best results.

Logging vs. Monitoring - Best Practice



Use Both concurrently!

Logging:

- Log only internal information, that helps you fix/debug software related problems in individual modules/services, i.e., exceptions and errors in your code.
- Don't log any business logic / process related information.
- Use highly localized logging so you don't have to enrich or do complex analysis.
- Any stack can be used, even simple and free ones such as ELK.

Monitoring:

- Monitor everything that has to do with your business logic and your processes.
- Will require lots of custom transformation, and connection anyway (e.g. custom machine learning cases).
- Think about feeding the results back into your system (i.e., self-controlling systems) for continuous improvements.

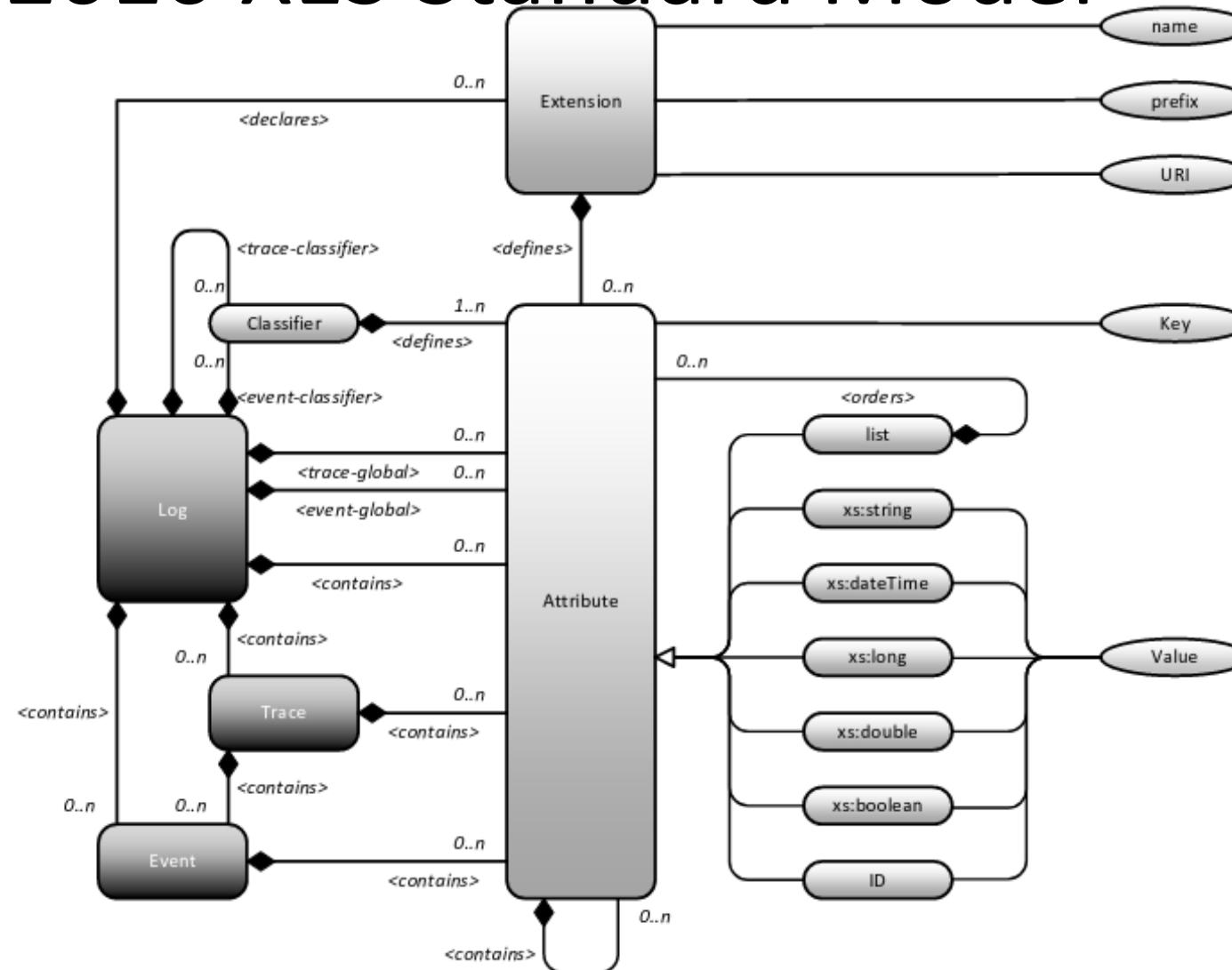
Logging vs. Monitoring - BPM



Use Process Management Engines!

- They separate business logic / processes from functionality / service implementations.
- Functionality / service implementations may log locally.
- The process engine will automatically create a unified, connected, and enriched data stream.
- Effort for custom analysis is the same, but maintainability is much better:
 - No need for custom duplicated code / configuration to collect, unify, and connect data.
 - Enrichment is part of the process model.
 - Analysis is much simpler and straight forward as the analysis pipeline is much shorter.

IEEE 1849-2016 XES Standard Model



IEEE 1849-2016 XES Standard Header



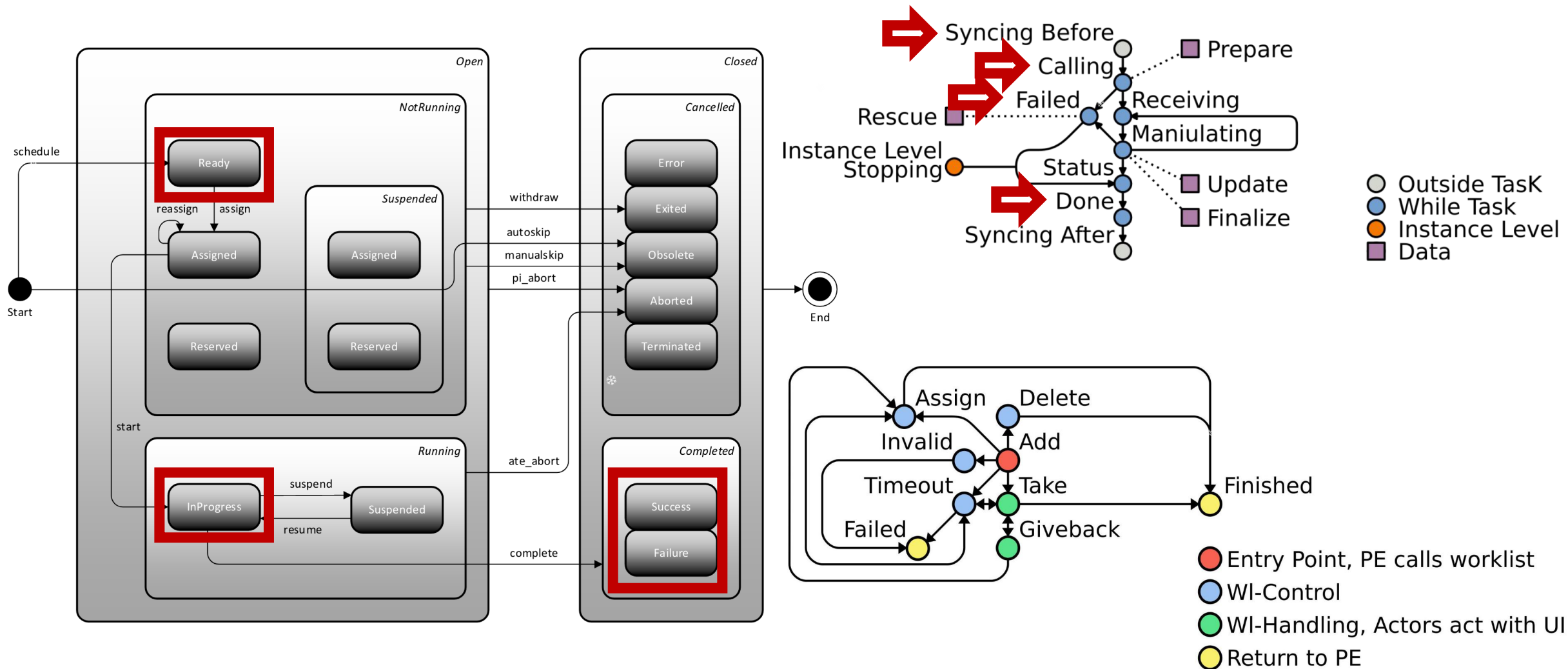
```
<log xes.version="1.0" xmlns="http://code.deckfour.org/xes" xes.creator="Fluxicon Nitro">
  <!-- Extensions describe possible attributes and their data types -->
  <extension name="Concept" prefix="concept" uri="http://code.deckfour.org/xes/concept.xesext"/>
  <extension name="Time" prefix="time" uri="http://code.deckfour.org/xes/time.xesext"/>
  <extension name="Organizational" prefix="org" uri="http://code.deckfour.org/xes/org.xesext"/>
  <global scope="trace"> <!-- Default values for attributes -->
    <string key="concept:name" value=""/>
  </global>
  <global scope="event"> <!-- Structure of events , a set of attributes -->
    <string key="concept:name" value="string"/>
    <string key="identifier:id" value="a0"/>
    <string key="lifecycle:transition" value="unknown"/>
    <date key="time:timestamp" value="1970-01-01T00:00:00.000+01:00"/>
    <list key="parameters">
      <string key="org:resource" value="string"/>
      <string key="Activity" value="string"/>
      <string key="Resource" value="string"/>
      <string key="Costs" value="string"/>
    </list>
  </global>
  <!-- One trace per instance, see lifecycle of instances in Chapter 3 of this lecture -->
  <trace></trace>
  <trace></trace>
</log>
```

IEEE 1849-2016 XES Standard Trace



```
<trace>
  <string key="concept:name" value="Instance 3"/>
  <string key="creator" value="Fluxicon Nitro"/>
  <event>
    <string key="concept:name" value="register request"/>
    <string key="identifier:id" value="a1"/>
    <string key="lifecycle:transition" value="start"/>
    <date key="time:timestamp" value="2010-12-30T14:32:00.000+01:00"/>
    <list key="parameters">
      <string key="org:resource" value="Pete"/>
      <string key="Activity" value="register request"/>
      <string key="Resource" value="Pete"/>
      <string key="Costs" value="50"/>
    </list>
  </event>
  ...
</trace>
```

XES Lifecycle vs. CPEE Lifecycle



XES Lifecycle vs. CPEE Lifecycle



In XES not much is about tasks, major focus on resources (i.e. worklists). Different POVs for worklists:

- XES: Worklist is a part of the process engine and its lifecycle.

VS.

- CPEE: Worklist is just another task, each task can have its own internal lifecycle.

CPEE Data Streams



PubSub, pushes data to HTTP endpoints.

Topics to subscribe to (<https://cpee.org/flow/engine/1/notifications/topics/>):

- Activity: see lifecycle in Chapter 3 of this lecture. Events are sent whenever an activity is called.
- Condition: Events are sent whenever a condition is evaluated.
- Position: Events are sent whenever a task is finished and the thread of control moves on to another task.
- Description: Events are sent whenever the process model is changed.
- State: see lifecycle in Chapter 3 of this lecture. Events are sent whenever the process engine starts/stops/finishes.
- Data elements: Events are sent whenever the process context changes.

CPEE Data Streams



- Endpoints: Events are sent whenever the list of functionalities invoked by a process model/instance changes.
- Attributes: Events are sent whenever attributes of the process model/instance change, e.g. a new author.
- Task: Events are sent whenever a functionality / service implementation wants to communicate internal lifecycle transitions through the process engine (i.e. lifecycle transition forwarding through the process engine).

CPEE Data Stream Subscriptions 1



Each process instance can have many subscriptions. The subscription format is as follows:

```
<subscriptions xmlns="http://riddl.org/ns/common-patterns/notifications-producer/2.0">
  <subscription id="_student" url="https://lehre.bpm.in.tum.de/~tumuser/log.php">
    <topic id="state">
      <event>change</event>
    </topic>
    <topic id="activity">
      <event>calling</event>
      <event>failed</event>
      <event>done</event>
    </topic>
  </subscription>
</subscriptions>
```

CPEE Data Stream Subscriptions 2



Each subscription has:

- an id: e.g. _student
- an endpoint where to push the messages
- a set of topics and events to be pushed

Subscription endpoints might be:

- Apache Kafka or other analysis frameworks
- Custom code for logging or visualization

Summary



Logging should deal with localized errors, **monitoring** should be about making the business logic / process observable.

Logs are stored in standardized log formats such as IEEE 1849-2016 XES.

- Logs have to be collected, unified, connected and enriched for ex-post for analysis.

Monitoring relies on data streams either from:

- Individual software components - these data streams have to be unified, connected and enriched before analysis.
- Process Engines - these data streams can be directly analysed.

Monitoring shows what is going on while it is going on.